

راهنمای توسعه‌دهنده

ساختار، کد و روش تغییر ابزار ParamGen

ParamGen — Developer Manual

نسخه‌ی ابزار ۰۱.۰۱.۰۱

آبان ۱۴۰۵

هر فایل، هر تابع و هر گام تغییر؛ برای برنامه‌نویسی که C می‌داند

فهرست مطالب

۱. درباره‌ی این راهنما ۹

۱.۱ این راهنما برای کیست ۹

۱.۲ چطور بخوانید ۹

۱.۳ نشانه‌گذاری ۹

۱.۴ پروژه‌ی نمونه ۱۰

۱.۵ نسخه و چند قاعده‌ی کلی ۱۱

۲. نگاه کلی ۱۲

۲.۱ سه بخش مخزن ۱۲

۲.۲ نقشه‌ی مخزن ۱۳

۲.۳ برنامه و پروژه ۱۴

۲.۴ دو قالب خروجی ۱۴

۲.۵ فایل‌های ریشه ۱۵

۳. محیط کار و روال یک تغییر ۱۶

۳.۱ آنچه روی رایانه لازم است ۱۶

۳.۲ اجرای ابزار از روی کد ۱۶

۳.۳ روال یک تغییر ۱۷

۳.۴ پیدا کردن چیزی در کد ۱۷

۳.۵ قراردادهای کد ۱۸

۳.۶ git ۱۸

۴. پایتون برای برنامه‌نویس C ۱۹

۴.۱ اجرای پایتون و ساختار یک فایل ۱۹

۴.۲ متغیر و نوع ۱۹

۴.۳ رشته‌ها و f-string ۲۰

۴.۴ شرط و حلقه ۲۱

۴.۵ لیست، تاپل، دیکشنری و مجموعه ۲۲

۴.۶ comprehension و generator ۲۳

۴.۷ تابع‌ها ۲۳

۴.۸ type hint و typing ۲۴

۴.۹ کلاس‌ها ۲۵

۴.۱۰ dataclass و Enum ۲۶
۴.۱۱ استثناها ۲۶
۴.۱۲ with و مدیر زمینه ۲۷
۴.۱۳ ماژول، پکیج و import ۲۸
۴.۱۴ عبارت منظم (re) ۲۸
۴.۱۵ فایل و مسیر ۲۹
۴.۱۶ JSON و XML ۳۰
۴.۱۷ کتابخانه‌های استاندارد دیگر ۳۰
۴.۱۸ دام‌های رایج برای برنامه‌نویس C ۳۱

۵. HTML، CSS و JavaScript برای برنامه‌نویس C ۳۲

۵.۱ HTML ۳۲
۵.۲ CSS ۳۳
۵.۳ JavaScript پایه ۳۳
۵.۴ شیء و آرایه ۳۴
۵.۵ DOM و رویدادها ۳۵
۵.۶ async/await و fetch ۳۶
۵.۷ ابزارهای توسعه‌ی مرورگر ۳۶

۶. قالب فایل‌ها: XML، JSON و الگوها ۳۸

۶.۱ XML ۳۸
۶.۲ JSON و JSONC ۳۹
۶.۳ `pl_project.json` — الگوی فایل پروژه ۴۰
۶.۴ `choices.json` — فهرست‌های کشویی پیش‌فرض شرکت ۴۰
۶.۵ `c.json` — snippetهای VS Code و قالب فایل‌های C شرکت ۴۱
۶.۶ پوشه‌ی var ۴۲

۷. فایل‌های bat و sh ۴۳

۷.۱ فایل bat ویندوز ۴۳
۷.۲ فایل sh لینوکس ۴۵
۷.۳ پایان سطر: CRLF و LF ۴۶

۸. معماری تولیدکننده: pl_codegen ۴۷

۸.۱ بخش‌ها و لایه‌ها ۴۷
۸.۲ شیءهایی که دست‌به‌دست می‌شوند ۴۸
۸.۳ سفر یک پارامتر از XML تا C ۴۸

۹. زیرساخت تولیدکننده‌ی کد ۵۰

۹.۱ `__init__.py` — شناسنامه‌ی پکیج ۵۰

۹.۲ `app_home.py` — برنامه کجاست ۵۱

۹.۳ `errors.py` — خطاها و هشدارها ۵۶

۹.۴ `logging_utils.py` — کنسول و رنگ ۵۹

۹.۵ `naming.py` — از نام پارامتر تا نام‌های C ۶۲

۹.۶ `config.py` — خواندن فایل پروژه ۶۴

۹.۷ `choices.py` — لیست‌های کشویی رایانه ۷۰

۱۰. مدل داده: پوشه‌ی model ۷۳

۱۰.۱ `__init__.py` — درِ ورودی پکیج model ۷۴

۱۰.۲ `types.py` — نوع‌های داده‌ی C ۷۴

۱۰.۳ `tags.py` — فهرست تگ‌ها ۷۶

۱۰.۴ `defines.py` — لیست‌های کشویی در حافظه ۷۸

۱۰.۵ `parameter.py` — یک گره از درخت ۸۰

۱۰.۶ `parameter_list.py` — کل فهرست ۸۵

۱۰.۷ `struct_type.py` — تایپ‌های استراکچر ۸۹

۱۰.۸ `struct_sync.py` — هماهنگ کردن استراکچرها با تایپشان ۹۱

۱۰.۹ `struct_migration.py` — تبدیل استراکچرهای قدیمی ۹۳

۱۱. فایل‌های XML: پوشه‌ی xml_model ۹۵

۱۱.۱ `__init__.py` — درِ ورودی پکیج xml_model ۹۵

۱۱.۲ `schema.py` — نام عنصرها و ویژگی‌ها ۹۵

۱۱.۳ `xml_reader.py` — از XML به مدل ۹۸

۱۱.۴ `xml_writer.py` — از مدل به XML ۱۰۱

۱۱.۵ `types_xml.py` — فایل تایپ‌های استراکچر ۱۰۳

۱۲. بررسی‌ها: پوشه‌ی validation ۱۰۵

۱۲.۱ `__init__.py` — درِ ورودی پکیج validation ۱۰۶

۱۲.۲ `__init__.py` — فهرست همه‌ی قاعده‌ها ۱۰۶

۱۲.۳ `validator.py` — اجرای همه‌ی قاعده‌ها ۱۰۶

۱۲.۴ `rule_base.py` — کلاس پایه‌ی قاعده‌ها و چند کمک‌تابع ۱۰۷

۱۲.۵ `defines_rules.py` — قاعده‌ی فهرست‌های کشویی ۱۰۹

۱۲.۶ `name_rules.py` — قاعده‌های نام‌ها و شناسه‌های ساخته‌شده از آن‌ها ۱۱۰

۱۲.۷ `struct_type_rules.py` — قاعده‌های تایپ‌های استراکچر ۱۱۲

۱۲.۸ type_rules.py — قاعده‌های نوع داده، Type و اندازه‌ی آرایه ۱۱۴
۱۲.۹ range_rules.py — قاعده‌های پیش‌فرض، کمینه، بیشینه و رزولوشن ۱۱۶
۱۲.۱۰ tag_rules.py — قاعده‌های تگ‌ها، نام نمایشی و شناسه‌ی لاگ ۱۱۸
۱۲.۱۱ param_table_rules.py — قاعده‌های قالب param_table ۱۲۰
۱۲.۱۲ افزودن یک قاعده‌ی تازه ۱۲۲

۱۳. چیدمان حافظه: پوشه‌ی layout ۱۲۳

۱۳.۱ __init__.py — درِ ورودی پکیج layout ۱۲۳
۱۳.۲ tag_offset.py — آفست هر مقدار در ناحیه‌ی هر تگ ۱۲۳
۱۳.۳ drift.py — آیا این ساخت چیزی را جابه‌جا کرده است؟ ۱۲۶

۱۴. تولید کد C: پوشه‌ی codegen ۱۳۰

۱۴.۱ __init__.py — درِ ورودی پکیج codegen ۱۳۱
۱۴.۲ c_style.py — شکل فایل‌های شرکت، از روی c.json ۱۳۱
۱۴.۳ descriptor_layout.py — ترتیب فیلدهای sParameterDescriptor ۱۳۳
۱۴.۴ literals.py — نوشتن عددها و رشته‌ها به زبان C ۱۳۴
۱۴.۵ پوشه‌ی blocks: آجرهای فایل C ۱۳۶
۱۴.۶ __init__.py — فهرست بلوک‌ها ۱۳۷
۱۴.۷ include.py — سطر #include ۱۳۷
۱۴.۸ define.py — سطرهای #define ۱۳۷
۱۴.۹ comment.py — کامنت‌ها ۱۳۸
۱۴.۱۰ enum.py — typedef enum ۱۳۸
۱۴.۱۱ variable.py — اعلان و تعریف متغیر ۱۳۹
۱۴.۱۲ function.py — پروتوتایپ و بدنه‌ی تابع ۱۳۹
۱۴.۱۳ table.py — جدول متنی ۱۴۰
۱۴.۱۴ file_banner.py — بنر بالای فایل ۱۴۰
۱۴.۱۵ file_footer.py — سطر آخر فایل ۱۴۰
۱۴.۱۶ include_guard.py — محافظ include دوباره ۱۴۱
۱۴.۱۷ cpp_guard.py — محافظ C++ ۱۴۱
۱۴.۱۸ section_marker.py — نشانگر بخش ۱۴۱
۱۴.۱۹ پوشه‌ی files: یک کلاس برای هر فایل ۱۴۲
۱۴.۲۰ __init__.py — کدام فایل‌ها ساخته شوند ۱۴۲
۱۴.۲۱ file_builder.py — کلاس پایه‌ی همه‌ی سازنده‌ها ۱۴۳
۱۴.۲۲ parameter_list_defines_h.py — فایل cg_parameter_list_defines.h ۱۴۵

۱۴.۲۳ parameter_types_h.py — فایل cg_parameter_list_types.h ۱۴۷
۱۴.۲۴ smart_data_h.py — فایل cg_smart_data.h ۱۴۷
۱۴.۲۵ smart_data_c.py — فایل cg_smart_data.c ۱۴۸
۱۴.۲۶ parameter_list_h.py — فایل cg_parameter_list.h ۱۴۹
۱۴.۲۷ parameter_list_c.py — فایل cg_parameter_list.c ۱۵۰
۱۴.۲۸ parameter_list_table_h.py — فایل cg_parameter_list_table.h ۱۵۱
۱۴.۲۹ param_table_h.py — قالب param_table: فایل nsrParamTables.h ۱۵۲
۱۴.۳۰ param_offsets_h.py — قالب param_table: فایل nsrParamOffsets.h ۱۵۴
۱۴.۳۱ system_names_h.py — قالب param_table: فایل nsrSystemNames.h ۱۵۵

۱۵. خط‌لوله و خط فرمان ۱۵۷

۱۵.۱ pipeline.py — خط‌لوله‌ی مشترک ۱۵۷
۱۵.۲ cli.py — خط فرمان تولیدکننده ۱۶۰

۱۶. ویرایشگر: نگاه کلی ۱۶۳

۱۶.۱ __init__.py — درِ ورودی پکیج pl_gui ۱۶۴
۱۶.۲ browser.py — پیدا کردن مرورگر و باز کردن آن بی‌شبکه ۱۶۴
۱۶.۳ native_dialog.py — پنجره‌های Open و Browse خود ویندوز ۱۶۵

۱۷. سرور، فضای کار و فایل پروژه ۱۶۸

۱۷.۱ server.py — سرور HTTP کوچک ۱۶۸
۱۷.۲ workspace.py — پروژه‌ی باز و پروژه‌های اخیر ۱۷۰
۱۷.۳ config_file.py — نوشتن فهرست ساب‌لیست‌ها در pl_project.json، با حفظ کامنت‌ها ۱۷۳

۱۸. سند در حال ویرایش: document.py ۱۷۵

۱۸.۱ document.py — سند: فهرست باز، فایلش و Undo ۱۷۵

۱۹. API ویرایشگر: api.py ۱۹۳

۱۹.۱ api.py — یک متد برای هر کار صفحه ۱۹۳

۲۰. صفحه‌ی ویرایشگر: پوشه‌ی web ۲۰۲

۲۰.۱ index.html — اسکلت صفحه ۲۰۲
۲۰.۲ style.css — همه‌ی ظاهر ۲۰۳
۲۰.۳ api.js — تنها جایی که با سرور حرف می‌زند ۲۰۴
۲۰.۴ grid.js — جدول سمت چپ ۲۰۵
۲۰.۵ editor.js — فرم سمت راست ۲۰۷
۲۰.۶ app.js — هماهنگ‌کننده‌ی صفحه ۲۰۹

۲۱. راه‌اندازی ابزار: run.bat، run.sh و پوشه‌ی bin ۲۱۵

۲۱.۱ run.bat — تنها فایلی که کاربر ویندوز اجرا می‌کند ۲۱۵

۲۱.۲ find_python.bat — پیدا کردن پایتونی که واقعاً اجرا شود ۲۱۶

۲۱.۳ run.sh — همتای لینوکسی run.bat ۲۱۶

۲۱.۴ run.py — منوی همه‌ی کارها ۲۱۷

۲۱.۵ چهار درِ ورودی: run_gui، run_generate، run_tests و run_setup ۲۱۸

۲۱.۶ setup_environment.py — آیا این رایانه می‌تواند ابزار را اجرا کند؟ ۲۱۹

۲۱.۷ تغییر نسخه‌ی ابزار: set_version.bat و set_version.py ۲۲۰

۲۲. کتابخانه‌ی C: smart_data و parameter_descriptor ۲۲۳

۲۲.۱ smart_data.h — رابط smart data ۲۲۳

۲۲.۲ smart_data.c — پیاده‌سازی smart data ۲۲۶

۲۲.۳ parameter_descriptor.h — شکل یک ردیف جدول پارامترها ۲۲۷

۲۲.۴ parameter_descriptor.c — نوشتن از راه جدول و رویداد تغییر ۲۲۸

۲۲.۵ parameter_list_usage_example.c — نمونه‌ی استفاده از کد تولیدی در فرم‌ور ۲۲۹

۲۳. آزمون‌ها: پوشه‌ی tests ۲۳۱

۲۳.۱ run_validation_test.py — قاعده‌های بررسی ۲۳۲

۲۳.۲ run_editor_test.py — ویرایشگر ۲۳۳

۲۳.۳ run_settings_test.py — هر تنظیم اثری دارد؟ ۲۳۷

۲۳.۴ run_docs_test.py — راهنماها با کد می‌خوانند؟ ۲۳۸

۲۳.۵ run_version_test.py — set_version فقط نسخه را عوض می‌کند ۲۳۸

۲۳.۶ run_struct_test.py — تایپ‌های استراکچر ۲۳۹

۲۳.۷ run_paramtable_test.py — قالب param_table ۲۴۰

۲۳.۸ run_compile_test.py — کد تولیدی کامپایل و اجرا می‌شود ۲۴۳

۲۴. دستورالعمل‌های تغییر ۲۴۵

۲۴.۱ افزودن یک نوع داده‌ی C ۲۴۶

۲۴.۲ افزودن یک تگ ۲۴۶

۲۴.۳ افزودن یک قاعده‌ی بررسی ۲۴۷

۲۴.۴ افزودن یک فایل تولیدی ۲۴۷

۲۴.۵ تغییر محتوای یک فایل تولیدی ۲۴۸

۲۴.۶ تغییر بنر، نشانگرها و کپی‌رایت ۲۴۸

۲۴.۷ افزودن کلید به فایل پروژه ۲۴۸

۲۴.۸ افزودن فیلد به پارامتر ۲۴۹

۲۴.۹ تغییر نام یک ویژگی XML بی شکستن فایل‌های قدیمی ۲۴۹
۲۴.۱۰ افزودن یک Type ۲۵۰
۲۴.۱۱ افزودن مقدار به فهرست‌های کشویی پیش‌فرض ۲۵۰
۲۴.۱۲ تغییر مقدارهای شروع پارامتر تازه ۲۵۱
۲۴.۱۳ افزودن ستون به جدول ویرایشگر ۲۵۱
۲۴.۱۴ افزودن خانه به فرم ویرایشگر ۲۵۱
۲۴.۱۵ افزودن کار تازه به ویرایشگر ۲۵۲
۲۴.۱۶ افزودن میان‌بر صفحه‌کلید ۲۵۲
۲۴.۱۷ تغییر رنگ‌ها و ظاهر ۲۵۲
۲۴.۱۸ افزودن گزینه به منوی run.bat ۲۵۳
۲۴.۱۹ افزودن گزینه به خط فرمان تولیدکننده ۲۵۳
۲۴.۲۰ تغییر پیشوند نام‌های تولیدی ۲۵۳
۲۴.۲۱ تغییر نام یک مفهوم در همه‌جا ۲۵۳
۲۴.۲۲ بالا بردن کمینه‌ی نسخه‌ی پایتون ۲۵۳
۲۴.۲۳ تغییر یک پیام خطا ۲۵۴
۲۴.۲۴ آماده کردن نسخه‌ی تازه برای تحویل ۲۵۴

۲۵. اشکال‌زدایی ۲۵۵

۲۵.۱ خواندن traceback پایتون ۲۵۵
۲۵.۲ پیام‌های ابزار ۲۵۵
۲۵.۳ لاگ و چاپ ۲۵۵
۲۵.۴ اشکال‌زدایی ویرایشگر ۲۵۶
۲۵.۵ آزمودن تکه‌ای از کد در پایتون تعاملی ۲۵۶
۲۵.۶ توقف و قدم‌به‌قدم: breakpoint ۲۵۷
۲۵.۷ وقتی کد C تولیدی کامپایل نمی‌شود ۲۵۷
۲۵.۸ ردیابی یک مشکل از صفحه تا فایل تولیدی ۲۵۸
۲۵.۹ مشکل‌های رایج ۲۵۸

پیوست الف. فهرست همه‌ی فایل‌ها ۲۶۰

پیوست ب. پیام‌های ابزار ۲۶۶

پیوست پ. واژه‌نامه ۲۷۱

پیوست ت. نمایه ۲۷۳

۱. درباره‌ی این راهنما

این راهنما برای کسی است که باید **ابزار ParamGen را تغییر دهد**: قابلیتی اضافه کند، باگی را رفع کند یا رفتاری را عوض کند. هدفش این است که برای هیچ تغییری به اینترنت یا هوش مصنوعی نیاز نباشد: هر فایل، هر تابع، هر قرارداد و هر روال تغییر اینجا نوشته شده است.

۱.۱ این راهنما برای کیست

فرض شده خواننده **زبان C را کمی می‌داند** (متغیر، تابع، struct، اشاره‌گر، `#include`) و پایتون، JavaScript، HTML، CSS و XML را **نمی‌داند**. برای همین:

- فصل ۴ پایتون را از صفر، با مقایسه با C و فقط با چیزهایی که در همین پروژه به کار رفته، آموزش می‌دهد.
- فصل ۵ همین کار را برای HTML، CSS و JavaScript صفحه‌ی ویرایشگر می‌کند، و فصل‌های ۶ و ۷ برای قالب‌های XML و JSON و اسکریپت‌های bat و sh.
- هر مثال، تا جای ممکن، از **کد واقعی پروژه** یا **خروجی واقعی ابزار** آمده است، نه مثال‌های ساختگی.

برای **استفاده** از ابزار (ساختن پروژه، ویرایش پارامترها، تولید کد) راهنمای کاربر را بخوانید؛ این راهنما فرض می‌کند با کار با ویرایشگر آشنا هستید یا راهنمای کاربر را کنار دستتان دارید.

۱.۲ چطور بخوانید

بخوانید	اگر می‌خواهید...
اول فصل ۲۴ (دستورالعمل‌ها)؛ هر دستورالعمل به فصل‌های لازم ارجاع می‌دهد	یک تغییر مشخص بدهید (مثلاً یک تگ تازه)
فصل‌های ۲، ۳ و ۸، بعد ۹ تا ۱۵ (تولیدکننده) و ۱۶ تا ۲۰ (ویرایشگر)	ابزار را از پایه بفهمید
فصل‌های ۴ تا ۷، پیش از هر فصل دیگر	زبان‌ها را یاد بگیرید
نمایه‌ی پایان راهنما (پیوست ت): نام تابع ← صفحه	بدانید یک تابع چه می‌کند
پیوست ب: هر کد پیام، با متن و جای آن در کد	بدانید یک پیام خطا از کجا می‌آید
فایلش را باز کنید: مسیر فایل بالای بخش آن فایل (و در پیوست الف) آمده است؛ نام تابع را در آن جست‌وجو کنید	کد خود یک تابع را ببینید
فصل ۲۵ (اشکال‌زدایی)	مشکلی را پیدا کنید

۱.۳ نشانه‌گذاری

- `متن با این قلم` نام فایل، تابع، متغیر، کد یا متنی است که دقیقاً همان‌طور تایپ می‌شود.

- **Generate code** برچسب یک دکمه، منو یا خانه در ویرایشگر است، همان‌طور که روی صفحه دیده می‌شود.
- شماره‌ها در متن فارسی‌اند؛ در کد و خروجی‌ها لاتین، چون همان‌طور تایپ و چاپ می‌شوند.
- مسیرها نسبت به ریشه‌ی مخزن نوشته شده‌اند، مثل `paramgen/src/pl_codegen/naming.py`؛ در ویندوز `/` همان `\` است.

هر فایل بخش خودش را دارد که با نام فایل و مسیرش شروع می‌شود. هر تابع، کلاس یا متد یک **مدخل** دارد: نام، نوع (تابع، متد، کلاس، …)، **امضای واقعی** از کد (در کادر خاکستری)، توضیح، و در صورت نیاز جدول ورودی‌ها، خروجی، خطا و مثال. تابع‌های کوچک‌تر در جدول‌های دوستونی آمده‌اند. هیچ تابعی از قلم نیفتاده است؛ ابزار ساخت این راهنما این را بررسی می‌کند.

کادرهای رنگی:

> **نکته:** اطلاعات کمکی که دانستنش مفید است.

> **مهم:** چیزی که اگر رعایت نشود، کار درست انجام نمی‌شود.

> **هشدار:** چیزی که اشتباه در آن به خرابی، داده‌ی غلط روی سخت‌افزار یا از دست رفتن کار می‌انجامد.

> **پیشنهاد:** راه ساده‌تر یا بهتر.

۱.۴ پروژه‌ی نمونه

همه‌ی مثال‌ها از همان پروژه‌ی نمونه‌ی راهنمای کاربر، **Flight Computer** (`D:\Projects\Flight_Computer`)، گرفته شده‌اند. این پروژه دو ساب‌لیست دارد. `core.xml`:

تگ‌های مهم	شناسه‌ی لاگ	واحد	بازه	اندازه	نوع	پارامتر
Loggable	۱۰۰	m/s	۰ تا ۱۲۰	۱	`float32_t`	`Airspeed`
ResetProof، Loggable	۱۰۱	m	۰ تا ۶۰۰۰ (پیش‌فرض ۳۰۰۰)	۱	`uint16_t`	`MaxAltitude`
ResetProof	—	—	۰ تا ۵	۱	`uint8_t`	`FlightMode`
Loggable	۲۰۰ تا ۲۰۳	V	۰ تا ۵	۴	`float32_t`	`CellVoltage`
Loggable	۳۰۰ تا ۳۰۲	—	—	۱	استراکچر `sPosition`	`Position`
Loggable	۴۰۰ تا ۴۰۷	—	—	۴	استراکچر `sMotor`	`Motor`
ResetProof	—	V	۹ تا ۱۲٫۶ (پیش‌فرض ۱۰٫۵)	۱	`float32_t`	`LowBatteryVoltage`

تایپ `sPosition` سه فیلد دارد (`Lat` و `Lon` از نوع `float64_t` بر حسب درجه، و `Alt` از نوع `float32_t` بر حسب متر) و `sMotor` دو فیلد (`Rpm` از نوع `int16_t` و `Temp` از نوع `int8_t`). ساب‌لیست دوم، `power.xml`، دو پارامتر دارد:

`BatteryVoltage` (۰ تا ۳۰ V، شناسه‌ی ۵۰۰) و `ChargeCurrent` (۵۰- تا ۵۰ A، شناسه‌ی ۵۰۱). کل ساخت ۲۱ مقدار دارد.

> **پیشنهاد:** برای آزمودن هر تغییر، از پروژه‌ی خودتان یک **کپی** بسازید یا پروژه‌ی نمونه را طبق راهنمای کاربر دوباره بسازید؛ هرگز روی پروژه‌ی واقعی فرم‌ور آزمایش نکنید.

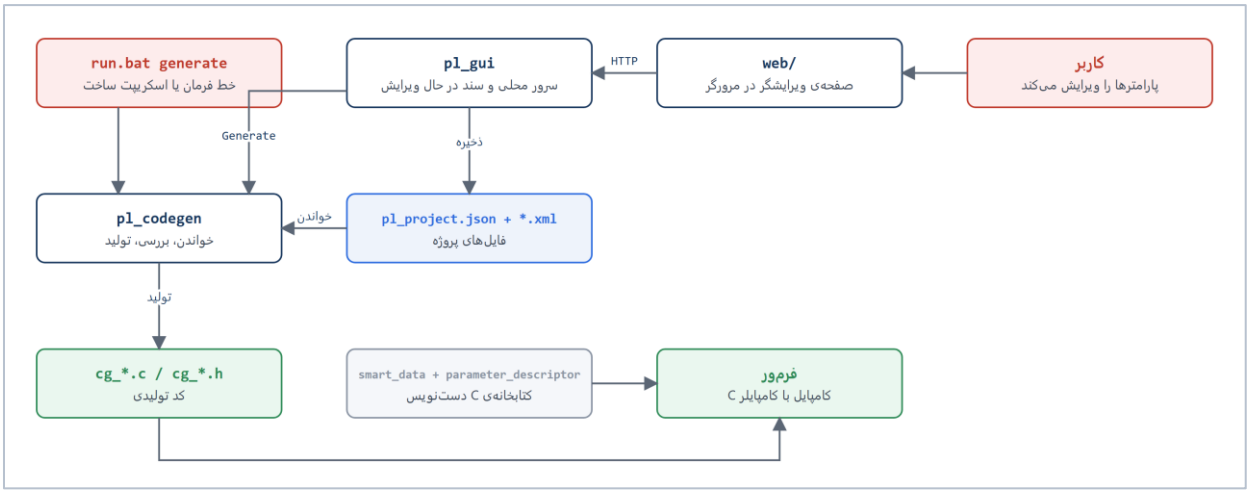
۱.۵ نسخه و چند قاعده‌ی کلی

این راهنما برای نسخه‌ی **۰۱.۰۱.۰۱** ابزار نوشته شده است. قاعده‌هایی که در سراسر کد رعایت شده‌اند و هر تغییری باید رعایتشان کند:

- **فقط کتابخانه‌ی استاندارد پایتون.** رایانه‌های شرکت اینترنت ندارند؛ هیچ `pip install`ی ممکن نیست. آزمون `test_no_external_dependency` هر import بیرونی را رد می‌کند.
- **پایتون ۳٫۱۱ یا تازه‌تر.** ویژگی‌های تازه‌تر از ۳٫۱۱ به کار نروند.
- **بی‌اینترنت در اجرا هم.** صفحه‌ی ویرایشگر هیچ فایلی از بیرون بار نمی‌کند و سرور فقط به `127.0.0.1` گوش می‌دهد.
- **کتابخانه‌ی C (`smart_data` و `parameter_descriptor`) مال فرم‌ور است.** تغییرش فقط با هماهنگی صاحب فرم‌ور (فصل ۲۲).
- **مسیرها نسبی.** هیچ مسیر کاملی از یک رایانه در فایل‌های پروژه یا کد تولیدی نوشته نمی‌شود؛ پروژه باید با جابه‌جایی پوشه‌اش کار کند.
- **هر رفتار تازه یک آزمون.** و پیش از تحویل همه‌ی آزمون‌ها (فصل ۲۳).

۲. نگاه کلی

ParamGen دو کار می‌کند: (۱) یک **ویرایشگر** که فهرست پارامترهای یک فرم‌ور را در قالب چند فایل XML (ساب‌لیست) نگه می‌دارد، و (۲) یک **تولیدکننده** که از آن فایل‌ها کد C می‌سازد: شیء smart data برای هر مقدار و یک جدول ثابت که هر پارامتر را توصیف می‌کند. فرم‌ور آن کد را کنار دو کتابخانه‌ی دست‌نویس `smart_data` و `parameter_descriptor` کامپایل می‌کند.



شکل ۱ — بخش‌های ابزار و مسیر داده از ویرایشگر تا فرم‌ور

۲.۱ سه بخش مخزن

زبان	چیست	پوشه
پایتون، JavaScript، HTML، CSS	خود ابزار: ویرایشگر و تولیدکننده (به آن «برنامه» یا app می‌گوییم)	`paramgen/`
C	کتابخانه‌ی دست‌نویس مقدارهای هوشمند؛ مال فرم‌ور	`smart_data/`
C	کتابخانه‌ی دست‌نویس توصیف‌گر پارامتر؛ مال فرم‌ور	`parameter_descriptor/`

درون `paramgen/src` دو پکیج پایتون هست که در سراسر راهنما با نامشان می‌آیند:

- `pl_codegen`: هسته؛ خواندن و نوشتن XML، مدل داده، بررسی، چیدمان حافظه و تولید کد، و خط فرمان. هیچ چیزی از ویرایشگر نمی‌داند (فصل‌های ۸ تا ۱۵).
- `pl_gui`: ویرایشگر؛ سرور محلی، API و صفحه‌ی وب. از `pl_codegen` استفاده می‌کند، نه برعکس (فصل‌های ۱۶ تا ۲۰).

۲.۲ نقشه‌ی مخزن

پوشه‌ها و فایل‌های اصلی مخزن (جز فایل‌های موقت پایتون __pycache__):

فصل	چیست	مسیر
۲	معرفی کوتاه مخزن	README.md
۲	تنظیمات git: پایان سطرها، فایل‌های نادیده	.gitattributes ،.gitignore
۲۲	smart_data.h و smart_data.c	smart_data/
۲۲	parameter_descriptor.h و .c	parameter_descriptor/
۲۱	تنها فایل‌هایی که کاربر اجرا می‌کند	paramgen/run.bat ،run.sh
۲	معرفی برنامه	paramgen/README.md
۲۱	درهای ورودی: run.py ،run_*.py ،set_version	paramgen/bin/
۲۱	بررسی رایانه	paramgen/src/setup_environment.py
۸ و ۹	هسته و تولیدکننده؛ ۸ فایل پایه (app_home ،config ،errors ،naming ،...)	paramgen/src/pl_codegen/
۱۰	مدل داده	.../pl_codegen/model/
۱۱	خواندن و نوشتن XML	.../pl_codegen/xml_model/
۱۲	قاعده‌های بررسی	.../pl_codegen/validation/rules/
۱۳	آفست‌ها و جابه‌جایی	.../pl_codegen/layout/
۱۴	تولید کد C: blocks/ و files/	.../pl_codegen/codegen/
۱۵	خط‌لوله و خط فرمان	.../pl_codegen/pipeline.py ،cli.py
۱۶ تا ۱۹	ویرایشگر	paramgen/src/pl_gui/
۲۰	صفحه: index.html ،style.css ،*.js	.../pl_gui/web/
۶	الگوی پروژه، فهرست‌های کشویی، c.json	paramgen/templates/
۲۳	هشت مجموعه آزمون	paramgen/tests/
۲۲	راهنماها و docs/example (نمونه‌ی C)	paramgen/docs/
۶	وضعیتی که برنامه می‌نویسد: settings.json، choices.json	paramgen/var/

نکته: فهرست کامل فایل‌ها با بخش مربوط به هرکدام در پیوست الف آمده است.

۲.۳ برنامه و پروژه

مهم‌ترین مفهوم برای فهم ساختار: **برنامه** (پوشه‌ی `paramgen/`) یک بار روی هر رایانه کپی می‌شود و هنگام کار در آن چیزی نوشته نمی‌شود (جز `var/`). **پروژه** پوشه‌ای جداست، هر جای دیگری، با یک فایل `pl_project.json`؛ مثل ویرایشگر متنی که یک بار نصب می‌شود و فایل‌های زیادی را باز می‌کند.

یک پروژه پس از اولین تولید کد، درون `D:\Projects\Flight_Computer\`:

چیست	مسیر
تنظیمات پروژه	`pl_project.json`
یک ساب‌لیست و تایپ‌هایش	`parameter_lists\core.xml`، `core_types.xml`
ساب‌لیست دوم	`parameter_lists\power.xml`
خروجی تولید	`cg_parameter_list\cg_parameter_list.c`، `.h`، …
کپی مدل ادغام‌شده، برای مقایسه‌ی ساخت بعدی	`cg_parameter_list\parameter_list.xml`

مال کیست	کجاست	چیز
پروژه	پوشه‌ی پروژه	ساب‌لیست‌ها، تایپ‌ها، تنظیمات، کد تولیدی
رایانه؛ همه‌ی پروژه‌ها	`paramgen/var/choices.json` (پیش‌فرض: `templates/choices.json`)	فهرست‌های کشویی (`SYSTEM NAMES`، …)
رایانه	`paramgen/var/settings.json`	پروژه‌های اخیر و پروژه‌ی باز
برنامه	`paramgen/templates/`	الگوی فایل پروژه و بنر فایل‌های C

۲.۴ دو قالب خروجی

کلید `output.format` فایل پروژه یکی از دو قالب را انتخاب می‌کند:

برای	خروجی	قالب
فرم‌ور تازه	هفت فایل `cg_*`: شیءهای smart data و جدول توصیف‌گرها	`smart_data` (پیش‌فرض)
فرم‌ور قدیمی؛ بی استراکچر، آرایه و بولی	`nsrParamTables.h` (سطرهای `ADD_PARAM`)، `nsrParamOffsets.h` (فقط کامنت) و `nsrSystemNames.h` (نام سیستم‌ها با عددشان)	`param_table`

ساب‌لیست‌ها در هر دو قالب همان XMLاند؛ پس یک ساب‌لیست در هر دو نوع پروژه باز می‌شود، ولی در `param_table` با قاعده‌های بیشتری بررسی می‌شود (بخش ۱۲.۱۱).

۲.۵ فایل‌های ریشه

README.md — معرفی مخزن

مسیر: README.md

به انگلیسی، سه بخش مخزن را معرفی می‌کند و خواننده را به paramgen/docs/ و paramgen/README.md می‌فرستد. کدی ندارد.

paramgen/README.md — معرفی برنامه

مسیر: paramgen/README.md

راه‌اندازی (run.bat و run.sh و چهار گزینه‌ی منو)، مفهوم برنامه و پروژه، دو قالب خروجی و پوشه‌ها. آزمون run_docs_test.py این فایل را هم بررسی می‌کند (فصل ۲۳).

> **نکته:** این README درباره‌ی لینوکس می‌گوید run.sh «python3 نسخه‌ی ۳٫۸ یا تازه‌تر» را پیدا می‌کند؛ کد امروز ۳٫۱۱ را می‌خواهد. اگر README را به‌روز می‌کنید، این عدد را هم درست کنید.

.gitattributes — پایان سطرها در git

مسیر: .gitattributes

به git می‌گوید پایان سطر کدام فایل‌ها ثابت بماند، حتی در کلون ویندوزی با core.autocrlf=true:

```
*.sh text eol=lf
*.bat text eol=crlf
paramgen/var/settings.json text eol=lf
paramgen/var/choices.json text eol=lf
```

run.sh با CRLF روی لینوکس اجرا نمی‌شود؛ و دو فایل var/ را برنامه همیشه با LF می‌نویسد، پس بی این سطرها git همیشه آن‌ها را «تغییرکرده» نشان می‌داد.

.gitignore — فایل‌هایی که در git نمی‌روند

مسیر: .gitignore

__pycache__/ و *.py[cod] (فایل‌های کامپایل‌شده‌ی پایتون که هر اجرا برای نسخه‌ی پایتون همان رایانه می‌سازد) و /paramgen/var/settings.json (پروژه‌های اخیر هر رایانه).

۳. محیط کار و روال یک تغییر

۳.۱ آنچه روی رایانه لازم است

چیز	برای چه	از کجا
پایتون ۳٫۱۱ یا تازه‌تر	اجرای ابزار و آزمون‌ها	نصب‌کننده‌ی رسمی python.org؛ گزینه‌ی «Add python.exe to PATH» را تیک بزنید
یک ویرایشگر متن	خواندن و تغییر کد	VS Code (پیشنهادی) یا Notepad++؛ هر ویرایشگری که UTF-8 و پایان سطر LF/CRLF را درست نگه دارد
gcc و g++ (مثلاً MinGW)	آزمون run_compile_test.py	بسته‌ی آفلاین MinGW-w64؛ پوشه‌ی bin آن در PATH
Microsoft Edge یا Chrome	پنجره‌ی ویرایشگر	معمولاً روی ویندوز هست

هیچ چیز دیگری لازم نیست؛ نه pip، نه اینترنت. اینکه پایتون رایانه کافی است را گزینه‌ی ۴ منو (run.bat setup) می‌گوید. اگر چند نسخه‌ی پایتون نصب است، در ویندوز با py -3.11 نسخه‌ی مشخص را اجرا کنید؛ py --list نسخه‌های نصب‌شده را نشان می‌دهد.

۳.۲ اجرای ابزار از روی کد

ابزار هیچ مرحله‌ی «ساخت» یا نصب ندارد: پایتون مستقیم از روی فایل‌های .py اجرا می‌شود. پس هر تغییری در کد، در اجرای **بعدی** دیده می‌شود. اگر ویرایشگر باز است، برای دیدن تغییر پایتون آن را ببندید (Ctrl+C در پنجره‌ی کنسول) و دوباره باز کنید؛ تغییر فایل‌های web/ فقط با بارگذاری دوباره‌ی صفحه (F5) دیده می‌شود.

کار	فرمان (از پوشه‌ی paramgen)
منو	run.bat
ویرایشگر با یک پروژه	run.bat editor D:\Projects\Flight_Computer
ویرایشگر در زبانه‌ی مرورگر، روی درگاه دیگر	py bin\run_gui.py --tab -p 8800
تولید کد یک پروژه	run.bat generate -c D:\Projects\Flight_Computer
فقط بررسی، با پیام‌های اطلاع	run.bat generate -c ... --check -v
همه‌ی آزمون‌ها	run.bat tests
یک مجموعه آزمون	py tests\run_editor_test.py

۳.۳ روال یک تغییر

هر تغییری، کوچک یا بزرگ، همین گام‌ها را دارد:

- **۱. بفهمید کجاست.** فصل مربوط را بخوانید؛ اگر نمی‌دانید، دستورالعمل‌های فصل ۲۴ را ببینید، یا متنی را که روی صفحه می‌بینید (پیام، برچسب دکمه) در کد جست‌وجو کنید (بخش بعد).
- **۲. آزمون‌ها را پیش از تغییر اجرا کنید** تا مطمئن شوید همه می‌گذرند؛ وگرنه نمی‌دانید شکستی که بعداً می‌بینید از شماست یا نه.
- **۳. تغییر را بدهید.** در همان سبک کد اطراف (بخش «قراردادهای کد» همین فصل).
- **۴. آزمون بنویسید** که رفتار تازه را می‌سنجد (فصل ۲۳). اگر باگ رفع می‌کنید، آزمونی که بدون رفع شکست می‌خورد.
- **۵. همه‌ی آزمون‌ها را اجرا کنید.** با کامپایلر نصب‌شده تا `run_compile_test.py` واقعاً اجرا شود.
- **۶. در ویرایشگر امتحان کنید**، روی **کپی** یک پروژه: همان کاری که کاربر می‌کند. نام‌های پیش‌فرض را هم امتحان کنید (`NewParameter` در دو سب‌لیست)؛ بسیاری از باگ‌ها فقط آنجا دیده می‌شوند.
- **۷. کد را تولید و کامپایل کنید** اگر تغییر به خروجی C می‌رسد.
- **۸. راهنماها را به‌روز کنید** (راهنمای کاربر اگر رفتار دیده‌شده عوض شده، و این راهنما)، و اگر نسخه‌ی تازه منتشر می‌شود `bin\set_version.bat` (بخش ۲۱.۶).

۳.۴ پیدا کردن چیزی در کد

سریع‌ترین راه پیدا کردن جای یک رفتار، جست‌وجوی **متنی است که می‌بینید**. هر پیام، برچسب و کد خطا دقیقاً یک جا در کد نوشته شده است.

جست‌وجو کنید	می‌خواهید بدانید...
`"TAG012"` در `src`	پیام `TAG012` از کجا می‌آید
`Next free` در `web/` (← `btn-next-log-id` در `editor.js`)، بعد `nextLogId`، بعد `next_log_id` در `api.py`	دکمه‌ی **Next free** چه می‌کند
یک تکه‌ی ثابت آن خط، مثل `Always` یا `fParameterList_GetBy` در `codegen/files`	یک خط فایل C تولیدی را کی می‌نویسد
نام تابع و `(` بعدش، مثلاً `merge_lists(`	چه کسی یک تابع را صدا می‌زند

در VS Code: پوشه‌ی مخزن را باز کنید (File ▸ Open Folder) و Ctrl+Shift+F. در خط فرمان ویندوز:

```
cd paramgen\src
findstr /s /n /c:"TAG012" *.py

pl_codegen\validation\rules\tag_rules.py:293:                    "TAG012",
```

(`/s` زیرپوشه‌ها، `/n` شماره‌ی سطر و `/c:` یعنی کل متن داخل گیومه یک عبارت است.)

۳.۵ قراردادهای کد

کد این پروژه یکدست نوشته شده و تغییرها باید همین‌طور بمانند:

نمونه	قرارداد	چیز
`build_offset_map`، `log_id`	`snake_case`	نام تابع، متغیر، ماژول
`ParameterList`، `LogIdRule`	`PascalCase`	نام کلاس
`TAG_REGISTRY`، `UNDO_DEPTH`	`UPPER_SNAKE_CASE`	ثابت سطح ماژول
`_copy_name`، `_change`	با _ شروع می‌شود؛ از بیرون صدا زده نشود	چیز «خصوصی» ماژول یا کلاس
`fParameterList_Init`، `sSmartDataU16`، `eSMART_DATA_OK`	قالب فرمور شرکت	نام تابع و ماکروی C تولیدی
`applyChanges`، `gridView`	`camelCase`	نام JS

- **هر فایل پایتونی که کد دارد با `from __future__ import annotations` شروع می‌شود** (فایل‌های `__init__.py` که فقط نام صادر می‌کنند نه) و تابع‌ها type hint دارند (بخش ۴.۸).
- **کامنت «چرا» را می‌گوید، نه «چه»**، و اغلب ماجرای باگی را که این شکل کد جلویش را می‌گیرد. پیش از ساده کردن کدی که «زیادی محتاط» به نظر می‌رسد، کامنتش را بخوانید.
- **پیام‌ها به انگلیسی** و به شکل جمله‌ای که کاربر بفهمد: چه شد، کجا، و چه کند (`hint`).
- **هر تصمیم یک جا.** نام یک ویژگی XML فقط در `schema.py`، ترتیب فیلدهای توصیف‌گر فقط در `descriptor_layout.py`، نسخه فقط در `__init__.py`، دو قالب خروجی فقط در `builders_for`. اگر چیزی را در دو جا می‌نویسید، احتمالاً جای درستش را پیدا نکرده‌اید.
- **ورودی کاربر هرگز بی‌صدا دور ریخته نمی‌شود:** یا پذیرفته می‌شود یا با پیام رد می‌شود.
- **فایل‌ها با UTF-8** خوانده و نوشته می‌شوند (`encoding="utf-8"`)؛ فایل‌های تولیدی با پایان سطر `\n`.

git ۳.۶

مخزن با git نگهداری می‌شود و کار git (commit، branch، push) با صاحب پروژه است. دو فایل `.gitattributes` و `.gitignore` (فصل ۲) پایان سطر فایل‌های حساس و فایل‌هایی را که نباید در git بروند تعیین می‌کنند. اگر فایل تازه‌ای با پایان سطر خاص اضافه می‌کنید (مثلاً اسکریپت `.sh`)، الگوی آن در `.gitattributes` هست یا نه را بررسی کنید.

۴. پایتون برای برنامه‌نویس C

این فصل همه‌ی پایتونی را که در کد این پروژه به کار رفته، از صفر و با مقایسه با C آموزش می‌دهد؛ نه بیشتر. هر بخش با کد واقعی پروژه تمام می‌شود. اگر C می‌دانید، بیشتر مفهوم‌ها آشنا هستند؛ تفاوت‌ها در شکل نوشتن و چند رفتار متفاوت است که در بخش ۴.۱۸ جمع شده‌اند.

۴.۱ اجرای پایتون و ساختار یک فایل

پایتون **مفسری** است: کامپایل و لینک جدا ندارد. `py -3.11 file.py` فایل را از بالا به پایین اجرا می‌کند. تابع `main` خودکار صدا زده نمی‌شود؛ رسم این است که پایان فایل بنویسیم:

```
def main() -> int:
    print("hello")
    return 0

if __name__ == "__main__":      # فقط وقتی همین فایل اجرا شده، نه import
    raise SystemExit(main())    # همان return main() در C؛ کد خروج
```

تفاوت‌های ظاهری با C:

پایتون	C
تورفتگی (چهار فاصله) بلوک را می‌سازد؛ تورفتگی غلط خطاست	بلوک `{ ... }`
پایان سطر؛ `;` لازم نیست	پایان دستور `;`
`#` تا پایان سطر	`//` و `/* ... */`
docstring: رشته‌ی `"""..."""` درست زیر `def`	توضیح تابع در کامنت بالایش
`if x:` (بدون پرانتز، با :)	`if (x) {`
`and`، `or`، `not`	`&&`، `||`، `!`
`None`	`NULL`
`False`/`True`	`false`/`true`

یک سطر طولانی را درون پرانتز می‌شود شکست؛ پایتون تا بسته شدن پرانتز سطر را ادامه می‌داند. برای آزمودن یک تکه کد، `py -3.11` را بی‌فایل اجرا کنید تا اعلان `>>>` بیاید و دستور به دستور بنویسید (بخش ۲۵.۵).

۴.۲ متغیر و نوع

متغیر **اعلان** نمی‌شود؛ با اولین مقداردهی ساخته می‌شود. نوع مال **مقدار** است، نه متغیر: یک متغیر می‌تواند بعداً مقداری از نوع دیگر بگیرد (این کار را نکنید، ولی ممکن است).

نوع پایتون	معادل C	نکته
`int`	`int` و همه‌ی نوع‌های صحیح	**بی‌اندازه**: سرریز ندارد؛ `2**64` درست حساب می‌شود. بازه‌ی نوع C را خودمان بررسی می‌کنیم (`RNG001`)
`float`	`double`	همیشه ۶۴ بیتی؛ برای `float32_t` گرد کردن را خودمان شبیه‌سازی می‌کنیم (`_as_float32`)
`bool`	`bool`	`False`/`True`؛ زیرنوع `int` است: `True == 1`
`str`	`char *`	متن یونیکد، تغییرناپذیر؛ طول با `len(s)`
`bytes`	`uint8_t[]`	بایت‌های خام؛ `b"..."`
`None`	`NULL`	«مقداری نیست»؛ با `is None` مقایسه می‌شود

تقسیم: `2 / 7` برابر `3.5` است (همیشه اعشاری) و `2 // 7` برابر `3` (تقسیم صحیح). باقی‌مانده `%`، توان `**`. عملگرهای بیتی (`&`، `|`، `~`، `<<`) مثل C‌اند. بررسی نوع در زمان اجرا: `isinstance(value, int)`. چون `bool` زیرنوع `int` است، کد پروژه برای «عدد صحیحِ واقعی» همیشه هر دو را می‌پرسد:

```
if not isinstance(log_id, int) or isinstance(log_id, bool):
    continue        # True هم int نیست؛ شناسه‌ی لاگ است
```

۴.۳ رشته‌ها و f-string

رشته با `'...'` یا `"..."` (یکسان) و رشته‌ی چندخطی با `"""..."""`. به‌جای `sprintf` از **f-string** استفاده می‌شود: `f` پیش از گیومه و عبارت‌ها درون `{}`:

```
name, size = "CellVoltage", 4
text = f"#define {name.upper()}_ARRAY_SIZE  ({size}U)"
# '#define CELLVOLTAGE_ARRAY_SIZE  (4U)'

f"{3.14159:.2f}"      # '3.14'   (مثل %.2f)
f"{42:>6}"            # '    42' (راست‌چین در ۶ نویسه)
f"{'Index':<10}|"     # 'Index     |' (چپ‌چین)
```

کار	نمونه	نتیجه
بزرگ/کوچک	`"Loggable".upper()`	`'LOGGABLE'`
حذف فاصله‌ی دو سر	`"  x ".strip()`	`'x'`
شکستن	`"Motor.Rpm".split(".")`	`['Motor', 'Rpm']`
چسباندن	`", ".join(["a", "b"])`	`'a, b'`
جایگزینی	`"a-b".replace("-", "_")`	`'a_b'`
شروع/پایان	`name.endswith(".xml")`	`False/True`
بخشی از رشته	`"Airspeed"[0:3]`	`'Air'`
وجود زیررشته	`"[" in path`	`False/True`

بک‌اسلش درون رشته‌ی معمولی escape است (`"\n"`)؛ برای مسیر ویندوز یا عبارت منظم از **رشته‌ی خام** `r"..."` استفاده می‌شود که `\` را همان‌طور نگه می‌دارد: `r"\d{2}"`.

۴.۴ شرط و حلقه

```
if size < 1:
    print("invalid")
elif size > 65535:          # else if
    print("too big")
else:
    print("ok")

for index in range(4):      # for (index = 0; index < 4; index++)
    print(index)            # 0 1 2 3

for parameter in model.parameters:        # روی عنصرهای لیست، بی شمارنده
    print(parameter.name)

for index, parameter in enumerate(model.parameters):   # با شمارنده
    print(index, parameter.name)

while index < count:
    index += 1              # ++ وجود ندارد
```

`break` و `continue` مثل C‌اند. `switch` نیست؛ زنجیره‌ی `elif`/`if` یا یک دیکشنری به کار می‌رود. عبارت شرطی کوتاه: `a if cond else b` (همان `cond ? a : b`):

```
kind = "struct" if parameter.is_struct else "value"
```

در شرط، هر مقداری درست یا نادرست حساب می‌شود («truthiness»): `None`، `0`، `""`، لیست و دیکشنری خالی **نادرست‌اند**، بقیه درست. پس `if not values:` یعنی «لیست خالی است». دقت کنید صفر هم نادرست است؛ برای همین کد پروژه برای «مقدار داده نشده» همیشه `is None` می‌پرسد، نه `not value`:

```
default = 0 if parameter.default_value is None else parameter.default_value
```

۴.۵ لیست، تاپل، دیکشنری و مجموعه

ویژگی	شبیه C	نوشتن	نوع
تغییرپذیر؛ `append`، `insert`، `pop`، `remove`، `len`	آرایه‌ی پویا	`[1, 2, 3]`	`list`
تغییرناپذیر؛ برای برگرداندن چند مقدار	struct بی‌نام کوچک	`(1, 2)`	`tuple`
جست‌وجوی سریع با کلید؛ ترتیب افزودن حفظ می‌شود	جدول کلید ← مقدار	`{"a": 1}`	`dict`
بی‌تکرار؛ «آیا هست؟» سریع	—	`{1, 2}`	`set`

```
names = ["Airspeed", "MaxAltitude"]
names.append("FlightMode")
names[0]          # 'Airspeed'
names[-1]         # 'FlightMode'  (از آخر: منفی)
names[1:]         # ['MaxAltitude', 'FlightMode']  (برش)
"Airspeed" in names    # True

sizes = {"uint8_t": 1, "uint16_t": 2}
sizes["uint16_t"]           # 2
sizes.get("bool_t")         # None  (به‌جای خطا)
sizes.get("bool_t", 1)      # 1     (پیش‌فرض)
for key, value in sizes.items():
    print(key, value)

old, new = 22, 30            # تاپل «باز کردن»
```

نمونه از پروژه (`tags.py`): دیکشنری «شناسه ← توصیف تگ» از روی لیست تگ‌ها:

```
TAGS_BY_ID: Dict[str, TagSpec] = {spec.tag_id: spec for spec in TAG_REGISTRY}
TAGS_BY_ID["Loggable"].label      # 'Tag Loggable'
```

`NamedTuple` تاپلی است که فیلدهایش نام دارند؛ شبیه یک struct ثابت. `bin/run.py` هر کار منو را چنین تعریف می‌کند:

```
class Action(NamedTuple):
    key: str
    names: Sequence[str]
    title: str
    ...

action = Action("3", ("tests", "test", "t"), "Tests", ...)
action.title        # 'Tests'
```

۴.۶ comprehension و generator

comprehension شکل کوتاه «یک حلقه که لیست می‌سازد» است و در این پروژه همه‌جا هست:

```
# حلقه‌ی معمولی
leaves = []
for node in enabled_nodes(model):
    if node.is_primitive:
        leaves.append(node)

# همان، به شکل comprehension (enabled_leaves در rule_base.py)
leaves = [node for node in enabled_nodes(model) if node.is_primitive]

{spec.tag_id: spec for spec in TAG_REGISTRY}      # دیکشنری
{node.name for node in siblings}                  # مجموعه
```

generator تابعی است که به‌جای `return`، با `yield` یکی‌یکی مقدار **تحویل** می‌دهد؛ هر بار که حلقه مقدار بعدی را می‌خواهد، تابع از همان جا ادامه می‌یابد. برای پیمایش درخت عالی است. `Parameter.walk` (فصل ۱۰):

```
def walk(self) -> Iterator["Parameter"]:
    """Yield this node and every node below it."""
    yield self
    for child in self.children:
        yield from child.walk()      # فرزند تحویل می‌دهد همه‌ی آنچه walk

for node in parameter.walk():        # Position, Lat, Lon, Alt
    print(node.name)
```

`any(...)` و `all(...)` روی یک generator: «آیا دست‌کم یکی/همه درست است». `sum(1 for item in items if ...)` تعداد را می‌شمارد. `sorted(items, key=...)` و `max(items, key=...)` مرتب‌سازی و بیشینه با معیار دلخواه.

۴.۷ تابع‌ها

```
def index_macro(name: str, prefix: str = "") -> str:          # naming.py
    """Name of the ``#define`` that holds the index of a parameter."""
    return f"{prefix}{to_macro_case(name)}"

index_macro("Airspeed")                  # 'AIRSPEED'
index_macro("Airspeed", prefix="IDX_")   # 'IDX_AIRSPEED'  (آرگومان نام‌دار)
```

- **آرگومان پیش‌فرض**: `prefix: str = ""`؛ اگر داده نشود همان است.
- **آرگومان نام‌دار**: هر آرگومانی را می‌شود با نامش داد، به هر ترتیبی.
- **فقط نام‌دار**: آرگومان‌های بعد از `*` فقط با نام داده می‌شوند: `def open_copy(work, *, empty=False)` ← `.open_copy(work, empty=True)`
- **چند خروجی**: `return a, b` یک تاپل برمی‌گرداند: `.node, indices = document.value_at(path)`

- `*args` و `**kwargs`: بقیه‌ی آرگومان‌های مکانی/نام‌دار؛ مثلاً `super().__init__(*args, **kwargs)` همه را به کلاس پدر می‌دهد.
- تابع هم یک **مقدار** است: می‌شود آن را در متغیر یا دیکشنری گذاشت و بعد صدا زد (مثل اشاره‌گر به تابع در C). `Api.handlers()` دیکشنری «نام ← متد» است.

lambda تابع یک‌خطی بی‌نام است؛ بیشتر برای کلید مرتب‌سازی:

```
drift.moved.sort(key=lambda move: (order.get(move.parameter, 0), move.tag_id))
```

تابع درون تابع (closure): تابع درونی به متغیرهای تابع بیرونی دسترسی دارد، حتی بعد از تمام شدن آن. `_make_handler(api)` در `server.py` کلاسی درون خودش می‌سازد که از `api` استفاده می‌کند؛ بی متغیر سراسری.

متغیرهای درون تابع محلی‌اند. برای **نوشتن** در یک متغیر سطح ماژول باید `global name` گفت (`global _can_show` با `native_dialog.available`)؛ خواندن بی آن هم ممکن است.

۴.۸ type hint و typing

نوشته‌های `str : ` و `-> int` **راهنمای نوع**اند: پایتون آن‌ها را **بررسی نمی‌کند**، فقط برای خواننده و ویرایشگر است. با این حال در این پروژه همه‌جا نوشته شده‌اند و تابع تازه هم باید داشته باشد.

یعنی	نوشته
یک `str` یا `None`	`Optional[str]`
لیستی از `Parameter`	`List[Parameter]`
دیکشنری `str` ← `int`	`Dict[str, int]`
تاپل دوتایی	`Tuple[str, int]`
هر چیز ترتیبی (لیست یا تاپل)	`Sequence[str]`
چیزی که می‌شود روی آن حلقه زد (مثل generator)	`Iterator[Parameter]`
تابعی که یک `Path` می‌گیرد و چیزی برنمی‌گرداند	`Callable[[Path], None]`
هر چیزی	`Any`
نامی که هنوز تعریف نشده (همان کلاسِ در حال تعریف)	`"Parameter"` (در گیومه)

این نام‌ها از `typing` import می‌شوند: `from typing import Dict, List, Optional`. سطر `from __future__ import annotations` در ابتدای فایل‌ها باعث می‌شود راهنماها اصلاً اجرا نشوند (فقط متن بمانند)؛ پس ترتیب تعریف کلاس‌ها مهم نیست.

۴.۹ کلاس‌ها

کلاس شبیه یک `struct` است که تابع‌هایش (**متد**) را هم با خودش دارد. `self` همان اشاره‌گر `me` در کد C شرکت است (`fSmartData_U16_Write(sSmartDataU16 * const me, ...)`)؛ فقط در پایتون خودکار داده می‌شود:

```
class OffsetMap:
    def __init__(self, areas):            # سازنده؛ مثل Ctor
        self._areas = areas               # فیلد: self.<نام>

    def total(self, tag_id: str) -> int:  # متد
        area = self._areas.get(tag_id)
        return area.total if area is not None else 0

offsets = OffsetMap(areas)        # ساختن شیء (بی new و بی free)
offsets.total("ResetProof")       # 7   ← total(offsets, "ResetProof")
```

نمونه در پروژه	معنی	چیز
`Document.__init__`	سازنده؛ هنگام ساختن شیء	`__init__`
`self._undo`	قرارداد «خصوصی»؛ پایتون جلوی دسترسی را نمی‌گیرد	`self._name`
`parameter.is_struct`، `document.dirty`	متدی که مثل فیلد خوانده می‌شود (بی پرانتز)	`@property`
`Document._hold_boolean_shape`	تابعی درون کلاس که `self` نمی‌گیرد	`@staticmethod`
`CStyle.load(...)`، `Config.load(...)`	متدی که به‌جای شیء، **کلاس** را می‌گیرد (`cls`)؛ برای سازنده‌های جایگزین	`@classmethod`
`StructTypeLibrary`	متدهای ویژه: `str(x)`، `len(x)`، `for ... in x`، `a in x`	`__str__`، `__len__`، `__iter__`، `__contains__`

ارث‌بری: `class ParamTableShapeRule(ParamTableRule):` همه‌ی متدهای پدر را دارد و می‌تواند بازنویسی کند. `super().__init__(...)` سازنده‌ی پدر را صدا می‌زند؛ مثلاً `ParameterListTableHeader.__init__`. **کلاس انتزاعی** (`ABC` از `abc`) متدهایی با `@abstractmethod` دارد که هر زیرکلاس **باید** بنویسد؛ از خود کلاس انتزاعی شیء ساخته نمی‌شود:

```
class ValidationRule(ABC):
    rule_id: str = "RULE"                       # ویژگی کلاس

    @abstractmethod
    def check(self, model, diagnostics) -> None:
        """Inspect *model* and append findings to *diagnostics*."""

class LogIdRule(ValidationRule):
    rule_id = "TAG_LOG_ID"                      # بازنویسی
    def check(self, model, diagnostics) -> None:
        ...
```

ویژگی کلاس (مثل `rule_id`) بین همه‌ی شیءها مشترک است و بی شیء هم خوانده می‌شود: `LogIdRule.rule_id`. `cli.py` با همین، فهرست قاعده‌ها را بی ساختن شیء چاپ می‌کند.

۴.۱۰ dataclass و Enum

`@dataclass` کلاسی می‌سازد که فقط فیلد دارد؛ پایتون سازنده و مقایسه را خودش می‌نویسد. نزدیک‌ترین چیز به `struct` در C:

```
@dataclass
class OffsetEntry:
    tag_id: str
    parameter: Parameter
    offset: int
    size: int

entry = OffsetEntry("ResetProof", node, 2, 1)
entry.offset        # 2
```

فیلد با مقدار پیش‌فرض (`array_size: int = 1`). برای پیش‌فرضِ **لیست یا دیکشنری** هرگز `[] =` ننویسید (بخش ۴.۱۸)؛ از `field(default_factory=list)` استفاده کنید تا هر شیء لیست خودش را بگیرد:

```
@dataclass
class OffsetArea:
    tag: TagSpec
    entries: List[OffsetEntry] = field(default_factory=list)
    total: int = 0
```

`@dataclass(frozen=True)` شیء تغییرناپذیر می‌سازد (مثل `CValueType`). `replace(obj, a=1)` از `dataclasses` کپی‌ای با یک فیلد عوض‌شده می‌سازد (`pipeline.py`).

`Enum` همان `enum` در C است، ولی مقدارهایش شیءاند و با نام کلاس نوشته می‌شوند:

```
class TagKind(Enum):
    BOOLEAN = "boolean"
    ENUM = "enum"
    FREE = "free"

spec.kind is TagKind.BOOLEAN     # مقایسه با is
TagKind.ENUM.value               # 'enum'
TagKind("free")                  # TagKind.FREE (از متن)
```

`IntEnum` (مثل `Severity` در `errors.py`) مقدارهای عددی دارد و مثل عدد مقایسه می‌شود: `Severity.ERROR > Severity.WARNING`.

۴.۱۱ استثناها

در C هر تابع کد خطا برمی‌گرداند و فراخوان باید بررسی کند. در پایتون خطا **پرتاب** می‌شود (`raise`) و تا اولین `try`ی که آن را بگیرد بالا می‌رود؛ اگر کسی نگیرد، برنامه با پیام و traceback متوقف می‌شود (بخش ۲۵.۱).

```
class CodeGeneratorError(Exception):      # استثنای خود پروژه
    """Unrecoverable error: the tool cannot continue at all."""

def require(self):
    if self.document is None:
        raise CodeGeneratorError("No project is open. ...")
    return self.document

try:
    path = self.document.duplicate(str(request.get("path") or ""))
except ValueError as error:               # فقط همین نوع
    return {"ok": False, "message": str(error)}
finally:
    pass                                  # همیشه اجرا می‌شود
```

قراردادهای این پروژه:

- `ValueError` با جمله‌ای برای آدم: «این ویرایش را نمی‌پذیرم و دلیلش این است». `Api` آن را به `{"ok": false, "message": ...}` تبدیل می‌کند.
- `CodeGeneratorError`: ابزار نمی‌تواند ادامه دهد (فایل نیست، XML خراب).
- مشکل‌های **فهرست** استثنا نیستند؛ به `DiagnosticCollector` اضافه می‌شوند تا همه با هم گزارش شوند (فصل ۹).
- `except Exception:` (گرفتن همه) فقط جایی که عمداً «هیچ چیز نباید این را متوقف کند»، با کامنت `# noqa: BLE001 - ...` که دلیل را می‌گوید (مثل `server.py`).
- `raise ... from None` پیام استثنای قبلی را از traceback حذف می‌کند.

۴.۱۲ with و مدیر زمینه

`with` تضمین می‌کند کاری در پایان بلوک انجام شود، حتی اگر استثنایی رخ دهد؛ مثل بستن فایل. معادل C آن `fopen` ... `fclose` با `goto cleanup` است:

```
with open(path, "r", encoding="utf-8") as handle:
    text = handle.read()
# این‌جا فایل بسته شده، حتی اگر read استثنا داده باشد

with tempfile.TemporaryDirectory() as raw:      # پوشه‌ی موقت؛ پایان بلوک پاک می‌شود
    case(Path(raw))
```

هر شیئی که `__enter__` و `__exit__` داشته باشد با `with` کار می‌کند. راه ساده‌تر ساختن آن، `@contextlib.contextmanager` روی یک generator است: کد پیش از `yield` در آغاز و کد بعد از آن در پایان اجرا می‌شود. `Document._change` (فصل ۱۸) همین است:

```
@contextlib.contextmanager
def _change(self, label):
    undo, redo = list(self._undo), list(self._redo)
    self.snapshot(label)
    before = self._undo[-1][1]
    try:
        yield                       # این‌جا اجرا می‌شود بدنه‌ی with
    except Exception:
        self.model = before         # برگرداندن همه‌چیز
        self._undo, self._redo = undo, redo
        raise
```

۴.۱۳ ماژول، پکیج و import

هر فایل `.py` یک **ماژول** است و هر پوشه‌ای که `__init__.py` دارد یک **پکیج**. `import` شبیه `#include` است با دو فرق: فقط یک بار اجرا می‌شود (هرچقدر import شود) و نام‌ها با نام ماژول قابل دسترسی‌اند:

```
import json                                  # json.loads(...)
from pathlib import Path                     # Path(...)
from pl_codegen.errors import DiagnosticCollector
import xml.etree.ElementTree as ET           # نام کوتاه: ET.Element

# import نسبی: یک نقطه = همین پوشه، دو نقطه = پوشه‌ی بالا
from .tag_offset import build_offset_map     # layout/tag_offset.py
from ..model.parameter import Parameter      # از layout/ به model/
```

پایتون ماژول‌ها را در پوشه‌های `sys.path` جست‌وجو می‌کند. برای همین هر فایل `bin/run_*.py` اول `paramgen/src` را به آن اضافه می‌کند (فصل ۲۱)؛ بی آن `import pl_codegen` پیدا نمی‌شود (`ModuleNotFoundError`). `__init__.py` یک پکیج معمولاً نام‌های مهم زیرماژول‌ها را دوباره صادر می‌کند تا بشود نوشت `from pl_codegen.model import Parameter`.

import دوری (circular): اگر `a.py` در سطر اولش `b` را import کند و `b.py` هم `a` را، یکی از آن‌ها نیمه‌کاره است و نامی پیدا نمی‌شود. راه این پروژه: import **درون تابع** (محلی)، که فقط هنگام صدا زدن تابع اجرا می‌شود؛ مثلاً `_numbered_per_system` در `drift.py`:

```
def _numbered_per_system(model):
    from ..codegen.files.param_table_h import system_of        # local: one way
    ...
```

۴.۱۴ عبارت منظم (re)

ماژول `re` الگوی متن را بررسی یا جست‌وجو می‌کند. الگوها با رشته‌ی خام `r"..."` نوشته می‌شوند و معمولاً یک بار با `re.compile` ساخته و در یک ثابت نگه داشته می‌شوند.

یعنی	الگو
یک رقم	`\d`
دقیقاً دو رقم	`\d{2}`
حرف، رقم یا _	`\w`
فاصله، تب یا سطرشکن	`\s`
هر نویسه (برای خود نقطه: `\.`)	`.`
یک یا بیشتر، صفر یا بیشتر، صفر یا یک	`+`، `*`، `?`
«کمترین» یک یا بیشتر	`+?`
هر تعداد نویسه به‌جز گیومه	`[^"]*`
گروه؛ بخشی که بعداً خوانده می‌شود، بی‌نام یا نام‌دار	`(...)` و `(?P<name>...)`
ابتدا و انتهای سطر (با `re.MULTILINE`)	`^` و `$`

```
VERSION_FORMAT = re.compile(r"\d{2}\.\d{2}\.\d{2}")       # set_version.py
VERSION_FORMAT.fullmatch("01.01.85")     # شیء match (درست)
VERSION_FORMAT.fullmatch("1.2.3")        # None

_COPY_NAME = re.compile(r"(?P<original>.+?)_Copy\d*")      # document.py
_COPY_NAME.fullmatch("Speed_Copy3").group("original")       # 'Speed'

re.sub(r"\[\d+\]", "", "Motor[1].Rpm")      # 'Motor.Rpm'   (strip_array_indices)
```

`fullmatch` کل متن، `match` از ابتدا و `search` هر جای متن را بررسی می‌کند؛ `findall` و `finditer` همه‌ی تطابق‌ها.

۴.۱۵ فایل و مسیر

مسیرها با `pathlib.Path` کار می‌شوند، نه با رشته. عملگر `/` مسیرها را به هم می‌چسباند و روی ویندوز و لینوکس درست کار می‌کند:

```
from pathlib import Path
project = Path("D:/Projects/Flight_Computer")
core = project / "parameter_lists" / "core.xml"
core.name        # 'core.xml'
core.stem        # 'core'
core.suffix      # '.xml'
core.parent      # Path('D:/Projects/Flight_Computer/parameter_lists')
core.is_file()   # True
core.resolve()   # مسیر کامل و واقعی (نقطه‌ها و پیوندها حل‌شده)
core.with_name("core_types.xml")
text = core.read_text(encoding="utf-8")
core.write_text(text, encoding="utf-8", newline="\n")   # پایان سطر LF حتی در ویندوز
sorted(folder.glob("*.xml"))    # همه‌ی xmlهای یک پوشه
folder.mkdir(parents=True, exist_ok=True)   # mkdir -p
```

همیشه `encoding="utf-8"` **بدهید.** بی آن، ویندوز با code page خودش می‌خواند و فارسی یا … خراب می‌شود. `open(..., newline="")` پایان سطرها را دست‌نخورده نگه می‌دارد (`set_version.py`). برای کپی و حذف: `shutil.copy2(src, dst)`، `path.unlink()`، `shutil.rmtree(folder)`.

۴.۱۶ JSON و XML

JSON با ماژول `json`: `json.loads(text)` متن را به دیکشنری و لیست پایتون تبدیل می‌کند و `json.dumps(data, indent=2)` برعکس. فایل‌های پروژه ممکن است کامنت داشته باشند، پس با `load_jsonc` (فصل ۹) خوانده می‌شوند. مقدارها: `true` ← `True`، `null` ← `None`، عدد ← `float/int`.

XML با `xml.etree.ElementTree` (معمولاً `ET`): فایل به درختی از `Element` تبدیل می‌شود.

```
import xml.etree.ElementTree as ET
root = ET.parse("core.xml").getroot()           # <ParameterListModel>
root.get("schemaVersion")                       # '1.0'   (اگر نیست None ویژگی؛)
params = root.find("Parameters")                # اولین فرزند با این نام
for element in params.findall("Parameter"):     # همه‌ی فرزندان با این نام
    print(element.get("name"), element.findtext("Description"))

new = ET.Element("Parameter", {"name": "Speed"})   # ساختن
ET.SubElement(new, "Limits").set("minimum", "0")
ET.tostring(new)          # b'<Parameter name="Speed"><Limits minimum="0" /></Parameter>'
```

متن یک عنصر در `element.text` است و ویژگی‌ها همیشه **رشته‌اند**؛ تبدیل به عدد با خود ماست (`_to_number` در `xml_reader.py`).

۴.۱۷ کتابخانه‌های استاندارد دیگر

کجا در پروژه	برای چه	ماژول
`cli.py`، `bin/run*.py`	خواندن آرگومان‌های خط فرمان و ساختن `--help`	`argparse`
`run.py`، `run_tests.py`، `server.py`، `native_dialog.py`	اجرای برنامه‌ی دیگر (`run`، `call`، `Popen`)	`subprocess`
`server.py`	سرور HTTP ساده	`http.server`
`server.py`	باز کردن مرورگر پیش‌فرض	`webbrowser`
Undo در `document.py`	`copy.deepcopy(x)`: کپی کامل یک شیء و هرچه درون آن است	`copy`
`_give_every_row_its_text`	`product(range(2), range(3))`: همه‌ی ترکیب‌ها	`itertools`
آزمون‌ها، `server.py`	پوشه و فایل موقت	`tempfile`
`document.py`، `browser.py`	کپی، حذف پوشه، `which` (پیدا کردن برنامه در PATH)	`shutil`
زمان تولید در بنر	تاریخ و ساعت	`datetime`

کجا در پروژه	برای چه	ماژول
`literals.py` در `_as_float32`	تبدیل عدد به بایت‌های C و برعکس (`pack`/`unpack`)	`struct`
`literals.py`	`isnan`، `isinf`	`math`
`native_dialog.py`، `server.py`	`Lock` (قفل)، `Timer` (اجرای بعدی)	`threading`
`setup_environment.py`، `run_version_test.py`	import با نام در زمان اجرا یا از مسیر فایل	`importlib`
`logging_utils.py`	پیام‌های لاگ با سطح	`logging`
`config.py`	نزدیک‌ترین نام («منظورتان … بود؟»)	`difflib`
`browser.py`، `pipeline.py`	متغیر محیطی (`os.environ`)، `os.walk`	`os`
آزمون‌های ویرایشگر	خواندن کد پایتون به‌صورت درخت، بی اجرا	`ast`

۴.۱۸ دام‌های رایج برای برنامه‌نویس C

- **متغیر اشاره است، نه کپی.** `b = a` برای لیست یا شیء، `b` را **همان** شیء می‌کند؛ تغییر `b` روی `a` اثر دارد. کپی: `list(a)` یا `copy.deepcopy(a)`. برای همین Undo از `deepcopy` استفاده می‌کند.
- **پیش‌فرض تغییرپذیر.** `def f(items=[])` یک لیست می‌سازد که بین **همه‌ی** فراخوان‌ها مشترک است. همیشه `items=None` و درون تابع `items = items or []`؛ یا در dataclass `field(default_factory=list)`.
- **`is` و `==`.** `==` برابری مقدار است؛ `is` «همان شیء». فقط برای `None` و اعضای Enum از `is` استفاده کنید.
- **تقسیم.** `/` همیشه اعشاری است؛ تقسیم صحیح `//`.
- **صفر نادرست است.** `if value:` برای صفر هم نادرست است؛ «داده نشده» را با `is None` بپرسید.
- **`bool` عدد است.** `isinstance(True, int)` درست است؛ عدد واقعی را با «int و نه bool» بپرسید.
- **عدد اعشاری.** `0.1 + 0.2 != 0.3`، مثل C. برای «صحیح بودن» `float(x).is_integer()`.
- **تورفتگی.** فاصله و تب را مخلوط نکنید؛ این پروژه همیشه چهار فاصله.
- **تغییر لیست هنگام حلقه روی آن.** روی کپی (`for x in list(items):`) حلقه بزنید.
- **رشته تغییرناپذیر است.** `s.upper()` رشته‌ی تازه برمی‌گرداند؛ `s` عوض نمی‌شود.
- **بازه‌ی `range` پایانش را ندارد.** `range(4)` یعنی ۰ تا ۳، مثل `i < 4`.

۵. HTML، CSS و JavaScript برای برنامه‌نویس C

صفحه‌ی ویرایشگر با سه زبان ساخته شده که مرورگر می‌فهمد: **HTML** (چه چیزی روی صفحه است)، **CSS** (چه شکلی است) و **JavaScript** (چه کاری می‌کند). هیچ کتابخانه‌ای به کار نرفته؛ همه‌ی کد همان است که در `paramgen/src/pl_gui/web/` می‌بینید (فصل ۲۰).

۵.۱ HTML

HTML درختی از **عنصر**هاست. هر عنصر یک برچسب باز و بسته دارد و ممکن است **ویژگی** داشته باشد؛ شبیه XML (فصل ۶)، ولی با برچسب‌هایی که مرورگر می‌شناسد:

```
<button id="btn-save" class="btn primary" title="Save the sub list (Ctrl+S)">Save</button>
<!--    ↑ شناسه‌ی یکتا   ↑ کلاس‌ها (برای CSS)   ↑ متن راهنمای موس      ↑ متن دکمه -->
```

چیست	برچسب
جعبه‌ی بی‌معنا برای گروه‌بندی؛ `div` بلوکی، `span` درون خط	`<div>`، `<span>`
دکمه	`<button>`
خانه‌ی ورودی	`<input type="text">`، `type="number"`، `type="checkbox"`
فهرست کشویی	`<select>` و `<option>`
خانه‌ی چندخطی	`<textarea>`
عنوان، بند، پررنگ، کد	`<h1>`…`<h3>`، `<p>`، `<b>`، `<code>`
بخش‌های صفحه (معنایی؛ مثل div)	`<header>`، `<main>`، `<section>`، `<footer>`
بار کردن JavaScript و CSS	`<script src="...">`، `<link rel="stylesheet">`

- `id` یکتاست؛ JavaScript با آن عنصر را پیدا می‌کند (`document.getElementById("btn-save")`).
- `class` می‌تواند چند نام با فاصله داشته باشد؛ CSS با آن ظاهر می‌دهد.
- ویژگی‌های `data-*` اطلاعات دلخواه روی عنصراند: `<button data-action="open-project">`؛ در JS با `element.dataset.action` خوانده می‌شوند. فرم ویرایشگر نام فیلد را در `data-field` نگه می‌دارد.
- `hidden` عنصر را پنهان می‌کند. `&`، `<`، `>` همان `&`، `<`، `>` هستند.

CSS ۵.۲

CSS قاعده‌هایی است به شکل «انتخابگر { ویژگی: مقدار; }»:

```
.btn.primary {                  /* هر عنصر با هر دو کلاس btn و primary */
  background: var(--accent);    /* متغیر CSS */
  color: var(--on-accent);
  border-radius: var(--radius);
}
#problems-pane { height: 34px; }     /* عنصر با id */
.grid-row.selected { ... }           /* ردیف انتخاب‌شده */
:root[data-theme="dark"] { --bg: #14171c; }    /* تم تیره */
```

یعنی	انتخابگر
همه‌ی عنصرهای `button`	`button`
عنصرهای با کلاس `name`	`.name`
عنصر با `id="name"`	`#name`
`B`ای که درون `A` است	`A B`
عنصرهای دارای ویژگی `hidden`	`[hidden]`
ریشه‌ی صفحه (`<html>`)؛ جای تعریف متغیرها	`:root`

متغیر CSS با `--name: value;` تعریف و با `var(--name)` خوانده می‌شود. همه‌ی رنگ‌های ویرایشگر متغیرند و تم تیره فقط همان متغیرها را دوباره تعریف می‌کند (بخش ۲۰.۲). وقتی دو قاعده یک ویژگی را برای یک عنصر تعیین کنند، آن که **دقیق‌تر** است (مثلاً id بر کلاس) و در تساوی، **آخری** برنده است؛ `!important` همه را کنار می‌زند.

چیدمان: `display: flex` فرزندان را در یک ردیف یا ستون می‌چیند و `display: grid` در جدول (جدول ویرایشگر با `grid-template-columns` ساخته می‌شود). اندازه‌ها با `px` (نقطه‌ی صفحه)، `%` یا `fr` (سهمی از فضای باقی‌مانده).

JavaScript ۵.۳ پایه

نحو JavaScript به C نزدیک است (`{}`، `;`، `if`، `for`، `while`، `switch`، `&&`، `||`، `!`)؛ ولی مثل پایتون نوع مال مقدار است و متغیر نوع اعلام‌شده ندارد.

```
const limit = 65535;            // ثابت (دوباره مقدار نمی‌گیرد)
let count = 0;                  // متغیر
count += 1;  count++;           // ++ هست

if (count === 1) { ... }        // === مقایسه‌ی دقیق؛ نه ==
const kind = node.isStruct ? "struct" : "value";

for (let index = 0; index < 4; index++) { ... }
for (const node of nodes) { ... }          // روی عنصرهای آرایه

const text = `${name} = ${value}`;         // رشته‌ی الگو، مثل f-string پایتون
```

یعنی	JS
«مقداری نیست»؛ `undefined` یعنی هرگز داده نشده	`null` و `undefined`
`a`، مگر اینکه `undefined`/`null` باشد؛ آن‌وقت `b`	`a ?? b`
`a`، مگر اینکه نادرست باشد (`0`، `""`، `null`، …)	`a || b`
برابری و نابرابری بی تبدیل نوع؛ همیشه این دو را به کار ببرید	`===` و `!==`
تبدیل نوع	`Number("5")`، `parseInt("5", 10)`، `String(5)`
`"number"`، `"string"`، `"boolean"`، `"object"`، …	`typeof x`

تابع: `function name(a, b) { return a + b; }`، یا **تابع پیکانی** `(a, b) => a + b` که برای callbackها به کار می‌رود: `values.map((value) => value.index)`. کامنت `//` و `/* */` مثل C.

۵.۴ شیء و آرایه

شیء مجموعه‌ای از «نام: مقدار» در `{}` است؛ شبیه struct که می‌تواند تابع هم داشته باشد. هر فایل صفحه یک شیء بزرگ تعریف می‌کند (`app`، `gridView`، `editorView`، `api`) که متدهایش با `this` به بقیه‌ی شیء دسترسی دارند:

```
const gridView = {
  filters: {},                  // ویژگی
  search: "",

  isOpen(path, isArray) {       // متد
    if (this.forceOpen.has(path)) return true;
    ...
  },
};

gridView.isOpen("Motor", true);
gridView.filters["unit"] = "V";   // افزودن یا تغییر کلید
```

آرایه `[...]` است و متدهای کاربردی دارد که در کد صفحه زیاد دیده می‌شوند:

نمونه	کار	متد
`out.push(row)`	افزودن به آخر	`push`
`values.map((v) => v.index)`	آرایه‌ی تازه از تبدیل هر عنصر	`map`
`all.filter((row) => this.matches(row))`	آرایه‌ی عنصرهایی که شرط را دارند	`filter`
`modules.find((m) => m.current)`	اولین عنصری که شرط را دارد، یا `undefined`	`find`
`values.some((v) => v.name === path)`	آیا یکی / همه شرط را دارند	`every` / `some`
`rows.join("")`	چسباندن به رشته	`join`
`IGNORED_TAGS.includes(spec.id)`	آیا عنصری هست	`includes`
`own.length`	طول	`length`

`Set` مجموعه‌ی بی‌تکرار است (`new Set()`، `add`، `has`، `delete`)؛ `toggled` و `forceOpen` در `grid.js` چنین‌اند. `Object.entries(obj)` جفت‌های کلید و مقدار را می‌دهد.

۵.۵ DOM و رویدادها

DOM نمای JavaScript از عنصرهای صفحه است. کد صفحه عنصرها را پیدا می‌کند، محتوایشان را می‌نویسد و به رویدادهایشان (کلیک، تایپ) گوش می‌دهد:

```
const button = document.getElementById("btn-save");
button.addEventListener("click", () => this.save());   // هر کلیک

const row = element.querySelector(".grid-row[data-path]");   // با انتخابگر CSS
for (const box of element.querySelectorAll("[data-field]")) { ... }

element.innerHTML = `<p class="empty">${escapeHtml(text)}</p>`;   // جایگزینی محتوا
element.textContent = "Nothing selected";      // فقط متن
button.disabled = true;
element.hidden = false;
element.classList.toggle("dirty", state.dirty);  // کلاس را بگذار یا بردار
input.value;  checkbox.checked;  element.dataset.field;
```

هشدار: هر متنی که از مدل یا کاربر می‌آید و در `innerHTML` گذاشته می‌شود **باید** از `escapeHtml` بگذرد (`grid.js`)؛ وگرنه نامی که `<` دارد صفحه را خراب می‌کند. `textContent` این مشکل را ندارد.

کی	رویداد
کلیک	`click`
مقدار فهرست کشویی یا تیک عوض شد	`change`
هر تغییر در جعبه‌ی متن (هر کلید)	`input`
جعبه فوکوس را از دست داد (کاربر بیرون رفت)؛ فرم ویرایشگر با همین می‌فرستد	`blur`
کلیدی فشرده شد؛ `event.code` (کلید فیزیکی، مثل `KeyS`) و `event.ctrlKey`	`keydown`
فشار و رهاسازی دکمه‌ی موس (`onSteadyClick`)	`mouseup` / `mousedown`

`event.preventDefault()` کار پیش‌فرض مرورگر را لغو می‌کند (مثلاً Ctrl+S ذخیره‌ی صفحه‌ی وب) و `event.stopPropagation()` نمی‌گذارد رویداد به عنصرهای بیرونی برسد.

۵.۶ async/await و fetch

فرستادن درخواست به سرور زمان می‌برد و مرورگر نباید در این مدت قفل شود. `fetch` درخواست را می‌فرستد و یک **Promise** («قولِ نتیجه‌ای در آینده») برمی‌گرداند. درون تابعی که `async` است، `await` تا رسیدن نتیجه صبر می‌کند بی‌آنکه صفحه قفل شود. این تمام `api.js` است:

```
const api = {
  async call(action, payload = {}) {
    const response = await fetch(`/api/${action}`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload),          // شیء ← متن JSON
    });
    if (!response.ok) { ... throw new Error(reason); }
    return response.json();                   // متن JSON ← شیء
  },
  update(path, changes) { return this.call("update", { path, changes }); },
};

// استفاده، درون یک تابع async:
const answer = await api.update("Airspeed", { unit: "km/h" });
if (!answer.ok) toast(answer.message, "bad");
```

`{ path, changes }` کوتاه‌شده‌ی `{ path: path, changes: changes }` است. خطای `await` را مثل پایتون با `try { ... } catch (error) { ... }` می‌گیریم؛ `app.failed` همین کار را برای همه‌ی کارهای صفحه می‌کند.

۵.۷ ابزارهای توسعه‌ی مرورگر

در پنجره‌ی ویرایشگر **F12** (یا Ctrl+Shift+I) ابزارهای توسعه‌ی مرورگر را باز می‌کند؛ مهم‌ترین ابزار اشکال‌زدایی صفحه:

- **Console**: خطاهای JavaScript با نام فایل و شماره‌ی سطر. می‌شود دستور هم نوشت: `app.state.tree[0]` وضعیت فعلی را نشان می‌دهد.

- **Network**: هر درخواست `/api/...` با بدنه و جواب؛ برای دیدن اینکه صفحه دقیقاً چه فرستاد و سرور چه جواب داد.
- **Elements**: درخت HTML و قاعده‌های CSS هر عنصر؛ می‌شود همان‌جا CSS را آزمایشی عوض کرد.
- **Sources**: کد JS با نقطه‌ی توقف (breakpoint).

تغییر فایل‌های `web/` با بارگذاری دوباره (F5) دیده می‌شود؛ سرور هر بار فایل را از دیسک می‌خواند و `Cache-Control: no-store` می‌فرستد تا مرورگر نسخه‌ی کهنه نگه ندارد.

۶. قالب فایل‌ها: XML، JSON و الگوها

ابزار سه قالب متنی می‌خواند و می‌نویسد: **XML** برای ساب‌لیست‌ها و تایپ‌ها، **JSON** برای فایل پروژه و فهرست‌های کشویی، و **JSONC** (JSON با کامنت) برای فایل‌هایی که آدم با دست می‌نویسد. این فصل نحو هر سه و فایل‌های پوشه‌ی `templates/` را شرح می‌دهد.

۶.۱ XML

XML درختی از **عنصر**هاست، شبیه HTML ولی با نام‌هایی که خود قالب تعریف می‌کند. بخشی از `power.xml` پروژه‌ی نمونه:

```
<?xml version='1.0' encoding='utf-8'?>                    ← اعلان نسخه و کدگذاری
<ParameterListModel schemaVersion="1.0" ...>              ← عنصر ریشه با ویژگی‌ها
  <SourceFiles />                                         ← عنصر خالی (خودبسته)
  <Parameters>
    <Parameter name="BatteryVoltage" kind="primitive" ... logId="500">
      <Limits default="0.0" minimum="0.0" maximum="30.0" resolution="0.01" />
      <Description>Voltage of the whole pack.</Description>  ← متن عنصر
    </Parameter>
  </Parameters>
</ParameterListModel>
```

- هر عنصر باز (`<Parameters>`) باید بسته شود (`</Parameters>`)، یا خودبسته باشد (`<Source />`).
- مقدار هر **ویژگی** (`name="..."`) همیشه رشته است، درون گیومه.
- بزرگی و کوچکی حروف مهم است: `<parameter>` همان `<Parameter>` نیست.
- پنج نویسه در متن escape می‌شوند: `&` (&)، `<` (<)، `>` (>)، `"` (")، `'` ('). `ElementTree` این کار را خودش می‌کند.
- کامنت: `<!-- ... -->`.
- فایل با `encoding='utf-8'` نوشته می‌شود؛ متن فارسی در توضیح‌ها بی‌مشکل است.

ساختار کامل فایل ساب‌لیست و فایل تایپ‌ها و همه‌ی نام‌های عنصر و ویژگی در فصل ۱۱ آمده است. XML خراب (مثلاً برچسبی بسته نشده) با پیامی شامل سطر و ستون رد می‌شود:

```
'D:\Projects\Flight_Computer\parameter_lists\core.xml' is not a valid XML file:
mismatched tag: line 5, column 4.  Open it in a text editor at that line; the
editor cannot load a file it cannot parse.
```

۶.۲ JSON و JSONC

JSON شش نوع مقدار دارد که مستقیم به نوع‌های پایتون تبدیل می‌شوند:

پایتون	نمونه	JSON
`dict`	`{"name": "Flight Computer"}`	شیء
`list`	`["core.xml", "power.xml"]`	آرایه
`str`	`"cg_"` (فقط با گیومه‌ی دوتایی)	رشته
`int`، `float`	`3`، `0.5`	عدد
`True`، `False`	`true`، `false`	درست/نادرست
`None`	`null`	تهی

JSON **کامنت ندارد** و **ویرگول بعد از آخرین عنصر** را نمی‌پذیرد. **JSONC** همان JSON است که این دو را اجازه می‌دهد (VS Code فایل‌های تنظیماتش را این‌طور می‌نویسد):

```
{
  // which sub lists are built, in this order
  "model_files": [
    "core.xml",
    "power.xml",          // ← ویرگول پایانی: در JSON خطا، در JSONC مجاز
  ],
}
```

ابزار هر فایل JSON را با `load_jsonc` (فصل ۹) می‌خواند، پس فایل پروژه‌ای که کسی با دست کامنت در آن گذاشته هم خوانده می‌شود. ولی هرچه **خود ابزار** می‌نویسد JSON ساده است (بی کامنت و بی ویرگول پایانی) تا هر ویرایشگر JSONی بی خط قرمز بازش کند؛ و وقتی فقط یک آرایه را عوض می‌کند، متن را دست‌کاری می‌کند تا کامنت‌های کاربر بمانند (`config_file.py`، فصل ۱۷).

۶.۳ `pl_project.json` — الگوی فایل پروژه

مسیر: `paramgen/templates/pl_project.json`

هر پروژه‌ی تازه از این فایل ساخته می‌شود (`Workspace.create`، فصل ۱۷): الگو خوانده، نام و قالب خروجی **به‌صورت داده** در آن گذاشته (جای‌خالی `{{NAME}}` جایگزین متنی نمی‌شود؛ مقدار `project.name` عوض می‌شود) و به‌صورت JSON ساده نوشته می‌شود. متن کامل:

```
{
  "project": { "name": "{{NAME}}", "description": "" },
  "inputs": { "model_files": [], "model_directory": "parameter_lists" },
  "output": {
    "format": "smart_data",
    "code_directory": "cg_parameter_list",
    "full_parameter_list_file": "cg_parameter_list/parameter_list.xml",
    "file_prefix": "cg_",
    "base_name": "parameter_list"
  },
  "style": { "company": "FAS", "indent": "    " },
  "codegen": {
    "emit_description": true, "emit_unit": true,
    "emit_generation_info": true, "emit_timestamp": true,
    "index_define_prefix": "", "offset_define_prefix": "PARAMETER_LIST_",
    "smart_data_prefix": "Cg_SmartData_", "struct_type_prefix": "sCgStruct_",
    "descriptor_symbol": "ParameterList", "quantity_define": "PARAMETER_LIST_QTY",
    "extra_includes": []
  },
  "validation": { "treat_warnings_as_errors": false, "require_unit": false }
}
```

(در فایل واقعی هر کلید سطر خودش را دارد.) معنای هر کلید و اینکه کدام فیلد `Config` را پر می‌کند در فصل ۹ (`config.py`) آمده است. **کلید تازه** باید هم در `config.py`، هم اینجا و هم در `run_settings_test.py` اضافه شود (فصل ۲۴). کلید ناشناخته خطا نیست ولی هشدار «Unknown configuration key» با پیشنهاد نزدیک‌ترین نام می‌گیرد.

۶.۴ `choices.json` — فهرست‌های کشویی پیش‌فرض شرکت

مسیر: `paramgen/templates/choices.json`

چهار فهرست کشویی ویرایشگر، همان‌طور که برنامه تحویل می‌شود:

کلید	مقدارها	ستون مدل
`paramTypes`	`MONITORING`، `COMMAND`، `SETTING`	`TYPE`
`systemNames`	`SYS_UNKNOWN`، `SYS_GS_COMPUTER`، … ، `SYS_AUP` (۲۱ مقدار)	`SYSTEM NAMES`
`paramModes`	`PARAM_MODE_NONE`، `PARAM_MODE_USER`، … ، `PARAM_MODE_TEMPORARY` (۷ مقدار)	`PARAM MODES`
`valueTypes`	`bool_t`، `uint8_t`، … ، `float64_t` (۱۱ نوع)	`VALUE TYPES`

ویرایشگر فهرست‌های **این رایانه** را در `paramgen/var/choices.json` (با همین شکل) نگه می‌دارد (پنجره‌ی ⚙ ▸ **Drop-down lists…**). هنگام خواندن (`load_choices`، فصل ۹)، فهرست کاربر برنده است و هر فهرستی که آنجا نیست از این الگو پر می‌شود؛ دکمه‌ی «Restore the company defaults» همین الگو را برمی‌گرداند.

نام‌های `systemNames` و `paramModes` در کد تولیدی `#define` می‌شوند، هر کدام با یک عدد. عددی که کسی داده زیر کلید `numbers`، کنار فهرست‌ها، نگه داشته می‌شود؛ خود فهرست‌ها فقط نام دارند. نامی که عدد ندارد، عدد نام بالایی به‌اضافه‌ی یک را می‌گیرد و اولی صفر را:

```
"systemNames": ["SYS_UNKNOWN", "SYS_NAV", "SYS_NEXT"],
"numbers": {"systemNames": {"SYS_UNKNOWN": 0, "SYS_NAV": 16}}

SYS_UNKNOWN = 0   SYS_NAV = 16   SYS_NEXT = 17
```

الگوی همراه برنامه هیچ عددی ندارد، پس شماره‌ها همان جایگاه در فهرست‌اند (`SYS_UNKNOWN` صفر، `PARAM_MODE_NONE` صفر، …).

> **هشدار:** نامی که عدد ندارد با جایگاهش شماره می‌گیرد: افزودن نام بی‌عدد در **وسط** فهرست، عدد همه‌ی نام‌های بی‌عدد بعدی را عوض می‌کند و فرم‌وری که با فهرست قدیمی ساخته شده، زیرسیستم‌ها را اشتباه می‌فهمد. پنجره‌ی **Drop-down lists…** هر سطر را با عددش نشان می‌دهد (`SYS_NAV = 16`)، پس بعد از اولین ذخیره همه‌ی عددها نوشته شده‌اند و جابه‌جایی سطرها عددی را عوض نمی‌کند. در `param_table` مقدارهای `PARAM MODES` هنوز با جایگاهشان شماره می‌گیرند.

۶.۵ `c.json` — snippetهای VS Code و قالب فایل‌های C شرکت

مسیر: `paramgen/templates/c.json`

فایل snippet همان VS Code که برنامه‌نویس‌های شرکت با تایپ `h-file` یا `c-file` اسکلت یک فایل C را می‌گیرند. JSONC است (کامنت‌های بالای فایل، تورفتگی با تب). سه snippet دارد:

نام snippet	prefix	سطرها	کاربرد در ابزار
`Create new header file`	`h-file`	۴۰	**بنر** فایل‌های تولیدی (اولین بلوک `/** */`) و نشانگرهای بخش
`Create new source file`	`c-file`	۴۴	نشانگرهای بخش `.c` و سطر پانویس (`EOF ===`)
`Create init/Enable block`	`ie-block`	۵	ابزار از آن استفاده نمی‌کند

شکل هر snippet:

```
"Create new header file": {
    "prefix": "h-file",
    "body": [
        "/**",
        "  ******************************************************************************",
        "  * @file    $TM_FILENAME",
        "  * @brief   ",
        ...
    ],
    "description": "Creates a new C header file."
}
```

جای‌خالی‌های VS Code (`$TM_FILENAME`، `$TM_FILENAME_BASE`، `$CURRENT_YEAR`) را `CStyle` (فصل ۱۴) پر می‌کند. **عوض کردن شکل بنر، نشانگرها یا سطر کپی‌رایت همه‌ی فایل‌های تولیدی = عوض کردن همین فایل**؛ کد پایتون لازم نیست عوض شود. نام دو snippet اول را عوض نکنید؛ `c_style.py` با همین نام‌ها پیدایشان می‌کند (اگر پیدا نشوند، نسخه‌ی داخلی همان شکل به کار می‌رود و تغییرهای شما بی‌اثر می‌شوند).

۶.۶ پوشه‌ی var

تنها جایی از برنامه که هنگام کار در آن نوشته می‌شود. هر دو فایل را برنامه می‌سازد و پاک کردنشان بی‌خطر است:

نویسنده	محتوا	فایل
`Workspace` (فصل ۱۷)	`recent` (پروژه‌های اخیر، نسبت به پوشه‌ی برنامه) و `current`	`var/settings.json`
`save_choices` (فصل ۹)	فهرست‌های کشویی این رایانه، با کلیدهای `choices.json`	`var/choices.json`

هر دو با پایان سطر LF نوشته می‌شوند (و `.gitattributes` همین را تعیین می‌کند).

۷. فایل‌های bat و sh

سه اسکریپت کوچک کار راه‌اندازی را انجام می‌دهند: `run.bat` و `bin/set_version.bat` در ویندوز (زبان دسته‌ای `cmd.exe`) و `run.sh` در لینوکس (زبان `sh`). هر دو زبان قدیمی و عجیب‌اند؛ این فصل فقط چیزهایی را که در این سه فایل آمده توضیح می‌دهد. شرح خود فایل‌ها در فصل ۲۱ است.

۷.۱ فایل bat ویندوز

`run.bat`، کوتاه‌شده:

```
@echo off
REM Comments start with REM.
setlocal
cd /d "%~dp0"

call "%~dp0bin\find_python.bat"
if not defined PYTHON (
    pause
    exit /b 9
)

"%PYTHON%" bin\run.py %*
set EXITCODE=%ERRORLEVEL%
if not "%~1"=="" goto :wait
exit /b %EXITCODE%

:wait
pause
exit /b %EXITCODE%
```

سطر به سطر: `@echo off` فرمان‌ها پیش از اجرا چاپ نشوند؛ `REM` کامنت؛ `setlocal` متغیرهای این فایل بیرون نشت نکنند؛ `cd /d "%~dp0"` برو به پوشه‌ی همین فایل (و درایوش)؛ `call` فایل `find_python.bat` را اجرا می‌کند و برمی‌گردد؛ اگر متغیر `PYTHON` تعریف نشد (پایتونی پیدا نشد)، `pause` («. . . Press any key to continue») و خروج با کد ۹؛ اجرای منو با همان پایتون و همه‌ی آرگومان‌ها؛ نگه داشتن کد خروج؛ اگر آرگومانی داده شده بود برو به برچسب `:wait` و مکث کن.

یعنی	نوشته
درایو و پوشه‌ی خود فایل bat، با `\` پایانی	`%~dp0`
همه‌ی آرگومان‌ها؛ `%1` اولی، `%~1` اولی بی گیومه	`%*`
مقدار متغیر؛ `set NAME=value` بی فاصله دور `=`؛ `set "NAME="` خالی‌اش می‌کند	`%NAME%`
آیا متغیر مقدار دارد؟	`if defined NAME`
کد خروج آخرین فرمان؛ `if errorlevel 1` یعنی «۱ یا بیشتر»	`%ERRORLEVEL%`، `if errorlevel 1`
خروجی و خطا به هیچ‌جا	`>nul 2>&1`

یعنی	نوشته
پایان این فایل (نه پنجره) با کد `N`	`exit /b N`
اجرای bat دیگر و **برگشتن**؛ بی `call` کنترل دیگر برنمی‌گردد	`call file.bat`
صدا زدن زیرروال درون همین فایل؛ `goto :eof` از آن (یا از فایل) برمی‌گردد	`call :label`، `goto :eof`
اجرای فرمان و خواندن هر سطر خروجی‌اش در `%%v` (در فایل `%%`، در خط فرمان `%`)	`for /f "delims=" %%v in ('cmd') do ...`
هر **پوشه**‌ای که با الگو جور است	`for /d %%d in ("C:\X\Py*") do ...`
`<` درون `for /f ('...')`، تا cmd آن را مال فرمان درونی بداند	`^>`

پیدا کردن پایتون: `find_python.bat`

`run.bat` و `bin/set_version.bat` پایتون را خودشان صدا نمی‌زنند: هر دو `bin/find_python.bat` را `call` می‌کنند، که متغیر `PYTHON` را به مسیر کامل یک `python.exe` می‌رساند که **واقعاً اجرا شود** و ۳.۱۱ یا تازه‌تر باشد. اولین `python` در PATH همیشه آن نیست: «Python install manager» یک میان‌بر `python.exe` در پوشه‌ی `WindowsApps` می‌گذارد که اول PATH است، و اگر آن میان‌بر خراب باشد اصلاً اجرا نمی‌شود («The system cannot execute the specified program») در حالی که یک `python.exe` سالم چند پوشه آن‌طرف‌تر است. پس همه به ترتیب امتحان می‌شوند:

```
set "PYTHON="
for /f "delims=" %%p in ('where python.exe 2^>nul') do call :try "%%~p"
if defined PYTHON goto :eof
for /f "delims=" %%p in ('py -3 -c "import sys; print(sys.executable)" 2^>nul') do call :try
"%%~p"
if defined PYTHON goto :eof
call :try "%LOCALAPPDATA%\Python\bin\python.exe"
for /d %%d in ("%LOCALAPPDATA%\Python\pythoncore-3*" ...) do call :try "%%~d\python.exe"
...
:try
if defined PYTHON goto :eof
if not exist "%~1" goto :eof
"%~1" -c "import sys; sys.exit(sys.version_info < (3, 11))" >nul 2>&1
if not errorlevel 1 (
    set "PYTHON=%~1"
    goto :eof
)
for /f "delims=" %%v in ('call "%~1" --version 2^>nul') do set "PYTHON_TOO_OLD=%%v (%~1)"
goto :eof
```

- ترتیب: هر `python.exe` در PATH (`where`)، بعد پایتون ۳ لانچر `py`، بعد پوشه‌هایی که پایتون خودش را در آن‌ها نصب می‌کند (`%LOCALAPPDATA%\Python\bin`، `pythoncore-3*`، `%LOCALAPPDATA%\Programs\Python\Python3*`، `%ProgramFiles%\Python3*`).
- `:try` یک زیرروال است: نامزد را با یک‌خطی پایتون می‌آزماید (کد خروج ۱ اگر قدیمی است یا اگر اصلاً اجرا نشد) و اولین خوب را در `PYTHON` می‌گذارد. نامزدی که اجرا می‌شود ولی قدیمی است در `PYTHON_TOO_OLD` نگه داشته می‌شود تا پیام بگوید کدام پیدا شد.

- فقط `python.exe`، نه `python`: یک shim از نوع `python.bat` (مثل pyenv-win) اگر بی `call` اجرا شود دیگر به این فایل برنمی‌گردد.
- `call` جلوی `for /f` درون `"%~1" --version`: بی آن، cmd گیومه‌های مسیری را که فاصله دارد برمی‌دارد.
- این فایل `setlocal` **ندارد**: `PYTHON` باید به فایلی که آن را صدا زده برسد.
- هر دو فایل آن را با `%~dp0` صدا می‌زنند (`call "%~dp0bin\find_python.bat"`) نه با نام خالی: روی رایانه‌ای که `NoDefaultCurrentDirectoryInExePath` روشن است، cmd فایلی را با نام خالی از پوشه‌ی جاری اجرا نمی‌کند.
- اگر هیچ پایتونی پیدا نشد، همان پیام همیشگی چاپ می‌شود («Python was not found on this machine.» یا «Python 3.11 or newer was not found») به‌اضافه‌ی آنچه پیدا شد) و `PYTHON` خالی می‌ماند.

۷.۲ فایل sh لینوکس

```
#!/bin/sh                               # «با /bin/sh اجرا کن» (سطر اول، shebang)
here=$0                                 # نام همین فایل
cd "$(dirname "$here")" || exit 1       # $(...) خروجی یک فرمان؛ || خورد اگر شکست

python=                                 # متغیر خالی (بی فاصله دور =)
for candidate in python3 python; do
    if command -v "$candidate" >/dev/null 2>&1; then    # آیا این فرمان هست؟
        python=$candidate
        break
    fi
done

if [ -z "$python" ]; then               # [ ... ] آزمون؛ -z یعنی رشته خالی است
    echo "Python 3.11 or newer was not found on this machine."
    exit 9
fi

trap : INT                              # این اسکریپت را نکشد Ctrl+C
"$python" bin/run.py "$@"               # همه‌ی آرگومان‌ها، هرکدام جدا "$@"
exitcode=$?                             # کد خروج آخرین فرمان
```

یعنی	نوشته
آرگومان اول، تعداد آرگومان‌ها، همه‌ی آرگومان‌ها	`$1`، `$#`، `"$@"`
رشته خالی نیست / خالی است	`[ -n "$x" ]`، `[ -z "$x" ]`
ورودی استاندارد یک ترمینال است	`[ -t 0 ]`
عدد نابرابر (`-eq`، `-lt`، `-gt` …)	`[ "$a" -ne 0 ]`
مثل `switch`	`case $x in ... esac`
تعریف تابع (`pause` در `run.sh`)	`name() { ... }`
این اسکریپت را با `cmd` جایگزین کن	`exec cmd`

همیشه متغیرها را در گیومه بگذارید (`"$here"`)؛ وگرنه مسیری که فاصله دارد به چند تکه شکسته می‌شود. `run.sh` عمداً فقط `sh` استاندارد (POSIX) به کار می‌برد، نه چیزهای مخصوص `bash`، تا روی dash (`sh` اوبونتو) هم اجرا شود.

۷.۳ پایان سطر: CRLF و LF

ویندوز پایان هر سطر را با دو نویسه‌ی `\r\n` (CRLF) می‌نویسد و لینوکس با یکی، `\n` (LF). بیشتر فایل‌ها با هر دو کار می‌کنند، جز:

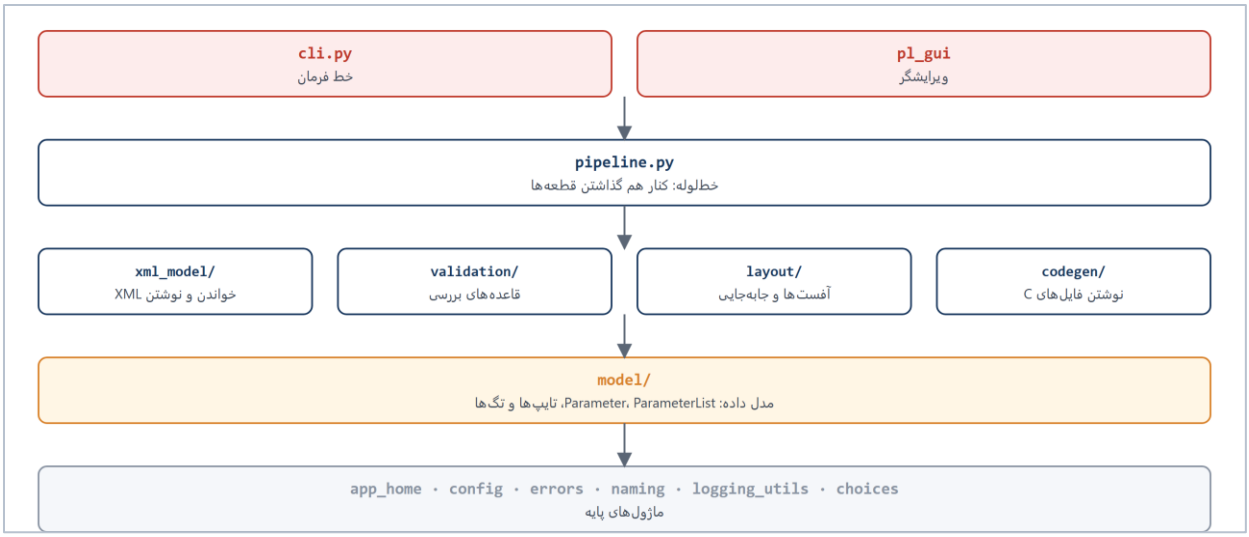
- `run.sh` **باید LF** باشد؛ با CRLF، لینوکس سطر اول را `/bin/sh\r` می‌خواند («bad interpreter») و هیچ چیز اجرا نمی‌شود.
- فایل‌های bat بهتر است CRLF بمانند.

`.gitattributes` این دو را برای git ثابت می‌کند (فصل ۲) و `run_docs_test.py` پایان سطر `run.sh` را بررسی می‌کند. در VS Code پایان سطر فایل باز در نوار پایین (`LF`/`CRLF`) نوشته شده و با کلیک روی آن عوض می‌شود. فایل‌هایی که ابزار تولید می‌کند همیشه LFاند (`newline="\n"`).

۸. معماری تولیدکننده: pl_codegen

فصل‌های ۹ تا ۱۵ تک‌تک فایل‌های `pl_codegen` را شرح می‌دهند. این فصل نقشه‌ی کلی را می‌دهد: پکیج چه بخش‌هایی دارد، کدام از کدام استفاده می‌کند، چه شیءهایی میان آن‌ها دست‌به‌دست می‌شوند و یک پارامتر از XML تا C چه مسیری را طی می‌کند.

۸.۱ بخش‌ها و لایه‌ها



شکل ۲ — لایه‌های pl_codegen؛ پیکان یعنی «استفاده می‌کند از»

فصل	کار	بخش
۹	مسیرهای برنامه، تنظیمات پروژه، پیام‌ها، نام‌های C، لاگ، فهرست‌های کشویی	ماژول‌های پایه
۱۰	شکل فهرست در حافظه؛ هیچ چیز از XML یا C نمی‌داند	`model/`
۱۱	تبدیل XML ↔ مدل	`xml_model/`
۱۲	قاعده‌های بررسی روی مدل	`validation/`
۱۳	آفست هر مقدار در ناحیه‌ی هر تگ، و مقایسه با ساخت قبلی	`layout/`
۱۴	نوشتن فایل‌های C از روی مدل	`codegen/`
۱۵	به ترتیب اجرا کردن همه، و خط فرمان	`cli.py`، `pipeline.py`

لایه‌ها تقریباً یک‌طرفه‌اند: هر بخش فقط از بخش‌های پایین‌ترش استفاده می‌کند و `model/` از هیچ بخش دیگری جز `errors.py` چیزی نمی‌داند. سه استثنا، همه برای قالب `param_table`: `validation/rules/param_table_rules.py` و `layout/drift.py` از `codegen/files/param_table_h.py` تابع `system_of` (زیرسیستم از نام فایل) و ثابت `ADDRESS_MAX` را می‌گیرند، و `drift.py` این کار را با import محلی می‌کند

تا وابستگی دوری پیش نیاید (بخش ۴.۱۳). هنگام افزودن کد، این جهت را رعایت کنید: مثلاً `model/` هرگز نباید `codegen/` را import کند.

۸.۲ شیءهایی که دست‌به‌دست می‌شوند

چه کسی می‌خواهد	چه دارد	از کجا می‌آید	شیء
تقریباً همه	همه‌ی تنظیمات پروژه، با مسیرهای کامل	`Config.load(pl_project.json)`	`Config`
بررسی، آفست، سازنده‌ها	درخت پارامترها، تایپ‌ها، فهرست‌های کشویی	`merge_lists` و `read_xml`	`ParameterList`
خواننده، قاعده‌ها، خط‌لوله	همه‌ی پیام‌های خطا و هشدار	`DiagnosticCollector()`	`DiagnosticCollector`
سازنده‌های فایل	آفست هر مقدار در هر تگ	`build_offset_map(model)`	`OffsetMap`
سازنده‌های فایل	بنر، نشانگرها، پانویس شرکت	`CStyle.load(c.json)`	`CStyle`
`cli.py` و ویرایشگر	تفاوت چیدمان با ساخت قبلی	`compare(previous, model)`	`LayoutDrift`
`cli.py` و ویرایشگر	موفقیت، پیام‌ها، فایل‌ها، جابه‌جایی	`generate` / `Pipeline.run`	`PipelineResult`

۸.۳ سفر یک پارامتر از XML تا C

پارامتر `Airspeed` پروژه‌ی نمونه را دنبال می‌کنیم. **۱. در فایل**: `core.xml`:

```
<Parameter name="Airspeed" kind="primitive" enable="true" userName="Airspeed"
           type="MONITORING" valueType="float32_t" size="1" unit="m/s" logId="100"
address="0">
  <Source />
  <Limits default="0.0" minimum="0.0" maximum="120.0" resolution="0.01" />
  <Tags>
    <Tag id="SystemName" kind="enum" value="SYS_NAV" />
    <Tag id="ResetProof" kind="boolean" value="false" />
    <Tag id="Loggable" kind="boolean" value="true" />
    <Tag id="LogEncryption" kind="boolean" value="false" />
    <Tag id="ParamMode" kind="enum" value="PARAM_MODE_READ_ONLY" />
  </Tags>
  <Description>Indicated airspeed.</Description>
</Parameter>
```

۲. در مدل: `read_xml` (فصل ۱۱) از آن یک `Parameter` می‌سازد: `name="Airspeed"`، `value_type_name="float32_t"`، `minimum=0.0`، `maximum=120.0`، `resolution=0.01`، `unit="m/s"`، `log_id=100` و `tags={"SystemName": "SYS_NAV", "Loggable": True, ...}`. عددها بر اساس نوع داده خوانده شده‌اند: `float` برای `float32_t`.

۳. ادغام: `merge_lists` (فصل ۱۰) آن را با `power.xml` یکی می‌کند؛ `Airspeed` اولین پارامتر اولین ساب‌لیست است.

۴. بررسی: هر ۳۲ قاعده (فصل ۱۲) اجرا می‌شوند؛ مثلاً `DefaultInRangeRule` می‌پرسد ۰ در [۰، ۱۲۰] هست؟ و `LogIdRule` می‌پرسد شناسه‌ی ۱۰۰ تکراری نیست؟ پیامی نمی‌آید.

۵. برگ‌ها: `flatten()` (فصل ۱۰) فهرست تخت ۲۱ مقدار را می‌سازد؛ `Airspeed` برگ شماره‌ی ۰ است.

۶. آفست‌ها: `build_offset_map` (فصل ۱۳): `Airspeed` تگ‌های `SystemName`، `Loggable` و `ParamMode` را دارد و در هر سه ناحیه آفست ۰ و اندازه‌ی ۴ بایت می‌گیرد.

۷. کد C (فصل ۱۴). در `cg_parameter_list_defines.h`:

```
#define AIRSPEED            (0U)   /*!< float32_t [m/s] */
```

در `cg_smart_data.h` و `cg_smart_data.c`:

```
extern sSmartDataF32 Cg_SmartData_Airspeed;  /*!< [m/s] Indicated airspeed. */

status = fSmartData_F32_Ctor(&Cg_SmartData_Airspeed, 0.0f, 120.0f, 0.0f, 0.01f, "m/s");
```

و ردیف ۰ جدول در `cg_parameter_list.c`:

```
    /* [0] AIRSPEED */
    {
        .pName = "Airspeed",
        .pUserName = "Airspeed",
        .Type = ePARAMETER_TYPE_MONITORING,
        .pSmartData = (void *)&Cg_SmartData_Airspeed,
        .LogId = 100U,
        .TagSystemName = (uint32_t)SYS_NAV,
        ...
        .pDescription = "Indicated airspeed."
    },
```

۸. کپی مدل: در پایان، کل مدل ادغام‌شده در `cg_parameter_list/parameter_list.xml` نوشته می‌شود تا ساخت بعدی بتواند بفهمد آفست `Airspeed` جابه‌جا شده یا نه (فصل ۱۳).

۹. در فرم‌ور: شیء `fParameterList_Init()` را با سازنده‌اش می‌سازد، `fSmartData_F32_Read(&Cg_SmartData_Airspeed, &value)` مقدار را می‌خواند و `fParameterList_GetByName("Airspeed")` ردیف جدول را پیدا می‌کند (فصل ۲۲).

> **پیشنهاد:** هر تغییری در خروجی را با همین سفر بسنجید: تغییر در کدام گام است؟ اگر در «فایل» است، فصل ۱۱؛ «مدل» فصل ۱۰؛ «قاعده» فصل ۱۲؛ «C» فصل ۱۴. دستورالعمل‌های فصل ۲۴ دقیقاً همین را برای تغییرهای رایج انجام داده‌اند.

۹. زیرساخت تولیدکننده‌ی کد

این فصل فایل‌های کوچکی را شرح می‌دهد که بقیه‌ی پکیج `pl_codegen` روی آن‌ها بنا شده‌اند: اینکه برنامه کجا نصب شده، خطاها چطور جمع و گزارش می‌شوند، خروجی کنسول چطور رنگی می‌شود، نام پارامترها چطور به نام‌های C تبدیل می‌شوند و فایل پروژه چطور خوانده می‌شود. هر تغییری در این فایل‌ها روی کل ابزار اثر می‌گذارد؛ پس پیش از تغییرشان این فصل را کامل بخوانید.

چه کسانی از آن استفاده می‌کنند	کار	فایل
خط فرمان، سرآیند فایل‌های تولیدی، XML، منو	شماره‌ی نسخه و نام ابزار	`__init__.py`
تقریباً همه‌ی نقطه‌های ورود و ویرایشگر	مسیر پوشه‌های برنامه و فایل تنظیمات ویرایشگر	`app_home.py`
همه‌ی قاعده‌های بررسی، خواننده‌ی XML، خط لوله، ویرایشگر	خطا، هشدار و جمع‌کننده‌ی آن‌ها	`errors.py`
خط فرمان، منو، تست‌ها	چاپ در کنسول و رنگ‌ها	`logging_utils.py`
سازنده‌های فایل، قاعده‌های نام، ویرایشگر	ساختن نام ماکروها و نمادهای C	`naming.py`
خط لوله، خط فرمان، ویرایشگر، تست‌ها	خواندن و بررسی `pl_project.json`	`config.py`
خواننده‌ی XML، ویرایشگر	لیست‌های کشویی رایانه	`choices.py`

۹.۱ `__init__.py` — شناسنامه‌ی پکیج

مسیر: `paramgen/src/pl_codegen/__init__.py`

هر پوشه‌ای که فایلی به نام `__init__.py` داشته باشد، برای پایتون یک **پکیج** است و می‌توان آن را import کرد (بخش ۴.۱۳). این فایل در پکیج `pl_codegen` سه چیز را تعریف می‌کند:

```
__version__ = "01.01.01"
__tool_name__ = "parameter_list_code_generator"

__all__ = ["__version__", "__tool_name__"]
```

نام	معنی
`__version__`	شماره‌ی نسخه‌ی ابزار. تنها جایی است که این شماره نوشته شده و همه‌ی جاهای دیگر آن را از اینجا می‌خوانند: منوی `run.bat`، دستور `--version`، سرآیند فایل‌های C تولیدی و ویژگی `generatorVersion` در XMLها. با `bin/set_version.bat` تغییرش دهید (بخش ۲۱.۶).
`__tool_name__`	نام ابزار در خروجی‌ها؛ مثلاً در سطر `Generator :` سرآیند فایل‌های C و ویژگی `generator` در XMLها.
`__all__`	فهرست نام‌هایی که `from pl_codegen import *` وارد می‌کند. در عمل فقط برای خوانایی است؛ کد پروژه همیشه نام‌ها را صریح import می‌کند.

بقیه‌ی متن فایل یک docstring است که ساختار پوشه‌های پکیج را خلاصه می‌کند. فایل‌های `__init__.py` زیرپوشه‌ها (`model/`، `xml_model/`، `validation/` و...) در فصل همان زیرپوشه آمده‌اند.

فایل `src/pl_codegen/README.md` یک یادداشت کوتاه انگلیسی درباره‌ی همین پکیج است: پوشه‌ها و اینکه خواندن کد را از `pipeline.py` شروع کنید. کد اجرایی ندارد.

۹.۲ `app_home.py` — برنامه کجاست

مسیر: `paramgen/src/pl_codegen/app_home.py`

ابزار دو نیمه دارد که در دو جای جدا زندگی می‌کنند. **برنامه** همین پوشه‌ی `paramgen` است: یک بار روی رایانه قرار می‌گیرد و هنگام کار با یک پروژه در آن چیزی نوشته نمی‌شود، جز پوشه‌ی `var`. **پروژه** پوشه‌ای جداست که فایل `pl_project.json`، ساب‌لیست‌ها و کد تولیدی در آن است. هر جای کد که به چیزی از «برنامه» نیاز دارد، مسیرش را از این فایل می‌گیرد؛ بنابراین هیچ پروژه‌ای لازم نیست بداند برنامه کجا نصب شده است.

مسیر برنامه از جای خود همین فایل حساب می‌شود: `app_home.py` در `paramgen/src/pl_codegen/` است، پس سه پوشه بالاتر خود `paramgen` است. به همین دلیل اگر پوشه‌ی `paramgen` را جابه‌جا یا تغییر نام دهید، هیچ مسیری در کد عوض نمی‌شود.

ثابت‌ها

نام	مقدار	کاربرد
`APP_DIR`	پوشه‌ی `paramgen`	ریشه‌ی همه‌ی مسیرهای برنامه
`SRC_DIR`	`paramgen/src`	کد برنامه؛ نقطه‌های ورود آن را به مسیر import اضافه می‌کنند
`BIN_DIR`	`paramgen/bin`	نقطه‌های ورود
`LAUNCHER`	`run.bat` در ویندوز، `./run.sh` در بقیه	در پیام‌هایی که به کاربر می‌گویند چه چیزی را اجرا کند
`VAR_DIR`	`paramgen/var`	چیزهایی که برنامه می‌نویسد

کاربرد	مقدار	نام
راهنماها	paramgen/docs	DOCS_DIR
الگوهای پروژه‌ی تازه	paramgen/templates	TEMPLATES_DIR
کتابخانه‌ی C؛ فقط تست‌ها از آن کپی می‌گیرند	paramgen کنار smart_data	SMART_DATA_DIR
همان	parameter_descriptor کنار paramgen	PARAMETER_DESCRIPTOR_DIR
قالب فایل‌های C شرکت	templates/c.json	SNIPPET_FILE
فایل پروژه‌ی تازه از روی آن ساخته می‌شود	templates/pl_project.json	PROJECT_TEMPLATE
نام فایل پروژه	"pl_project.json"	PROJECT_FILE_NAME
نام قدیمی فایل پروژه؛ هنوز باز می‌شود	"project.json"	LEGACY_PROJECT_FILE_NAME
دو نامی که خود ابزار به فایل پروژه می‌دهد؛ نام تنها معیار نیست (is_project_file)	هر دو نام بالا	PROJECT_FILES
پروژه‌ی باز و پروژه‌های اخیر ویرایشگر	var/settings.json	SETTINGS_FILE

برای نمونه، APP_DIR این‌طور تعریف شده است؛ __file__ مسیر همین فایل است و هر .parent یک پوشه بالا می‌رود:

```
APP_DIR = Path(__file__).resolve().parent.parent.parent
```

تابع‌ها

is_project_file تابع

```
def is_project_file(path: Path) -> bool:
```

آیا این فایل، فایل پروژه است؟ از روی **محتوایش**، نه نامش: یک فایل JSON که شیء است و دست‌کم یکی از بخش‌های پروژه (project، inputs، output، style، codegen، validation؛ ثابت SECTIONS در config.py) را دارد؛ همان آزمونی که باز کردن پروژه هم می‌کند. صاحب پروژه ممکن است pl_project.json را تغییر نام بدهد و پروژه باید باز هم پروژه بماند.

فایلی بزرگ‌تر از یک مگابایت (_PROJECT_FILE_MAX_BYTES) خوانده نمی‌شود: فایل پروژه چند صد بایت است و خواندن هر JSON بزرگی فقط برای «نه» گفتن، فهرست کردن یک پوشه را کند می‌کرد. هر خطای خواندن یعنی «پروژه نیست».

ورودی‌ها

path	مسیر یک فایل

خروجی: True یا False.

مثال:

```
is_project_file(Path("D:/Fc/flight_computer.json"))   # True - renamed
is_project_file(Path("D:/Fc/notes.json"))             # False - {"todo": []}
```

project_files_in تابع

```
def project_files_in(folder: Path) -> List[Path]:
```

همه‌ی فایل‌های پروژه‌ی **مستقیم** درون یک پوشه، به ترتیب نام؛ با is_project_file.

ورودی‌ها

folder	پوشه

خروجی: فهرستی از Path؛ خالی اگر هیچ نبود یا پوشه خوانده نشد.

project_file_in تابع

```
def project_file_in(folder: Path) -> Path:
```

فایل پروژه‌ی یک پوشه را برمی‌گرداند. اگر pl_project.json در پوشه باشد همان، وگرنه اگر project.json (نام قدیمی) باشد آن، وگرنه - چون صاحب پروژه ممکن است آن را تغییر نام داده باشد - **تنها** فایل پروژه‌ی پوشه (project_files_in). اگر هیچ نبود یا چند فایل پروژه‌ی تغییرنام‌داده بود، مسیر pl_project.json را برمی‌گرداند تا پیام خطای بعدی نام درست را بگوید (Workspace.open و Config.load در آن حالت نام همه را می‌گویند).

هر جای برنامه که از «پوشه» به «فایل پروژه» می‌رسد از همین تابع استفاده می‌کند؛ بنابراین ویرایشگر، خط فرمان و پنجره‌ی Open یک معنی از «پوشه‌ی پروژه» دارند.

ورودی‌ها

folder	پوشه‌ی پروژه

خروجی: مسیر فایل پروژه (Path)؛ ممکن است وجود نداشته باشد.

default_snippet_file تابع

```
def default_snippet_file() -> Optional[Path]:
```

مسیر قالب C شرکت (templates/c.json) را برمی‌گرداند، یا None اگر آن فایل نباشد. نبودن قالب خطا نیست؛ چون قالب فقط شکل ظاهری فایل‌های C را تعیین می‌کند و ابزار بدون آن هم کار می‌کند. اما اگر پروژه‌ای در style.snippet_file قالب دیگری نام ببرد و آن فایل نباشد، Config.validate خطا می‌دهد.

خروجی: Path یا None.

`copy_lib_into` تابع

```
def copy_lib_into(dest: Path) -> Path:
```

دو پوشه‌ی کتابخانه‌ی C (`smart_data` و `parameter_descriptor`) را در پوشه‌ی داده‌شده کپی می‌کند. خود برنامه هرگز این کار را برای یک پروژه‌ی واقعی نمی‌کند؛ فقط تست‌ها از آن استفاده می‌کنند تا پروژه‌ی موقت خود را قابل کامپایل کنند.

ورودی‌ها

`dest`	پوشه‌ی مقصد

خروجی: همان `dest`.

`read_settings` تابع

```
def read_settings(path: Optional[Path] = None) -> Dict[str, Any]:
```

فایل `var/settings.json` را می‌خواند و یک دیکشنری برمی‌گرداند. اگر فایل نباشد، خوانا نباشد یا JSON درستی نداشته باشد، دیکشنری خالی برمی‌گرداند؛ چون این فایل فقط برای راحتی است و نباید هیچ‌وقت جلوی اجرای ابزار را بگیرد.

ورودی‌ها

`path`	فایل تنظیمات دیگری به‌جای `SETTINGS_FILE`؛ تست‌ها از آن استفاده می‌کنند

خروجی: `dict`؛ مثلاً `{"recent": [...], "current": "..."}`.

`write_settings` تابع

```
def write_settings(data: Dict[str, Any], path: Optional[Path] = None) -> None:
```

دیکشنری تنظیمات را به شکل JSON در فایل می‌نویسد. پایان سطرها همیشه LF است (حتی در ویندوز) تا git فایل را بی‌دلیل تغییریافته نشان ندهد؛ فایل `.gitattributes` هم همین را می‌خواهد. اگر پوشه‌ی برنامه فقط‌خواندنی باشد، بی‌صدا چیزی نمی‌نویسد.

ورودی‌ها

`data`	تنظیمات
`path`	مانند `read_settings`

`store_recent_entry` تابع

```
def store_recent_entry(path: Path) -> str:
```

مسیر یک پروژه را به شکلی درمی‌آورد که در فهرست پروژه‌های اخیر ذخیره شود: نسبت به `APP_DIR` و با `./`. مسیر نسبی باعث می‌شود اگر برنامه و پروژه‌ها با هم در یک مخزن git باشند، فهرست اخیر روی رایانه‌ی دیگر هم درست باشد. فقط وقتی پروژه روی درایو دیگری باشد، مسیر کامل نوشته می‌شود.

ورودی‌ها

`path`	مسیر فایل پروژه

خروجی: رشته؛ مثلاً `.../projects/demo/pl_project.json`.

`resolve_recent_entry` تابع

```
def resolve_recent_entry(item: str) -> Path:
```

عکس تابع قبلی: یک مدخل فهرست اخیر را به مسیر واقعی تبدیل می‌کند. مدخل نسبی نسبت به `APP_DIR` خوانده می‌شود و مدخل کامل همان‌طور که هست. `\` هم به‌عنوان جداکننده پذیرفته می‌شود تا مدخلی که در ویندوز نوشته شده در لینوکس هم درست خوانده شود.

ورودی‌ها

`item`	یک مدخل `recent` یا `current`

خروجی: `Path`

`last_project` تابع

```
def last_project() -> Optional[Path]:
```

اولین پروژه‌ی فهرست اخیر را برمی‌گرداند که هنوز روی دیسک است. وقتی کاربر `run.bat generate` را بدون `-c` اجرا کند، خط فرمان همین پروژه را می‌سازد.

خروجی: `Path` فایل پروژه، یا `None`.

`project_name` تابع

```
def project_name(project_file: Path) -> str:
```

نام پروژه را از فایل پروژه می‌خواند (`project.name`). اگر نامی نباشد یا فایل خوانده نشود، نام پوشه را برمی‌گرداند. فهرست پروژه‌های اخیر ویرایشگر و منوی `run.bat` از آن استفاده می‌کنند.

این تابع `load_jsonc` را درون خودش import می‌کند، نه در بالای فایل؛ چون `config.py` خودش `app_home.py` را import می‌کند و import دوطرفه در بالای فایل‌ها خطا می‌دهد (بخش ۴.۱۳).

ورودی‌ها

`project_file`	مسیر فایل پروژه

خروجی: رشته.

۹.۳ `errors.py` — خطاها و هشدارها

مسیر: `paramgen/src/pl_codegen/errors.py`

ابزار دو نوع مشکل را از هم جدا می‌کند:

- **مشکل در فهرست پارامترها** (نام نامعتبر، بازه‌ی غلط و...): ابزار آن را به شکل یک `Diagnostic` ثبت می‌کند و ادامه می‌دهد. به این ترتیب یک اجرا همه‌ی مشکل‌ها را با هم گزارش می‌کند، نه فقط اولی را.
- **مشکلی که ادامه را ناممکن می‌کند** (فایل پروژه نیست، XML خراب است): ابزار یک استثنای `CodeGeneratorError` می‌اندازد و کار متوقف می‌شود.

در C معمولاً هر تابع یک کد خطا برمی‌گرداند. اینجا به‌جای آن، یک شیء `DiagnosticCollector` دست‌به‌دست می‌شود و هر بخش مشکل‌هایش را در آن می‌ریزد؛ مثل آرایه‌ای که هر تابع یک خانه به آن اضافه کند. در پایان، اگر حتی یک خطا (`ERROR`) در آن باشد، کدی نوشته نمی‌شود.

`CodeGeneratorError` کلاس

```
class CodeGeneratorError(Exception):
```

استثنای «ابزار نمی‌تواند ادامه دهد». هیچ کد اضافه‌ای ندارد؛ فقط از `Exception` ارث می‌برد تا بتوان آن را جدا از بقیه‌ی استثناها گرفت. متن پیامش همیشه برای کاربر نوشته شده است، چون خط فرمان و ویرایشگر همان متن را نشان می‌دهند.

مثال:

```
raise CodeGeneratorError(f"'{path}' is not valid JSON: {exc}")
```

`Severity` کلاس

```
class Severity(IntEnum):
```

سطح یک مشکل. چون از `IntEnum` ارث می‌برد، مقدارهایش عدد هم هستند و می‌توان آن‌ها را مقایسه کرد: `INFO = 0`، `WARNING = 1`، `ERROR = 2`؛ پس `severity >= Severity.WARNING` یعنی «هشدار یا خطا». این همان کاری است که `enum` در C می‌کند.

`Severity.label` ویژگی (property)

```
@property
def label(self) -> str:
```

نام کوچک سطح برای نمایش و JSON: `"info"`، `"warning"` یا `"error"`. ویرایشگر همین رشته‌ها را در پنل Problems نشان می‌دهد.

ثابت `_SEVERITY_COLOUR` رنگ هر سطح را در کنسول تعیین می‌کند: INFO فیروزه‌ای، WARNING زرد و ERROR قرمز روشن.

`SourceLocation` کلاس

```
@dataclass(frozen=True)
class SourceLocation:
```

جای یک مشکل. `frozen=True` یعنی بعد از ساخته شدن قابل تغییر نیست (مثل `const` در C) و می‌توان آن را با خیال راحت بین چند Diagnostic مشترک گذاشت. همه‌ی فیلدها اختیاری‌اند:

- `file`: فایل XML یا فایل پروژه.
- `sheet`، `row` و `column`: از دوره‌ای مانده‌اند که فهرست پارامترها در Excel بود؛ هنوز در رندر چاپ می‌شوند اگر کسی پرشان کند.
- `parameter`: مسیر نقطه‌دار پارامتر، مثل `Position.Home.Lat`. ویرایشگر با همین فیلد به ردیف مشکل‌دار می‌پرد.
- `struct_type`: نام تایپ استراکچر، وقتی مشکل مال یک تایپ است؛ ویرایشگر آن‌وقت پنجره‌ی Structure types را روی همان تایپ باز می‌کند.

`SourceLocation.__str__` متد

```
def __str__(self) -> str:
```

متن خوانای مکان؛ مثلاً `parameter 'Airspeed' parameter_lists/core.xml`. پایتون هر بار که شیء را به رشته تبدیل می‌کند (مثلاً در f-string) این متد را صدا می‌زند. اگر هیچ فیلدی پر نباشد، `<unknown location>` برمی‌گرداند.

`SourceLocation.to_dict` متد

```
def to_dict(self) -> Dict[str, object]:
```

مکان را به دیکشنری تبدیل می‌کند تا به شکل JSON به مرورگر فرستاده شود. نام کلید `structType` با حرف بزرگ است، چون JavaScript این قرارداد را دارد.

`Diagnostic` کلاس

```
@dataclass(frozen=True)
class Diagnostic:
```

یک مشکل: `severity`، `code` (مثل `RNG003`)، `message`، `location` و `hint` (راه رفع، اختیاری). این هم frozen است. `field(default_factory=SourceLocation)` یعنی اگر مکانی داده نشود، برای هر Diagnostic یک `SourceLocation` خالی تازه ساخته شود (بخش ۴.۱۰).

`Diagnostic.render` متد

```
def render(self) -> str:
```

متن چندسطری Diagnostic برای کنسول: سطر اول سطح و کد (رنگی) و پیام، سطر دوم `at` و مکان، و اگر راه‌حلی باشد سطر سوم `hint`. فقط سطح و کد رنگی می‌شوند، چون متن پیام را باید خواند و رنگ خواندن را سخت می‌کند.

مثال:

```
ERROR   RNG003: Default value 500.0 is outside the allowed range [0.0, 120.0].
        The smart-data constructor would refuse it at run time.
            at parameter 'Airspeed' D:\Projects\Flight_Computer\parameter_lists\core.xml
            hint: Leave Minimum and Maximum both empty (or both 0) to allow the
        whole range of the type.
```

این خروجی واقعی `run.bat generate --check` روی پروژه‌ی نمونه است، وقتی مقدار پیش‌فرض Airspeed برابر `500` باشد؛ سطرهای بلند برای جا شدن در صفحه شکسته شده‌اند.

`Diagnostic.to_dict` متد

```
def to_dict(self) -> Dict[str, object]:
```

Diagnostic را برای JSON آماده می‌کند؛ سطح به شکل رشته (`label`) و مکان با `to_dict` خودش. پنل Problems ویرایشگر دقیقاً همین دیکشنری‌ها را دریافت می‌کند.

`DiagnosticCollector` کلاس

```
class DiagnosticCollector:
```

ظرف همه‌ی Diagnosticهای یک اجرا. فهرست داخلی `_items` با `_` شروع می‌شود؛ یعنی «از بیرون مستقیم دست نزنید» (پایتون `private` واقعی ندارد؛ بخش ۴.۹). از بیرون با متدهای زیر کار کنید:

مثال:

```
diagnostics = DiagnosticCollector()
diagnostics.error("NAME001", "Name '2x' is not a valid C identifier.",
                  parameter.source, hint="Start the name with a letter.")
if diagnostics.has_errors:
    print(diagnostics.render())
```

کار	نام
یک فهرست خالی می‌سازد.	`DiagnosticCollector.__init__`
یک `Diagnostic` آماده را اضافه می‌کند.	`DiagnosticCollector.add`
یک خطا می‌سازد و اضافه می‌کند. اگر مکان داده نشود، مکان خالی.	`DiagnosticCollector.error`
همان، برای هشدار.	`DiagnosticCollector.warning`
همان، برای پیام اطلاعاتی.	`DiagnosticCollector.info`
ویژگی: یک **کپی** از فهرست؛ تغییر آن روی جمع‌کننده اثری ندارد.	`DiagnosticCollector.items`
تعداد مشکل‌های یک سطح.	`DiagnosticCollector.count`
ویژگی: آیا حتی یک خطا هست؟ خط لوله با همین تصمیم می‌گیرد کد بنویسد یا نه.	`DiagnosticCollector.has_errors`
ویژگی: آیا هشداری هست؟ برای گزینه‌ی `-W`.	`DiagnosticCollector.has_warnings`
تعداد کل؛ به همین دلیل `len(diagnostics)` کار می‌کند.	`DiagnosticCollector.__len__`
همه را با `to_dict` برای JSON آماده می‌کند.	`DiagnosticCollector.to_list`

`DiagnosticCollector.render` متد

```
def render(self, min_severity: Severity = Severity.INFO) -> str:
```

متن همه‌ی مشکل‌های هم‌سطح یا شدیدتر از `min_severity` را برای کنسول می‌سازد. مشکل‌ها اول بر اساس سطح (خطاها اول)، بعد فایل، سطر و کد مرتب می‌شوند. اگر چیزی نباشد، `No problems found.` برمی‌گرداند.

ورودی‌ها

`min_severity`	کمترین سطحی که نشان داده شود؛ پیش‌فرض همه

`DiagnosticCollector.summary` متد

```
def summary(self) -> str:
```

سطر جمع‌بندی، مثل `0 error(s), 2 warning(s), 1 info message(s)`. شمارش خطا و هشدار فقط وقتی رنگی می‌شود که صفر نباشد.

۹.۴ `logging_utils.py` — کنسول و رنگ

مسیر: `paramgen/src/pl_codegen/logging_utils.py`

همه‌ی چیزی که ابزار در کنسول چاپ می‌کند از این فایل می‌گذرد. کار اصلی‌اش رنگ است: در انبوه سطرهای INFO، سطر ERROR باید بدون خواندن پیدا شود. رنگ در ترمینال با **کدهای ANSI** ساخته می‌شود؛ رشته‌هایی که با کاراکتر Escape (`\033`) شروع می‌شوند و ترمینال به‌جای چاپ، رنگ را با آن‌ها عوض می‌کند. برای نمونه `"\033[31m"` یعنی «از اینجا قرمز».

سه قاعده برای رنگ رعایت می‌شود:

- فقط وقتی خروجی یک کنسول واقعی است. اگر خروجی به فایل یا CI برود، متن ساده نوشته می‌شود؛ کدهای ANSI در فایل فقط نویز هستند.
- اگر متغیر محیطی `NO_COLOR` تنظیم شده باشد، رنگ خاموش است؛ این یک قرارداد عمومی بین ابزارهاست.
- در ویندوز ۱۰ باید «پردازش ترمینال مجازی» یک بار روشن شود تا کنسول کدها را اجرا کند؛ اگر کنسول نپذیرد، رنگ کنار گذاشته می‌شود.

ثابت‌ها و متغیرهای سراسری

نام	معنی
`RESET`، `BOLD`، `RED`، `BRIGHT_RED`، `YELLOW`، `GREEN`، `CYAN`، `GREY`	کدهای ANSI رنگ‌ها. `RESET` رنگ را به حالت عادی برمی‌گرداند.
`_LOGGER_NAME`	نام logger ابزار: `"pl_codegen"`.
`_FORMAT`	قالب هر سطر log: نام سطح در ۸ نویسه، بعد پیام.
`_LEVEL_COLOURS`	رنگ هر سطح log؛ DEBUG خاکستری، WARNING زرد، ERROR قرمز.
`_colour_enabled`	آیا رنگ روشن است؟ `setup_logging` آن را تنظیم می‌کند و `paint` می‌خواندش. تست‌ها هم مستقیم آن را مقدار می‌دهند.

`_enable_windows_ansi` تابع

```
def _enable_windows_ansi() -> bool:
```

در ویندوز، با کتابخانه‌ی `ctypes` (فراخوانی مستقیم تابع‌های DLL ویندوز، مثل صدا زدن یک تابع C) حالت `ENABLE_VIRTUAL_TERMINAL_PROCESSING` را روی خروجی استاندارد روشن می‌کند. در سیستم‌های دیگر بی‌درنگ `True` برمی‌گرداند. هر خطایی یعنی «رنگ نه».

خروجی: آیا کنسول کدهای رنگ را اجرا می‌کند.

`_use_utf8` تابع

```
def _use_utf8() -> None:
```

خروجی و خطای استاندارد را روی UTF-8 تنظیم می‌کند. کنسول ویندوز به‌طور پیش‌فرض cp1252 است و نمی‌تواند نویسه‌هایی مثل ▾ یا مسیرهای فارسی را چاپ کند؛ خطای چاپ در میانه‌ی گزارش یک مشکل، خود مشکل را پنهان می‌کند. این تابع همان لحظه‌ی import فایل یک بار اجرا می‌شود (سطر `_use_utf8()` در سطح ماژول).

`colour_supported` تابع

```
def colour_supported(stream=None) -> bool:
```

آیا جریان خروجی داده‌شده (پیش‌فرض stdout) باید رنگ بگیرد: `NO_COLOR` تنظیم نشده باشد، خروجی یک ترمینال باشد (`isatty`) و در ویندوز حالت ANSI روشن شود.

ورودی‌ها

`stream`	جریان خروجی؛ `None` یعنی `sys.stdout`

`paint` تابع

```
def paint(text: str, colour: str) -> str:
```

متن را در رنگ می‌پیچد: `رنگ + متن + RESET`. اگر رنگ خاموش باشد یا رنگی داده نشود، همان متن را دست‌نخورده برمی‌گرداند. همه‌ی جاهای ابزار برای رنگی کردن از همین تابع استفاده می‌کنند.

مثال:

```
print(paint("FAILED", BRIGHT_RED))
```

`_ColourFormatter` کلاس

```
class _ColourFormatter(logging.Formatter):
```

قالب‌دهنده‌ی سطرهای log. از `logging.Formatter` ارث می‌برد و فقط `format` را عوض می‌کند.

`_ColourFormatter.format` متد

```
def format(self, record: logging.LogRecord) -> str:
```

برای سطح‌های عادی فقط نام سطح را رنگی می‌کند تا پیام خوانا بماند؛ برای ERROR و بالاتر کل سطر را. `super().format(record)` یعنی «اول کار پدر را انجام بده» (بخش ۴.۹).

`setup_logging` تابع

```
def setup_logging(verbose: bool = False, quiet: bool = False,
                  colour: bool | None = None) -> logging.Logger:
```

logger ابزار را آماده می‌کند: رنگ را تعیین می‌کند، سطح را از `verbose` و `quiet` می‌گیرد (DEBUG، INFO یا WARNING)، هندلرهای قبلی را پاک می‌کند و یک هندلر تازه روی stdout با `_ColourFormatter` می‌گذارد. کلیدواژه‌ی `global` در اولش لازم است، چون تابع متغیر سراسری `_colour_enabled` را **مقدار** می‌دهد (بخش ۴.۷).

ورودی‌ها

`verbose`	`True` برای دیدن پیام‌های DEBUG
`quiet`	`True` برای دیدن فقط هشدار و خطا
`colour`	`False`/`True` برای اجبار؛ `None` یعنی از روی خروجی تصمیم بگیر

خروجی: شیء `logging.Logger`.

`get_logger` تابع

```
def get_logger() -> logging.Logger:
```

همان logger با نام `pl_codegen` را برمی‌گرداند. هر فایلی که بخواهد چیزی log کند، این را صدا می‌زند: `get_logger().warning(...)`.

`banner` تابع

```
def banner(text: str, colour: str = CYAN) -> str:
```

یک تیتر سه‌سطری بین مرحله‌های خط لوله می‌سازد: یک خط ۷۸تایی =، متن، و دوباره خط.

۹.۵ `naming.py` — از نام پارامتر تا نام‌های C

مسیر: `paramgen/src/pl_codegen/naming.py`

هر قاعده‌ای که نام یک پارامتر را به یک نام C تبدیل می‌کند، فقط در این فایل است. اگر روزی قرارداد نام‌گذاری شرکت عوض شود (مثلاً پیشوند ماکروها)، فقط همین فایل تغییر می‌کند. سازنده‌های فایل و قاعده‌های بررسی هر دو از همین تابع‌ها استفاده می‌کنند؛ بنابراین نامی که بررسی می‌شود همان نامی است که نوشته می‌شود.

ثابت‌ها

نام	معنی
`_BOUNDARY_LOWER_UPPER`	عبارت منظمی که جای بین حرف کوچک یا رقم و حرف بزرگ را پیدا می‌کند (`TestVar` ← `Test_Var`).
`_BOUNDARY_ACRONYM`	جای بین دو حرف بزرگ وقتی دومی اول یک کلمه است (`HTTPServer` ← `HTTP_Server`).
`_C_IDENTIFIER`	یک شناسه‌ی معتبر C: حرف یا _ و بعد حرف، رقم یا _.
`C_RESERVED_KEYWORDS`	کلیدواژه‌های C99 و C++ که نام پارامتر نمی‌توانند باشند؛ چون هدرهای تولیدی باید در C++ هم include شوند.
`_ARRAY_INDEX`	یک ایندکس آرایه مثل `[2]`.

`is_valid_c_identifier` تابع

```
def is_valid_c_identifier(name: str) -> bool:
```

آیا نام می‌تواند شناسه‌ی C باشد. رشته‌ی خالی نامعتبر است.

مثال:

```
is_valid_c_identifier("_x9")    # True
is_valid_c_identifier("2abc")   # False
```

`is_reserved_keyword` تابع

```
def is_reserved_keyword(name: str) -> bool:
```

آیا نام یکی از کلیدواژه‌های C یا C++ است. بزرگی و کوچکی حروف مهم است: `class` کلیدواژه است و `Class` نه.

`to_macro_case` تابع

```
def to_macro_case(name: str) -> str:
```

قلب این فایل: هر نامی را به شکل `UPPER_SNAKE_CASE` درمی‌آورد. ترتیب کار:

- `-`، فاصله و `.` به `_` تبدیل می‌شوند؛ `]` به `_` و `[` حذف می‌شود.
- بین دو کلمه‌ی CamelCase یک `_` گذاشته می‌شود (دو عبارت منظم بالا).
- `_`های پشت سر هم یکی می‌شوند و همه‌چیز با حروف بزرگ نوشته می‌شود.

ورودی	خروجی
`TestVarU16`	`TEST_VAR_U16`
`MaxAltitude`	`MAX_ALTITUDE`
`HTTPServer`	`HTTP_SERVER`
`Position.Home.Lat`	`POSITION_HOME_LAT`
`CellVoltage[0]`	`CELL_VOLTAGE_0`
`Motor[1].Rpm`	`MOTOR_1_RPM`
`PR_GSM_TABLE_NUM`	`PR_GSM_TABLE_NUM`
`my-name x`	`MY_NAME_X`

`index_macro` تابع

```
def index_macro(name: str, prefix: str = "") -> str:
```

نام ماکروی ایندکس یک مقدار: پیشوند (`codegen.index_define_prefix`) به‌علاوه‌ی `to_macro_case`. مثلاً با پیشوند `PL_`، `Airspeed` می‌شود `PL_AIRSPEED`.

`strip_array_indices` تابع

```
def strip_array_indices(name: str) -> str:
```

ایندکس‌های آرایه را از یک نام برمی‌دارد: `Wheels[2].Samples[1]` ← `Wheels.Samples`. عنصرهای یک آرایه‌ی استراکچر هرکدام مقدار جدایی‌اند، ولی «اعلان» یکسانی دارند؛ هرچه مال اعلان است (مثل اندازه‌ی آرایه) باید بدون ایندکس نام‌گذاری شود.

`array_size_macro` تابع

```
def array_size_macro(name: str, prefix: str = "") -> str:
```

نام ماکروی اندازه‌ی آرایه: `CellVoltage` ← `CELL_VOLTAGE_ARRAY_SIZE`. چون از `strip_array_indices` می‌گذرد، `Wheels[0].Samples` و `Wheels[1].Samples` هر دو `WHEELS_SAMPLES_ARRAY_SIZE` را می‌گیرند؛ همان ماکرویی که typedef با آن نوشته شده است.

`smart_data_symbol` تابع

```
def smart_data_symbol(name: str, prefix: str = "Cg_SmartData_") -> str:
```

نام شیء smart data یک پارامتر سطح بالا: پیشوند (پیش‌فرض `Cg_SmartData_`) به‌علاوه‌ی نام خود پارامتر، بدون تغییر حروف: `Cg_SmartData_Airspeed`.

`struct_type_name` تابع

```
def struct_type_name(dotted_name: str, prefix: str = "sCgStruct_") -> str:
```

نام C یک تایپ استراکچر: پیشوند (پیش‌فرض `sCgStruct_`) به‌علاوه‌ی نام با `.` تبدیل‌شده به `_`: `Position.Home` ← `sCgStruct_Position_Home`.

`offset_total_macro` تابع

```
def offset_total_macro(tag_id: str, prefix: str = "") -> str:
```

نام ماکروی اندازه‌ی کل یک تگ: `PARAMETER_LIST_TAG_RESET_PROOF_SIZE` برای `ResetProof`. پیشوندش هنوز از تنظیمی به نام `codegen.offset_define_prefix` می‌آید؛ نامی که از دوره‌ای مانده که ماکروهای آفست تک‌تک پارامترها هم وجود داشتند. تغییر نام کلید، فایل پروژه‌های موجود را خراب می‌کرد.

`tag_size_function` تابع

```
def tag_size_function(tag_id: str, prefix: str = "fParameterList_Get") -> str:
```

نام تابع C که اندازه‌ی کل یک تگ را برمی‌گرداند: `ResetProof` ← `fParameterList_GetResetProofSize`.

`include_guard` تابع

```
def include_guard(file_name: str) -> str:
```

نام include guard یک فایل: پسوند حذف و بقیه با حروف بزرگ، به‌علاوه‌ی `_H`: `cg_parameter_list_defines.h` ← `CG_PARAMETER_LIST_DEFINES_H`.

۹.۶ `config.py` — خواندن فایل پروژه

مسیر: `paramgen/src/pl_codegen/config.py`

این فایل `pl_project.json` را می‌خواند، مسیرهای آن را کامل می‌کند، کلیدهای ناشناخته را هشدار می‌دهد و نتیجه را در یک شیء `Config` تحویل می‌دهد. از اینجا به بعد هیچ بخش دیگری فایل JSON را نمی‌خواند؛ همه از `Config` استفاده می‌کنند.

اصل مهم این فایل: **فایل پروژه، خود پروژه است.** پوشه‌ای که فایل در آن است ریشه‌ی پروژه است و هر مسیر نسبی در فایل از همان‌جا حساب می‌شود. به همین دلیل می‌توان پوشه‌ی پروژه را جابه‌جا کرد و هنوز کار کند.

ساختار Config

شیء `Config` از چند dataclass ساخته شده که هرکدام یک بخش فایل پروژه است. جدول زیر نشان می‌دهد هر کلید JSON به کدام فیلد می‌رسد (توضیح معنای هر کلید در فصل ۱۴ راهنمای کاربر آمده است):

کلید در JSON	فیلد در Config	پیش‌فرض
`project.name`	`config.project.name`	نام پوشه
`project.description`	`config.project.description`	خالی
`inputs.model_files` (یا `model_file`)	`config.inputs.model_files` (فهرست `Path`)	خالی
`inputs.model_directory`	`config.inputs.model_directory`	`parameter_lists`
`output.format`	`config.output.format`	`smart_data`
`output.code_directory`	`config.output.code_directory`	`cg_parameter_list`
`output.full_parameter_list_file` (و سه نام قدیمی)	`config.output.xml_file`	`full_parameter_list/parameter_list.xml`
`output.file_prefix`	`config.output.file_prefix`	`cg_`
`output.base_name`	`config.output.base_name`	نام اولین ساب‌لیست، یا `parameter_list`
`style.snippet_file`	`config.style.snippet_file`	برنامه `templates/c.json`
`style.company`	`config.style.company`	`FAS`
`style.indent`	`config.style.indent`	چهار فاصله
`codegen.*`	`config.codegen.*` (همان نام‌ها)	جدول `CodegenConfig`
`validation.*`	`config.validation.*` (همان نام‌ها)	هر دو `False`

`load_jsonc` تابع

```
def load_jsonc(path: Path) -> Dict[str, Any]:
```

یک فایل JSON را می‌خواند که می‌تواند توضیح `//` و ویرگول اضافه پس از آخرین عضو داشته باشد (قالبی که VS Code به آن JSONC می‌گوید). کارش ساده است: اول با دو عبارت منظم سطرهای توضیح و ویرگول‌های اضافه را پاک می‌کند، بعد بقیه را به `json.loads` می‌دهد. `utf-8-sig` یعنی اگر فایل BOM داشته باشد، نادیده گرفته شود.

دقت کنید: فقط توضیحی حذف می‌شود که **کل سطر** باشد. توضیح در انتهای یک سطر کد (`"a": 1 // x`) پذیرفته نیست.

ورودی‌ها

`path`	مسیر فایل

خروجی: دیکشنری محتوای فایل.

خطا: `CodeGeneratorError` اگر فایل خوانده نشود یا JSON درستی نباشد.

`relative_to_root` تابع

```
def relative_to_root(path: Union[str, Path], root: Path) -> str:
```

مسیر یک فایل را آن‌طور که پروژه نامش می‌برد برمی‌گرداند: نسبت به ریشه‌ی پروژه و با `/`. فایل‌های تولیدی ساب‌لیست‌ها را با همین شکل نام می‌برند تا دو رایانه از یک پروژه خروجی یکسانی بسازند.

خروجی	ریشه	مسیر
`parameter_lists/core.xml`	`C:\Users\me\projects\RHS`	`C:\Users\me\projects\RHS\parameter_lists\core.xml`
`../../shared/x.xml`	همان	`C:\Users\me\shared\x.xml`
`D:/other/x.xml`	همان (درایو دیگر)	`D:\other\x.xml`

`ProjectConfig` کلاس

```
@dataclass
class ProjectConfig:
```

بخش `project`: نام و توضیح پروژه.

`InputConfig` کلاس

```
@dataclass
class InputConfig:
```

بخش `inputs`: فهرست ساب‌لیست‌ها به ترتیب ساخت (`model_files`) و پوشه‌ی ساب‌لیست‌ها (`model_directory`).

`InputConfig.model_file` ویژگی (property)

```
@property
def model_file(self) -> Optional[Path]:
```

اولین ساب‌لیست، یا `None` برای پروژه‌ای که هنوز ساب‌لیستی ندارد. خطا نمی‌دهد، چون پروژه‌ی تازه‌ساخته‌شده واقعاً همین حالت را دارد و ویرایشگر باید بتواند آن را نشان دهد.

دو ثابت `FORMAT_SMART_DATA = "smart_data"` و `FORMAT_PARAM_TABLE = "param_table"` دو قالب خروجی‌اند و `OUTPUT_FORMATS` هر دو را با هم دارد. هر جای کد که رفتارش به قالب بستگی دارد، با این ثابت‌ها مقایسه می‌کند، نه با رشته‌ی دستی.

`OutputConfig` کلاس

```
@dataclass
class OutputConfig:
```

بخش `output`. نکته‌ی مهم: فیلد `xml_file` همان کلید `full_parameter_list_file` است. این کلید سه نام قدیمی‌تر هم دارد (`last_build_file`، `snapshot_file`، `xml_file`) که هنوز خوانده می‌شوند، چون فایل پروژه‌های قدیمی از آن‌ها استفاده می‌کنند.

`StyleConfig` کلاس

```
@dataclass
class StyleConfig:
```

بخش `style`: قالب فایل‌های C، نام شرکت و تورفتگی.

`CodegenConfig` کلاس

```
@dataclass
class CodegenConfig:
```

بخش `codegen`: کلیدهای JSON دقیقاً هم‌نام فیلدهای این کلاس‌اند؛ برای همین با `_dataclass_from_dict` ساخته می‌شود. افزودن یک کلید تازه به `codegen` یعنی فقط افزودن یک فیلد به این کلاس (دستورالعمل در فصل ۲۴).

`ValidationConfig` کلاس

```
@dataclass
class ValidationConfig:
```

بخش `validation`: `treat_warnings_as_errors` و `require_unit`.

`Config` کلاس

```
@dataclass
class Config:
```

کل تنظیمات یک پروژه. `path` مسیر کامل خود فایل پروژه است و `path.parent` ریشه‌ی پروژه.

`Config.load` متد کلاس

```
@classmethod
def load(cls, config_path: Union[str, Path]) -> "Config":
```

تنها راه ساختن `Config` از روی فایل. `@classmethod` یعنی روی خود کلاس صدا زده می‌شود، نه روی یک شیء: `Config.load("...")` (بخش ۴.۹). گام‌ها به ترتیب:

- مسیر کامل می‌شود. اگر فایل نباشد، پیام خطا فرق می‌گذارد بین «این پوشه پروژه نیست»، «این پوشه چند فایل پروژه دارد، یکی را نام ببرید» (با نام همه) و «این مسیر وجود ندارد». نام فایل مهم نیست: هر JSON که پروژه باشد باز می‌شود.
- فایل با `load_jsonc` خوانده و با `_refuse_a_file_that_is_not_a_project` بررسی می‌شود.
- بخش‌ها و کلیدهای ناشناخته هشدار می‌گیرند (با پیشنهاد نزدیک‌ترین نام درست).

- هر بخش به dataclass خودش تبدیل می‌شود و مسیرها کامل می‌شوند. نام تنهای یک ساب‌لیست در `model_directory` جست‌وجو می‌شود و مسیری که پوشه دارد نسبت به فایل پروژه.
- `format` بررسی می‌شود؛ اگر غلط باشد خطا با پیشنهاد نزدیک‌ترین مقدار.
- در پایان `validate()` صدا زده می‌شود.

ورودی‌ها

`config_path`	مسیر `pl_project.json` (رشته یا `Path`)

خروجی: یک `Config` کامل.

خطا: `CodeGeneratorError` برای هر مشکلی که ادامه را ناممکن کند.

مثال:

```
config = Config.load(r"D:\Projects\Flight_Computer\pl_project.json")
config.output.format            # 'smart_data'
config.inputs.model_files[0]    #
Path('D:/Projects/Flight_Computer/parameter_lists/core.xml')
```

`Config.validate` متد

```
def validate(self) -> None:
```

چیزهایی را بررسی می‌کند که اگر غلط باشند، کد C تولیدی کامپایل نمی‌شود: قالب C نام‌برده وجود داشته باشد، `indent` فقط فاصله و Tab باشد، و `base_name` و `file_prefix` فقط حرف، رقم و `_` داشته باشند (چون به نام فایل و include guard تبدیل می‌شوند).

خطا: `CodeGeneratorError` با پیامی که می‌گوید کدام تنظیم را چطور درست کنید.

`Config.generated_file_name` متد

```
def generated_file_name(self, suffix: str, extension: str) -> str:
```

نام یک فایل تولیدی: پیشوند + نام پایه + `_` + پسوند + `.` + نوع.

مثال:

```
config.generated_file_name("defines", "h")   # 'cg_parameter_list_defines.h'
config.generated_file_name("", "c")          # 'cg_parameter_list.c'
```

`Config.smart_data_file_name` متد

```
def smart_data_file_name(self, extension: str) -> str:
```

نام فایل smart data: پیشوند + `smart_data.` + نوع؛ مثلاً `cg_smart_data.h`. این فایل `base_name` نمی‌گیرد.

`_closest` تابع

```
def _closest(key: str, candidates: Any) -> Optional[str]:
```

نزدیک‌ترین نام شناخته‌شده به یک کلید غلط را با `difflib.get_close_matches` پیدا می‌کند (با حداقل ۷۰٪ شباهت)؛ برای پیام‌هایی مثل «Did you mean "emit_timestamp"?».

ثابت `SECTIONS` شش بخش مجاز فایل پروژه را دارد: `project`، `inputs`، `output`، `style`، `codegen` و `validation`.

`_refuse_a_file_that_is_not_a_project` تابع

```
def _refuse_a_file_that_is_not_a_project(path: Path, data: Dict[str, Any]) -> None:
```

فایلی را که اصلاً فایل پروژه نیست رد می‌کند: اگر JSON یک شیء (`{}`) نباشد، یا هیچ‌کدام از شش بخش را نداشته باشد. پیش از این، هر فایل JSON دیگری هم به‌عنوان پروژه باز می‌شد و تولید بعدی، فهرست پارامترها را در پوشه‌ی آن فایل می‌نوشت.

`_warn_unknown_sections` تابع

```
def _warn_unknown_sections(data: Dict[str, Any]) -> None:
```

برای هر بخش سطح بالای ناشناخته یک هشدار log می‌کند؛ بخشی که نامش غلط تایپ شده هیچ اثری ندارد و کاربر باید بداند.

`_warn_unknown_keys` تابع

```
def _warn_unknown_keys(section: str, data: Dict[str, Any], known: Any) -> None:
```

همان کار، برای کلیدهای داخل بخش‌هایی که dataclass نیستند (`project`، `inputs`، `output`، `style`).

`_dataclass_from_dict` تابع

```
def _dataclass_from_dict(cls: type, data: Dict[str, Any], section: str = "") -> Any:
```

یک dataclass را از دیکشنری می‌سازد و کلیدهای ناشناخته را هشدار می‌دهد. نام فیلدها را از `cls.__dataclass_fields__` می‌خواند؛ پس با افزودن یک فیلد به `CodegenConfig`، کلید JSON آن خودبه‌خود شناخته می‌شود.

ورودی‌ها

`cls`	کلاس dataclass
`data`	دیکشنری آن بخش
`section`	نام بخش، برای پیام هشدار

خروجی: شیء ساخته‌شده.

۹.۷ `choices.py` — لیست‌های کشویی رایانه

مسیر: `paramgen/src/pl_codegen/choices.py`

چهار فهرست Parameter types، System names، Parameter modes و Value types مال شرکت‌اند، نه یک پروژه؛ پس در پوشه‌ی برنامه نگه داشته می‌شوند: نسخه‌ی رایانه در `var/choices.json` و نسخه‌ی پیش‌فرض شرکت در `templates/choices.json`. در حافظه، این فهرست‌ها در یک شیء `Defines` نگه داشته می‌شوند (فصل ۱۰).

ثابت	معنی
`CHOICES_FILE`	`var/choices.json`؛ فهرست‌های این رایانه.
`CHOICES_TEMPLATE`	`templates/choices.json`؛ پیش‌فرض‌های شرکت.
`COLUMNS`	کلید JSON هر فهرست ← نام ستون آن در `Defines` (مثلاً `systemNames` ← `SYSTEM NAMES`). نام ستون‌ها از دوره‌ی Excel مانده‌اند.
`FALLBACK`	حداقل فهرست‌ها، اگر فایل الگو هم پاک شده باشد؛ تا پروژه همیشه باز شود (`paramModes`: `PARAM_MODE_NONE` و `PARAM_MODE_USER`).
`NUMBERED`	کلید فهرست‌هایی که نام‌هایشان `#define` عددی می‌شوند: `systemNames` و `paramModes` (از `GENERATED_DEFINES` فصل ۱۰).
`NUMBERS_KEY`	`"numbers"`: کلیدی که عدد آن نام‌ها زیرش، کنار فهرست‌ها، نوشته می‌شود.

`default_choices` تابع

```
def default_choices() -> Dict[str, List[str]]:
```

فهرست‌های پیش‌فرض شرکت را از `templates/choices.json` می‌خواند؛ هر فهرستی که در آن نباشد از `FALLBACK` می‌آید. دکمه‌ی Restore the company defaults ویرایشگر همین را برمی‌گرداند.

خروجی: دیکشنری با چهار کلید `COLUMNS`.

`default_defines` تابع

```
def default_defines() -> Defines:
```

همان `default_choices`، به شکل `Defines` و **با عددها**: دکمه‌ی Restore the company defaults سطرهای System names و Parameter modes را با عددشان (`SYS_NAV = 16`) نشان می‌دهد.

خروجی: یک `Defines`.

`load_choices` تابع

```
def load_choices() -> Defines:
```

فهرست‌های این رایانه را به شکل `Defines` برمی‌گرداند. فایل رایانه برنده است؛ هر فهرستی که در آن نباشد یا خالی باشد، از الگو پر می‌شود. عددهای یک فهرست از **همان** فایلی می‌آیند که خود فهرست از آن آمده. فاصله‌های اول و آخر هر مقدار حذف و مقدارهای خالی کنار گذاشته می‌شوند.

`_take` تابع

```
def _take(defines: Defines, data: Dict[str, object], key: str, column: str,
          fallback: List[str]) -> None:
```

یک فهرست از داده‌ی خوانده‌شده - و اگر از `NUMBERED` است، عددهایش - را در `Defines` می‌گذارد. عددی که با دست در فایل نوشته شده نگه داشته می‌شود به شرط اینکه عدد صحیح باشد؛ بیرون بازه بودنش را اعتبارسنجی می‌گوید (`DEF006`، فصل ۱۲).

ورودی‌ها

`defines`	مقصد
`data`	محتوای یک فایل choices
`key`	کلید فهرست
`column`	ستون آن در `Defines`
`fallback`	اگر فهرست در داده نبود

`parse_numbered` تابع

```
def parse_numbered(lines: List[str]) -> Tuple[List[str], Dict[str, int]]:
```

سطرهای جعبه‌ی پنجره‌ی Drop-down lists را به نام‌ها و عددها تبدیل می‌کند: هر سطر `NAME = number`. سطر بی `=` (یا با `=` و بی عدد) نامی است که عدد سطر بالایی به‌اضافه‌ی یک را می‌گیرد، پس در عددها چیزی برایش نوشته نمی‌شود. عدد می‌تواند دهدهی یا هگز (`0x10`) باشد، با `U` پایانی یا بی آن، و باید در uint32_t جا شود.

ورودی‌ها

`lines`	سطرهای جعبه، همان‌طور که تایپ شده‌اند

خروجی: `(names, numbers)`: فهرست نام‌ها به ترتیب، و `{name: number}` فقط برای سطرهای عددی.

خطا: `ValueError` با نام سطر: «Line 2: 'abc' after SYS_NAV is not a whole number…»، «Line 2: 'SYS_NAV = =' has a number but no name…»، یا «Line 5: '1 does not fit…- = SYS_NAV».

مثال:

```
parse_numbered(["SYS_UNKNOWN = 0", "SYS_NAV = 0x10", "SYS_NEXT"])
# (['SYS_UNKNOWN', 'SYS_NAV', 'SYS_NEXT'], {'SYS_UNKNOWN': 0, 'SYS_NAV': 16})
```

`save_choices` تابع

```
def save_choices(defines: Defines) -> None:
```

فهرست‌ها را در `var/choices.json` می‌نویسد (UTF-8، پایان سطر LF)، و عددهایی که کسی داده زیر `numbers`. اگر پوشه فقط‌خواندنی باشد، بی‌صدا چیزی نمی‌نویسد.

`seed_from` تابع

```
def seed_from(defines: Optional[Defines]) -> None:
```

تحویل یک‌باره‌ی فهرست‌ها از فایل‌های قدیمی: ساب‌لیست‌هایی که با نسخه‌های قبلی ساخته شده‌اند بخش `<Defines>` خودشان را دارند. اگر این رایانه هنوز `var/choices.json` نداشته باشد، اولین ساب‌لیستی که `<Defines>` غیرخالی دارد، فهرست‌هایش را به این فایل می‌دهد. اگر فایل از قبل باشد، هیچ کاری نمی‌کند.

> **نکته:** به همین دلیل اگر `var/choices.json` را پاک کنید و پروژه‌ای قدیمی را باز کنید، فهرست‌های آن پروژه فهرست‌های رایانه می‌شوند، نه پیش‌فرض‌های شرکت.

`_read` تابع

```
def _read(path) -> Dict[str, object]:
```

یک فایل JSON را می‌خواند و اگر نباشد، خراب باشد یا شیء نباشد، دیکشنری خالی برمی‌گرداند.

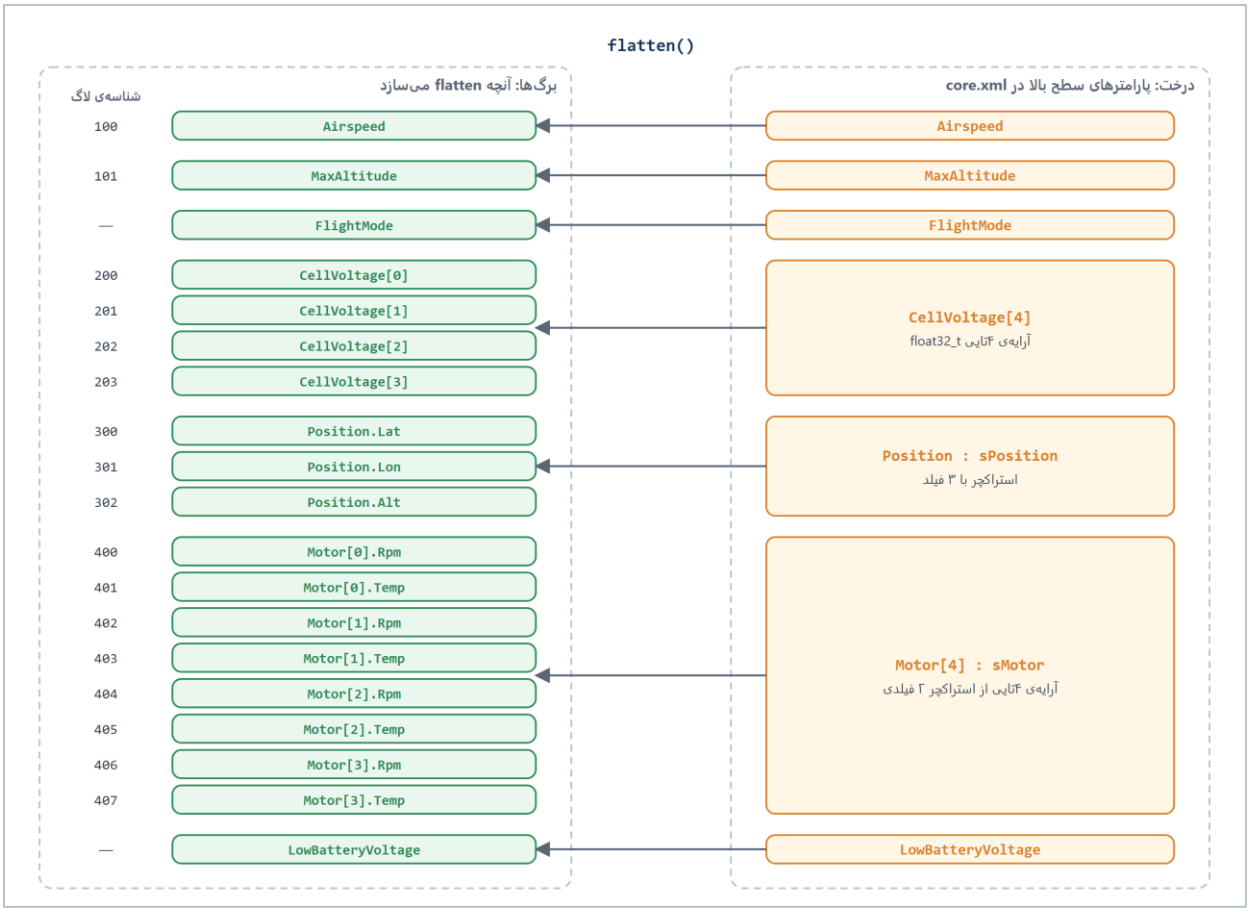
۱۰. مدل داده: پوشه‌ی model

مدل، شکل فهرست پارامترها در حافظه است: یک درخت از اشیای `Parameter` به‌علاوه‌ی لیست‌های کشویی و تایپ‌های استراکچر. مدل نه XML می‌شناسد، نه C و نه ویرایشگر؛ خواننده‌ی XML آن را می‌سازد، قاعده‌های بررسی آن را می‌خوانند، سازنده‌های فایل از روی آن کد می‌نویسند و ویرایشگر آن را تغییر می‌دهد. اگر بخواهید بفهمید «یک پارامتر چه ویژگی‌هایی دارد»، جواب در همین فصل است.

درخت و برگ‌ها

دو نما از یک فهرست وجود دارد و فرق آن‌ها کلید فهم کل ابزار است:

- **درخت**: همان چیزی که کاربر می‌سازد و در XML ذخیره می‌شود. هر پارامتر سطح بالا یک گره است؛ استراکچر فرزند دارد و آرایه فقط یک گره با `array_size > 1` است.
- **برگ‌ها** (`flatten()`): فهرست تخت همه‌ی مقدارها، به همان ترتیبی که در جدول C می‌آیند. هر عنصر آرایه و هر فیلد هر عنصر یک برگ جداست، با نام کامل مثل `Motor[1].Rpm`. سازنده‌های فایل فقط برگ‌ها را می‌بینند.



شکل ۳ — درخت پارامترهای پروژه‌ی نمونه و برگ‌هایی که flatten از آن می‌سازد

نمونه‌ی واقعی: ساب‌لیست `core.xml` پروژه‌ی نمونه هفت گره سطح بالا دارد، ولی `flatten()` از آن ۱۹ برگ می‌سازد:

array_declaration	شناسه‌ی لاگ	از کدام گره	برگ
`Airspeed`	`100`	`Airspeed`	`Airspeed`
`CellVoltage`	`200` … `203`	`CellVoltage` (آرایه‌ی ۴تایی)	`CellVoltage[0]` … `CellVoltage[3]`
`sPosition.Lat` و…	`300` … `302`	`Position` (تایپ `sPosition`)	`Position.Lat`، `Position.Lon`، `Position.Alt`
`sMotor.Rpm` و `sMotor.Temp`	`400` … `407`	`Motor` (آرایه‌ی ۴تایی از `sMotor`)	`Motor[0].Rpm` … `Motor[3].Temp`

۱۰.۱ `__init__.py` — درِ ورودی پکیج model

مسیر: `paramgen/src/pl_codegen/model/__init__.py`

این فایل نام‌های پرکاربرد زیرماژول‌ها را دوباره صادر می‌کند تا بتوان نوشت `from pl_codegen.model import Parameter` به‌جای `from pl_codegen.model.parameter import Parameter`. تابعی ندارد. نام‌هایی که صادر می‌کند: `Defines`، `type_enum_member`، `Parameter`، `ParameterKind`، `ParameterList`، `PATH_SEPARATOR`، `TAG_REGISTRY`، `TAGS_BY_ID`، `TagKind`، `TagSpec`، `VALUE_TYPES`، `CValueType` و `resolve_value_type`.

۱۰.۲ `types.py` — نوع‌های داده‌ی C

مسیر: `paramgen/src/pl_codegen/model/types.py`

هرچه ابزار درباره‌ی یک نوع داده‌ی C لازم دارد اینجاست: اندازه، علامت‌دار بودن، بازه، پسوند لیترال و نام تابع‌های smart data. افزودن یک نوع داده‌ی تازه یعنی افزودن یک مدخل به `VALUE_TYPES` (دستورالعمل در فصل ۲۴).

`CValueType` کلاس

```
@dataclass(frozen=True)
class CValueType:
```

توصیف ثابت (frozen) یک نوع داده. فیلدها:

- `c_name`: نام C، مثل `uint16_t`.
- `suffix`: پسوند smart data، مثل `U16`؛ از روی آن نام‌های `sSmartDataU16` و `fSmartData_U16_Ctor` ساخته می‌شود.
- `kind`: یکی از `"bool"`، `"int"` یا `"float"`.
- `signed` و `bits`: علامت‌دار بودن و تعداد بیت.

- `literal_suffix`: پسوندی که به عددهای این نوع در کد C اضافه می‌شود؛ مثل `U` برای `uint8_t` و `f` برای `float32_t`.
- `native_min` و `native_max`: کل بازه‌ی نوع.
- `limit_min_macro`، `limit_max_macro` و `limit_header`: ماکروهای استاندارد C برای دو سر بازه (مثل `INT16_MIN` و `INT16_MAX` از `stdint.h`). پارامتری که بازه‌ی خودش را ندارد، با این ماکروها تولید می‌شود، نه با عدد.
- `default_resolution`: رزولوشن وقتی کاربر خالی بگذارد؛ ۱ برای عدد صحیح، `0.001` برای `float32_t` و `0.000001` برای `float64_t`.

کار	نام
نام struct smart data: `sSmartData` + پسوند.	`CValueType.smart_data_struct`
نام سازنده: `fSmartData_<پسوند>_Ctor`.	`CValueType.constructor`
نام عضو enum نوع: `eSMART_DATA_TYPE_<پسوند>`.	`CValueType.enum_name`
اندازه‌ی یک مقدار به بایت: `bits // 8`؛ برای `bool_t` یک.	`CValueType.value_size`
آیا کمینه، بیشینه و رزولوشن دارد؛ برای `bool_t` نه.	`CValueType.has_range`
آیا اعشاری است.	`CValueType.is_float`
آیا `bool_t` است.	`CValueType.is_bool`
آیا هر دو ماکروی بازه تعریف شده‌اند.	`CValueType.has_limit_macros`

`_int` تابع

```
def _int(c_name: str, suffix: str, bits: int, signed: bool, literal_suffix: str) ->
CValueType:
```

سازنده‌ی کمکی برای نوع‌های صحیح. بازه را از تعداد بیت حساب می‌کند و نام ماکروها را از نام نوع می‌سازد: `uint16_t` ← `UINT16_MAX` و `0U`؛ `int16_t` ← `INT16_MIN` و `INT16_MAX` (`stdint.h`) برای نوع‌های بی‌علامت ماکروی MIN ندارد).

`VALUE_TYPES` و `TYPE_ALIASES`

`VALUE_TYPES` دیکشنری همه‌ی نوع‌های پشتیبانی‌شده است، از نام C به `CValueType`:

نوع	پسوند	بیت	پسوند لیترال	ماکروهای بازه	رزولوشن پیش‌فرض
`bool_t`	`B8`	۸	—	—	—
`uint8_t`	`U8`	۸	`U`	`UINT8_MAX` … `0U`	1
`int8_t`	`I8`	۸	—	`INT8_MAX` … `INT8_MIN`	1
`uint16_t`	`U16`	۱۶	`U`	`UINT16_MAX` … `0U`	1

نوع	پسوند	بیت	پسوند لیترال	ماکروهای بازه	رزولوشن پیش‌فرض
int16_t	I16	۱۶	—	INT16_MAX … INT16_MIN	1
uint32_t	U32	۳۲	UL	UINT32_MAX … 0UL	1
int32_t	I32	۳۲	L	INT32_MAX … INT32_MIN	1
uint64_t	U64	۶۴	ULL	UINT64_MAX … 0ULL	1
int64_t	I64	۶۴	LL	INT64_MAX … INT64_MIN	1
float32_t	F32	۳۲	f	FLT_MAX … -FLT_MAX	0.001
float64_t	F64	۶۴	—	DBL_MAX … -DBL_MAX	0.000001

> **نکته:** برای نوع اعشاری «کل بازه» از `-FLT_MAX` تا `FLT_MAX` است. `FLT_MIN` کوچک‌ترین عدد **مثبت** است، نه پایین‌ترین عدد؛ استفاده از آن اشتباهی رایج و بی‌صداست.

`TYPE_ALIASES` نام‌های دیگری را که در ورودی پذیرفته می‌شوند به نام اصلی نگاشت می‌کند: `bool`، `boolean` و `b8` ← `bool_t`؛ `float` و `float_t` ← `float32_t`؛ `double` ← `float64_t`؛ و شکل‌های کوتاه مثل `u16`، `s16`، `i16`، `f32`.

`resolve_value_type` تابع

```
def resolve_value_type(name: Optional[str]) -> Optional[CValueType]:
```

از یک نام، `CValueType` را پیدا می‌کند: اول در `VALUE_TYPES` با همان شکل، بعد با حروف کوچک در `TYPE_ALIASES`. اگر پیدا نشود (مثلاً نام یک تایپ استراکچر است)، `None` برمی‌گرداند. خیلی از جاهای کد با همین «`None` بودن» تشخیص می‌دهند که یک نام، نام تایپ استراکچر است.

مثال:

```
resolve_value_type("U16").c_name     # 'uint16_t'
resolve_value_type("sPosition")       # None
```

`supported_type_names` تابع

```
def supported_type_names() -> str:
```

نام همه‌ی نوع‌ها را مرتب و با ویرگول جدا برمی‌گرداند؛ برای پیام‌های خطا.

۱۰.۳ `tags.py` — فهرست تگ‌ها

مسیر: `paramgen/src/pl_codegen/model/tags.py`

تگ‌ها برچسب‌های دسته‌بندی هر پارامترند. همه‌ی تگ‌ها در یک جا، `TAG_REGISTRY`، تعریف شده‌اند و بقیه‌ی ابزار (خواننده‌ی XML، قاعده‌های بررسی، محاسبه‌ی آفست، جدول C، فرم ویرایشگر) از روی همین فهرست کار می‌کند.

افزودن یک تگ تازه سه گام دارد که docstring فایل هم می‌گوید: افزودن یک `TagSpec`، افزودن فیلد هم‌نام به `parameter_descriptor.h` با همان ترتیب، و یک برچسب برای ویرایشگر.

> **مهم:** شناسه‌ی لاگ عمداً در این فهرست **نیست**؛ چون تگ نیست. یک `uint16_t` ساده در توصیف‌گر است که ناحیه‌ی حافظه‌ی خودش را ندارد. در مدل، فیلد `log_id` روی `Parameter` است.

`TagKind` کلاس

```
class TagKind(Enum):
```

نوع مقدار یک تگ: `BOOLEAN` (TRUE/FALSE)، `ENUM` (یکی از مقدارهای یک فهرست کشویی) یا `FREE` (یک عدد یا یک نام C، بدون بررسی بیشتر).

`TagSpec` کلاس

```
@dataclass(frozen=True)
class TagSpec:
```

توصیف ثابت یک تگ: `tag_id` (شناسه‌ی داخلی که در XML هم می‌آید)، `label` (نامی که ویرایشگر نشان می‌دهد)، `descriptor_field` (نام فیلد در `sParameterDescriptor`)، `kind`، `mandatory`، `defines_column` (برای تگ‌های ENUM: نام فهرست کشویی) و `description`.

`TagSpec.is_boolean` ویژگی (property)

```
@property
def is_boolean(self) -> bool:
```

آیا تگ از نوع بله/خیر است.

محتوای `TAG_REGISTRY`، به همان ترتیب فیلدها در `sParameterDescriptor`:

tag_id	label	فیلد توصیف‌گر	نوع	الزامی	فهرست کشویی
`SystemName`	Tag System Name	`TagSystemName`	ENUM	بله	`SYSTEM NAMES`
`ResetProof`	Tag Reset Proof	`TagResetProof`	BOOLEAN	بله	—
`Loggable`	Tag Loggable	`TagLoggable`	BOOLEAN	بله	—
`LogEncryption`	Tag Log Encryption	`TagLogEncryption`	BOOLEAN	بله	—
`ParamMode`	Tag ParamMode	`TagParamMode`	ENUM	بله	`PARAM MODES`
`Tag4` … `Tag1`	Tag4 … Tag1	`Tag4` … `Tag1`	FREE	خیر	—

ثابت	مقدار و کاربرد
`TAGS_BY_ID`	دیکشنری `tag_id` ← `TagSpec`؛ برای پیدا کردن سریع یک تگ با شناسه.
`LOGGABLE_TAG`	`"Loggable"`؛ تگی که تعیین می‌کند `log_id` معنی دارد یا نه.
`LOG_ID_FIELD`	`"LogId"`؛ نام فیلد شناسه‌ی لاگ در توصیف‌گر (بدون پیشوند Tag).
`LOG_ID_LABEL`	`"LogID"`؛ نامی که ویرایشگر نشان می‌دهد.

نام	کار
`mandatory_tags`	فهرست تگ‌های الزامی (پنج تگ اول).
`optional_tags`	فهرست تگ‌های اختیاری (`Tag1` تا `Tag4`).

۱۰.۴ `defines.py` — لیست‌های کشویی در حافظه

مسیر: `paramgen/src/pl_codegen/model/defines.py`

شیء `Defines` فهرست‌های کشویی را در حافظه نگه می‌دارد. نام هر فهرست («ستون») از دوره‌ی Excel مانده است:

فهرست	مقدار	ثابت
Parameter types	`"TYPE"`	`COLUMN_TYPES`
Value types	`"VALUE TYPES"`	`COLUMN_VALUE_TYPES`
System names	`"SYSTEM NAMES"`	`COLUMN_SYSTEM_NAMES`
Parameter modes	`"PARAM MODES"`	`COLUMN_PARAM_MODES`
استفاده نمی‌شود؛ مانده از قالب قدیمی	`"BOOLEANS"`	`COLUMN_BOOLEANS`

`GENERATED_DEFINES` می‌گوید نام‌های کدام فهرست‌ها در کد تولیدی `#define` می‌شوند، هر کدام با یک عدد: `SYSTEM NAMES` و `PARAM MODES`. در smart data هر دو در `cg_parameter_list_defines.h` (فصل ۱۴)، در param_table فقط `SYSTEM NAMES`، در `nsrSystemNames.h`. (پیش‌تر دو `typedef enum` بودند که عدد را جایگاه در فهرست تعیین می‌کرد و با فرم‌وری که خودش `SYS_NAV` را تعریف کرده بود تداخل داشتند.) `DEFINE_NUMBER_MAX` (`0xFFFFFFFF`) بزرگ‌ترین عددی است که یکی از آن‌ها می‌تواند بگیرد، چون در تگی از نوع uint32_t نوشته می‌شود.

`Defines` کلاس

```
@dataclass
class Defines:
```

یک dataclass با دو فیلد: `values`، دیکشنری نام ستون ← فهرست مقدارها (نام ستون همیشه با حروف بزرگ)، و `numbers`، دیکشنری نام ستون ← `{نام: عدد}` برای عددهایی که **کسی داده** (فقط فهرست‌های `GENERATED_DEFINES`). نامی که عدد ندارد با `numbered` شماره می‌گیرد.

`Defines.numbered` متد

```
def numbered(self, column: str) -> List[Tuple[str, int]]:
```

هر نام فهرست با عددی که `#define` می‌شود، به ترتیب فهرست. نامی که عدد ندارد عدد نام بالایی به‌اضافه‌ی یک را می‌گیرد و اولی صفر را؛ یعنی فهرستی که هیچ‌کس شماره‌گذاری نکرده همان عددهایی را دارد که `enum` قدیمی می‌داد.

ورودی‌ها

`column`	نام ستون

خروجی: فهرستی از `(name, number)`.

مثال:

```
defines.values["SYSTEM NAMES"] = ["SYS_A", "SYS_NAV", "SYS_NEXT"]
defines.numbers["SYSTEM NAMES"] = {"SYS_NAV": 16}
defines.numbered("SYSTEM NAMES")
# [('SYS_A', 0), ('SYS_NAV', 16), ('SYS_NEXT', 17)]
```

کار	نام
یک مقدار را به یک ستون اضافه می‌کند، اگر از قبل نباشد.	`Defines.add`
فهرست یک ستون؛ اگر نباشد، فهرست خالی.	`Defines.get`
آیا مقداری در یک ستون هست.	`Defines.contains`
عددی که کسی به یک نام داده، یا `None` اگر عدد بعدی را می‌گیرد.	`Defines.number_of`
همه‌ی مقدارها و عددهای یک `Defines` دیگر را اضافه می‌کند (بدون تکرار؛ عدد اولی که آمده می‌ماند).	`Defines.merge`
ویژگی: همان `.get(COLUMN_TYPES)`.	`Defines.param_types`
ویژگی: همان `.get(COLUMN_VALUE_TYPES)`.	`Defines.types`
ویژگی: همان `.get(COLUMN_SYSTEM_NAMES)`.	`Defines.system_names`
ویژگی: همان `.get(COLUMN_PARAM_MODES)`.	`Defines.param_modes`

`type_enum_member` تابع

```
def type_enum_member(param_type: str) -> str:
```

نام عضو enum برای یک Parameter type: `MONITORING` ← `ePARAMETER_TYPE_MONITORING`. این نام‌ها در `parameter_descriptor.h` تعریف شده‌اند؛ برای همین هر Parameter type تازه باید آنجا هم اضافه شود.

۱۰.۵ `parameter.py` — یک گره از درخت

مسیر: `paramgen/src/pl_codegen/model/parameter.py`

مهم‌ترین کلاس مدل. هر پارامتر، هر فیلد استراکچر و هر برگ `flatten` یک شیء `Parameter` است. پارامتر یا **primitive** است (یک مقدار، یا آرایه‌ای از مقدارها) یا **struct** (فرزند دارد).

> **مهم:** تگ‌ها و Type مال گره سطح بالا هستند، نه فیلدهای داخل استراکچر. یک استراکچر یک چیز است که به یک زیرسیستم تعلق دارد؛ فیلدهایش همان دسته‌بندی را به ارث می‌برند. هر فیلد فقط چیزهای خودش را دارد: نام، نوع، اندازه، واحد، بازه و توضیح.

نام	تعریف	معنی
`TagValue`	`Union[bool, int, str, None]`	مقداری که یک تگ می‌تواند داشته باشد.
`NumericValue`	`Union[int, float, bool, None]`	مقدار عددی (پیش‌فرض، کمینه، بیشینه، رزولوشن).

`element_key` تابع

```
def element_key(indices: Sequence[int]) -> str:
```

کلید «مسیر عنصری» یک مقدار را می‌سازد: ایندکس‌های آرایه‌ها از بیرون به درون، با `.` جدا. فقط آرایه‌ها عددی اضافه می‌کنند؛ استراکچر تکی چیزی به مسیر نمی‌افزاید.

مثال:

```
element_key([3])      # '3'    -> Speed[3]
element_key([1])      # '1'    -> Wheels[1].Speed
element_key([1, 5])   # '1.5'  -> Wheels[1].Samples[5]
element_key([])       # ''     -> a value that is nobody's element
```

`nearest_text` تابع

```
def nearest_text(entries: Dict[str, str], own: Optional[str],
                 indices: Sequence[int]) -> Optional[str]:
```

متن نزدیک‌ترین ردیفی را برمی‌گرداند که چیزی گفته است؛ این همان قاعده‌ی «نام نمایشی و توضیح هر ردیف» است (بخش ۷.۴ راهنمای کاربر). برای ایندکس‌های `(1, 5)` اول کلید `"1.5"` (خود مقدار)، بعد `"1"` (ردیف بالاتر) و در آخر `own` (متن اعلان) را امتحان می‌کند. متنی که فقط فاصله باشد، چیزی نگفته است.

ورودی‌ها

`entries`	متن‌های ردیف‌های یک گره (کلید: مسیر عنصری)
`own`	متن خود اعلان
`indices`	مسیر عنصری مقدار

خروجی: متن، یا `None` اگر هیچ‌کدام چیزی نگفته باشند.

`ParameterKind` کلاس

```
class ParameterKind(Enum):
```

نوع گره: `PRIMITIVE` (`"primitive"`) یا `STRUCT` (`"struct"`). همین رشته‌ها در ویژگی `kind` در XML نوشته می‌شوند.

`Parameter` کلاس

```
@dataclass
class Parameter:
```

یک گره درخت. فیلدهای اصلی (همان ستون‌های فرم ویرایشگر):

فیلد	نوع و پیش‌فرض	معنی
`name`	`str`	نام؛ برای برگ‌ها نام کامل نقطه‌دار (`Position.Lat`).
`kind`	`ParameterKind`، PRIMITIVE	primitive یا struct.
`enable`	`bool`، `True`	آیا در کد ساخته می‌شود.
`user_name`	`str` یا `None`	نام نمایشی اعلان.
`param_type`	`str` یا `None`	Type (مثل `MONITORING`)؛ فقط برای گره سطح بالا.
`value_type_name`	`str` یا `None`	نام نوع داده؛ برای struct، نام تایپ استراکچر.
`array_size`	`int`، `1`	تعداد عنصرها.
`default_value`، `minimum`، `maximum`، `resolution`	`NumericValue`، `None`	مقدارهای بخش Value.
`unit`	`str` یا `None`	واحد.
`log_id`	`int` یا `None`	شناسه‌ی لاگ؛ برای استراکچر و آرایه، اولین شناسه‌ی بلوک.
`address`	`int`، `0`	ستون آدرس جدول پارامتر؛ همه‌ی ساب‌لیست‌ها نگهش می‌دارند.
`tags`	`Dict[str, TagValue]`	مقدار تگ‌ها با `tag_id`.
`description`	`str` یا `None`	توضیح اعلان.

فیلد	نوع و پیش‌فرض	معنی
`element_descriptions`	`Dict[str, str]`	توضیح هر ردیف، با کلید مسیر عنصری (`element_key`).
`element_user_names`	`Dict[str, str]`	نام نمایشی هر ردیف، با همان کلید.
`children`	`List[Parameter]`	فیلدهای استراکچر.
`source`	`SourceLocation`	جای گره؛ برای پیام‌های خطا.

چهار فیلد دیگر را فقط `flatten()` روی برگ‌ها پر می‌کند و روی گره‌های درخت معنایی ندارند:

فیلد	معنی	نمونه برای `Motor[1].Rpm`
`c_object`	عبارت C که به این مقدار می‌رسد (بدون پیشوند)	`Motor[1].Rpm`
`element_index`	اگر خود اعلان آرایه باشد، ایندکس این عنصر؛ وگرنه `None`	`None` (خود Rpm آرایه نیست)
`declared_array_size`	اندازه‌ی آرایه‌ی اعلانی که این برگ از آن آمده	`1`
`array_declaration`	نام اعلانی که ماکروی اندازه از آن ساخته می‌شود	`sMotor.Rpm`

نام	کار
`Parameter.is_struct`	ویژگی: آیا struct است.
`Parameter.is_primitive`	ویژگی: آیا primitive است.
`Parameter.value_type`	ویژگی: `CValueType` گره، یا `None` برای struct یا نوع ناشناخته.
`Parameter.is_array`	ویژگی: آیا `array_size > 1`.
`Parameter.tag`	مقدار یک تگ با شناسه‌اش؛ اگر نباشد `None`.
`Parameter.effective_tag`	مقدار تگ آن‌طور که کد تولیدی می‌بیند؛ امروز همان `tag`.
`Parameter.walk`	خود گره و همه‌ی گره‌های زیر آن را یکی‌یکی برمی‌گرداند (generator، بخش ۴.۶).
`Parameter.enabled_children`	فرزندهایی که `enable` آن‌ها روشن است.
`Parameter.__str__`	متن کوتاه برای اشکال‌زدایی، مثل `Parameter(Lat, float64_t, x1)`.

`Parameter.range_disabled` ویژگی (property)

```
@property
def range_disabled(self) -> bool:
```

آیا این مقدار «بازه ندارد»: کمینه و بیشینه هر دو صفر یا هر دو خالی‌اند. چنین مقداری در کد تولیدی کل بازه‌ی نوعش را می‌گیرد، نه `0, 0`؛ چون سازنده‌ی smart data مقدار پیش‌فرض بیرون از بازه را رد می‌کند.

`Parameter.effective_minimum` ویژگی (property)

```
@property
def effective_minimum(self) -> NumericValue:
```

کمینه‌ای که سازنده‌ی تولیدی واقعاً می‌گیرد: اگر بازه خاموش باشد کمینه‌ی خود نوع، اگر فقط یکی از دو خانه پر باشد خانه‌ی خالی صفر، وگرنه همان کمینه. برای `bool_t` و struct مقدار `None`.

`Parameter.effective_maximum` ویژگی (property)

```
@property
def effective_maximum(self) -> NumericValue:
```

همان، برای بیشینه.

`Parameter._effective_limit` متد

```
def _effective_limit(self, value: NumericValue, native: bool, low: bool) -> NumericValue:
```

پیاده‌سازی مشترک دو ویژگی بالا. `native=True` یعنی وقتی بازه خاموش است، حد خود نوع را بده؛ `low` تعیین می‌کند کمینه است یا بیشینه.

`Parameter.reserved_size` ویژگی (property)

```
@property
def reserved_size(self) -> int:
```

تعداد بایتی که مقدار در ناحیه‌ی هر تگ اشغال می‌کند: `value_size × array_size`. برای `CellVoltage` (چهار `float32_t`) برابر ۱۶. شناسه‌ی لاگ این را نمی‌شمارد؛ شناسه مقدارها را می‌شمارد، نه بایت‌ها.

`Parameter.effective_log_id` ویژگی (property)

```
@property
def effective_log_id(self) -> Optional[int]:
```

شناسه‌ی لاگ آن‌طور که کد تولیدی می‌بیند: فقط اگر تگ Loggable روشن باشد؛ وگرنه `None`. شناسه‌ای که روی مقدار غیرلاگ‌شونده مانده، نباید جایی بگیرد یا با چیزی تداخل کند. `LOGGABLE_TAG` را درون تابع import می‌کند تا import دوطرفه پیش نیاید.

`Parameter.has_active_tag` متد

```
def has_active_tag(self, tag_id: str) -> bool:
```

آیا مقدار این تگ را «دارد»؛ همان چیزی که تعیین می‌کند در ناحیه‌ی حافظه‌ی آن تگ جا بگیرد یا نه. تگ بله/خیر وقتی `True` باشد. هر تگ دیگر وقتی مقداری داشته باشد، **حتی صفر**؛ چون صفر یک مقدار enum درست است و اگر «بدون تگ» حساب شود، آفست همه‌ی مقدارهای بعدی جابه‌جا می‌شود.

`Parameter.copy_as_leaf` متد

```
def copy_as_leaf(self, dotted_name: str, c_object: str,
                 tags: Dict[str, TagValue]) -> "Parameter":
```

یک کپی تخت از گره برای فهرست برگ‌ها می‌سازد، با نام کامل، `c_object` و تگ‌های گره سطح بالا. از `dataclasses.replace` استفاده می‌کند که یک کپی با چند فیلد تغییریافته می‌سازد. `log_id` عمداً `None` می‌شود؛ چون شناسه‌ها بعداً یکی‌یکی پخش می‌شوند (`_distribute_log_ids`).

ورودی‌ها

`dotted_name`	نام کامل برگ
`c_object`	عبارت C
`tags`	تگ‌های گره سطح بالا

`Parameter.description_at` متد

```
def description_at(self, indices: Sequence[int]) -> Optional[str]:
```

توضیح یک مقدار: متن ردیف خودش اگر باشد، وگرنه توضیح اعلان. برخلاف `description_over`، ردیف‌های میانی (مثل `Wheels[1].Samples`) را نگاه نمی‌کند؛ فقط خود مقدار و اعلان.

`Parameter.user_name_at` متد

```
def user_name_at(self, indices: Sequence[int]) -> Optional[str]:
```

همان، برای نام نمایشی. اگر هیچ‌کدام چیزی نگویند `None`؛ استراکچری که گره در آن است ممکن است نام بدهد.

`Parameter.user_name_over` متد

```
def user_name_over(self, indices: Sequence[int]) -> Optional[str]:
```

نام نمایشی‌ای که **این گره** به مقدار یا ردیف `indices` می‌دهد: نزدیک‌ترین ردیف خودش (`nearest_text`) وگرنه اعلانش. `flatten` با همین متد قاعده‌ی «هر ردیف به زیرمجموعه‌ی خودش» را اجرا می‌کند.

`Parameter.description_over` متد

```
def description_over(self, indices: Sequence[int]) -> Optional[str]:
```

همان، برای توضیح.

`_is_zero_or_empty` تابع

```
def _is_zero_or_empty(value: NumericValue) -> bool:
```

برای یک حد بازه، خانه‌ی خالی و صفر یکسان‌اند: `None`، رشته‌ی خالی یا عدد صفر.

۱۰.۶ `parameter_list.py` — کل فهرست

مسیر: `paramgen/src/pl_codegen/model/parameter_list.py`

`ParameterList` کل یک ساب‌لیست (یا چند ساب‌لیست ادغام‌شده) است: پارامترهای سطح بالا، لیست‌های کشویی، تایپ‌های استراکچر و چند فهرست کمکی که ادغام پر می‌کند. مهم‌ترین متدش `flatten` است.

ثابت `PATH_SEPARATOR` برابر `"."` است؛ جداکننده‌ی نام استراکچر و فیلد در نام‌های کامل.

`ForeignType` کلاس

```
class ForeignType(NamedTuple):
```

یک تایپ استراکچر که یک ساب‌لیست بدون تعریف کردنش استفاده کرده است (خطای `TYP017`). `NamedTuple` است: یک تاپل که فیلدهایش نام دارند (بخش ۴.۵). فیلدها: `list_file`، `where` (استراکچر، یا `Type.field`)، `type_name` و `in_type` (تایپ خود ساب‌لیست که فیلد مال آن است).

`ParameterList` کلاس

```
@dataclass
class ParameterList:
```

فیلدها: `parameters` (گره‌های سطح بالا به ترتیب)، `defines`، `source_files`، `generator_version` و `generated_at` (از سرآیند XML)، `struct_types` (یک `StructTypeLibrary`)، و سه فیلدی که ادغام پر می‌کند: `type_clashes` (نام تایپ‌هایی که دو ساب‌لیست تعریف کرده‌اند)، `type_owners` (هر تایپ را کدام فایل‌ها تعریف کرده‌اند) و `foreign_types`.

کار	نام
پیمایش روی پارامترهای سطح بالا: `for p in model:`.	`ParameterList.__iter__`
تعداد پارامترهای سطح بالا.	`ParameterList.__len__`
یک پارامتر به انتها اضافه می‌کند.	`ParameterList.add`
همه‌ی گره‌های درخت، پدر پیش از فرزند.	`ParameterList.walk`
ویژگی: پارامترهای سطح بالای فعال.	`ParameterList.enabled`
ویژگی: پارامترهای سطح بالای غیرفعال.	`ParameterList.disabled`
دیکشنری نام ← پارامتر سطح بالا.	`ParameterList.by_name`
پسوند نوع‌هایی که برگ‌ها استفاده کرده‌اند، بدون تکرار؛ برای پروژه‌ی نمونه `.F32، U16، U8، F64، I16، I8`.	`ParameterList.used_data_type_suffixes`
نام تایپ ← استراکچرهای فعال آن تایپ، به شکل `(مسیر، گره)`.	`ParameterList.structures_by_type`

`ParameterList.extend` متد

```
def extend(self, other: "ParameterList") -> None:
```

یک ساب‌لیست دیگر را به انتهای این یکی اضافه می‌کند: پارامترها پشت سر هم، لیست‌های کشویی ادغام، نام فایل‌ها اضافه. اگر تایپ استراکچری در هر دو تعریف شده باشد، اولی نگه داشته و نامش در `type_clashes` ثبت می‌شود تا بررسی گزارشش کند. نام تکراری و شناسه‌ی لاگ تکراری را بررسی بعدی پیدا می‌کند، نه این متد.

`ParameterList.find` متد

```
def find(self, dotted_name: str) -> Optional[Parameter]:
```

یک گره را با مسیر نقطه‌دار پیدا می‌کند؛ مثلاً `find("Position.Lat")`. مسیر عنصری (`Motor[1]`) را نمی‌فهمد؛ فقط نام اعلان‌ها.

خروجی: `Parameter` یا `None`.

`ParameterList.assign_paths` متد

```
def assign_paths(self) -> None:
```

مسیر کامل هر گره را در `source.parameter` آن می‌نویسد تا هر پیام خطا بگوید درباره‌ی کدام پارامتر است. `flatten` اول همین را صدا می‌زند.

`ParameterList.flatten` متد

```
def flatten(self) -> List[Parameter]:
```

همه‌ی برگ‌های فعال را به ترتیب جدول C برمی‌گرداند. برای هر پارامتر سطح بالای فعال:

- `_collect` برگ‌های آن را با نام کامل، تگ‌ها و Type خود پارامتر سطح بالا می‌سازد؛ هر عنصر آرایه و هر فیلد هر عنصر یک برگ.
- `_distribute_log_ids` شناسه‌های لاگ را از `log_id` پارامتر سطح بالا، یکی برای هر مقدار، پخش می‌کند.

خروجی: فهرست برگ‌ها (`List[Parameter]`). هر بار صدا زدن، فهرست تازه‌ای می‌سازد؛ تغییر برگ‌ها روی درخت اثری ندارد.

مثال:

```
for leaf in model.flatten():
    print(leaf.name, leaf.log_id)
# Airspeed 100
# CellVoltage[0] 200
# ...
# Motor[3].Temp 407
```

`ParameterList.struct_parameters` متد

```
def struct_parameters(self) -> List[Tuple[str, Parameter]]:
```

همه‌ی استراکچرهای فعال به شکل (مسیر، گره)، عمیق‌ترین اول؛ چون در C استراکچر درونی باید پیش از بیرونی تعریف شود.

`ParameterList.array_nodes` متد

```
def array_nodes(self) -> List[Tuple[str, Parameter]]:
```

همه‌ی گره‌های آرایه‌ی فعال، از روی **درخت** نه برگ‌ها: `Wheels[0].Samples` و `Wheels[1].Samples` دو مقدار از یک اعلان‌اند و یک ماکرو بیشتر لازم ندارند.

`ParameterList.struct_types_in_use` متد

```
def struct_types_in_use(self) -> List[StructType]:
```

تایپ‌های استراکچری که استراکچرهای فعال استفاده می‌کنند، هرکدام یک بار و درونی‌ترین اول. از این فهرست یک `typedef` برای هر **تایپ** ساخته می‌شود، نه برای هر پارامتر. فیلدی که در تایپ خاموش است حساب نمی‌شود. تابع درونی `visit` نام تایپ را پیش از دیدن فیلدهایش در `seen` می‌گذارد تا تایپ خودارجاع حلقه‌ی بی‌پایان نسازد.

`ParameterList.array_declarations` متد

```
def array_declarations(self) -> List[Tuple[str, int, Optional[Parameter]]]:
```

همه‌ی **اعلان‌های** آرایه، یعنی چیزهایی که ماکروی `..._ARRAY_SIZE` از آن‌ها ساخته می‌شود، به شکل (نام اعلان، تعداد، گرهی که خطا روی آن گزارش شود). آرایه‌ی یک پارامتر سطح بالا مال خودش است (`CellVoltage`)، و آرایه‌ی یک فیلد مال **تایپ** آن (`sMotor.Samples`)؛ چون همه‌ی پارامترهای آن تایپ همان فیلد را دارند.

`merge_lists` تابع

```
def merge_lists(parts: Sequence[Tuple[object, ParameterList]]) -> ParameterList:
```

ساب‌لیست‌های یک ساخت را به ترتیب ساخت ادغام می‌کند. تولیدکننده و ویرایشگر هر دو از همین تابع ادغام می‌کنند تا درباره‌ی ترتیب ایندکس‌ها اختلافی پیش نیاید. پیش از ادغام، تا وقتی هر ساب‌لیست جداست، دو چیز ثبت می‌شود: مالک‌های هر تایپ (`type_owners`، برای `TYP016`) و تایپ‌های بیگانه (`foreign_types`، برای `TYP017`).

ورودی‌ها

`parts`	فهرست (آن ParameterList، فایل ساب‌لیست) به ترتیب `inputs.model_files`

خروجی: یک `ParameterList` ادغام‌شده. توجه: اولین ساب‌لیست خودش نتیجه می‌شود و بقیه به آن اضافه می‌شوند؛ پس شیء اول تغییر می‌کند.

`_undeclared_types` تابع

```
def _undeclared_types(part: ParameterList, list_file: str) -> List[ForeignType]:
```

تایپ‌هایی را پیدا می‌کند که یک ساب‌لیست بدون تعریفشان استفاده کرده است: استراکچری که تایپش در کتابخانه‌ی همان ساب‌لیست نیست، و فیلدی از یکی از تایپ‌های آن ساب‌لیست که نه نوع داده است و نه تایپ تعریف‌شده. استراکچری که تایپش گم است فقط یک بار گزارش می‌شود.

`_collect` تابع

```
def _collect(node: Parameter, path: List[str], tags: Dict[str, TagValue],
             param_type: Optional[str], leaves: List[Parameter],
             owner_type: Optional[str] = None,
             indices: Tuple[int, ...] = (),
             inherited_user_name: Optional[str] = None,
             inherited_description: Optional[str] = None) -> None:
```

قلب `flatten`: یک گره را بازگشتی (recursive) پیمایش و برگ‌هایش را به `leaves` اضافه می‌کند.

- گره غیرفعال هیچ برگی ندارد.
- **استراکچر**: برای هر عنصر (یا یک بار اگر آرایه نباشد)، فرزندها را با مسیر `Name[i]` صدا می‌زند. نام نمایشی و توضیحی که این ردیف به فرزندان می‌دهد (`description_over`/`user_name_over`) به‌عنوان `inherited_...` پایین می‌رود.
- **primitive**: برای هر عنصر یک برگ با `copy_as_leaf` می‌سازد؛ نام نمایشی و توضیح برگ، نزدیک‌ترین ردیف خودش وگرنه آنچه از بالا آمده است. `array_size` برگ همیشه ۱ است؛ اندازه‌ی واقعی در `declared_array_size` می‌ماند.

ورودی‌ها

`node`	گره
`path`	مسیر تا اینجا به شکل C، مثل `['Wheels[1]', 'Speed']`
`tags`	تگ‌های پارامتر سطح بالا
`param_type`	Type پارامتر سطح بالا
`leaves`	فهرست خروجی
`owner_type`	تایپ استراکچری که این گره را تعریف کرده
`indices`	مسیر عنصری تا اینجا
`inherited_user_name`	نام نمایشی که ردیف‌های بالاتر داده‌اند
`inherited_description`	توضیحی که ردیف‌های بالاتر داده‌اند

`_distribute_log_ids` تابع

```
def _distribute_log_ids(block: List[Parameter], root: Parameter) -> None:
```

شناسه‌های لاگ یک پارامتر سطح بالا را بین برگ‌هایش پخش می‌کند: `log_id` پارامتر، شناسه‌ی اولین برگ است و هر برگ بعدی یکی بیشتر. اگر پارامتر شناسه نداشته باشد (یا عدد صحیح نباشد)، کاری نمی‌کند. نوع داده در این شمارش نقشی ندارد.

```
Position          log_id = 300
  Lat     float64_t  -> 300
  Lon     float64_t  -> 301
  Alt     float32_t  -> 302
```

`_stamp` تابع

```
def _stamp(node: Parameter, path: List[str]) -> None:
```

پیاده‌سازی بازگشتی `assign_paths`.

`_collect_arrays` تابع

```
def _collect_arrays(node: Parameter, path: List[str],
                    found: List[Tuple[str, Parameter]]) -> None:
```

پیاده‌سازی بازگشتی `array_nodes`.

`_collect_structs` تابع

```
def _collect_structs(node: Parameter, path: List[str],
                     found: List[Tuple[str, Parameter]]) -> None:
```

پیاده‌سازی بازگشتی `struct_parameters`؛ فرزندها پیش از پدر اضافه می‌شوند.

۱۰.۷ `struct_type.py` — تایپ‌های استراکچر

مسیر: `paramgen/src/pl_codegen/model/struct_type.py`

تایپ استراکچر دقیقاً همان `struct` در C است: نام و فیلدها، و دیگر هیچ. کمینه و بیشینه، تگ‌ها، شناسه‌ی لاگ، واحد و توضیح مال هر **استفاده** از تایپ‌اند، نه خود تایپ. هر ساب‌لیست تایپ‌هایش را در فایل `<نام>_types.xml` کنار خودش دارد و هیچ ساب‌لیست دیگری نمی‌تواند از آن‌ها استفاده کند.

`StructField` کلاس

```
@dataclass
class StructField:
```

یک فیلد تایپ: `name`، `value_type_name` (یک نوع داده یا نام تایپ استراکچر دیگر)، `array_size` و `enable`. روشن یا خاموش بودن فیلد مال تایپ است، چون همه‌ی پارامترهای یک تایپ باید فیلدهای یکسان داشته باشند تا یک typedef مشترک داشته باشند.

`StructField.copy` متد

```
def copy(self) -> "StructField":
```

یک کپی مستقل از فیلد.

`StructType` کلاس

```
@dataclass
class StructType:
```

یک تایپ: `name` و فهرست `fields`.

کار	نام
کپی عمیق: فیلدها هم کپی می‌شوند.	`StructType.copy`
فیلدی با یک نام، یا `None`.	`StructType.field_named`
متن کوتاه برای اشکال‌زدایی.	`StructType.__str__`

`StructType.shape` ویژگی (property)

```
@property
def shape(self) -> tuple:
```

«شکل» تایپ به‌صورت یک تاپل: برای هر فیلد به ترتیب `(نام، نوع، اندازه، روشن)`. دو تایپ با شکل یکسان یکی حساب می‌شوند؛ تبدیل فایل‌های قدیمی (`struct_migration`) از این برای پیدا کردن تایپ تکراری استفاده می‌کند.

`StructTypeLibrary` کلاس

```
class StructTypeLibrary:
```

کتابخانه‌ی تایپ‌های یک مدل: یک دیکشنری داخلی نام ← تایپ. نام یک تایپ را مشخص می‌کند؛ پس دو بار تعریف `Position` دو تایپ نیست، یک تایپ است که دو بار تعریف شده.

کار	نام
یک کتابخانه، خالی یا از تایپ‌های داده‌شده.	`StructTypeLibrary.__init__`
تعداد تایپ‌ها.	`StructTypeLibrary.__len__`
پیمایش روی تایپ‌ها به ترتیب نام.	`StructTypeLibrary.__iter__`
`"sPosition" in library`.	`StructTypeLibrary.__contains__`
همه‌ی تایپ‌ها به ترتیب الفبایی نام.	`StructTypeLibrary.all`
نام همه‌ی تایپ‌ها به ترتیب الفبایی.	`StructTypeLibrary.names`
تایپی با یک نام، یا `None`؛ نام خالی هم `None`.	`StructTypeLibrary.get`
یک تایپ را اضافه یا جایگزین می‌کند.	`StructTypeLibrary.put`
یک تایپ را حذف می‌کند؛ آیا بود؟	`StructTypeLibrary.remove`

کار	نام
نام یک تایپ را عوض می‌کند. فیلدهای تایپ‌های دیگر که به آن ارجاع دارند را **تغییر نمی‌دهد**؛ آن کار را `uses_of` و فراخواننده انجام می‌دهند.	`StructTypeLibrary.rename`
کپی عمیق کل کتابخانه.	`StructTypeLibrary.copy`
نام تایپ‌هایی که فیلدی از نوع یک تایپ دارند؛ برای تغییر نام.	`StructTypeLibrary.uses_of`

`StructTypeLibrary.merge` متد

```
def merge(self, other: "StructTypeLibrary") -> List[str]:
```

تایپ‌های کتابخانه‌ی دیگری را اضافه می‌کند و نام‌هایی را که از قبل بوده‌اند برمی‌گرداند. نام تکراری همیشه تداخل است، حتی اگر دو تعریف یکسان باشند؛ تعریف اول نگه داشته و تداخل برای گزارش برگردانده می‌شود.

۱۰.۸ `struct_sync.py` — هماهنگ کردن استراکچرها با تایپشان

مسیر: `paramgen/src/pl_codegen/model/struct_sync.py`

وقتی تایپی تغییر می‌کند، هر استراکچر آن تایپ باید همان لحظه شکل تازه را بگیرد. این فایل همین کار را انجام می‌دهد: فیلدهای هر استراکچر را دقیقاً از روی تایپ می‌چیند و مقدارهای خود پارامتر را، با تطبیق **نام** فیلد، نگه می‌دارد.

مال پارامتر (حفظ می‌شود)	مال تایپ (هر بار از تایپ دوباره نوشته می‌شود)
کمینه، بیشینه، پیش‌فرض، رزولوشن، شناسه‌ی لاگ، واحد، توضیح، نام نمایشی و متن‌های هر ردیف	نام فیلد، نوع داده، اندازه‌ی آرایه، روشن بودن، primitive یا struct بودن

چون تطبیق با نام است، تغییر نام یک فیلد در تایپ تنها ویرایشی است که مقدارهای آن فیلد را از دست می‌دهد؛ دقیقاً مثل تغییر نام عضو یک struct در C.

`apply_types` تابع

```
def apply_types(parameters: List[Parameter], library: StructTypeLibrary) -> None:
```

همه‌ی استراکچرهای یک فهرست پارامتر را با تایپشان هماهنگ می‌کند. خواننده‌ی XML بعد از خواندن و ویرایشگر بعد از هر تغییر تایپ این را صدا می‌زنند.

ورودی‌ها

`parameters`	پارامترهای سطح بالا
`library`	تایپ‌های همان ساب‌لیست

`_reshape` تابع

```
def _reshape(node: Parameter, library: StructTypeLibrary, busy: Set[str]) -> None:
```

یک گره را بازگشتی هماهنگ می‌کند:

- primitive فرزند ندارد؛ اگر داشته باشد (پارامتری که از struct به مقدار برگشته)، پاک می‌شوند.
- اگر تایپ پیدا نشود، فیلدها دست‌نخورده می‌مانند (خطا را بررسی گزارش می‌کند)؛ دور ریختن محتوا به‌خاطر یک فایل گم‌شده، کار کاربر را از بین می‌برد.
- اگر تایپ در `busy` باشد، تایپ خودش را در بر دارد؛ همان‌جا می‌ایستد تا بازگشت بی‌پایان نشود.
- وگرنه برای هر فیلد تایپ، فرزند هم‌نام موجود را برمی‌دارد (یا تازه می‌سازد)، نیمه‌ی تایپ را رویش می‌نویسد و خودش را بازگشتی هماهنگ می‌کند. فرزندهایی که در تایپ نیستند کنار می‌روند.

ورودی‌ها

`busy`	مجموعه‌ی تایپ‌هایی که در مسیر فعلی بازگشت باز هستند

`_new_field` تابع

```
def _new_field(member: StructField) -> Parameter:
```

فیلدی که پارامتر هنوز نداشت: فقط با نام ساخته می‌شود. ویرایشگر بعداً مقدارهای شروع (صفر، بدون بازه) را به آن می‌دهد.

`_apply_field` تابع

```
def _apply_field(child: Parameter, member: StructField) -> None:
```

نیمه‌ی تایپ را روی فرزند می‌نویسد: نام، نوع، اندازه، روشن بودن و kind. `param_type` و `tags` فیلد پاک می‌شوند، چون فیلد تگ‌های استراکچر را می‌گیرد. اگر فیلد struct باشد، مقدارهای عددی و واحدش هم پاک می‌شوند.

`_names_a_struct` تابع

```
def _names_a_struct(member: StructField) -> bool:
```

آیا نوع فیلد، نام یک تایپ استراکچر است؛ یعنی نوع داده‌ی شناخته‌شده نیست.

`type_of` تابع

```
def type_of(node: Parameter, library: StructTypeLibrary) -> Optional[StructType]:
```

تایپی که یک گره struct نمونه‌ی آن است، یا `None`.

`types_used_by` تابع

```
def types_used_by(parameters: List[Parameter], library: StructTypeLibrary) ->
StructTypeLibrary:
```

تایپ‌هایی که پارامترهای داده‌شده واقعاً استفاده می‌کنند، به‌علاوه‌ی تایپ‌هایی که آن‌ها استفاده می‌کنند و الی آخر. فایل تایپ‌های ادغام‌شده‌ای که کنار فهرست کامل ساخت نوشته می‌شود، دقیقاً همین را دارد. استراکچرهای غیرفعال هم حساب می‌شوند، چون در فهرست کامل هستند.

۱۰.۹ `struct_migration.py` — تبدیل استراکچرهای قدیمی

مسیر: `paramgen/src/pl_codegen/model/struct_migration.py`

پیش از اینکه تایپ‌ها فایل خودشان را داشته باشند، یک استراکچر در XML فقط فیلدهایش بود و نام تایپ نداشت. این فایل‌ها هنوز باید باز شوند. هنگام خواندن:

- استراکچری که نام تایپ دارد دست نمی‌خورد.
- استراکچری که تایپ ندارد، از روی شکل فعلی‌اش یک تایپ می‌گیرد که نامش از نام خود پارامتر است. هیچ چیز دیگری از پارامتر تغییر نمی‌کند.
- اگر تایپی با همان شکل از قبل باشد، همان استفاده می‌شود؛ سه پارامتر `Position` با شکل یکسان یک تایپ می‌شوند که سه بار استفاده شده.

نتیجه با اولین ذخیره در فایل نوشته می‌شود؛ پروژه با یک بار باز و ذخیره شدن خودش را تبدیل می‌کند. فایلی که فقط خوانده شود، روی دیسک همان شکل قدیمی را نگه می‌دارد.

`adopt_types` تابع

```
def adopt_types(model: "ParameterList", legacy: List[Parameter],
                path: "Path", diagnostics: Optional[DiagnosticCollector] = None) -> None:
```

به هر استراکچر مدل یک تایپ نام‌دار می‌دهد. اول «الگوهای» قدیمی فایل (`legacy`) را مستقیم به تایپ تبدیل می‌کند، بعد هر پارامتر را با `_adopt` می‌گذراند.

ورودی‌ها

`model`	مدل خوانده‌شده
`legacy`	الگوهای قدیمی درون فایل، اگر بود
`path`	فایل، برای پیام
`diagnostics`	برای پیام اطلاعاتی `TYP001`

`_adopt` تابع

```
def _adopt(node: Parameter, model: "ParameterList", made: Dict[str, str],
           path: "Path", diagnostics: Optional[DiagnosticCollector]) -> None:
```

یک گره را بازگشتی تبدیل می‌کند؛ فرزندها اول، تا استراکچر تودرتو پیش از بیرونی تایپ بگیرد. برای هر تایپ تازه یک پیام اطلاعاتی `TYP001` ثبت می‌کند.

`_type_from_node` تابع

```
def _type_from_node(node: Parameter, name: str) -> StructType:
```

تایپ را از روی یک گره struct می‌سازد: فقط نام، نوع، اندازه و روشن بودن فیلدها؛ نه واحد و توضیح.

`_free_name` تابع

```
def _free_name(wanted: str, model: "ParameterList", node: Parameter) -> str:
```

نامی برای تایپ تازه پیدا می‌کند: نام خود پارامتر، اگر آزاد باشد یا تایپ هم‌نام همان شکل را داشته باشد؛ وگرنه `Position_2`، `Position_3` و الی آخر.

۱۱. فایل‌های XML: پوشه‌ی xml_model

فایل XML هر ساب‌لیست، **قرارداد** بین ویرایشگر و تولیدکننده است: ویرایشگر دقیقاً همین قالب را می‌نویسد و تولیدکننده چیزی جز آن نمی‌خواند. تولیدکننده هرگز نمی‌داند فایل را چه کسی نوشته است؛ ویرایشگر، یا کسی با یک ویرایشگر متن. این پوشه چهار فایل دارد: نام همه‌ی عنصرها و ویژگی‌ها (schema.py)، خواندن (xml_reader.py)، نوشتن (xml_writer.py) و فایل تایپ‌ها (types_xml.py). اصول نحو XML در فصل ۶ آمده است.

۱۱.۱ __init__.py — درِ ورودی پکیج xml_model

مسیر: paramgen/src/pl_codegen/xml_model/__init__.py

نام‌های پرکاربرد را صادر می‌کند: schema (خود ماژول)، read_xml، write_xml و model_to_element. تابعی ندارد.

۱۱.۲ schema.py — نام عنصرها و ویژگی‌ها

مسیر: paramgen/src/pl_codegen/xml_model/schema.py

هر نام عنصر و ویژگی XML فقط اینجا نوشته شده است و خواننده و نویسنده هر دو از همین ثابت‌ها استفاده می‌کنند؛ مثلاً sch.ATTR_USER_NAME به‌جای رشته‌ی "userName". پس تغییر نام یک ویژگی یک تغییر یک‌جایی است (دستورالعمل «تغییر نام یک ویژگی XML بدون خراب کردن فایل‌های قدیمی» در فصل ۲۴). SCHEMA_VERSION برابر "1.0" است؛ نسخه‌ی **ساختار** فایل، نه نسخه‌ی ابزار.

ساختار یک فایل ساب‌لیست

بخشی واقعی از `core.xml` پروژه‌ی نمونه، شامل یک استراکچر:

```
<?xml version='1.0' encoding='utf-8'?>
<ParameterListModel schemaVersion="1.0" generator="parameter_list_code_generator"
                    generatorVersion="01.01.01" generatedAt="2026-10-16T13:12:51+03:30">
  <SourceFiles />
  <Parameters>
    <Parameter name="Position" kind="struct" enable="true" userName="Position"
               type="MONITORING" valueType="sPosition" size="1" logId="300" address="0">
      <Source />
      <Limits />
      <Tags>
        <Tag id="SystemName" kind="enum" value="SYS_NAV" />
        <Tag id="ResetProof" kind="boolean" value="false" />
        <Tag id="Loggable" kind="boolean" value="true" />
        <Tag id="LogEncryption" kind="boolean" value="false" />
        <Tag id="ParamMode" kind="enum" value="PARAM_MODE_READ_ONLY" />
      </Tags>
      <Description>Where the aircraft is.</Description>
      <Fields>
        <Parameter name="Lat" kind="primitive" enable="true" userName="Latitude"
                   valueType="float64_t" size="1" unit="deg">
          <Source />
          <Limits default="0.0" minimum="-90.0" maximum="90.0" resolution="1e-07" />
          <Tags />
          <Description>Latitude, WGS-84.</Description>
        </Parameter>
        ...
      </Fields>
    </Parameter>
  </Parameters>
</ParameterListModel>
```

عنصرها

کجا و برای چه	عنصر	ثابت
ریشه‌ی فایل ساب‌لیست	`ParameterListModel`	`ROOT`
فقط در فهرست کامل ساخت: ساب‌لیست‌هایی که ادغام شده‌اند (ویژگی `path`)	`SourceFile` / `SourceFiles`	`SOURCE_FILE` / `SOURCE_FILES`
فهرست‌های کشویی؛ فقط در فایل‌های قدیمی. خوانده می‌شود، دیگر نوشته نمی‌شود	`Value` / `DefineGroup` / `Defines`	`DEFINE_VALUE` / `DEFINE_GROUP` / `DEFINES`
ظرف پارامترهای سطح بالا؛ حتی خالی باید باشد	`Parameters`	`PARAMETERS`
یک پارامتر یا یک فیلد	`Parameter`	`PARAMETER`
فیلدهای یک استراکچر، درون `Parameter`	`Fields`	`FIELDS`
الگوهای قدیمی؛ فقط خوانده و به تایپ تبدیل می‌شوند	`StructTemplates`	`STRUCT_TEMPLATES`

کجا و برای چه	عنصر	ثابت
منشأ پارامتر؛ معمولاً خالی	`Source`	`SOURCE`
پیش‌فرض، کمینه، بیشینه و رزولوشن	`Limits`	`LIMITS`
تگ‌ها	`Tag` / `Tags`	`TAG` / `TAGS`
توضیح اعلان (متن عنصر)	`Description`	`DESCRIPTION`
نام نمایشی و توضیح هر ردیف (بخش ۱۰.۵)	`Element` / `Elements`	`ELEMENT` / `ELEMENTS`
ریشه‌ی فایل تایپ‌ها	`ParameterListTypes`	`TYPES_ROOT`
یک تایپ و فیلدهایش در فایل تایپ‌ها	`Field` / `StructType`	`STRUCT_FIELD` / `STRUCT_TYPE`

ویژگی‌ها

معنی	روی	ویژگی	ثابت
نسخه‌ی ساختار؛ اگر بخش اصلی آن فرق کند، هشدار `XML001`	ریشه	`schemaVersion`	`ATTR_SCHEMA_VERSION`
ابزار و نسخه‌ای که فایل را نوشته؛ خوانده نمی‌شود	ریشه	`generator`، `generatorVersion`	`ATTR_GENERATOR`، `ATTR_GENERATOR_VERSION`
زمان نوشتن	ریشه	`generatedAt`	`ATTR_GENERATED_AT`
نام	Parameter، StructType، Field، DefineGroup	`name`	`ATTR_NAME`
`struct`/`primitive`؛ برای Tag نوع تگ	Parameter، Tag	`kind`	`ATTR_KIND`
`false`/`true`	Parameter، Field	`enable`	`ATTR_ENABLE`
نام نمایشی	Parameter، Element	`userName`	`ATTR_USER_NAME`
نام قدیمی `userName`؛ فقط خوانده می‌شود	Parameter	`gsName`	`ATTR_OLD_USER_NAME`
Parameter type	Parameter	`type`	`ATTR_TYPE`
نوع داده یا نام تایپ	Parameter، Field	`valueType`	`ATTR_DATA_TYPE`
اندازه‌ی آرایه	Parameter، Field	`size`	`ATTR_ARRAY_SIZE`
نام قدیمی `size`؛ فقط خوانده می‌شود	Parameter، Field	`arraySize`	`ATTR_OLD_ARRAY_SIZE`
واحد	Parameter	`unit`	`ATTR_UNIT`

معنی	روی	ویژگی	ثابت
شناسه‌ی لاگ؛ روی Parameter است نه در Tags، چون تگ نیست	Parameter	`logId`	`ATTR_LOG_ID`
ستون آدرس جدول پارامتر؛ همیشه نوشته می‌شود، صفر هم	Parameter سطح بالا	`address`	`ATTR_ADDRESS`
مقدارهای بازه	Limits	`default`، `minimum`، `maximum`، `resolution`	`ATTR_DEFAULT`، `ATTR_MINIMUM`، `ATTR_MAXIMUM`، `ATTR_RESOLUTION`
شناسه و مقدار	Tag، Value	`id`، `value`	`ATTR_ID`، `ATTR_VALUE`
مسیر عنصری، مثل `3` یا `1.5`	Element	`at`	`ATTR_AT`
منشأ؛ `sheet` و `row` از دوره‌ی Excel	Source	`file`، `sheet`، `row`	`ATTR_FILE`، `ATTR_SHEET`، `ATTR_ROW`
مسیر یک ساب‌لیست	SourceFile	`path`	`ATTR_PATH`
تعریف شده ولی امروز استفاده نمی‌شود؛ توضیح متن عنصر `Description` است	—	`description`	`ATTR_DESCRIPTION`

۱۱.۳ `xml_reader.py` — از XML به مدل

مسیر: `paramgen/src/pl_codegen/xml_model/xml_reader.py`

تنها راه تولیدکننده برای خواندن یک ساب‌لیست. خواندن با ماژول `xml.etree.ElementTree` پایتون انجام می‌شود (بخش ۴.۱۶): فایل به یک درخت از عنصرها تبدیل می‌شود و کد با `find` و `findall` و `get` در آن می‌گردد.

قاعده‌ی مهم این فایل: **هر چیز ناخوانا گزارش می‌شود، هرگز بی‌صدا دور ریخته نمی‌شود.** مثلاً اگر `maximum` عدد نباشد و بی‌صدا خالی فرض شود، بازه خاموش می‌شود و فرمور مقدارهایی را می‌پذیرد که فهرست ممنوع کرده بود؛ پس این فایل برایش خطای `XML012` ثبت می‌کند.

ثابت `_ELEMENT_PATH` عبارت منظم یک مسیر عنصری معتبر است: عددها با `.` جدا (`3`، `1.5`).

تابع `read_xml`

```
def read_xml(path: Path, diagnostics: DiagnosticCollector) -> ParameterList:
```

یک فایل ساب‌لیست را می‌خواند و `ParameterList` آن را می‌سازد. گام‌ها:

- اگر فایل نباشد، XML خراب باشد یا ریشه `ParameterListModel` نباشد، `CodeGeneratorError` (متن خطای XML شماره‌ی سطر و ستون را دارد).

- اگر نسخه‌ی اصلی ساختار فرق کند، هشدار `XML001`.
- `generatorVersion`، `generatedAt` و `SourceFiles` خوانده می‌شوند.
- `<Defines>` قدیمی خوانده و به `seed_from` داده می‌شود؛ بعد فهرست‌های رایانه با `load_choices` در `model.defines` گذاشته می‌شوند.
- تایپ‌ها از فایل `_types.xml` کنار ساب‌لیست خوانده می‌شوند.
- `<StructTemplates>` قدیمی، اگر باشد، خوانده می‌شود.
- اگر `<Parameters>` نباشد خطای `XML002`؛ وگرنه هر `<Parameter>` با `_read_parameter`.
- در پایان `adopt_types` (تبدیل استراکچرهای قدیمی) و `apply_types` (هماهنگی با تایپ).

ورودی‌ها

`path`	مسیر فایل
`diagnostics`	جمع‌کننده‌ی مشکل‌ها

خروجی: `ParameterList` ساب‌لیست.

مثال:

```
diagnostics = DiagnosticCollector()
model = read_xml(Path("parameter_lists/core.xml"), diagnostics)
[p.name for p in model.parameters]
# ['Airspeed', 'MaxAltitude', 'FlightMode', 'CellVoltage', 'Position', 'Motor', ...]
```

`_read_defines` تابع

```
def _read_defines(root: ET.Element) -> Defines:
```

عنصر قدیمی `<Defines>` را به یک `Defines` تبدیل می‌کند؛ اگر نباشد، خالی.

`_read_parameter` تابع

```
def _read_parameter(element: ET.Element, path: Path,
                    diagnostics: DiagnosticCollector,
                    prefix: str = "") -> Optional[Parameter]:
```

یک `<Parameter>` را می‌خواند (و فیلدهایش را بازگشتی). `prefix` مسیر پدر است تا هر پیام بگوید درباره‌ی کدام پارامتر است. کارهای مهم:

- نبود `name` خطای `XML010` و `None`.
- `kind` ناشناخته هشدار `XML016` و primitive فرض می‌شود.
- `userName` وگرنه نام قدیمی `gsName`؛ `size` وگرنه `arraySize`.
- `address` که عدد صحیح نیست هشدار `XML017` و صفر.
- عددهای `<Limits>` با `_to_number` بر اساس نوع داده. فقط `default` در `bool_t` بولی خوانده می‌شود.

- تگ ناشناخته هشدار `XML011`؛ بقیه با `convert_tag`.
- `<Element>`ها: مسیر نامعتبر هشدار `XML018`؛ متن و نام خالی نگه داشته نمی‌شوند.
- فیلدهای `<Fields>` بازگشتی خوانده و با `_drop_inherited` پاک‌سازی می‌شوند.

خروجی: `Parameter`، یا `None` اگر نام نداشت.

`_as_template` تابع

```
def _as_template(template: Parameter) -> Parameter:
```

یک الگوی قدیمی را به شکل تایپ درمی‌آورد: struct، روشن، بدون تگ، Type، اندازه و شناسه‌ی لاگ؛ تا یک فایل دست‌کاری‌شده نتواند این‌ها را به هر کپی بفرستد.

`_drop_inherited` تابع

```
def _drop_inherited(field: Parameter, structure: str,
                    diagnostics: DiagnosticCollector) -> None:
```

تگ‌ها، Type و شناسه‌ی لاگ را از یک فیلد پاک می‌کند، چون مال پارامتر سطح بالا هستند؛ برای هرکدام که بود هشدار می‌دهد: `XML013` (تگ‌ها)، `XML014` (Type) و `XML015` (شناسه‌ی لاگ).

`convert_tag` تابع

```
def convert_tag(kind: TagKind, raw: Optional[str]):
```

متن یک تگ را به شکل مدل درمی‌آورد: تگ بولی به `bool`، تگ آزاد اگر عدد صحیح باشد به `int` وگرنه همان متن، و تگ enum همان متن. عمومی است چون ویرایشگر هم تگی را که کاربر تایپ می‌کند با همین تبدیل می‌کند؛ تا ویرایشگر و ساخت یک برداشت داشته باشند.

مثال:

```
convert_tag(TagKind.BOOLEAN, "true")    # True
convert_tag(TagKind.FREE, "5")          # 5
convert_tag(TagKind.FREE, "APP_FLAG")   # 'APP_FLAG'
```

کار	نام
`true`، `1` یا `yes` (بدون توجه به بزرگی حروف) ← `True`؛ هر متن دیگر `False`؛ نبود ← پیش‌فرض.	`_to_bool`
عدد صحیح، یا پیش‌فرض اگر خوانده نشود.	`_to_int`
خطای `XML012` برای عددی که خوانده نمی‌شود، با نام برچسب و متن.	`_report_bad_number`

`_to_optional_int` تابع

```
def _to_optional_int(raw: Optional[str]) -> Optional[int]:
```

متن را به عدد صحیح تبدیل می‌کند یا `None`. `int(text, 0)` یعنی پیشوندهای `0x` (هگز) و `0b` (دودویی) هم پذیرفته‌اند. عدد اعشاری که مقدارش صحیح است (`5.0`) هم پذیرفته می‌شود.

`_to_number` تابع

```
def _to_number(raw: Optional[str], is_float: bool, is_bool: bool, *,
               label: str = "", parameter=None, type_name: Optional[str] = None,
               diagnostics: Optional[DiagnosticCollector] = None):
```

یک ویژگی عددی `<Limits>`: برای نوع اعشاری `float`، برای نوع صحیح `int`، و برای پیش‌فرض `bool_t` مقدار بولی. عدد غیرصحیح برای نوع صحیح (مثل `1.2`) **نگه داشته می‌شود**، نه دور ریخته؛ تا ویرایشگر همان را نشان دهد و قاعده‌ی `RNG017` درباره‌اش توضیح دهد. فقط متنی که اصلاً عدد نیست خطای `XML012` می‌گیرد.

۱۱.۴ `xml_writer.py` — از مدل به XML

مسیر: `paramgen/src/pl_codegen/xml_model/xml_writer.py`

عکس خواننده: مدل را به XML تبدیل می‌کند. ویرایشگر هنگام ذخیره و تولیدکننده هنگام نوشتن فهرست کامل ساخت (`full_parameter_list_file`) از آن استفاده می‌کنند. خروجی همیشه به یک شکل است؛ پس اگر فایلی را بخوانید و بدون تغییر بنویسید، همان فایل بیرون می‌آید (جز سرآیند).

`_text` تابع

```
def _text(value: object) -> str:
```

یک مقدار را به متن ویژگی تبدیل می‌کند: بولی به `false`/`true`، عدد اعشاری صحیح با `repr` (تا `5.0` بماند، نه `5`) و بقیه با `str`.

`_set` تابع

```
def _set(element: ET.Element, name: str, value: object) -> None:
```

یک ویژگی را می‌نویسد، مگر اینکه مقدار `None` باشد.

`model_to_element` تابع

```
def model_to_element(model: ParameterList, *, emit_timestamp: bool = True,
                     home: Optional[Path] = None,
                     project_root: Optional[Path] = None) -> ET.Element:
```

درخت XML یک مدل را می‌سازد (بدون نوشتن). `<Defines>` دیگر نوشته نمی‌شود، چون فهرست‌ها مال رایانه‌اند.

ورودی‌ها

`model`	مدل
`emit_timestamp`	آیا `generatedAt` نوشته شود
`home`	فایلی که درخت در آن نوشته می‌شود؛ برای نسبی کردن مسیرهای `<Source>`
`project_root`	فقط برای فهرست کامل ساخت: مسیرها نسبت به ریشه‌ی پروژه

خروجی: عنصر ریشه (`ET.Element`).

`_parameter_to_element` تابع

```
def _parameter_to_element(parameter: Parameter,
                          home: Optional[Path] = None,
                          top_level: bool = True,
                          project_root: Optional[Path] = None) -> ET.Element:
```

یک پارامتر (و فیلدهایش، بازگشتی) را به `<Parameter>` تبدیل می‌کند. `address` فقط روی سطح بالا و همیشه (حتی صفر) نوشته می‌شود. تگ‌ها به ترتیب `TAG_REGISTRY` و فقط اگر مقدار داشته باشند. `<Elements>` فقط وقتی نوشته می‌شود که ردیفی متن خودش را داشته باشد.

`_element_order` تابع

```
def _element_order(at: str) -> Tuple[int, ...]:
```

کلید مرتب‌سازی مسیرهای عنصری به‌صورت عدد: `1.10` بعد از `1.9` می‌آید، نه قبلش (برخلاف مرتب‌سازی متنی).

`_source_file` تابع

```
def _source_file(file: Optional[str], home: Optional[Path],
                 project_root: Optional[Path] = None) -> Optional[str]:
```

چه چیزی در `<Source file=...>` نوشته شود: هیچ، اگر پارامتر از همان فایلی آمده که نوشته می‌شود (حالت عادی)؛ وگرنه مسیر نسبی، یا مسیر کامل اگر روی درایو دیگری باشد. مسیر کامل یک رایانه در فایل، با جابه‌جایی پوشه‌ی پروژه غلط می‌شد.

`write_xml` تابع

```
def write_xml(model: ParameterList, path: Path, *, emit_timestamp: bool = True,
              project_root: Optional[Path] = None) -> Path:
```

مدل را در فایل می‌نویسد، **و** تایپ‌هایش را در فایل `_types.xml` کنارش؛ هر دو با هم، تا هیچ لحظه‌ای نباشد که فقط یکی روی دیسک باشد. اگر فایل تایپ‌ها با نام قدیمی (`_structures.xml`) باشد، پاک می‌شود.

ورودی‌ها

`model`	مدل
`path`	مسیر فایل
`emit_timestamp`	آیا زمان نوشته شود
`project_root`	مانند `model_to_element`

خروجی: مسیر فایل نوشته‌شده.

`_indent` تابع

```
def _indent(element: ET.Element, level: int = 0, indent: str = "  ") -> None:
```

به عنصرها تورفتگی دو فاصله‌ای می‌دهد تا فایل خوانا باشد. جایگزین `ET.indent` است که از پایتون ۳.۹ به بعد هست؛ از دوره‌ای مانده که ابزار روی ۳.۸ هم اجرا می‌شد.

۱۱.۵ `types_xml.py` — فایل تایپ‌های استراکچر

مسیر: `paramgen/src/pl_codegen/xml_model/types_xml.py`

کنار هر ساب‌لیست `foo.xml` فایل `foo_types.xml` است که تایپ‌های آن ساب‌لیست را دارد. قالبش عمداً کوچک است؛ چون تایپ چیزی جز نام و فیلد ندارد. نمونه‌ی واقعی `core_types.xml`:

```
<?xml version='1.0' encoding='utf-8'?>
<ParameterListTypes schemaVersion="1.0" generator="parameter_list_code_generator"
                    generatorVersion="01.01.01">
  <StructType name="sMotor">
    <Field name="Rpm" valueType="int16_t" />
    <Field name="Temp" valueType="int8_t" />
  </StructType>
  <StructType name="sPosition">
    <Field name="Lat" valueType="float64_t" />
    <Field name="Lon" valueType="float64_t" />
    <Field name="Alt" valueType="float32_t" />
  </StructType>
</ParameterListTypes>
```

`size` فقط وقتی نوشته می‌شود که با ۱ فرق کند و `enable` فقط وقتی `false` باشد.

ثابت	مقدار	معنی
`TYPES_SUFFIX`	`"_types.xml"`	پسوند فایل تایپ‌ها: `power.xml` ← `power_types.xml`
`LEGACY_TYPES_SUFFIX`	`"_structures.xml"`	نام قدیمی؛ خوانده می‌شود و با ذخیره‌ی بعدی پاک

`is_types_file` تابع

```
def is_types_file(path: Path) -> bool:
```

آیا یک مسیر، فایل تایپ‌هاست (نه ساب‌لیست). هر جایی که «فایل‌های XML این پوشه» را نشان می‌دهد باید این را بپرسد؛ وگرنه فایل تایپ‌ها هم به‌عنوان ساب‌لیست فهرست می‌شود.

کار	نام
نام فایل تایپ‌هایی که **نوشته** می‌شود: `<stem>_types.xml`.	`types_file_for`
همان فایل با نام قدیمی `<stem>_structures.xml`.	`legacy_types_file_for`
فایلی که باید **خوانده** شود: نام تازه، وگرنه اگر فقط نام قدیمی هست، آن.	`existing_types_file_for`
اندازه‌ی یک فیلد: `size`، وگرنه نام قدیمی `arraySize`.	`_size_of`
مثل خواننده‌ی اصلی، با این فرق که `no`/`0`/`false` صریحاً `False` و هر متن دیگر پیش‌فرض است.	`_to_bool`
عدد صحیح یا پیش‌فرض.	`_to_int`
تورفتگی، مثل `xml_writer._indent`.	`_indent`

`read_types` تابع

```
def read_types(path: Path, diagnostics: Optional[DiagnosticCollector] = None
               ) -> StructTypeLibrary:
```

یک فایل تایپ‌ها را می‌خواند. نبودن فایل خطا نیست (کتابخانه‌ی خالی)؛ ساب‌لیستی که استراکچر ندارد هیچ‌وقت این فایل را ندارد. XML خراب یا ریشه‌ی غلط `CodeGeneratorError` می‌دهد. تایپ بی‌نام `TYP010`، فیلد بی‌نام `TYP011` و تایپ تکراری در یک فایل `TYP012`.

خروجی: `StructTypeLibrary`.

`types_to_element` تابع

```
def types_to_element(library: StructTypeLibrary) -> ET.Element:
```

درخت XML یک کتابخانه، دقیقاً همان‌طور که نوشته می‌شود. جدا از `write_types` است تا ویرایشگر بتواند بپرسد «ذخیره چه می‌نوشت» بدون اینکه بنویسد؛ ویرایشگر با همین می‌فهمد ساب‌لیست تغییر ذخیره‌نشده دارد یا نه.

`write_types` تابع

```
def write_types(library: StructTypeLibrary, path: Path, *,
                emit_timestamp: bool = True) -> Optional[Path]:
```

کتابخانه را می‌نویسد. کتابخانه‌ی خالی فایل را **پاک** می‌کند، نه یک فایل خالی بنویسد؛ فایل کهنه‌ای پر از تایپ‌هایی که کسی استفاده نمی‌کند، همان چیزی است که بعداً جایی کپی می‌شود و تداخل نام می‌سازد.

خروجی: مسیر فایل، یا `None` اگر چیزی نوشته نشد.

۱۲. بررسی‌ها: پوشه‌ی validation

بعد از خواندن XML و پیش از نوشتن حتی یک خط C، فهرست بررسی می‌شود. هدف ساده است: **هرچه به کد C نامعتبر یا فرمور اشتباه می‌انجامد، باید اینجا با پیامی روشن گرفته شود**، نه در کامپایلر و نه روی سخت‌افزار. هر بررسی یک «قاعده» (rule) است: یک کلاس کوچک با یک کار. همین قاعده‌ها دو جا اجرا می‌شوند:

- در تولیدکننده (`pipeline.py`): اگر خطایی بماند، هیچ فایلی نوشته نمی‌شود.
- در ویرایشگر (`Document.validate` در `pl_gui/document.py`): نتیجه همان فهرست پنل **Problems** است.

چون هر دو یک کد را اجرا می‌کنند، ویرایشگر هرگز چیزی را «سالم» نشان نمی‌دهد که تولیدکننده رد کند.

سه شدت

اثر	در کد	شدت
ساخت متوقف می‌شود. برای چیزی که کد نادرست یا ناکامپایل می‌سازد.	`diagnostics.error(...)`	خطا
ساخت ادامه دارد؛ مگر اینکه `treat_warnings_as_errors` روشن باشد یا گزینه‌ی `-W` داده شود. برای چیزی که شاید عمدی باشد.	`diagnostics.warning(...)`	هشدار
فقط با `-v` در خروجی دیده می‌شود. برای توضیح، نه ایراد.	`diagnostics.info(...)`	اطلاع

قاعده‌ی انتخاب شدت در این پروژه: «هشدار برای چیزی است که نویسنده شاید منظورش همین بوده؛ کسی `1.2` نمی‌نویسد که `1` بگیرد». پس هرچه خروجی را بی‌صدا عوض می‌کند، خطاست.

یک پیام واقعی

در کپی پروژه‌ی نمونه، شناسه‌ی لاگ `BatteryVoltage` در `power.xml` را ۱۰۰ کردیم که مال `Airspeed` در `core.xml` است، و فقط بررسی را اجرا کردیم (`--check`):

```
> py bin\run_generate.py --check -q -c ...\Flight_Computer
ERROR   TAG012: Log id 100 is already used by 'Airspeed'.
            at parameter 'BatteryVoltage' ...\parameter_lists\power.xml
            hint: Every value needs an id of its own, so change one of the two.
                  'Use next free id' in the editor gives a free one.
                  'Airspeed' comes from ...\parameter_lists\core.xml.

1 error(s), 0 warning(s), 0 info message(s)

Code generation was aborted because the model contains errors.
```

هر پیام چهار بخش دارد که از `diagnostics.error` می‌آید: **کد** (ثابت و یکتا؛ آزمون‌ها و راهنمای کاربر با آن به پیام اشاره می‌کنند)، **متن** (چه شده)، **مکان** (`SourceLocation`: فایل و پارامتر؛ ویرایشگر با آن روی ردیف می‌پرد) و **راهنما** (`hint`: چه باید کرد). کلاس‌های این‌ها در فصل ۹ (`errors.py`) آمده‌اند. فهرست کامل کدها در پیوست ب است.

> **مهم:** متن پیام‌ها انگلیسی است و همین‌طور باید بماند؛ آزمون‌ها بخش‌هایی از متن را می‌جویند. کد یک پیام را هرگز عوض نکنید: کد شناسه است و ممکن است جایی (آزمون، راهنما، عادت کاربر) به آن تکیه کرده باشد.
> پیشوند `GRP` در کدهای Type به همین دلیل مانده، با اینکه نام قدیمی «Group» دیگر نیست.

۱۲.۱ `__init__.py` — درِ ورودی پکیج validation

مسیر: `paramgen/src/pl_codegen/validation/__init__.py`

سه نام را صادر می‌کند: `ALL_RULES`، `ValidationRule` و `Validator`. تابعی ندارد.

۱۲.۲ `__init__.py` — فهرست همه‌ی قاعده‌ها

مسیر: `paramgen/src/pl_codegen/validation/rules/__init__.py`

فقط یک چیز مهم دارد: لیست `ALL_RULES`، یعنی کلاس همه‌ی قاعده‌ها به ترتیب اجرا. **قاعده‌ای که در این لیست نباشد، هرگز اجرا نمی‌شود**؛ هر قاعده‌ی تازه باید هم import شود و هم به این لیست اضافه. ترتیب فقط ترتیب پیام‌ها در گزارش را عوض می‌کند، نه نتیجه را. پنج قاعده‌ی آخر فقط در پروژه‌ای با `output.format` برابر `param_table` کاری می‌کنند.

همین لیست را گزینه‌ی `--list-rules` چاپ می‌کند (شناسه و `description` هر قاعده):

```
> py bin\run_generate.py --list-rules
Validation rules:
  DEFINES                 Values that become C defines must be unique, valid identifiers
with numbers of their own.
  NAME                    Parameter and field names must be valid, non-reserved C
identifiers.
  NAME_UNIQUE             Names must be unique inside their parent (and at the top level).
  ...
  TAG_LOG_ID              A loggable value needs a log id, and no two values may share one.
  PARAM_TABLE_SHAPE       The parameter-table format has no structures and no arrays.
  ...
```

۱۲.۳ `validator.py` — اجرای همه‌ی قاعده‌ها

مسیر: `paramgen/src/pl_codegen/validation/validator.py`

`Validator` کلاس

```
class Validator:
```

همه‌ی قاعده‌های `ALL_RULES` را روی یک مدل اجرا می‌کند. هنگام ساخته شدن از هر کلاس قاعده یک شیء با `Config` پروژه می‌سازد؛ پس هر قاعده به تنظیمات پروژه (پیشوندها، قالب خروجی، `require_unit`) دسترسی دارد.

کار	نام
شیء هر قاعده را می‌سازد: `[rule(config) for rule in ALL_RULES]`.	`Validator.__init__`
کپی لیست شیءهای قاعده (property).	`Validator.rules`

`Validator.validate` متد

```
def validate(self, model: ParameterList, diagnostics: DiagnosticCollector) -> bool:
```

مسیر همه‌ی گره‌ها را تازه می‌کند (`assign_paths`)، هر قاعده را صدا می‌زند و دو بررسی کلی هم خودش دارد:

- `VAL002` (خطا): پروژه اصلاً ساب‌لیستی ندارد؛ نه «فهرست خالی است» بلکه «فهرستی نیست». راهنمایش کاربر را به ساختن ساب‌لیست می‌فرستد.
- `VAL001` (خطا): ساب‌لیست هست ولی هیچ پارامتر روشنی ندارد؛ نام فایل‌ها در پیام می‌آید.

ورودی‌ها

`model`	مدل ادغام‌شده‌ی همه‌ی ساب‌لیست‌ها
`diagnostics`	جمع‌کننده‌ای که پیام‌ها به آن اضافه می‌شوند

خروجی: `True` اگر تولید کد مجاز است: هیچ خطایی نیست، و اگر `treat_warnings_as_errors` روشن است، هیچ هشداری هم نیست.

مثال:

```
diagnostics = DiagnosticCollector()
ok = Validator(config).validate(model, diagnostics)
if not ok:
    print(diagnostics.to_list())   # هر پیام یک دیکشنری
```

۱۲.۴ `rule_base.py` — کلاس پایه‌ی قاعده‌ها و چند کمک‌تابع

مسیر: `paramgen/src/pl_codegen/validation/rules/rule_base.py`

`ValidationRule` کلاس

```
class ValidationRule(ABC):
```

کلاس پایه‌ی انتزاعی (abstract، بخش ۴.۹) همه‌ی قاعده‌ها. هر زیرکلاس باید متد `check` را بنویسد؛ پایتون نمی‌گذارد از کلاسی که `check` ندارد شیء ساخته شود. دو ویژگی کلاس (class attribute) دارد که هر زیرکلاس عوض می‌کند:

- `rule_id`: شناسه‌ی کوتاه و ثابت قاعده، مثل `TAG_LOG_ID`؛ در `--list-rules` دیده می‌شود.
- `description`: یک جمله درباره‌ی کار قاعده.

کار	نام
تنظیمات پروژه را در `self.config` نگه می‌دارد.	`ValidationRule.__init__`
متد انتزاعی: مدل را بررسی کن و پیام‌ها را به `diagnostics` بیفزا. چیزی برنمی‌گرداند.	`ValidationRule.check`
متن `rule_id: description`؛ فقط برای اشکال‌زدایی.	`ValidationRule.__str__`

بقیه‌ی فایل کمک‌تابع‌هایی است که «کدام گره‌ها را بررسی کنم» را جواب می‌دهند. انتخاب درست میان آن‌ها مهم‌ترین تصمیم هنگام نوشتن یک قاعده است:

برای چه قاعده‌ای	چه می‌دهد	تابع
نام‌ها (نام گره خاموش هم مهم است)	همه‌ی گره‌ها، خاموش‌ها هم	`model.walk()`
ویژگی استراکچرها و پارامترها	همه‌ی گره‌های روشن، استراکچرها هم	`enabled_nodes`
بازه، نوع داده، اندازه	گره‌های روشنِ غیراستراکچر (یک بار برای هر اعلان)	`enabled_leaves`
تگ‌ها و Type (به ارث رسیده از پدر)	برگ‌ها، ولی یکی برای هر **اعلان**	`declared_values`
چیزی که مال مقدار است: شناسه‌ی لاگ، آفست	همه‌ی برگ‌ها؛ یکی برای هر **مقدار**	`model.flatten()`

`declared_values` تابع

```
def declared_values(model: ParameterList) -> List[Parameter]:
```

مثل `flatten()`، ولی برای هر اعلان یک برگ: `CellVoltage[0]` تا `CellVoltage[3]` یک اعلان‌اند، چون یک ردیف در فهرست‌اند. کلید یکتایی، فایل منبع به‌علاوه‌ی نام بدون اندیس (`[\d+]` پاک می‌شود) است. چرا مهم است: تگِ خالی روی یک آرایه‌ی ۷۰۰۰۰تایی **یک** اشتباه است؛ با `flatten()` می‌شد ۳۵۰۰۰۰ پیام یکسان. تگ‌ها و Type به ارث رسیده نگه داشته می‌شوند، چون قاعده‌های تگ دقیقاً همین را باید ببینند.

مثال:

```
[v.name for v in declared_values(model)]
# ['Airspeed', 'MaxAltitude', 'FlightMode', 'CellVoltage[0]',
#  'Position.Lat', 'Position.Lon', 'Position.Alt',
#  'Motor[0].Rpm', 'Motor[0].Temp', 'LowBatteryVoltage']
```

از هر اعلان، اولین مقدارش نماینده است؛ برای همین `CellVoltage[0]` می‌آید و `CellVoltage[1]` نه.

کار	نام
همه‌ی گره‌های روشن به ترتیب درخت. فرزندان گره خاموش هم کنار می‌روند.	`enabled_nodes`
از `enabled_nodes` فقط آن‌هایی که primitive هستند.	`enabled_leaves`
از `enabled_nodes` فقط استراکچرها.	`enabled_structs`
مسیر نقطه‌دار هر گره روشن (`Position.Lat`)، به ترتیب درخت.	`dotted_names`
پیمایش بازگشتی برای `enabled_nodes`؛ در گره خاموش متوقف می‌شود.	`_walk_enabled`
پیمایش بازگشتی برای `dotted_names`.	`_walk_paths`

۱۲.۵ `defines_rules.py` — قاعده‌ی فهرست‌های کشویی

مسیر: `paramgen/src/pl_codegen/validation/rules/defines_rules.py`

`DefinesSheetRule` کلاس

```
class DefinesSheetRule(ValidationRule):
```

دو فهرست کشویی (`SYSTEM NAMES` و `PARAM MODES`، ثابت `GENERATED_DEFINES` در فصل ۱۰) مستقیم به `#define` در هدر تولیدی تبدیل می‌شوند، هر نام با یک عدد. پس هر مقدارشان باید شناسه‌ی معتبر C باشد؛ وگرنه مثلاً `#define GS / PC (1U)` نوشته می‌شد و کامپایل نمی‌شد، بی‌آنکه ابزار چیزی بگوید. یک مقدار در دو فهرست هم نمی‌تواند باشد. و دو نام یک فهرست نباید یک عدد داشته باشند، وگرنه فرمور آن‌ها را از هم تشخیص نمی‌دهد.

کی	شدت	کد
یکی از این فهرست‌ها خالی است؛ هیچ `#define` آن ساخته نمی‌شود.	خطا	`DEF001`
مقداری شناسه‌ی معتبر C نیست، یا کلمه‌ی رزروشده‌ی C/C++ است.	خطا	`DEF002`
یک مقدار در دو فهرست آمده؛ دو `#define` با هم برخورد می‌کنند.	خطا	`DEF004`
دو نام یک فهرست یک عدد دارند (مثلاً `SYS_NAV = 16` و `SYS_GPS = 16`).	خطا	`DEF005`
عدد یک نام بیرون بازه‌ی uint32_t است (۰ تا `0xFFFFFFFF`).	خطا	`DEF006`

`DEF005` و `DEF006` را `_check_numbers` می‌گوید، با عددهای `Defines.numbered`؛ پس نامی که عدد ندارد و عدد بعدی را می‌گیرد هم می‌تواند به عدد نام دیگری برسد. این دو فقط جایی بررسی می‌شوند که عددها واقعاً نوشته می‌شوند: در smart data هر دو فهرست، در param_table فقط `SYSTEM NAMES` (برای `nsrSystemNames.h`)؛ در `PARAM MODES` param_table هنوز با جایگاه شماره می‌گیرد.

۱۲.۶ `name_rules.py` — قاعده‌های نام‌ها و شناسه‌های ساخته‌شده از آن‌ها

مسیر: `paramgen/src/pl_codegen/validation/rules/name_rules.py`

ثابت `RESERVED_MACROS` نام ماکروهایی است که `smart_data.h` خودش تعریف می‌کند (`TRUE`، `FALSE`، `NULL`)؛ هیچ پارامتری نباید به همین نام برسد.

`NameSyntaxRule` کلاس

```
class NameSyntaxRule(ValidationRule):
```

هر نام (حتی گره خاموش) باید شناسه‌ی معتبر C باشد و کلمه‌ی رزروشده نباشد؛ چون همان نام شناسه و ماکرو می‌شود. توابع `is_valid_c_identifier` و `is_reserved_keyword` در `naming.py` (فصل ۹) هستند.

کی	شدت	کد
نام شناسه‌ی معتبر C نیست (حرف و رقم و _، بی‌رقم در ابتدا).	خطا	`NAME001`
نام کلمه‌ی رزروشده‌ی C یا C++ است (مثل `int` یا `class`).	خطا	`NAME002`

`NameUniquenessRule` کلاس

```
class NameUniquenessRule(ValidationRule):
```

نام‌ها میان خواهر و برادرها (فرزندان یک پدر، و سطح بالا) یکتا باشند. چون پروژه چند ساب‌لیست ادغام‌شده است، دو نام تکراری اغلب در دو فایل‌اند؛ برای هر نسخه یک پیام جدا می‌آید که فایل خودش و فایل(های) دیگر را نام می‌برد.

کی	شدت	کد
یک نام چند بار در یک سطح آمده؛ پیام هر نسخه فایل بقیه را می‌گوید.	خطا	`NAME003`

`IndexMacroCollisionRule` کلاس

```
class IndexMacroCollisionRule(ValidationRule):
```

هر شناسه‌ای که تولیدکننده می‌سازد در یک جدول جمع می‌شود، با دلیل ساخته شدنش؛ نامی که دو بار ادعا شود خطاست. دو مشکل را می‌گیرد که هر دو **کامپایل می‌شوند** ولی غلط‌اند:

- دو پارامتر به یک ماکرو می‌رسند: `TestVar` و `Test_Var` هر دو `TEST_VAR` می‌شوند و `#define` دوم بی‌صدا برنده است.
- یک پارامتر با ماکرویی که به دلیل دیگری ساخته می‌شود برخورد می‌کند: ماکروی تعداد (`PARAMETER_LIST_QTY`)، اندازه‌ی آرایه‌ی پارامتر دیگر، اندازه‌ی ناحیه‌ی یک تگ، نام typedef یک تایپ، یا `NULL`/`FALSE`/`TRUE`.

کی	شدت	کد
یک شناسه برای دو چیز ساخته می‌شود؛ پیام همه‌ی صاحبانش را می‌شمارد.	خطا	`NAME004`

ادعاکننده‌ها، به ترتیب جمع‌آوری: ماکروی تعداد؛ سه ماکروی رزرو؛ ماکروی اندازه‌ی ناحیه‌ی هر تگ؛ یک ماکروی اندازه برای هر **اعلان** آرایه (فیلد آرایه‌ای یک تایپ، در همه‌ی استراکچرهای آن تایپ یک ماکروست، نه چند تا)؛ یک typedef برای هر تایپ در حال استفاده؛ و برای هر برگ، ماکروی اندیس و نام شیء smart data.

`GeneratedIdentifierRule` کلاس

```
class GeneratedIdentifierRule(ValidationRule):
```

نامی که خودش شناسه‌ی معتبر C است، هنوز می‌تواند ماکروی نامعتبر بسازد: `to_macro_case` زیرخط‌های دو سر را حذف می‌کند، پس پارامتری به نام `_1` ماکروی `1` می‌سازد. این قاعده شناسه‌های **ساخته‌شده** را بررسی می‌کند: ماکروی اندازه‌ی آرایه (یک بار برای هر اعلان)، ماکروی اندیس و ریشه‌ی نام شیء smart data. برای هر اعلان فقط یک پیام، با فهرست همه‌ی شناسه‌هایی که خراب می‌شوند؛ و اگر خود نام را `NAME001` گرفته، چیزی تکرار نمی‌شود.

کی	شدت	کد
شناسه‌ای که از نام ساخته می‌شود شناسه‌ی معتبر C نیست.	خطا	`NAME005`

`_name_segments` تابع

```
def _name_segments(dotted_name: str) -> List[str]:
```

نام‌های تکی درون یک مسیر، بدون اندیس عنصر.

مثال:

```
_name_segments("Wheels[1].Speed")   # ['Wheels', 'Speed']
```

`_root_identifier` تابع

```
def _root_identifier(expression: str) -> str:
```

ریشه‌ی یک عبارت عضو تولیدی؛ تنها بخشی که کامپایلر به‌صورت شناسه‌ی **اعلان‌شده** می‌بیند.

مثال:

```
_root_identifier("Cg_SmartData_Position.Home.Lat")   # 'Cg_SmartData_Position'
_root_identifier("Cg_SmartData_Wheels[0].Speed")    # 'Cg_SmartData_Wheels'
```

`StructRule` کلاس

```
class StructRule(ValidationRule):
```

هر استراکچر روشن باید دست‌کم یک فیلد روشن و اندازه‌ی آرایه‌ی ۱ یا بیشتر داشته باشد. (کد `STR003` و قاعده‌ی قدیمی `StructFieldRule` با کدهای `STR005`/`STR004` حذف شده‌اند؛ توضیحشان در کامنت‌های فایل است.)

کی	شدت	کد
استراکچر هیچ فیلد روشنی ندارد.	خطا	STR001
اندازه‌ی آرایه‌ی استراکچر کمتر از ۱ است.	خطا	STR002

کار	نام
همه‌ی گره‌های زیر یک ریشه (خود ریشه نه). امروز کسی صدایش نمی‌زند.	_fields_of
فقط نام فایل ساب‌لیست یک گره، یا this sub list؛ برای پیام‌های NAME003.	_file_of
همه‌ی گره‌ها، خاموش‌ها هم: list(model.walk()).	_all_nodes

۱۲.۷ struct_type_rules.py — قاعده‌های تایپ‌های استراکچر

مسیر: paramgen/src/pl_codegen/validation/rules/struct_type_rules.py

پارامتر استراکچر نام یک تایپ را دارد، همان‌طور که در C یک متغیر از یک struct است. این قاعده‌ها می‌پرسند آن نام معنایی دارد، معنایش استراکچر است، و خود اعلان‌ها درست‌اند. دو قاعده‌ای که کاربر بیشتر می‌بیند، دو دنیا را جدا نگه می‌دارند: پارامتر ساده نمی‌تواند نام تایپ استراکچر بگیرد و استراکچر نمی‌تواند float32_t بگیرد.

StructTypeRule کلاس

```
class StructTypeRule(ValidationRule):
```

هر استراکچر تایپی دارد، آن تایپ وجود دارد و استراکچر است؛ و هیچ پارامتر ساده‌ای نام تایپ استراکچر را نگرفته است.

کی	شدت	کد
استراکچر تایپ ندارد.	خطا	TYP002
تایپ استراکچر در این پروژه نیست (شاید فایل _types.xml جا مانده).	خطا	TYP003
پارامتر ساده نام یک تایپ استراکچر را به‌عنوان نوع داده دارد.	خطا	TYP004
استراکچر نوع داده‌ی ساده (مثل float32_t) دارد، نه تایپ استراکچر.	خطا	TYP005

_TypeProblems کلاس

```
class _TypeProblems:
```

کمک‌کلاسی که ایراد یک **تایپ** را با شدت درستش گزارش می‌کند: اگر ساخت از آن تایپ typedef می‌سازد، خطا؛ اگر هیچ استراکچری از آن استفاده نمی‌کند، هشدار (چون چیزی از آن ساخته نمی‌شود). قبلاً همیشه خطا بود و به‌خاطر تایپی که کسی استفاده نمی‌کرد، ساخت را می‌بست. مکان پیام نام تایپ را دارد (struct_type) تا ویرایشگر پنجره‌ی **Structure types…** را روی آن باز کند.

کار	نام
کتابخانه، تایپ‌های در حال استفاده، استراکچرها و صاحب هر تایپ را از مدل برمی‌دارد.	`_TypeProblems.__init__`
یک پیام با شدت درست؛ مکان: فایلی که تایپ را اعلان کرده و (برای تایپ در حال استفاده) اولین استراکچری که آن را دارد.	`_TypeProblems.report`
اولین استراکچر روشنی که تایپ را در خود دارد، در هر عمقی.	`_TypeProblems._holder`

`StructTypeDeclarationRule` کلاس

```
class StructTypeDeclarationRule(ValidationRule):
```

خود اعلان‌ها: نام تایپ، فیلدها و نوع فیلدها. همه با `_TypeProblems` گزارش می‌شوند؛ پس برای تایپی که کسی استفاده نمی‌کند، هشدارند.

کی	شدت	کد
نام تایپ شناسه‌ی معتبر C نیست.	خطا/ه شدار	`TYP006`
تایپ هیچ فیلدی ندارد.	خطا/ه شدار	`TYP007`
دو فیلد یک تایپ هم‌نام‌اند.	خطا/ه شدار	`TYP008`
فیلدی نوع داده ندارد.	خطا/ه شدار	`TYP009`
نوع یک فیلد نه نوع داده است و نه تایپ استراکچر.	خطا/ه شدار	`TYP013`
اندازه‌ی آرایه‌ی یک فیلد کمتر از ۱ است.	خطا/ه شدار	`TYP014`

`StructTypeCycleRule` کلاس

```
class StructTypeCycleRule(ValidationRule):
```

تایپی که خودش را (مستقیم یا با چند واسطه) در خود دارد، اندازه ندارد و در C ممکن نیست.

کی	شدت	کد
تایپ خودش را در بر دارد؛ پیام زنجیره را نشان می‌دهد، مثل `sA -> sB -> sA`.	خطا/ه شدار	`TYP015`

`StructTypeClashRule` کلاس

```
class StructTypeClashRule(ValidationRule):
```

هر تایپ مال یک ساب‌لیست است. دو ساب‌لیست که یک نام را اعلان کنند خطاست، حتی اگر هر دو یک چیز بگویند؛ چون فایل تایپ ادغام‌شده و typedef فقط یکی را جا می‌دهند. فهرست برخوردها را ادغام (`model.type_clashes`، فصل ۱۰) جمع کرده است؛ پیام روی ساب‌لیست بعدی می‌آید.

کی	شدت	کد
یک نام تایپ در بیش از یک ساب‌لیست اعلان شده.	خطا	`TYP016`

`StructTypeOwnerRule` کلاس

```
class StructTypeOwnerRule(ValidationRule):
```

هر ساب‌لیست فقط تایپ‌هایی را استفاده می‌کند که خودش اعلان کرده. فهرست استفاده‌های بیگانه را ادغام (`model.foreign_types`) جمع کرده است. برای استراکچر همیشه خطا؛ برای فیلدی درون یک تایپ، با `_TypeProblems`.

کی	شدت	کد
استراکچر یا فیلدی تایپی دارد که ساب‌لیست خودش اعلان نکرده.	خطا/ه شدار	`TYP017`

کار	نام
فقط نام فایل یک ساب‌لیست، مثل `power.xml`؛ یا `?`.	`_list_name`
یک تایپ و همه‌ی تایپ‌هایی که در هر عمقی درونش هستند (پیمایش با صف).	`_reachable`
زنجیره‌ی نام‌هایی که به خود تایپ برمی‌گردد، یا لیست خالی (پیمایش عمقی بازگشتی).	`_cycle_from`

۱۲.۸ `type_rules.py` — قاعده‌های نوع داده، Type و اندازه‌ی آرایه

مسیر: `paramgen/src/pl_codegen/validation/rules/type_rules.py`

معنی	مقدار	ثابت
Typeهایی که `parameter_descriptor.h` برایشان enumerator دارد. افزودن Type تازه یعنی تغییر این‌جا **و** در هدر C (فصل ۲۴).	`("MONITORING", "SETTING", "COMMAND")`	`KNOWN_TYPES`
بیشترین اندازه‌ی آرایه؛ هر عنصر یک ردیف جدول است.	`65535`	`MAX_ARRAY_SIZE`

ValueTypeRule کلاس

```
class ValueTypeRule(ValidationRule):
```

هر برگ روشن نوع داده‌ای از نوع‌های پشتیبانی‌شده (VALUE_TYPES، فصل ۱۰) دارد.

کی	شدت	کد
نوع داده خالی است.	خطا	TYPE001
نوع داده پشتیبانی نمی‌شود.	خطا	TYPE002

DeclaredTypeRule کلاس

```
class DeclaredTypeRule(ValidationRule):
```

اگر فهرست کشویی TYPES پر باشد، نوع داده‌ی هر برگ باید در آن هم باشد. فقط هشدار.

کی	شدت	کد
نوع داده معتبر است ولی در فهرست TYPES نیست.	هشدار	TYPE003

ParameterTypeRule کلاس

```
class ParameterTypeRule(ValidationRule):
```

Type (MONITORING و...) هر اعلان. فیلد Type را از استراکچر بالایش به ارث می‌برد؛ برای همین روی declared_values کار می‌کند.

کی	شدت	کد
Type خالی است و هیچ استراکچری بالای آن هم Type ندارد.	خطا	GRP001
Type در KNOWN_TYPES نیست، پس enumerator ندارد و کد کامپایل نمی‌شود.	خطا	GRP002
Type معتبر است ولی در فهرست کشویی TYPE نیست.	هشدار	GRP003

ArraySizeRule کلاس

```
class ArraySizeRule(ValidationRule):
```

اندازه‌ی آرایه‌ی هر برگ روشن بین ۱ و MAX_ARRAY_SIZE.

کی	شدت	کد
اندازه کمتر از ۱.	خطا	ARR001
اندازه بیشتر از ۶۵۵۳۵.	خطا	ARR002

۱۲.۹ `range_rules.py` — قاعده‌های پیش‌فرض، کمینه، بیشینه و رزولوشن

مسیر: `paramgen/src/pl_codegen/validation/rules/range_rules.py`

پیش از این قاعده‌ها، سه ویژگی `Parameter` را به یاد بیاورید (فصل ۱۰): `range_disabled` (هر دو حد خالی یا هر دو صفر یعنی «بی‌بازه»؛ کل بازه‌ی نوع)، و `effective_maximum` / `effective_minimum` (حدی که سازنده‌ی smart data واقعاً می‌گیرد؛ خانه‌ی خالی کنار خانه‌ی پر، صفر حساب می‌شود). قاعده‌ها همیشه با مقدار **مؤثر** حساب می‌کنند، چون فرمور همان را می‌بیند.

`NativeRangeRule` کلاس

```
class NativeRangeRule(ValidationRule):
```

پیش‌فرض، کمینه، بیشینه و رزولوشن باید در بازه‌ی خود نوع C جا شوند (`native_min` تا `native_max`). نوع بولی بررسی نمی‌شود.

کی	شدت	کد
مقداری در نوع C جا نمی‌شود؛ مثلاً بیشینه‌ی ۳۰۰ برای `uint8_t`.	خطا	`RNG001`

`MinMaxOrderRule` کلاس

```
class MinMaxOrderRule(ValidationRule):
```

کمینه‌ی مؤثر از بیشینه‌ی مؤثر بزرگ‌تر نباشد؛ وگرنه سازنده هر مقداری را رد می‌کند.

کی	شدت	کد
کمینه بزرگ‌تر از بیشینه است (پیام می‌گوید اگر خانه‌ی خالی صفر حساب شده).	خطا	`RNG002`

`DefaultInRangeRule` کلاس

```
class DefaultInRangeRule(ValidationRule):
```

پیش‌فرض درون بازه‌ی مؤثر باشد. خانه‌ی خالی پیش‌فرض، صفر تولید می‌شود؛ پس پیش‌فرض خالی با بازه‌ی [۱۰، ۱۰۰] هم خطاست، چون `fParameterList_Init()` روی سخت‌افزار شکست می‌خورد.

کی	شدت	کد
پیش‌فرض بیرون بازه است.	خطا	`RNG003`

کلاس `IntegralValueRule`

```
class IntegralValueRule(ValidationRule):
```

نوع صحیح فقط عدد صحیح می‌گیرد. `1.2` در یک لیترال صحیح C همان `1` است؛ پس فرمور بی‌صدا عدد دیگری از آنچه روی صفحه است به کار می‌برد. ثابت کلاس `FIELDS` چهار خانه‌ی بررسی‌شده و نام نمایشی‌شان است.

کی	شدت	کد
یکی از چهار خانه برای نوع صحیح عدد غیرصحیح دارد.	خطا	`RNG017`

کلاس `RequiredValuesRule`

```
class RequiredValuesRule(ValidationRule):
```

برای نوع غیربولی، خانه‌های لازم و معقول بودن رزولوشن. برای نوع بولی متد `_check_boolean` اجرا می‌شود: `bool_t` کمینه و بیشینه و رزولوشن ندارد؛ نوشتن ۱/۱/۰ (ثابت `BOOLEAN_HARMLESS`) بی‌صدا پذیرفته است و هر چیز دیگر یک هشدار، با فهرست همه‌ی خانه‌ها.

کی	شدت	کد
در ردیف بولی، کمینه/بیشینه/رزولوشنی جز ۱/۱/۰ نوشته شده (نادیده گرفته می‌شود).	هشدار	`RNG010`
پیش‌فرض خالی است؛ صفر به کار می‌رود.	هشدار	`RNG011`
بیشینه هست ولی کمینه خالی؛ صفر به کار می‌رود.	هشدار	`RNG012`
کمینه هست ولی بیشینه خالی؛ صفر به کار می‌رود.	هشدار	`RNG013`
رزولوشن خالی است؛ پیش‌فرض نوع به کار می‌رود.	هشدار	`RNG014`
رزولوشن مثبت نیست.	هشدار	`RNG015`
رزولوشن از کل بازه بزرگ‌تر است؛ فقط یک مقدار ممکن است.	هشدار	`RNG016`

کلاس `MetadataRule`

```
class MetadataRule(ValidationRule):
```

فقط واحد، و فقط وقتی `validation.require_unit` در فایل پروژه روشن باشد. توضیح هرگز لازم نیست و درباره‌اش هیچ پیامی داده نمی‌شود.

کی	شدت	کد
واحد خالی است و پروژه آن را لازم دانسته (برای نوع غیربولی).	خطا	`META002`

۱۲.۱۰ `tag_rules.py` — قاعده‌های تگ‌ها، نام نمایشی و شناسه‌ی لاگ

مسیر: `paramgen/src/pl_codegen/validation/rules/tag_rules.py`

تولیدکننده معنای تگ‌ها را نمی‌داند (آن کار لایه‌ی کاربرد در فرم‌ور است)؛ فقط بررسی می‌کند مقدارها و فهرست‌های کشویی با هم جورند تا فایل C همیشه کامپایل شود.

چرا	مقدار	ثابت
هر تگ یک فیلد `uint32_t` در `sParameterDescriptor` است.	`0xFFFFFFFF`	`TAG_VALUE_MAX`
فیلد `LogId` در توصیف‌گر `uint16_t` است.	`0xFFFF`	`LOG_ID_MAX`

کار	نام
راهنمای پیام `TAG001`: چه چیزی در تگ‌های خالی بنویسیم؛ تگ‌های بولی با هم («TRUE یا FALSE») و برای enumها مقدارهای مجاز فهرست.	`_how_to_fill`
فهرست انگلیسی با «and»: `['a','b','c']` ← `'a', 'b' and 'c'`.	`_listed`

`MandatoryTagRule` کلاس

```
class MandatoryTagRule(ValidationRule):
```

تگ‌های اجباری (`mandatory=True` در `TAG_REGISTRY`) برای هر اعلان پر باشند. یک پیام برای هر پارامتر، با فهرست همه‌ی تگ‌های خالی؛ نه یک پیام برای هر تگ.

کی	شدت	کد
یک یا چند تگ اجباری خالی است.	خطا	`TAG001`

`TagEnumValueRule` کلاس

```
class TagEnumValueRule(ValidationRule):
```

مقدار تگ‌های enum (مثل `SystemName`) از فهرست کشویی خودش باشد و شناسه‌ی معتبر C؛ چون همان‌طور که هست در کد نوشته می‌شود. یک استثنا: مقداری که **قالب** برای همه‌ی پارامترها می‌نویسد (`PARAM_MODE_NONE` در smart data؛ `fixed_tags` فصل ۱۴) اگر در فهرست نباشد یک بار گفته می‌شود، روی اولین پارامتر و با جایی که باید اضافه شود («Parameter modes" add PARAM_MODE_NONE to (⚙ › ...Drop-down lists)»)، نه یک بار برای هر پارامتر.

کی	شدت	کد
مقدار شناسه‌ی معتبر C نیست.	خطا	`TAG002`
مقدار در فهرست کشویی آن تگ نیست.	خطا	`TAG003`

کلاس `TagValueRangeRule`

```
class TagValueRangeRule(ValidationRule):
```

مقدار عددی هر تگ در `uint32_t` جا شود.

کی	شدت	کد
مقدار منفی یا بزرگ‌تر از `0xFFFFFFFF`.	خطا	`TAG004`

کلاس `FreeTagSyntaxRule`

```
class FreeTagSyntaxRule(ValidationRule):
```

تگ‌های آزاد (`Tag1` تا `Tag4`) یا عددند یا شناسه‌ی C (نام یک ثابت)، چون بی‌تغییر در کد نوشته می‌شوند.

کی	شدت	کد
تگ آزاد نه عدد است و نه شناسه‌ی C.	خطا	`TAG005`

کلاس `UserNameRule`

```
class UserNameRule(ValidationRule):
```

پارامتر قابل لاگ باید نام نمایشی داشته باشد، چون لاگ مقدار را با همین نام می‌شناسد؛ «نام لاگ» جدایی وجود ندارد. داوری برای هر **مقدار** است (هر عنصر یا فیلد می‌تواند نام خودش را داشته باشد)، ولی گزارش برای هر اعلان یک بار. در پروژه‌ی `param_table` اجرا نمی‌شود، چون آن قالب چیزی لاگ نمی‌کند.

کی	شدت	کد
پارامتر قابل لاگ نام نمایشی ندارد.	خطا	`TAG011`

کلاس `LogIdRule`

```
class LogIdRule(ValidationRule):
```

شناسه‌ی لاگ مال **مقدار** است، پس روی `flatten()` کار می‌کند: آرایه‌ی ۱۰تایی با شناسه‌ی ۴۱۰، شناسه‌های ۴۱۰ تا ۴۱۹ را می‌گیرد، مستقل از نوع داده. فقط پارامتر قابل لاگ شناسه دارد؛ عدد روی پارامتر غیرقابل‌لاگ هیچ بازه‌ای نمی‌گیرد و صفر در جدول نوشته می‌شود. (این عمداً قاعده‌ی آفست حافظه نیست: آفست بر حسب بایت است و با `sizeof` رشد می‌کند.)

کی	شدت	کد
پارامتر قابل لاگ شناسه ندارد.	خطا	TAG010
دو مقدار یک شناسه دارند (نمونه‌ی واقعی بالای فصل).	خطا	TAG012
پارامتر غیرقابل‌لاگ شناسه دارد؛ نادیده گرفته می‌شود.	اطلاع	TAG014
شناسه بیرون ۰ تا ۶۵۵۳۵ است (اغلب عنصر آخر یک آرایه).	خطا	TAG015

۱۲.۱۱ `param_table_rules.py` — قاعده‌های قالب param_table

مسیر: `paramgen/src/pl_codegen/validation/rules/param_table_rules.py`

جدول فرم‌ور قدیمی فهرست تختی از مقدارهای تکی است: نه استراکچر دارد، نه آرایه، نه نوع بولی، و چیزی در آن لاگ یا رمز نمی‌شود. پس فهرستی که این‌ها را دارد در پروژه‌ی `smart_data` درست و در این پروژه غلط است. هر قاعده‌ی این فایل اول قالب پروژه را می‌پرسد و اگر `param_table` نباشد هیچ کاری نمی‌کند. همین، سازوکار «یک فهرست، دو مجموعه قاعده بسته به پروژه‌ای که بازش کرده» است.

`ParamTableRule` کلاس

```
class ParamTableRule(ValidationRule):
```

کلاس پایه‌ی این قاعده‌ها. property `wanted` فقط وقتی `True` است که `config.output.format == FORMAT_PARAM_TABLE`؛ هر `check` با `if not self.wanted: return` شروع می‌شود.

`ParamTableShapeRule` کلاس

```
class ParamTableShapeRule(ParamTableRule):
```

نه استراکچر، نه آرایه: هر ردیف یک مقدار.

کی	شدت	کد
پارامتر استراکچر است.	خطا	PTB001
پارامتر آرایه است.	خطا	PTB002

`ParamTableTypeRule` کلاس

```
class ParamTableTypeRule(ParamTableRule):
```

enum نوع‌های جدول، بولی ندارد؛ به‌جایش `uint8_t` با بازه‌ی ۰ تا ۱.

کی	شدت	کد
پارامتر `bool_t` است.	خطا	PTB006

ParamTableAddressRule کلاس

```
class ParamTableAddressRule(ParamTableRule):
```

ستون آدرس هر پارامتر سطح بالا عدد صحیح ۰ تا ADDRESS_MAX (۴۲۹۴۹۶۷۲۹۵، از param_table_h.py) باشد. صفر یعنی «فرم‌ور خودش جا می‌دهد».

کی	شدت	کد
آدرس عدد صحیح نیست یا بیرون بازه است.	خطا	PTB009

ParamTableSystemRule کلاس

```
class ParamTableSystemRule(ParamTableRule):
```

در این قالب، **نام فایل** می‌گوید ساب‌لیست مال کدام زیرسیستم است: <system>_paramTable.xml. تابع system_of (فصل ۱۴) آن را از نام فایل درمی‌آورد.

کی	شدت	کد
نام فایل زیرسیستمی از فهرست SYSTEM NAMES را نام نمی‌برد.	خطا	PTB005

ParamTableSystemTagRule کلاس

```
class ParamTableSystemTagRule(ParamTableRule):
```

هر پارامتر یک ساب‌لیست باید در تگ SystemName همان زیرسیستمِ نام فایل را داشته باشد. در این پروژه اثری ندارد (ستون SYS از نام فایل می‌آید)، ولی پروژه‌ی smart data که همان فایل را بخواند زیرسیستم را از تگ می‌گیرد. ویرایشگر هنگام باز کردن و ذخیره این تگ را خودش می‌نویسد؛ این قاعده ساب‌لیست‌هایی را پیدا می‌کند که هنوز آن‌طور ذخیره نشده‌اند.

کی	شدت	کد
پارامترهایی از ساب‌لیست، زیرسیستم فایل را در تگ ندارند.	هشدار	PTB008

۱۲.۱۲ افزودن یک قاعده‌ی تازه

فرض کنید شرکت بخواهد واحدها کوتاه بمانند، چون در صفحه‌ی ایستگاه زمینی جای کمی دارند: واحدی بلندتر از ۸ نویسه هشدار بگیرد. **این قاعده در پروژه نیست**؛ فقط نمونه است. سه گام:

گام ۱. کلاس را در فایل مناسب بنویسید؛ اینجا `range_rules.py` که `MetadataRule` (قاعده‌ی واحد) هم در آن است:

```
class UnitLengthRule(ValidationRule):
    """A unit longer than 8 characters does not fit the ground station."""

    rule_id = "UNIT_LENGTH"
    description = "Units must be at most 8 characters long."

    def check(self, model: ParameterList, diagnostics: DiagnosticCollector) -> None:
        for parameter in enabled_leaves(model):
            if parameter.unit and len(parameter.unit) > 8:
                diagnostics.warning(
                    "META003",
                    f"Unit '{parameter.unit}' is longer than 8 characters.",
                    parameter.source,
                    hint="Use a shorter unit, such as 'deg/s'.")
```

گام ۲. در `rules/__init__.py` آن را import کنید (در سطر `from .range_rules import ...`) و نامش را به `ALL_RULES` بیفزایید؛ مثلاً بعد از `MetadataRule`.

گام ۳. آزمونش را بنویسید (فصل ۲۳). برای قاعده‌ای که **خطا** می‌دهد، کافی است یک ردیف به لیست `CASES` در `tests/run_validation_test.py` اضافه کنید؛ آن آزمون مدل را در حافظه می‌سازد و کدهای خطای برگشتی را می‌شمارد:

```
{"name": "long_unit", "expect": "META003",
 "model": lambda: model(value("Speed", unit="metres/second"))},
```

ولی `run_case` در آن فایل فقط **خطاها** را جمع می‌کند؛ قاعده‌ی نمونه‌ی ما هشدار می‌دهد. برای هشدار، آزمونی مثل آزمون‌های `tests/run_editor_test.py` بنویسید که فهرست را در ویرایشگر باز می‌کند و با `document.validate_codes("META003")` می‌شمارد که پیام دقیقاً یک بار آمده است. اگر لازم است، کد تازه را به جدول پیام‌ها در راهنمای کاربر هم اضافه کنید.

نکته‌های انتخاب:

- کد تازه نباید با کد موجود یکی باشد؛ پیوست ب همه‌ی کدهای موجود را دارد.
- گره‌ها را با تابع درست انتخاب کنید (جدول `rule_base.py` بالاتر). اگر برای هر عنصر آرایه پیام بدهید، آرایه‌ی ۱۰۰۰تایی ۱۰۰۰ پیام یکسان می‌سازد.
- `parameter.source` را به‌عنوان مکان بدهید تا ویرایشگر با کلیک روی پیام به ردیف برود.
- اگر قاعده فقط برای یک قالب خروجی است، از `ParamTableRule` ارث ببرید یا اول قالب را بپرسید.

۱۳. چیدمان حافظه: پوشه‌ی layout

این پوشه به دو پرسش جواب می‌دهد: (۱) هر مقدار در ناحیه‌ی هر تگ از کدام بایت شروع می‌شود (`MemoryOffset`)، و (۲) آیا این ساخت چیزی را جابه‌جا کرده که فرم‌ورِ روی بردهای قبلی در حافظه دارد. پرسش دوم خطرناک‌ترین اشتباه ممکن با این ابزار را می‌گیرد؛ پس این فصل را حتی اگر قصد تغییرش را ندارید بخوانید.

۱۳.۱ `__init__.py` — درِ ورودی پکیج layout

مسیر: `paramgen/src/pl_codegen/layout/__init__.py`

نام‌های `tag_offset.py` را صادر می‌کند: `OffsetArea`، `OffsetEntry`، `OffsetMap`، `build_offset_map`، `log_id_span`، `reserved_log_ids` و `reserved_size`. ماژول `drift` جدا import می‌شود. تابعی ندارد.

۱۳.۲ `tag_offset.py` — آفست هر مقدار در ناحیه‌ی هر تگ

مسیر: `paramgen/src/pl_codegen/layout/tag_offset.py`

هر تگ (`SystemName`، `ResetProof`، `Loggable` و…) یک **ناحیه** دارد و هر ناحیه یک شمارنده‌ی مستقل. قاعده برای همه‌ی تگ‌ها یکی است:

- شمارنده از صفر شروع می‌شود.
- برای هر مقداری که تگ را «دارد»، آفستش مقدار فعلی شمارنده است و شمارنده به اندازه‌ی `sizeof(نوع) × اندازه‌ی آرایه` جلو می‌رود.
- مقداری که تگ را ندارد آفست صفر می‌گیرد و جایی اشغال نمی‌کند.

«داشتن تگ» را `Parameter.has_active_tag` (فصل ۱۰) تعیین می‌کند: تگ بولی وقتی `True` است؛ هر تگ دیگر وقتی مقداری نوشته شده باشد، **حتی صفر**. (صفر یک enumerator معتبر است؛ اگر «بی‌تگ» حساب می‌شد، مقدار از ناحیه بیرون می‌ماند و آفست همه‌ی بعدی‌ها جابه‌جا می‌شد.) اندازه‌ها از استاندارد C می‌آیند (`uint16_t` همه‌جا ۲ بایت است)؛ پس آفست‌ها روی هر کامپایلری یکسان‌اند.

نمونه‌ی واقعی

در `core.xml` پروژه‌ی نمونه، سه پارامتر تگ `ResetProof` را `TRUE` دارند. ناحیه‌ی آن تگ:

اندازه	آفست	نوع	مقدار
۲	۰	`uint16_t`	`MaxAltitude`
۱	۲	`uint8_t`	`FlightMode`
۴	۳	`float32_t`	`LowBatteryVoltage`
۷			**جمع ناحیه**

و ابتدای ناحیه‌ی `Loggable` (۱۷ مقدار، جمع ۵۴ بایت). `FlightMode` و `LowBatteryVoltage` لاگ نمی‌شوند، پس در این ناحیه نیستند:

اندازه	آفست	مقدار
۴	۰	`Airspeed`
۲	۴	`MaxAltitude`
۴ هرکدام	۶، ۱۰، ۱۴، ۱۸	`CellVoltage[0]` … `CellVoltage[3]`
۸، ۸، ۴	۲۲، ۳۰، ۳۸	`Position.Lat`، `Position.Lon`، `Position.Alt`
۲، ۱، …	۴۲، ۴۴، …	`Motor[0].Rpm`، `Motor[0].Temp`، …

> **مهم:** آفست **موقعیتی** است: جمع اندازه‌ی همه‌ی مقدارهای قبلی همان ناحیه. پس افزودن یک پارامتر در وسط، خاموش کردن یکی، جابه‌جایی، یا تغییر نوع یا اندازه‌ی آرایه‌ی یکی، آفست **همه‌ی مقدارهای بعدی** را عوض می‌کند. `drift.py` برای همین است.

`reserved_size` تابع

```
def reserved_size(parameter: Parameter) -> int:
```

چند بایت یک مقدار در ناحیه‌ی تگی که دارد اشغال می‌کند. خود عدد در مدل حساب می‌شود (`Parameter.reserved_size`، فصل ۱۰) چون فیلد استراکچر باید عضوهایش را جمع بزند. این تابع فقط همان را برمی‌گرداند.

مثال:

```
reserved_size(float64_t[4])   # 32
reserved_size(uint8_t[4])     # 4
```

`reserved_log_ids` تابع

```
def reserved_log_ids(parameter: Parameter) -> int:
```

چند شناسه‌ی لاگ پشت سر هم یک مقدار می‌گیرد. شناسه‌ی لاگ **مقدار** را می‌شمارد نه بایت را؛ پس برخلاف `reserved_size` به نوع داده بستگی ندارد: مقدار تکی همیشه ۱، و آرایه یکی برای هر عنصر. ویرایشگر برای «اولین شناسه‌ی آزاد» از همین استفاده می‌کند.

مثال:

```
reserved_log_ids(float64_t)       # 1
reserved_log_ids(float64_t[10])   # 10  (410 تا 419؛ آزاد بعدی 420)
```

`log_id_span` تابع

```
def log_id_span(parameter: Parameter) -> int:
```

چند شناسه‌ی لاگ یک **مدخل سطح بالا** می‌گیرد. برای مقدار ساده همان `reserved_log_ids`؛ برای استراکچر کل بلوک: `اندازه‌ی آرایه × جمع بلوک فرزندان`. گره خاموش صفر.

مثال:

```
log_id_span(Position)   # 3   (Lat، Lon، Alt)
log_id_span(Motor)      # 8   (4 × (Rpm + Temp))
```

سازنده‌ی `parameter_list_table_h.py` (فصل ۱۴) با این تابع بازه‌ی شناسه‌ی هر ردیف را در جدول کامنت‌ها می‌نویسد.

`OffsetEntry` کلاس

```
@dataclass
class OffsetEntry:
```

dataclass: آفست یک مقدار در ناحیه‌ی یک تگ. فیلدها: `tag_id`، `parameter` (برگ)، `offset` و `size` (بایت).

`OffsetArea` کلاس

```
@dataclass
class OffsetArea:
```

dataclass: همه‌ی آنچه برای یک تگ جا داده شده. فیلدها: `tag` (`TagSpec`)، `entries` (لیست `OffsetEntry` به ترتیب جدول؛ `field(default_factory=list)`، بخش ۴.۱۰) و `total` (کل بایت‌های ناحیه).

`OffsetMap` کلاس

```
class OffsetMap:
```

نتیجه‌ی نهایی: جست‌وجوی «(تگ، نام مقدار) ← آفست». هنگام ساخته شدن یک دیکشنری با کلید تاپل `(tag_id, parameter.name)` می‌سازد تا هر پرسش در زمان ثابت جواب بگیرد.

کار	نام
ناحیه‌ها را نگه می‌دارد و دیکشنری جست‌وجو را می‌سازد.	`OffsetMap.__init__`
دیکشنری `tag_id ← OffsetArea` (property).	`OffsetMap.areas`
ناحیه‌ی یک تگ، یا `None`.	`OffsetMap.area`
آیا آن مقدار در ناحیه‌ی آن تگ جا دارد.	`OffsetMap.has_entry`
آفست؛ صفر اگر مقدار تگ را ندارد.	`OffsetMap.offset`
اندازه؛ صفر اگر مقدار تگ را ندارد.	`OffsetMap.size`
کل بایت‌های ناحیه‌ی یک تگ؛ همان عددی که در ماکروی اندازه‌ی ناحیه نوشته می‌شود، مثل `PARAMETER_LIST_TAG_RESET_PROOF_SIZE`.	`OffsetMap.total`

`build_offset_map` تابع

```
def build_offset_map(model: ParameterList) -> OffsetMap:
```

برای هر تگ `TAG_REGISTRY` یک ناحیه می‌سازد: روی `model.flatten()` می‌چرخد و به هر برگی که تگ را دارد، آفست فعلی را می‌دهد و شمارنده را به اندازه‌ی `reserved_size` جلو می‌برد. خط‌لوله (`pipeline.py`) آن را یک بار صدا می‌زند و نتیجه را به همه‌ی سازنده‌های فایل می‌دهد.

مثال:

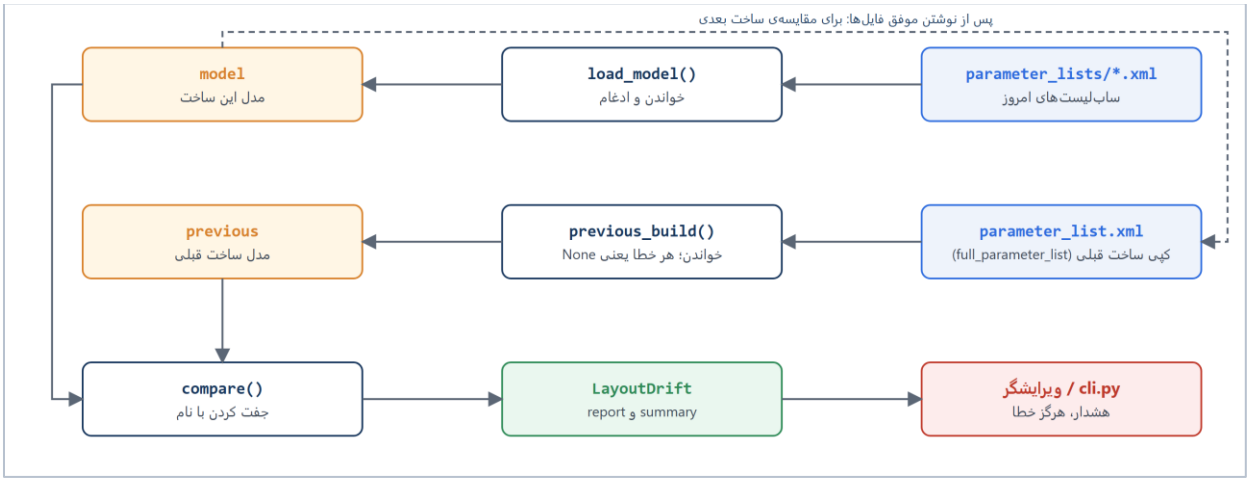
```
offsets = build_offset_map(model)
offsets.offset("ResetProof", "FlightMode")   # 2
offsets.total("ResetProof")                  # 7
offsets.has_entry("Loggable", "FlightMode")  # False
```

۱۳.۳ `drift.py` — آیا این ساخت چیزی را جابه‌جا کرده است؟

مسیر: `paramgen/src/pl_codegen/layout/drift.py`

خطر خاموش یک تولیدکننده‌ی فهرست پارامتر: یک پارامتر وسط فهرست اضافه می‌شود؛ کد کامپایل می‌شود، آزمون‌ها رد نمی‌شوند و فرم‌ور بالا می‌آید. ولی بردی که قبلاً در میدان بوده، مقداری را که در آن آفست ذخیره کرده بود می‌خواند، و آن آفست حالا مال پارامتر دیگری است. «حد سرعت» به‌صورت «شماره‌ی سریال» خوانده می‌شود. در diff کد منبع هم هیچ نشانی نیست.

راه‌حل: هر اجرای موفق، مدل ادغام‌شده را کنار کد تولیدی می‌نویسد (فایل `full_parameter_list_file` در تنظیمات، پیش‌فرض `full_parameter_list/parameter_list.xml`) و اجرای بعدی خودش را با آن مقایسه می‌کند. هر مقداری که بوده و آفستش عوض شده، با نام و آفست قدیم و جدید گزارش می‌شود. این همیشه **هشدار** است، هرگز خطا: جابه‌جایی کاملاً مجاز است (پروژه‌ای که هنوز تحویل نشده، تگی که کسی ذخیره نمی‌کند، چیدمان دوباره‌ی عمدی). کار ابزار این است که کسی **بی‌خبر** این کار را نکند.



شکل ۴ — مقایسه‌ی چیدمان: ساخت قبلی، ساخت فعلی و گزارش جابه‌جایی

نمونه‌ی واقعی

در پروژه‌ی نمونه، اندازه‌ی آرایه‌ی `CellVoltage` را از ۴ به ۶ بردیم. دو عنصر تازه بعد از `CellVoltage[3]` می‌آیند، پس همه‌ی مقدارهای بعدی ۸ بایت (دو `float32_t`) جلو می‌روند. خلاصه‌ی یک‌خطی (`summary`):

```
35 offset(s) MOVED, 12 index(es) changed, 2 added
```

و بخشی از گزارش کامل (`report`):

```
The stored position of these values CHANGED:
    Position.Lat  [Loggable]  22 -> 30
    Position.Lat  [ParamMode]  23 -> 31
    Position.Lat  [SystemName]  23 -> 31
    Position.Lon  [Loggable]  30 -> 38
    ...
    ... and 23 more

A board that already has data in flash will read the wrong
value at these offsets. Before flashing, decide what to do:
  * erase the stored area, or
  * migrate it, or
  * put the new parameters at the END instead of in the middle.

These values kept their offset but changed table index:
    Position.Lat  7 -> 9
    ...
Added in this build:
    CellVoltage[4]
    CellVoltage[5]
```

اگر همان دو عنصر تازه را در **پایان** فهرست (مثلاً به‌صورت پارامتر جدا) گذاشته بودیم، هیچ آفستی جابه‌جا نمی‌شد و گزارش فقط «Added» داشت.

کلاس `OffsetMove`

```
@dataclass
class OffsetMove:
```

dataclass: یک مقدار که آفستش در ناحیه‌ی یک تگ عوض شده. فیلدها: `parameter` (نام)، `tag_id`، `old` و `new`. متد `__str__` آن را به شکل یک سطر گزارش درمی‌آورد: `Position.Lat [Loggable] 22 -> 30`.

کلاس `LayoutDrift`

```
@dataclass
class LayoutDrift:
```

dataclass: همه‌ی تفاوت‌های ساخت قبلی و این ساخت. فیلدها:

- `added`: نام مقدارهایی که در ساخت قبلی نبودند.
- `removed`: مقدارهایی که بودند و نیستند (پاک، خاموش یا تغییرنام).
- `reindexed`: تاپل‌های `(نام، اندیس قدیم، اندیس جدید)`؛ جای مقدار در جدول عوض شده.
- `moved`: لیست `OffsetMove`.
- `resystemed`: فقط در `param_table`: `(نام، زیرسیستم قدیم، جدید)`.
- `first_run`: `True` اگر ساخت قبلی‌ای برای مقایسه نبود.
- `table`: `True` اگر پروژه `param_table` تولید می‌کند. در آن قالب آفست ذخیره‌شده‌ای نیست؛ آنچه بردها را خراب می‌کند **شماره‌ی پارامتر در زیرسیستمش** است.

کار	نام
هیچ تفاوتی نیست (property).	`LayoutDrift.is_empty`
آیا چیزی که برد میدانی به آن تکیه دارد جابه‌جا شده: در `moved`، در smart data `param_table` یا `reindexed` `resystemed` (property).	`LayoutDrift.breaks_stored_data`
یک سطر برای لاگ یا نوار وضعیت؛ مثل نمونه‌ی بالا، یا «the memory layout is unchanged».	`LayoutDrift.summary`
گزارش کامل به‌صورت لیست سطرها؛ از هر دسته حداکثر `limit` (۱۲) سطر و بعد «... and N more». در اجرای اول یا بی‌تفاوت، لیست خالی.	`LayoutDrift.report`

تابع `compare`

```
def compare(previous: Optional[ParameterList], current: ParameterList,
            table: bool = False) -> LayoutDrift:
```

دو مدل را مقایسه می‌کند. مقدارها **با نام** جفت می‌شوند، چون تنها هویتشان نام است؛ پس تغییر نام یک «removed» و یک «added» است. این صادقانه است: ابزار نمی‌تواند تغییر نام را از حذف و افزودن تشخیص دهد و در هر دو حالت خواننده باید نگاه کند. گام‌ها:

- اگر `previous` نباشد: `LayoutDrift(first_run=True)`.

- `added` و `removed` از مقایسه‌ی نام برگ‌ها.
- در smart data: اندیس تغییرکرده در `reindexed`؛ سپس برای هر تگ و هر نام مشترک، اگر در هر دو ساخت در ناحیه‌ی تگ بوده و آفستش فرق دارد، یک `OffsetMove`. (اگر تگ برای آن مقدار روشن یا خاموش شده، جابه‌جایی بی‌صدا نیست بلکه تغییر نیت است و گزارش نمی‌شود.)
- در `param_table`: شماره‌ی هر مقدار درون زیرسیستمش (`_numbered_per_system`) مقایسه می‌شود.
- در پایان همه‌چیز به ترتیب جدول جدید مرتب می‌شود.

ورودی‌ها

`previous`	مدل ساخت قبلی، یا `None`
`current`	مدل این ساخت
`table`	آیا پروژه `param_table` تولید می‌کند

خروجی: یک `LayoutDrift`.

`_numbered_per_system` تابع

```
def _numbered_per_system(model: ParameterList) -> Dict[str, Tuple[str, int]]:
```

زیرسیستم هر مقدار و شماره‌اش درون آن، دقیقاً با همان شماره‌گذاری جدول تولیدی: بر اساس فایلی که پارامتر از آن خوانده شده، از صفر در هر زیرسیستم. `system_of` را درون تابع import می‌کند (import محلی) تا وابستگی دوری بین `layout` و `codegen` پیش نیاید (بخش ۴.۱۳).

`previous_build` تابع

```
def previous_build(path: Path) -> Optional[ParameterList]:
```

مدلی که آخرین اجرای موفق نوشته، یا `None` در اجرای اول. **هر** خطایی در خواندن هم `None` حساب می‌شود (`except Exception`)؛ این یک بررسی کمکی است و کپی خراب یک مدل قدیمی هرگز نباید جلوی تولید کد را بگیرد.

چه کسی این‌ها را صدا می‌زند: `Pipeline.generate` (فصل ۱۵) پیش از نوشتن فایل‌ها `compare` را اجرا می‌کند و نتیجه را در `PipelineResult.drift` می‌گذارد؛ `cli.py` آن را با `_print_drift` چاپ می‌کند و ویرایشگر در پنجره‌ی پایان Generate نشان می‌دهد.

۱۴. تولید کد C: پوشه‌ی codegen

این پوشه مدل بررسی‌شده را به فایل‌های `.c` و `.h` تبدیل می‌کند. طراحی آن عمداً ساده است و در دو پوشه خلاصه می‌شود:

- `blocks/`: برای هر ساختار C یک کلاس کوچک (include، define، comment، enum، تابع و…) که فقط یک تکه متن برمی‌گرداند.
- `files/`: برای هر فایل تولیدی یک کلاس، که متد `generate()` آن فایل را **از بالا به پایین** با `text += ...` می‌نویسد.

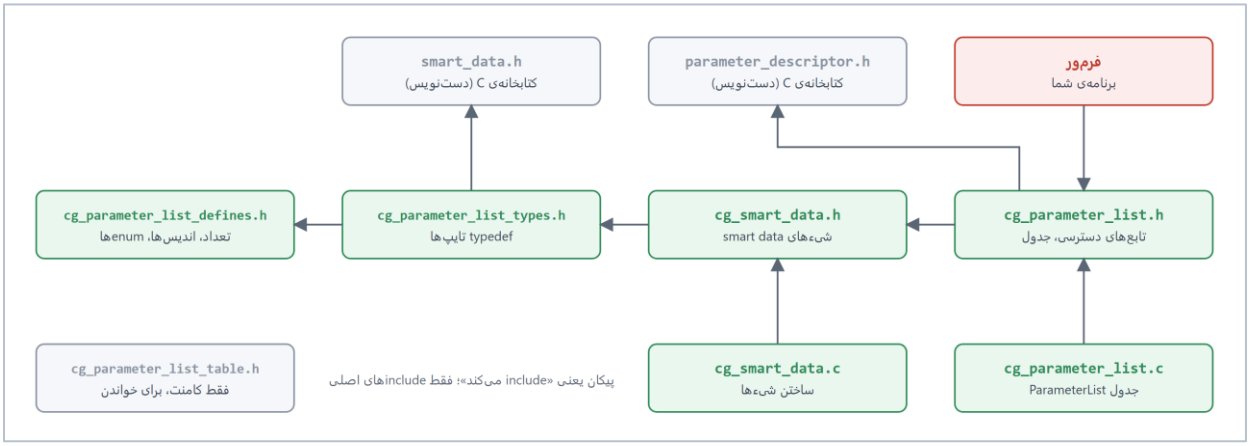
نتیجه‌ی این طراحی: اگر بخواهید بدانید یک فایل تولیدی چه شکلی است، یا یک سطرش را عوض کنید، فقط کافی است `generate()` همان فایل را از بالا بخوانید؛ ترتیب کد پایتون همان ترتیب فایل C است. همه‌ی نمونه‌های این فصل خروجی واقعی پروژه‌ی نمونه با نسخه‌ی ۰۱.۰۱.۰۱ ابزار است.

فایل‌هایی که ساخته می‌شوند

در قالب `smart_data` (پیش‌فرض) هفت فایل، با پیشوند `cg_` و نام پایه‌ی `parameter_list` (هر دو از تنظیمات `output.file_prefix` و `output.base_name`):

محتوا	سازنده	فایل
تعداد، اندیس هر مقدار، اندازه‌ی آرایه‌ها، اندازه‌ی ناحیه‌ی تگ‌ها، و نام‌های `#define` System names و Parameter modes (هر کدام با `#ifndef`)	`ParameterListDefinesHeader`	`cg_parameter_list_defines.h`
یک `typedef struct` برای هر تایپ استراکچر	`ParameterTypesHeader`	`cg_parameter_list_types.h`
اعلان `extern` شیء smart data هر پارامتر و `fCgSmartData_Init`	`SmartDataHeader`	`cg_smart_data.h`
تعریف شیءها و ساختنشان با سازنده‌های smart data	`SmartDataSource`	`cg_smart_data.c`
ماکروی weak، تابع‌های دسترسی و اعلان جدول توصیف‌گرها	`ParameterListHeader`	`cg_parameter_list.h`
جدول `ParameterList[]` و بدنه‌ی تابع‌های دسترسی	`ParameterListSource`	`cg_parameter_list.c`
فقط کامنت: فهرست به‌صورت جدول متنی، برای خواندن	`ParameterListTableHeader`	`cg_parameter_list_table.h`

در قالب `param_table` سه فایل: `nsrParamTables.h` (سطرهای `ADD_PARAM`)، `nsrParamOffsets.h` (فقط کامنت) و `nsrSystemNames.h` (نام سیستم‌ها با عددشان). در هر دو قالب، کپی مدل ادغام‌شده (`parameter_list.xml`) هم کنارشان نوشته می‌شود؛ ولی آن را خط‌لوله می‌نویسد، نه این پوشه (فصل ۱۵).



شکل ۵ — فایل‌های تولیدی قالب smart_data و اینکه هرکدام کدام را include می‌کند

۱۴.۱ `__init__.py` — درِ ورودی پکیج codegen

مسیر: `paramgen/src/pl_codegen/codegen/__init__.py`

`CStyle`، `FileBuilder` و `builders_for` را صادر می‌کند. تابعی ندارد. توضیح بالای فایل همان طراحی دو پوشه‌ای است که در ابتدای فصل آمد.

۱۴.۲ `c_style.py` — شکل فایل‌های شرکت، از روی c.json

مسیر: `paramgen/src/pl_codegen/codegen/c_style.py`

فایل‌های تولیدی باید دقیقاً شبیه فایلی باشند که برنامه‌نویس با تایپ `h-file` یا `c-file` در VS Code می‌گیرد. به‌جای نوشتن آن شکل در پایتون، این ماژول سه چیز را از فایل snippet شرکت (`paramgen/templates/c.json`، ثابت `SNIPPET_FILE` در `app_home.py`) بیرون می‌کشد:

- **بنر:** بلوک doxygen بالای هر فایل (`/** ... */` اول snippet «Create new header file»).
- **نشانگر بخش‌ها:** سطرهایی مثل `/* Includes ------- */`؛ از هر دو snippet.
- **پانویس:** سطر `/* === Copyright (c) 2026 FAS === EOF === */`.

اگر `c.json` نباشد یا خوانده نشود، نسخه‌ی داخلی همان شکل (`_FALLBACK_BANNER` و `_FALLBACK_FOOTER`) به کار می‌رود؛ پس تولیدکننده همیشه کار می‌کند. پس برای عوض کردن شکل بنر یا پانویس همه‌ی فایل‌ها، **`c.json` را عوض کنید، نه پایتون را.**

معنی	ثابت
نام دو snippet در c.json: Create new header file و Create new source file	_SNIPPET_HEADER، _SNIPPET_SOURCE
عبارت منظم سطر نشانگر: /* + نام + دست‌کم دو - + */؛ نام در گروه name	_MARKER
پهنای نشانگری که در c.json نیست: ۷۸	_DEFAULT_MARKER_WIDTH

CStyle کلاس

```
class CStyle:
```

شکل شرکت، آماده برای کلاس‌های بلوک. فیلدها: banner (سطرهای بنر با جای‌خالی‌ها)، markers (دیکشنری نام ← سطر نشانگر)، footer، company و indent (تورفتگی؛ پیش‌فرض چهار فاصله، از تنظیم style.indent).

CStyle.load متد کلاس

```
@classmethod
def load(cls, snippet_file: Optional[Path], company: str = "FAS",
         indent: str = "    ") -> "CStyle":
```

متد کلاس (@classmethod، بخش ۴.۹): یک CStyle از روی فایل snippet می‌سازد. فایل با load_jsonc خوانده می‌شود (JSON با کامنت، فصل ۶). اولین بلوک /** */ بنر می‌شود؛ هر سطری که با _MARKER جور باشد یک نشانگر (اولین نسخه‌ی هر نام می‌ماند)؛ و سطری که EOF === دارد پانویس.

مثال:

```
style = CStyle.load(SNIPPET_FILE, company="FAS", indent="    ")
style.marker("Includes")
# '/* Includes ------------------------------------------------------------- */'
```

CStyle.banner_lines متد

```
def banner_lines(self, file_name: str, brief: str) -> List[str]:
```

سطرهای بنر یک فایل، با پر کردن جای‌خالی‌های VS Code: $TM_FILENAME (نام فایل)، $TM_FILENAME_BASE (بدون پسوند) و $CURRENT_YEAR (سال جاری). سطر @brief با متن داده‌شده جایگزین می‌شود و اگر company چیزی جز FAS باشد، FAS با آن عوض می‌شود.

کار	نام
سطر نشانگر یک نام؛ اگر در c.json نبود، با خط تیره تا پهنای ۷۸ ساخته می‌شود.	CStyle.marker
سطر پانویس با سال جاری و نام شرکت.	CStyle.footer_line
سطرهای body یک snippet از دیکشنری c.json؛ اگر نبود لیست خالی.	_snippet_body
اولین بلوک /** ... */ یک snippet؛ اگر snippet با /** شروع نشود، لیست خالی.	_extract_banner

۱۴.۳ `descriptor_layout.py` — ترتیب فیلدهای sParameterDescriptor

مسیر: `paramgen/src/pl_codegen/codegen/descriptor_layout.py`

جدول تولیدی با **مقداردهی نام‌دار** (designated initializer) نوشته می‌شود: `.pName = "Airspeed",` .. این شکل در C99 و در C++20 هر دو معتبر است، **به شرط اینکه فیلدها به همان ترتیب اعلانشان در** `parameter_descriptor.h` **بیایند** (در C++20 ترتیب اجباری است). این فایل آن ترتیب را نگه می‌دارد:

توضیح	فیلد C	ثابت
نام پارامتر (برای `fParameterList_GetByName`)	`pName`	`FIELD_NAME`
نام نمایشی؛ برای هر مقدار جدا	`pUserName`	`FIELD_USER_NAME`
Type (`ePARAMETER_TYPE_MONITORING` و…)	`Type`	`FIELD_TYPE`
اشاره‌گر به شیء smart data	`pSmartData`	`FIELD_SMART_DATA`
شناسه‌ی لاگ؛ پیش از تگ‌ها چون تگ نیست	`LogId`	`FIELD_LOG_ID`
یک فیلد `uint32_t` برای هر تگ، به ترتیب `TAG_REGISTRY`	`TagSystemName` … `Tag4`	(از `TAG_REGISTRY`)
توضیح؛ آخرین فیلد	`pDescription`	`FIELD_DESCRIPTION`

`LEADING_FIELDS` پنج فیلد اول و `TRAILING_FIELDS` فیلد آخر است. عمداً `ArraySize` و `pUnit` ندارد: هر توصیف‌گر یک **مقدار** است (آرایه‌ی ۴تایی چهار ردیف است) و واحد روی خود شیء smart data است.

`descriptor_field_order` تابع

```
def descriptor_field_order() -> List[str]:
```

ترتیب کامل: فیلدهای اول + یک فیلد برای هر تگ + فیلد آخر.

مثال:

```
descriptor_field_order()
# ['pName', 'pUserName', 'Type', 'pSmartData', 'LogId', 'TagSystemName',
#  'TagResetProof', 'TagLoggable', 'TagLogEncryption', 'TagParamMode',
#  'Tag1', 'Tag2', 'Tag3', 'Tag4', 'pDescription']
```

> **هشدار:** اگر فیلدی به `sParameterDescriptor` در `parameter_descriptor.h` اضافه شود یا ترتیبش عوض شود، این فایل **و** `parameter_list_c.py` در `_table_entry` باید هم‌زمان عوض شوند. آزمون `run_compile_test.py` (فصل ۲۳) فایل `.c` را با `gcc -std=c99` می‌سازد و نام فیلد غلط را می‌گیرد؛ ولی C99 ترتیب غلط را می‌پذیرد و هیچ آزمونی ترتیب را جدا بررسی نمی‌کند. ترتیب را خودتان با هدر مقایسه کنید. دستورالعمل کامل در فصل ۲۴.

۱۴.۴ `literals.py` — نوشتن عددها و رشته‌ها به زبان C

مسیر: `paramgen/src/pl_codegen/codegen/literals.py`

درست نوشتن لیترال‌ها همان چیزی است که کد تولیدی را بی‌هشدار نگه می‌دارد؛ پس همه‌اش یک جاست. اگر روزی کامپایلری روی یک عدد تولیدی هشدار داد، اینجا را ببینید.

ثابت	مقدار	معنی
`FLOAT32_DIGITS`	۹	بیشترین رقم معنادار لازم برای بازگشت دقیق یک `float`
`FLOAT64_DIGITS`	۱۷	همان برای `double`

`escape_string` تابع

```
def escape_string(text: Optional[str]) -> str:
```

متن را به لیترال رشته‌ی C تبدیل می‌کند؛ `None` ← `NULL`. بک‌اسلش، گیومه، `\r`، `\n`، `\t` و **هر** `?` escape می‌شوند: `??/` در C99 یک trigraph است که به بک‌اسلش تبدیل می‌شود و نویسه‌ی بعد را می‌بلعد.

مثال:

```
escape_string('Say "hi"??/ ok\n')   # "Say \"hi\"\?\?/ ok\n"
escape_string(None)                   # NULL
```

`safe_comment` تابع

```
def safe_comment(text: Optional[str]) -> str:
```

متن را برای گذاشتن درون `/* ... */` امن می‌کند: `*/` و `/*` شکسته می‌شوند (وگرنه کامنت زود بسته می‌شد و فایل کامپایل نمی‌شد) و سطرشکن‌ها و تب‌ها فاصله می‌شوند. توضیح‌ها نثرند و نثر چنین چیزهایی دارد.

مثال:

```
safe_comment("range 0..100 /* see spec */\nnext")
# 'range 0..100 / * see spec * / next'
```

نام	کار
`bool_literal`	`TRUE` یا `FALSE` (ماکروهای `smart_data.h`).
`unsigned_literal`	عدد با پسوند `U`: `5` ← `5U`.
`_as_float32`	یک عدد پایتون (که همیشه double است) را به نزدیک‌ترین `float` ۳۲بیتی گرد می‌کند (با `struct.pack`).

`float_literal` تابع

```
def float_literal(value: float, value_type: CValueType) -> str:
```

لیترال اعشاری برای `float32_t` یا `float64_t`: کوتاه‌ترین متنی که دقیقاً همان عدد را برمی‌گرداند (`_shortest_exact`)، همیشه با نقطه یا توان، و پسوند نوع (`f` برای `float32_t`). NaN و بی‌نهایت `ValueError` می‌دهند. چرا «رقم معنادار» و نه «تعداد رقم اعشار»: `round(1.602176634e-19, 9)` صفر می‌شد، بی‌هیچ پیامی.

مثال:

```
float_literal(0.1, float32_t)              # 0.1f
float_literal(10, float32_t)               # 10.0f
float_literal(1e-7, float64_t)             # 1.0e-07
float_literal(1.602176634e-19, float64_t)  # 1.602176634e-19
float_literal(100000000.0, float32_t)      # 1.0e+08f
float_literal(1/3, float32_t)              # 0.33333333f
```

`_shortest_exact` تابع

```
def _shortest_exact(value: float, value_type: CValueType) -> str:
```

کوتاه‌ترین متن دهدهی که دقیقاً به همان عدد برمی‌گردد. نامزدها: `repr` پایتون و `%g` با ۱ تا ۹ (یا ۱۷) رقم؛ از آن‌هایی که دقیق برمی‌گردند کوتاه‌ترین برنده است و در تساوی، آنکه توان ندارد. برای `float32_t` «دقیق» یعنی پس از گرد شدن به ۳۲ بیت برابر باشد.

`integer_literal` تابع

```
def integer_literal(value: int, value_type: CValueType) -> str:
```

لیترال صحیح، با علامت بیرون پسوند و درون پرانتز. منفی‌ترین مقدار هر نوع علامت‌دار ویژه است: `-9223372036854775808LL` یک لیترال نیست بلکه منفیِ `9223372036854775808LL` است که در `long long` جا نمی‌شود؛ پس به شکل قابل‌حمل `(-9223372036854775807LL - 1)` نوشته می‌شود.

مثال:

```
integer_literal(5, uint16_t)          # 5U
integer_literal(-40, int8_t)          # (-40)
integer_literal(-128, int8_t)         # (-127 - 1)
integer_literal(-2**63, int64_t)      # (-9223372036854775807LL - 1)
```

`numeric_literal` تابع

```
def numeric_literal(value: NumericValue, value_type: CValueType,
                    fallback: NumericValue = 0) -> str:
```

یک مقدار را با لیترال درست نوعش می‌نویسد؛ اگر `fallback` بود، `None` بولی ← `bool_literal`، اعشاری ← `float_literal`، صحیح ← `integer_literal`.

مثال:

```
numeric_literal(None, bool_t)       # FALSE
numeric_literal(None, float32_t, 0) # 0.0f
```

`constructor_arguments` تابع

```
def constructor_arguments(parameter: Parameter, value_type: CValueType,
                          with_unit: bool = True) -> str:
```

آرگومان‌های سازنده‌ی `fSmartData_XX_Ctor` برای یک پارامتر (بدون `me`)، به ترتیب: کمینه، بیشینه، پیش‌فرض، رزولوشن، واحد. نکته‌ها:

- «بی‌بازه» (`range_disabled`) با **ماکروهای استاندارد** نوشته می‌شود: `INT32_MIN`، `UINT16_MAX`، `-DBL_MAX`؛ نه عددها. سطر تولیدی می‌گوید منظورش چیست.
- رزولوشن خالی، رزولوشن پیش‌فرض نوع می‌شود (۱ برای صحیح ولی کسری کوچک برای اعشاری؛ وگرنه هر مقدار به عدد صحیح گرد می‌شد).
- واحد آخرین آرگومان است. با `with_unit=False` (تنظیم `codegen.emit_unit`) رشته‌ی خالی می‌شود؛ تا کسی که برای صرفه‌جویی در فلش واحدها را خاموش کرده، واقعاً از آن‌ها خلاص شود.
- `bool_t` استثناست: سازنده‌اش فقط مقدار پیش‌فرض را می‌گیرد.

مثال:

```
constructor_arguments(MaxAltitude, uint16_t)
# '0U, 6000U, 3000U, 1U, "m"'
```

و همان در کد تولیدی: `fSmartData_U16_Ctor(&Cg_SmartData_MaxAltitude, 0U, 6000U, 3000U, 1U, "m");`

`limit_headers` تابع

```
def limit_headers(parameters) -> list:
```

هدرهای استانداردی که ماکروهای بازه‌ی پارامترهای «بی‌بازه» لازم دارند (`stdint.h`، `float.h`)؛ فقط آن‌هایی که واقعاً استفاده می‌شوند، تا پروژه‌ی بی‌اعشاری `float.h` نگیرد.

`tag_value_literal` تابع

```
def tag_value_literal(value: object) -> str:
```

مقدار یک تگ برای فیلد `uint32_t` توصیف‌گر.

مثال:

```
tag_value_literal(None)          # 0U
tag_value_literal(True)          # (uint32_t)TRUE
tag_value_literal(5)             # 5U
tag_value_literal("APP_FLAG_X")  # (uint32_t)APP_FLAG_X
```

۱۴.۵ پوشه‌ی blocks: آجرهای فایل C

هر فایل این پوشه **یک** کلاس کوچک دارد که **یک** نوع ساختار C را می‌نویسد. هر کلاس یک `__init__` و یک یا چند متد `generate...` دارد که فقط متن برمی‌گردانند؛ هیچ‌کدام چیزی در فایل نمی‌نویسند. `FileBuilder` از هر کلاس یک نمونه می‌سازد (`self.include`، `self.define`، …) و سازنده‌های فایل آن‌ها را پشت سر هم صدا می‌زنند.

۱۴.۶ `__init__.py` — فهرست بلوک‌ها

مسیر: `paramgen/src/pl_codegen/codegen/blocks/__init__.py`

دوازده کلاس را صادر می‌کند. تابعی ندارد. توضیح بالایش فهرست فایل‌ها و کار هرکدام است.

۱۴.۷ `include.py` — سطر #include

مسیر: `paramgen/src/pl_codegen/codegen/blocks/include.py`

کار	نام
یک سطر include می‌سازد.	`Include`
کاری نمی‌کند.	`Include.__init__`
`standard=True` ← `#include <stdint.h>`؛ وگرنه `#include "smart_data.h"`.	`Include.generate`

۱۴.۸ `define.py` — سطرهای #define

مسیر: `paramgen/src/pl_codegen/codegen/blocks/define.py`

کار	نام
سطرهای `#define` می‌سازد.	`Define`
کاری نمی‌کند.	`Define.__init__`
یک سطر: `#define NAME VALUE /*!< comment */`.	`Define.generate`
گروهی از سه‌تایی‌های `(نام، مقدار، کامنت)` با ستون‌های هم‌تراز؛ پهنای هر ستون پهنای بلندترین مقدار آن است.	`Define.generate_aligned_block`
جفت‌های `(نام، مقدار)`، هر کدام فقط `#ifndef` نوشته می‌شود (`#ifndef X` / `#define X v` / `#endif`)؛ برای نامی که فرمور ممکن است خودش تعریف کرده باشد: تعریف فرمور می‌ماند و هیچ‌کدام هشدار redefinition نمی‌دهند. مقدارها هم‌ترازند.	`Define.generate_guarded_block`

خروجی `generate_aligned_block` در `cg_parameter_list_defines.h`:

```
#define AIRSPEED            (0U)   /*!< float32_t [m/s] */
#define MAX_ALTITUDE        (1U)   /*!< uint16_t [m] */
#define CELL_VOLTAGE_0      (3U)   /*!< float32_t (element 0 of 4) [V] */
```

۱۴.۹ `comment.py` — کامنت‌ها

مسیر: `paramgen/src/pl_codegen/codegen/blocks/comment.py`

نام	کار
`Comment`	همه‌ی انواع کامنت فایل‌های تولیدی.
`Comment.__init__`	کاری نمی‌کند.
`Comment.generate_line`	یک سطر: `/* text */`.
`Comment.generate_doc`	بلوک doxygen با `@brief`، `@param`ها، `@note`ها و `@return`.
`Comment.generate_separator`	جداکننده‌ی بزرگ پیش از بدنه‌ی تابع‌ها در فایل `.c` (`##### Exported Functions #####`).
`Comment.generate_block`	بلوک `/* ... */` با چند سطر متن آزاد و عنوان اختیاری؛ برای جدول‌های متنی.

همه‌ی متن‌ها از `safe_comment` می‌گذرند. یک اثر جانبی: سطرشکن درون متن `@param` یا `@note` به فاصله تبدیل می‌شود؛ پس متنی که در پایتون با `\n` دو سطر نوشته شده، در فایل یک سطر است.

۱۴.۱۰ typedef enum — `enum.py`

مسیر: `paramgen/src/pl_codegen/codegen/blocks/enum.py`

امروز هیچ سازنده‌ای از آن استفاده نمی‌کند: تنها کاربرش دو enum فهرست‌های کشویی بود که حالا `#define` شده‌اند (`generate_guarded_block`). مانده تا فایلی که enum لازم دارد آن را آماده داشته باشد.

نام	کار
`Enum`	یک `typedef enum { ... } eName;` می‌سازد.
`Enum.__init__`	تورفتگی را نگه می‌دارد.
`Enum.generate`	از جفت‌های `(نام، مقدار)`؛ نام‌ها هم‌تراز. لیست خالی ← متن خالی.

۱۴.۱۱ `variable.py` — اعلان و تعریف متغیر

مسیر: `paramgen/src/pl_codegen/codegen/blocks/variable.py`

نام	کار
`Variable`	یک سطر متغیر می‌سازد.
`Variable.__init__`	کاری نمی‌کند.
`Variable.generate_declaration`	سطر `extern` برای هدر: `extern sSmartDataF32 Cg_SmartData_Airspeed; /*!< ... */`.
`Variable.generate_definition`	همان بدون `extern`، برای فایل `.c`.
`Variable._line`	سازنده‌ی مشترک: پیشوند، `qualifier` (مثل `const`)، نوع، نام، `[اندازه]` و کامنت.

۱۴.۱۲ `function.py` — پروتوتایپ و بدنه‌ی تابع

مسیر: `paramgen/src/pl_codegen/codegen/blocks/function.py`

نام	کار
`Function`	تکه‌های یک تابع C. هدر از `generate_prototype` استفاده می‌کند؛ فایل `.c` اول `generate_begin`، بعد سطرهای بدنه، بعد `generate_end`.
`Function.__init__`	تورفتگی را نگه می‌دارد.
`Function.generate_prototype`	`uint8_t fParameterList_Init(void);`.
`Function.generate_begin`	امضا و `{` و یک سطر خالی.
`Function.generate_end`	سطر خالی و `}`.
`Function.generate_body`	سطرهای بدنه را `level` بار تورفتگی می‌دهد؛ سطر خالی خالی می‌ماند.
`Function._signature`	متد ایستا: `qualifier` + نوع خروجی + نام + `(آرگومان‌ها)`.

۱۴.۱۳ `table.py` — جدول متنی

مسیر: `paramgen/src/pl_codegen/codegen/blocks/table.py`

`Table` کلاس

```
class Table:
```

جدول ASCII با حاشیه، برای کشیدن فهرست درون کامنت. سطر اول سرستون است. هر خانه **کامل** نوشته می‌شود و ستون به اندازه‌ی آن پهن می‌شود؛ قبلاً خانه‌ها در ۲۸ نویسه بریده می‌شدند و نام بلند به شکل `Position.Home.Latitu~` درمی‌آمد که کسی نمی‌تواند جست‌وجویش کند.

`Table.generate` متد

```
def generate(self, rows: Sequence[Sequence[str]]) -> List[str]:
```

جدول را به‌صورت لیست سطرها برمی‌گرداند (بی‌سطرشکن پایانی). سطرهای کوتاه‌تر با خانه‌ی خالی پر می‌شوند.

مثال:

```
Table().generate([["#", "Name"], ["0", "Airspeed"]])
# ['+---+----------+',
#  '| # | Name     |',
#  '+---+----------+',
#  '| 0 | Airspeed |',
#  '+---+----------+']
```

۱۴.۱۴ `file_banner.py` — بنر بالای فایل

مسیر: `paramgen/src/pl_codegen/codegen/blocks/file_banner.py`

کار	نام
بلوک `/** ... */` شروع هر فایل؛ شکلش از `c.json` (`CStyle`).	`FileBanner`
`CStyle` را نگه می‌دارد.	`FileBanner.__init__`
بنر را پر می‌کند و سطرهای `notes` (اطلاعات تولید) را زیر `@brief` می‌گذارد.	`FileBanner.generate`

۱۴.۱۵ `file_footer.py` — سطر آخر فایل

مسیر: `paramgen/src/pl_codegen/codegen/blocks/file_footer.py`

کار	نام
سطر `/* === Copyright (c) 2026 FAS === EOF === */`.	`FileFooter`
`CStyle` را نگه می‌دارد.	`FileFooter.__init__`
`style.footer_line()` و سطرشکن.	`FileFooter.generate`

۱۴.۱۶ include_guard.py — محافظ include دوباره

مسیر: paramgen/src/pl_codegen/codegen/blocks/include_guard.py

کار	نام
#ifndef / #define / #endif یک هدر.	IncludeGuard
کاری نمی‌کند.	IncludeGuard.__init__
سطر نشانگر «Define to prevent recursive inclusion»، #ifndef GUARD و #define GUARD.	IncludeGuard.generate_open
#endif /* GUARD */.	IncludeGuard.generate_close

۱۴.۱۷ cpp_guard.py — محافظ C++

مسیر: paramgen/src/pl_codegen/codegen/blocks/cpp_guard.py

کار	نام
پوشش #ifdef __cplusplus / extern "C" { که هدر را در C++ قابل استفاده می‌کند.	CppGuard
کاری نمی‌کند.	CppGuard.__init__
سه سطر باز کردن.	CppGuard.generate_open
سه سطر بستن.	CppGuard.generate_close

۱۴.۱۸ section_marker.py — نشانگر بخش

مسیر: paramgen/src/pl_codegen/codegen/blocks/section_marker.py

کار	نام
یک سطر نشانگر مثل /* Includes ---- */؛ متن دقیقش از c.json.	SectionMarker
CStyle را نگه می‌دارد.	SectionMarker.__init__
style.marker(name) و سطرشکن.	SectionMarker.generate

نشانگرهایی که سازنده‌ها استفاده می‌کنند: در هدرها Includes، Exported defines، Exported macros، Exported types، Exported functions، Exported variables؛ در فایل‌های .c علاوه بر این‌ها

`Private defines`، `Private macros`، `Private types`، `Private variables` و `Private functions`. این همان ترتیب قالب شرکت است و حتی بخش‌های خالی هم نوشته می‌شوند.

۱۴.۱۹ پوشه‌ی files: یک کلاس برای هر فایل

۱۴.۲۰ `__init__.py` — کدام فایل‌ها ساخته شوند

مسیر: `paramgen/src/pl_codegen/codegen/files/__init__.py`

دو لیست دارد، به ترتیب ساخت: `SMART_DATA_BUILDERS` (هفت کلاس، جدول ابتدای فصل) و `PARAM_TABLE_BUILDERS` (`ParamTableHeader`، `ParamOffsetsHeader` و `SystemNamesHeader`). **فایل تولیدی تازه یعنی یک ماژول تازه در این پوشه و افزودن کلاسش به یکی از این دو لیست** (فصل ۲۴).

`builders_for` تابع

```
def builders_for(config: Config) -> List[Type[FileBuilder]]:
```

کلاس‌های سازنده‌ی قالب این پروژه. **تنها جایی** است که دو قالب از هم جدا می‌شوند؛ هیچ جای دیگر نباید `output.format` را برای انتخاب فایل‌ها بپرسد.

مثال:

```
for builder_class in builders_for(config):
    builder = builder_class(model, config, offsets, style)
    path = builder.save(directory)
```

این سه سطر عیناً همان کاری است که `Pipeline.generate` می‌کند (فصل ۱۵).

تصمیم دیگری هم که قالب‌ها را از هم جدا می‌کند همین‌جاست: **تگ‌هایی که قالب برای همه‌ی پارامترها ثابت می‌کند.** در smart data حالت دسترسی (`ParamMode`) همیشه `PARAM_MODE_NONE` است (ثابت `SMART_DATA_FIXED_TAGS`)؛ ساب‌لیست مقدار خودش را نگه می‌دارد، چون همان فایل ممکن است در یک پروژه‌ی param_table هم ساخته شود که حالت دسترسی آنجا ستون واقعی `ADD_PARAM` است. مقدار ثابت فقط **در حافظه** جای مقدار فایل را می‌گیرد، در راه اعتبارسنجی و تولید کد.

`fixed_tags` تابع

```
def fixed_tags(config: Config) -> Dict[str, str]:
```

تگ‌هایی که قالب این پروژه ثابت می‌کند: در smart data `{"ParamMode": "PARAM_MODE_NONE"}`، در param_table هیچ.

ورودی‌ها

`config`	تنظیمات پروژه

خروجی: دیکشنری `{tag id: مقدار}`؛ خالی برای هیچ.

`apply_fixed_tags` تابع

```
def apply_fixed_tags(model: ParameterList, config: Config) -> ParameterList:
```

تگ‌های ثابت قالب را در همه‌ی گره‌های سطح بالای مدل می‌نویسد؛ فیلد استراکچر تگ‌هایش را از گره‌ای که در آن است می‌گیرد، پس سطح بالا کافی است. **خود مدل را تغییر می‌دهد**؛ پس فقط روی کپی‌ای صدا زده شود که ذخیره نمی‌شود. `Pipeline.load_model` و `Pipeline.generate` (فصل ۱۵) و `Document.merged_model` (فصل ۱۸) آن را صدا می‌زنند.

ورودی‌ها

`model`	مدل (کپی)
`config`	تنظیمات پروژه

خروجی: همان مدل.

۱۴.۲۱ `file_builder.py` — کلاس پایه‌ی همه‌ی سازنده‌ها

مسیر: `paramgen/src/pl_codegen/codegen/files/file_builder.py`

سه کار می‌کند و نه بیشتر: مدل و تنظیمات و آفست‌ها را در دسترس نگه می‌دارد؛ از هر کلاس بلوک یک نمونه می‌سازد؛ و متنی را که `generate()` برگرداند ذخیره می‌کند. ثابت `GENERATED_MARKER` سطر «This file is generated automatically - DO NOT EDIT IT BY HAND.» است؛ هر جا که بخواهد خروجی این ابزار را بشناسد (مثلاً کپی رهاشده در پوشه‌ی خروجی قدیمی، فصل ۱۵) با همین سطر می‌شناسد.

`FileBuilder` کلاس

```
class FileBuilder(ABC):
```

کلاس پایه‌ی انتزاعی. هر زیرکلاس باید دو چیز بنویسد: property `file_name` و متد `generate()`.

`FileBuilder.__init__` متد

```
def __init__(self, model: ParameterList, config: Config, offsets: OffsetMap,
             style: CStyle) -> None:
```

مدل، تنظیمات، `OffsetMap` و `CStyle` را نگه می‌دارد، زمان تولید را یک بار حساب می‌کند (تا همه‌ی فایل‌های یک اجرا یک زمان داشته باشند) و این ویژگی‌ها را می‌سازد:

- `self.banner`، `self.footer`، `self.marker`، `self.guard`، `self.cpp`، `self.include`، `self.define`، `self.comment`، `self.enum`، `self.variable`، `self.function`، `self.table`؛ و `self.tab` (تورفتگی).

کار	نام
انتزاعی: نام فایل، مثل `cg_parameter_list.h`.	`FileBuilder.file_name`

کار	نام
انتزاعی: کل متن فایل.	`FileBuilder.generate`
پوشه را (اگر نیست) می‌سازد و متن را با UTF-8 و پایان سطر `\n` (حتی روی ویندوز) می‌نویسد؛ مسیر را برمی‌گرداند.	`FileBuilder.save`
برگ‌های `flatten()`، یک بار حساب و نگه داشته می‌شود (property).	`FileBuilder.parameters`
نام محافظ include: نام فایل بی‌پسوند با حروف بزرگ + `_H`.	`FileBuilder.guard_name`
سطرهای اطلاعات تولید زیر `@brief` (اگر `emit_generation_info` روشن است): نشانگر، نسخه‌ی ابزار، زمان (اگر `emit_timestamp`)، ساب‌لیست‌ها **نسبت به ریشه‌ی پروژه** (تا بنر روی هر رایانه یکی باشد) و تعداد مقدارها.	`FileBuilder.generation_notes`
نام فایل defines: `cg_parameter_list_defines.h`.	`FileBuilder.defines_header`
`cg_parameter_list.h`.	`FileBuilder.list_header`
`cg_parameter_list.c`.	`FileBuilder.list_source`
`cg_parameter_list_table.h`.	`FileBuilder.table_header`
`cg_parameter_list_types.h`.	`FileBuilder.types_header`
`cg_smart_data.h`.	`FileBuilder.smart_data_header`
`cg_smart_data.c`.	`FileBuilder.smart_data_source`
ماکروی اندیس یک برگ: `Motor[1].Rpm` ← `MOTOR_1_RPM`.	`FileBuilder.index_of`
ماکروی اندازه‌ی آرایه‌ی اعلان یک برگ؛ فیلد آرایه‌ای یک تایپ به نام تایپ: `Stest.alive` ← `STEST_ALIVE_ARRAY_SIZE`.	`FileBuilder.array_size_of`
همان، از روی مسیر نقطه‌دار؛ اندیس عنصر حذف می‌شود.	`FileBuilder.array_size_of_name`
عبارت C شیء یک برگ: `Cg_SmartData_Position.Lat` یا `Cg_SmartData_Motor[1].Rpm`.	`FileBuilder.smart_data_of`
شیءهای سطح فایل: برای هر پارامتر ساده یک شیء smart data و برای هر استراکچر **یک** شیء از typedef تایپش. چهارتایی‌های (کامنت، None، ماکروی اندازه یا، نام، C نوع).	`FileBuilder.top_level_objects`
نام typedef یک تایپ: `sPosition` ← `sCgStruct_sPosition`.	`FileBuilder.struct_type_of`
نام تابع اندازه‌ی ناحیه‌ی یک تگ: `fParameterList_GetResetProofSize`.	`FileBuilder.tag_size_function_of`
تگ‌هایی که دست‌کم یک مقدار دارد؛ تگ بی‌استفاده نه ماکرو می‌گیرد نه تابع.	`FileBuilder.sized_tags`
نام ماکروی اندازه‌ی ناحیه‌ی یک تگ.	`FileBuilder.offset_total_of`
نام ماکروی تعداد: `PARAMETER_LIST_QTY`.	`FileBuilder.quantity`
نام آرایه‌ی توصیف‌گرها: `ParameterList`.	`FileBuilder.descriptor_symbol`
توضیح برای جدول؛ **رشته‌ی خالی، هرگز** `None`: `NULL` در `pDescription` یک `printf("%s", ...)` ساده را رفتار تعریف‌نشده می‌کرد.	`FileBuilder.description_of`

کار	نام
واحد برای جدول؛ همین قاعده.	`FileBuilder.unit_of`

۱۴.۲۲ `parameter_list_defines_h.py` — فایل cg_parameter_list_defines.h

مسیر: `paramgen/src/pl_codegen/codegen/files/parameter_list_defines_h.py`

ثابت `BRIEF` متن `@brief` فایل است (هر سازنده یکی دارد). `generate()` به ترتیب: بنر و محافظ‌ها، `#include <stdint.h>`، ماکروی تعداد، ماکروی اندیس هر برگ، ماکروی اندازه‌ی هر اعلان آرایه، ماکروی اندازه‌ی ناحیه‌ی هر تگ، و `#define` هر نام System names و Parameter modes با عددش (`Defines.numbered`، فصل ۱۰)، هر کدام فقط `#ifndef`: فرم‌ور ممکن است `SYS_NAV` را خودش تعریف کرده باشد و آن‌وقت تعریف خودش به کار می‌رود. بنر واقعی این فایل:

```
/**
  ******************************************************************************
  * @file    cg_parameter_list_defines.h
  * @brief   Quantities, parameter indices, tag sizes, system names and parameter modes.
  *
  * This file is generated automatically - DO NOT EDIT IT BY HAND.
  * Generator      : parameter_list_code_generator v01.01.01
  * Generated at   : 2026-10-17 12:15:15 +0330
  * Input file(s)  : parameter_lists/core.xml
  *                  parameter_lists/power.xml
  * Parameters     : 21 value(s) in 2 structure(s)
  ******************************************************************************
  * @attention
  *  ...  (Copyright از c.json و مجوز)
  */
```

و بخش ماکروها:

```
#define PARAMETER_LIST_QTY  (21U)  /*!< Number of enabled parameters */

#define AIRSPEED             (0U)   /*!< float32_t [m/s] */
#define MAX_ALTITUDE         (1U)   /*!< uint16_t [m] */
...
#define MOTOR_3_TEMP         (17U)  /*!< int8_t [C] */
#define LOW_BATTERY_VOLTAGE  (18U)  /*!< float32_t [V] */

#define CELL_VOLTAGE_ARRAY_SIZE  (4U)  /*!< Number of elements of 'CellVoltage' */
#define MOTOR_ARRAY_SIZE         (4U)  /*!< Number of elements of 'Motor' */

#define PARAMETER_LIST_TAG_SYSTEM_NAME_SIZE  (67U)  /*!< Bytes used by tag 'SystemName' (21
parameter(s)) */
#define PARAMETER_LIST_TAG_RESET_PROOF_SIZE  (7U)   /*!< Bytes used by tag 'ResetProof' (3
parameter(s)) */
...

/* Values of the 'SYSTEM NAMES' drop-down list (kept where already defined) */
#ifndef SYS_UNKNOWN
#define SYS_UNKNOWN          (0U)
#endif
#ifndef SYS_GS_COMPUTER
#define SYS_GS_COMPUTER      (1U)
#endif
...
/* Values of the 'PARAM MODES' drop-down list (kept where already defined) */
#ifndef PARAM_MODE_NONE
#define PARAM_MODE_NONE          (0U)
#endif
...
```

پیش‌تر این نام‌ها دو `typedef enum` بودند (`eParameterListSystemName` و `eParameterListParamMode`) که عددشان را جایگاه در فهرست تعیین می‌کرد؛ هر دو برداشته شده‌اند. جدول توصیف‌گر آن‌ها را مثل قبل با `(uint32_t)SYS_NAV` می‌نویسد.

کار	نام
سازنده‌ی این فایل.	`ParameterListDefinesHeader`
سه‌تایی‌های ماکروی اندیس: نام، `(iU)` و کامنت نوع داده، «element i of n» برای عنصر آرایه، و واحد.	`ParameterListDefinesHeader._index_entries`
یک ماکرو برای هر **اعلان** آرایه (`model.array_declarations()`).	`ParameterListDefinesHeader._array_size_entries`
یک ماکرو برای هر تگی که ناحیه‌اش خالی نیست.	`ParameterListDefinesHeader._tag_total_entries`
برای هر فهرست `GENERATED_DEFINES` یک کامنت و `generate_guarded_block` از `Defines.numbered`های `(نام، (nU))`.	`ParameterListDefinesHeader._list_defines`

آفست هر مقدار زمانی در همین فایل ماکرو بود؛ امروز فقط در `cg_parameter_list_table.h` (کامنت) کشیده می‌شود، چون فیلد تگ در توصیف‌گر دیگر آفست ندارد.

۱۴.۲۳ `parameter_types_h.py` — فایل cg_parameter_list_types.h

مسیر: `paramgen/src/pl_codegen/codegen/files/parameter_types_h.py`

یک `typedef struct` برای هر **تایپ** در حال استفاده، نه برای هر پارامتر؛ همان‌طور که در C++ هست: یک بار اعلان، و هر استراکچرِ آن تایپ یک نمونه. تایپی که درون تایپ دیگر است اول اعلان می‌شود (`struct_types_in_use` همین ترتیب را می‌دهد). فیلد خاموش در typedef نیست. واحد و توضیح فیلدها عمداً در typedef نیستند: مال یک **استفاده** از تایپ‌اند، نه خود تایپ. خروجی واقعی:

```
/**
 * @brief Structure type 'sPosition'.
 *
 * @note  Used by: Position
 */
typedef struct {

    sSmartDataF64 Lat;
    sSmartDataF64 Lon;
    sSmartDataF32 Alt;

} sCgStruct_sPosition;
```

کار	نام
سازنده‌ی این فایل. اگر هیچ استراکچری نباشد، فقط کامنت « The parameter list contains no structure.»	`ParameterTypesHeader`
یک typedef با کامنت doxygen «Used by:» (مسیر استراکچرهایی که از آن استفاده می‌کنند).	`ParameterTypesHeader._generate_struct`
سه‌تایی‌های (`None` نام فیلد، ماکروی اندازه یا `C` نوع): نوع داده ← struct smart data آن، تایپ دیگر ← typedef آن.	`ParameterTypesHeader._members`

۱۴.۲۴ `smart_data_h.py` — فایل cg_smart_data.h

مسیر: `paramgen/src/pl_codegen/codegen/files/smart_data_h.py`

برای هر پارامتر سطح بالا یک اعلان `extern` و پروتوتایپ `fCgSmartData_Init`. ثابت‌های `INIT_FUNCTION`، `INIT_RETURN`، `INIT_DOC_RETURN` و `INIT_BRIEF` را `smart_data_c.py` هم استفاده می‌کند تا نام تابع یک جا باشد. خروجی واقعی:

```
uint8_t fCgSmartData_Init(void);

extern sSmartDataF32 Cg_SmartData_Airspeed;  /*!< [m/s] Indicated airspeed. */
extern sSmartDataF32 Cg_SmartData_CellVoltage[CELL_VOLTAGE_ARRAY_SIZE];  /*!< [V] ... */
extern sCgStruct_sPosition Cg_SmartData_Position;  /*!< Where the aircraft is. */
extern sCgStruct_sMotor Cg_SmartData_Motor[MOTOR_ARRAY_SIZE];  /*!< One of the four motors.
*/
```

کار	نام
سازنده‌ی این فایل؛ شیءها را از `top_level_objects()` می‌گیرد.	`SmartDataHeader`

۱۴.۲۵ `smart_data_c.py` — فایل cg_smart_data.c

مسیر: `paramgen/src/pl_codegen/codegen/files/smart_data_c.py`

تعریف شیءها و تابع `fCgSmartData_Init` که هرکدام را با سازنده‌ی smart data نوعش می‌سازد. عنصرهای یک آرایه‌ی ساده با یک حلقه ساخته می‌شوند (همه آرگومان یکسان دارند)؛ ولی آرایه‌ی استراکچر فیلد به فیلد باز می‌شود. خروجی واقعی:

```
uint8_t fCgSmartData_Init(void) {

    eSmartDataStatus status = eSMART_DATA_OK;
    uint16_t index = 0U;

    status = fSmartData_F32_Ctor(&Cg_SmartData_Airspeed, 0.0f, 120.0f, 0.0f, 0.01f, "m/s");
    if (status != eSMART_DATA_OK) {
        return (uint8_t)status;
    }

    for (index = 0U; index < CELL_VOLTAGE_ARRAY_SIZE; index++) {
        status = fSmartData_F32_Ctor(&Cg_SmartData_CellVoltage[index], 0.0f, 5.0f, 0.0f,
0.01f, "V");
        if (status != eSMART_DATA_OK) {
            return (uint8_t)status;
        }
    }

    status = fSmartData_I8_Ctor(&Cg_SmartData_Motor[0].Temp, (-40), 125, 0, 1, "C");
    ...
    return (uint8_t)status;

}
```

ثابت‌های `LOOP_INDEX` (`index`) و `STATUS` (`status`) نام دو متغیر محلی‌اند.

`SmartDataSource._init_body` متد

```
def _init_body(self) -> str:
```

بدنه‌ی `fCgSmartData_Init`. روی برگ‌ها می‌رود؛ اگر برگ عنصر صفرِ یک آرایه‌ی ساده است، یک حلقه برای کل آرایه می‌نویسد و از همه‌ی عنصرهایش می‌پرد؛ وگرنه یک سطر سازنده. متغیر `index` **فقط اگر واقعاً حلقه‌ای نوشته شد** اعلان می‌شود: متغیر بی‌استفاده هشدار می‌داد و با `-Werror` شرکت، فایل کامپایل نمی‌شد.

کار	نام
سازنده‌ی این فایل. هدرهای `limit_headers` را هم include می‌کند.	`SmartDataSource`
`[0]` آخر یک نماد را حذف می‌کند: `Cg_SmartData_PosDrone[0]` ← `Cg_SmartData_PosDrone`.	`_without_element`

۱۴.۲۶ `parameter_list_h.py` — فایل cg_parameter_list.h

مسیر: `paramgen/src/pl_codegen/codegen/files/parameter_list_h.py`

هدر عمومی که برنامه‌ی فرمور include می‌کند. ثابت‌های نام تابع‌ها اینجاست و `parameter_list_c.py` هم از همین‌ها استفاده می‌کند:

مقدار	ثابت
`PARAMETER_LIST_WEAK`	`WEAK_MACRO`
`fParameterList_Init`	`INIT_FUNCTION`
`fParameterList_InitCompletedCallback`	`INIT_CALLBACK`
`fParameterList_GetQuantity`	`GET_QUANTITY_FUNCTION`
`fParameterList_GetByIndex`	`GET_BY_INDEX_FUNCTION`
`fParameterList_GetByName`	`GET_BY_NAME_FUNCTION`
`const sParameterDescriptor *`	`DESCRIPTOR_TYPE`

ماکروی weak به‌شکل مستقل از کامپایلر تعریف می‌شود (GCC/Clang/ARMCC با `__attribute__((weak))`، IAR با `__weak`)، تا فرمور بتواند `InitCompletedCallback` را بی‌تغییر فایل تولیدی بازنویسی کند. خروجی واقعی بخش تابع‌ها:

```
uint8_t fParameterList_Init(void);
PARAMETER_LIST_WEAK uint8_t fParameterList_InitCompletedCallback(void);
uint16_t fParameterList_GetQuantity(void);
const sParameterDescriptor * fParameterList_GetByIndex(uint16_t index);
const sParameterDescriptor * fParameterList_GetByName(const char *pName);

uint32_t fParameterList_GetSystemNameSize(void);
uint32_t fParameterList_GetResetProofSize(void);
...
extern const sParameterDescriptor ParameterList[PARAMETER_LIST_QTY];
```

کار	نام
سازنده‌ی این فایل؛ برای هر تگ `sized_tags()` یک پروتوتایپ تابع اندازه.	`ParameterListHeader`

۱۴.۲۷ `parameter_list_c.py` — فایل cg_parameter_list.c

مسیر: `paramgen/src/pl_codegen/codegen/files/parameter_list_c.py`

مهم‌ترین فایل تولیدی: جدول ثابت توصیف‌گرها و بدنه‌ی تابع‌های دسترسی. یک ردیف واقعی جدول:

```
const sParameterDescriptor ParameterList[PARAMETER_LIST_QTY] = {
    /* [0] AIRSPEED */
    {
        .pName = "Airspeed",
        .pUserName = "Airspeed",
        .Type = ePARAMETER_TYPE_MONITORING,
        .pSmartData = (void *)&Cg_SmartData_Airspeed,
        .LogId = 100U,
        .TagSystemName = (uint32_t)SYS_NAV,
        .TagResetProof = (uint32_t)FALSE,
        .TagLoggable = (uint32_t)TRUE,
        .TagLogEncryption = (uint32_t)FALSE,
        .TagParamMode = (uint32_t)PARAM_MODE_READ_ONLY,
        .Tag1 = 0U,
        ...
        .pDescription = "Indicated airspeed."
    },
```

و `fParameterList_GetByName`:

```
const sParameterDescriptor * fParameterList_GetByName(const char *pName) {

    uint16_t index = 0U;

    if (pName == NULL) {
        return NULL;
    }

    for (index = 0U; index < (uint16_t)PARAMETER_LIST_QTY; index++) {
        if (strcmp(ParameterList[index].pName, pName) == 0) {
            return &ParameterList[index];
        }
    }

    return NULL;

}
```

`ParameterListSource._table_entry` متد

```
def _table_entry(self, parameter: Parameter):
```

مقداردهی‌های نام‌دار یک ردیف، به ترتیب اعلان (`descriptor_layout.py`). نام برگ خودش عبارت C رسیدن به آن است (`Motor[1].Rpm`)، پس `pSmartData` مستقیم به عنصر خودش اشاره می‌کند. `LogId` از `effective_log_id` است: عدد روی مقدار غیرقابل‌لاگ صفر نوشته می‌شود. تگ بولی خالی `FALSE` می‌شود. Type خالی `MONITORING` فرض می‌شود (که بررسی‌ها پیش‌تر جلویش را گرفته‌اند).

کار	نام
سازنده‌ی این فایل.	`ParameterListSource`

کار	نام
هدرهای codegen.extra_includes. تگ آزادی که یک کلمه است (APP_FLAG_X) به شکل (uint32_t)APP_FLAG_X نوشته می‌شود و کامپایلر باید بداند چیست؛ فقط برنامه می‌داند کدام هدر آن را دارد. <x.h> و "x.h" و نام خالی هر سه پذیرفته‌اند.	ParameterListSource._generate_extra_includes
آرایه‌ی ParameterList[] با کامنت /* [i] MACRO */ بالای هر ردیف.	ParameterListSource._generate_table
برای هر تگ استفاده‌شده تابعی که فقط ماکروی اندازه را برمی‌گرداند؛ پس عدد هرگز از آفست‌ها جدا نمی‌شود.	ParameterListSource._generate_tag_sizes
fParameterList_Init: اول fCgSmartData_Init، بعد callback؛ اولین خطا برگردانده می‌شود.	ParameterListSource._generate_init
پیاده‌سازی weak که فقط صفر برمی‌گرداند.	ParameterListSource._generate_init_callback
fParameterList_GetQuantity.	ParameterListSource._generate_get_quantity
fParameterList_GetByIndex: اندیس بیرون بازه ← NULL.	ParameterListSource._generate_get_by_index
fParameterList_GetByName: جست‌وجوی خطی با strcmp؛ فیلد استراکچر با نام نقطه‌دار (Position.Lat).	ParameterListSource._generate_get_by_name

۱۴.۲۸ parameter_list_table_h.py — فایل cg_parameter_list_table.h

مسیر: paramgen/src/pl_codegen/codegen/files/parameter_list_table_h.py

این فایل **هیچ کدی ندارد**: فهرست را به‌صورت سه جدول متنی در کامنت می‌کشد تا برنامه‌نویس بدون باز کردن ابزار، کل فهرست را کنار کد ببیند. ورودی‌های خاموش هم نشان داده می‌شوند. سه جدول: مقدارها، تگ‌ها (با بازه‌ی شناسه‌ی لاگ) و آفست هر مقدار در هر تگ. بخشی از جدول سوم، واقعی:

```
+-------+-------------------+-------+------------+------------+----------+-----------+
| Index | Name              | Bytes | SystemName | ResetProof | Loggable | ParamMode |
+-------+-------------------+-------+------------+------------+----------+-----------+
| 0     | Airspeed          | 4     | 0          | -          | 0        | 0         |
| 1     | MaxAltitude       | 2     | 4          | 0          | 4        | 4         |
| 2     | FlightMode        | 1     | 6          | 2          | -        | 6         |
| 3     | CellVoltage[0]    | 4     | 7          | -          | 6        | 7         |
...
```

کار	نام
سازنده‌ی این فایل.	ParameterListTableHeader
جدول ۱: یک ردیف برای هر گره درخت (فیلدها با تورفتگی): اندیس، روشن، نام، نام نمایشی، Type، نوع، اندازه، پیش‌فرض، کمینه، بیشینه، رزولوشن، واحد.	ParameterListTableHeader._value_rows
جدول ۲: تگ‌های هر پارامتر سطح بالا و ستون «LogID range».	ParameterListTableHeader._tag_rows

کار	نام
سه‌تایی‌های (عمق، مسیر، گره) برای همه‌ی گره‌ها به ترتیب.	ParameterListTableHeader._all_entries
اندیس جدول یک گره، یا - برای استراکچر و گره خاموش.	ParameterListTableHeader._index_of
دیکشنری «مسیر ← اندیس»، اولین عنصر هر اعلان (property).	ParameterListTableHeader._index_by_path
100، یا 200..203 برای بلوک چندتایی؛ با log_id_span.	ParameterListTableHeader._log_id_range
جدول ۳: برای هر برگ، اندازه و آفست در هر تگ استفاده‌شده، و ردیف جمع.	ParameterListTableHeader._offset_rows
یادداشت زیر جدول ۱ (و اگر آرایه‌ی استراکچر هست، توضیح اندیس اولین عنصر).	ParameterListTableHeader._value_note
یادداشت زیر جدول ۳: قاعده‌ی شمارنده‌ی هر تگ.	ParameterListTableHeader._offset_note
عدد برای جدول: None ← -، بولی ← FALSE/TRUE، اعشاری صحیح بی‌نقطه.	_number
تگ برای جدول: None ← -، بولی ← x یا فاصله.	_tag

۱۴.۲۹ param_table_h.py — قالب param_table: فایل nsrParamTables.h

مسیر: paramgen/src/pl_codegen/codegen/files/param_table_h.py

فرمور قدیمی smart data و جدول توصیف‌گر ندارد؛ یک هدر پر از سطرهای ADD_PARAM(...) دارد که با #if DESIRED_SUB_SYSTEM == ... به زیرسیستم‌ها تقسیم شده، و خود برنامه پیش از include تعریف می‌کند ADD_PARAM چه معنایی دارد. دو تفاوت مهم با قالب دیگر:

- **زیرسیستم از نام فایل می‌آید**: sys_ac_power_paramTable.xml ← SYS_AC_POWER. تگ SystemName درون فایل اینجا نادیده گرفته می‌شود.
- **اندیس در هر بلوک از صفر شروع می‌شود**؛ چون شماره‌ی پارامتر در زیرسیستم خودش است.

نام‌های SYS_... که سطرهای #if با آن‌ها مقایسه می‌کنند در این فایل تعریف نمی‌شوند: فرمور خودش تعریفشان می‌کند، یا nsrSystemNames.h را include می‌کند (پایین‌تر). این فایل آن را include نمی‌کند.

خروجی واقعی یک پروژه‌ی `param_table` کوچک (کپی `power.xml` با نام `sys_ac_power_paramTable.xml`):

```
//WARNING: Order of subsytems should comply SYS_GS_ enum (at least for last system)
//WARNING: DO NOT ADD NEW ITEMS BETWEEN OLD ITEM

//ADD_PARAM(data type, TYPE_ENUM, PR_NAME, "gs name", SYS_NAME, param index, ...)

#define PARAM_MODE_NONE             (0)
#define PARAM_MODE_USER             (1)
#define PARAM_MODE_READ_ONLY        (2)
...
#if DESIRED_SUB_SYSTEM == SYS_AC_POWER || DESIRED_SUB_SYSTEM == SYS_ALL
ADD_PARAM(float, TYPE_FLOAT, BatteryVoltage, "Battery voltage", SYS_AC_POWER, 0, 0.0, 0.0  ,
30.0, 0.01, PARAM_MODE_READ_ONLY, 0, "Voltage of the whole pack.")
ADD_PARAM(float, TYPE_FLOAT, ChargeCurrent , "Charge current" , SYS_AC_POWER, 1, 0.0, -50.0,
50.0, 0.01, PARAM_MODE_READ_ONLY, 0, "Positive while charging.")

#endif
```

مقدار و معنی	ثابت
`nsrParamTables.h`؛ ثابت، چون فرم‌ور همین نام را include می‌کند	`FILE_NAME`
`_PARAMTABLE`؛ از آخر نام فایل حذف می‌شود تا زیرسیستم بماند	`NAME_SUFFIX`
`DESIRED_SUB_SYSTEM` و `SYS_ALL`	`SELECTOR`، `EVERYTHING`
دو هشدار و راهنمای ستون‌ها، کلمه به کلمه از فایل دست‌نویس قدیمی (حتی غلط املایی `subsytems`)	`WARNINGS`، `LEGEND`
`float32_t` ← `float` و `TYPE_FLOAT`؛ `float64_t` ← `double` و `TYPE_DOUBLE`	`_C_NAME`، `_TYPE_ENUM`
`0xFFFFFFFF`؛ بیشترین آدرس	`ADDRESS_MAX`
`ResetProof`، `Loggable`، `LogEncryption`؛ ستونی در `ADD_PARAM` ندارند	`IGNORED_TAGS`
`SystemName`؛ از نام فایل می‌آید و ویرایشگر آن را فقط‌خواندنی نشان می‌دهد	`FILE_OWNED_TAGS`، `READ_ONLY_TAGS`
۱۳ ستون؛ همه جز آخری (توضیح) در **کل فایل** هم‌تراز می‌شوند تا هر ویرگول زیر ویرگول بالایی باشد	`_COLUMNS`، `_ALIGNED`

`system_of` تابع

```
def system_of(list_file: Path) -> str:
```

زیرسیستم یک ساب‌لیست از روی نام فایلش، با حروف بزرگ (بزرگی و کوچکی حروف نام فایل مهم نیست). `drift.py` و قاعده‌های `param_table` هم از همین استفاده می‌کنند.

مثال:

```
system_of(Path("sys_gs_manager_paramTable.xml"))   # 'SYS_GS_MANAGER'
```

تابع **`blocks_by_system`**

```
def blocks_by_system(parameters: List[Parameter],
                     model_files: Sequence[Path]) -> List[Tuple[str, List[Parameter]]]:
```

پارامترها، گروه‌بندی‌شده بر اساس فایلی که از آن آمده‌اند، به ترتیب ساخت. جدا نوشته شده چون دو فایل از آن ساخته می‌شوند (جدول و یادداشت آفست‌ها) و هر دو باید دقیقاً یک گروه‌بندی داشته باشند.

خروجی: لیست جفت‌های `(زیرسیستم ، پارامترها)`.

کار	نام
نوع C در این قالب: `float32_t` ← `float`، بقیه بی‌تغییر.	`c_type_of`
`uint8_t` ← `TYPE_UINT8`؛ دو نوع اعشاری از `_TYPE_ENUM`.	`type_enum_of`
عدد ساده‌ی بی‌پسوند (بی`U`، بی`f`، بی‌پرانتز)، مثل فایل دست‌نویس؛ ردیف اعشاری نقطه را نگه می‌دارد (`70.0`)، ردیف صحیح نه (`31`).	`plain_number`
ستون آدرس: فیلد `address` پارامتر یا صفر («خودت حساب کن»).	`_address_of`
سازنده‌ی این فایل.	`ParamTableHeader`
`#define PARAM_MODE_NONE (0)` برای هر مقدار فهرست `PARAM MODES`، شماره به **ترتیب** فهرست (نه عدد پنجره‌ی Drop-down lists)، بی `#ifndef`؛ پهنا +۵ تا با فایل قدیمی یکی باشد.	`ParamTableHeader._modes`
`blocks_by_system` روی برگ‌های این ساخت.	`ParamTableHeader._blocks`
خانه‌های ردیف‌های یک بلوک، شماره از صفر.	`ParamTableHeader._rows`
پهنای هر ستون برای **کل** فایل.	`ParamTableHeader._widths`
یک بلوک `#if ... #endif` با ستون‌های هم‌تراز.	`ParamTableHeader._block`
۱۳ خانه‌ی یک `ADD_PARAM`. حدها همان‌طور که **نوشته شده‌اند**، نه مؤثر: بازه‌ی خالی `0, 0` می‌شود، مثل فایل دست‌نویس.	`ParamTableHeader._cells`

۱۴.۳۰ `param_offsets_h.py` — قالب param_table: فایل nsrParamOffsets.h

مسیر: `paramgen/src/pl_codegen/codegen/files/param_offsets_h.py`

جدول `ADD_PARAM` نمی‌گوید هر پارامتر **کجای حافظه** می‌نشیند. فرمور مقدارهای یک زیرسیستم را پشت سر هم می‌چیند، پس آفست سومی جمع اندازه‌ی دو تای بالایی است؛ حسابی که با دست به‌راحتی غلط می‌شود. این فایل

سراسر کامنت است و include کردنش به هیچ کدی کامپایل نمی‌شود. پارامتر خاموش در آن نیست، پس خاموش کردن یکی، اندیس و آفست همه‌ی بعدی‌های آن زیرسیستم را جابه‌جا می‌کند. خروجی واقعی:

```
/* ---- SYS_AC_POWER ----------------------------------------------------
     index  offset  size   type  parameter
         0       0     4  float  BatteryVoltage
         1       4     4  float  ChargeCurrent

     2 parameter(s), 8 byte(s) in total
*/
```

ثابت‌ها: `FILE_NAME` (`nsrParamOffsets.h`) و `HEADINGS` (سرستون‌ها).

کار	نام
سازنده‌ی این فایل.	`ParamOffsetsHeader`
دقیقاً همان گروه‌بندی جدول (`blocks_by_system`)؛ یادداشتی که با چیزی که توضیح می‌دهد نخواند، از نبودنش بدتر است.	`ParamOffsetsHeader._blocks`
جدول یک زیرسیستم: اندیس، آفست (جمع جاری، بی‌padding)، اندازه (`اندازه‌ی آرایه × value_size`)، نوع و نام.	`ParamOffsetsHeader._block`

۱۴.۳۱ `system_names_h.py` — قالب param_table: فایل nsrSystemNames.h

مسیر: `paramgen/src/pl_codegen/codegen/files/system_names_h.py`

فایل سوم قالب param_table، کنار جدول و یادداشت آفست‌ها، و فقط یک چیز در آن است: هر نام فهرست `SYSTEM NAMES` با عددش. سطرهای `#if DESIRED_SUB_SYSTEM == SYS_NAV` جدول همین نام‌ها را در پیش‌پردازنده مقایسه می‌کنند و پیش‌پردازنده فقط ماکرو می‌فهمد (ثابت enum آنجا بی‌صدا صفر حساب می‌شود)؛ پس `#define` هستند. عدد همان عدد پنجره‌ی Drop-down lists است، همان که smart data هم می‌نویسد، و هر نام فقط `#ifndef`. هیچ فایل دیگری آن را include نمی‌کند؛ فرمور هر جا لازم دارد include می‌کند. خروجی واقعی:

```
/* Define to prevent recursive inclusion -------------------------------------*/
#ifndef NSRSYSTEMNAMES_H
#define NSRSYSTEMNAMES_H

/* Every SYSTEM NAMES entry, for the #if lines of nsrParamTables.h (kept where already
defined) */
#ifndef SYS_UNKNOWN
#define SYS_UNKNOWN          (0)
#endif
#ifndef SYS_GS_COMPUTER
#define SYS_GS_COMPUTER      (1)
#endif
...

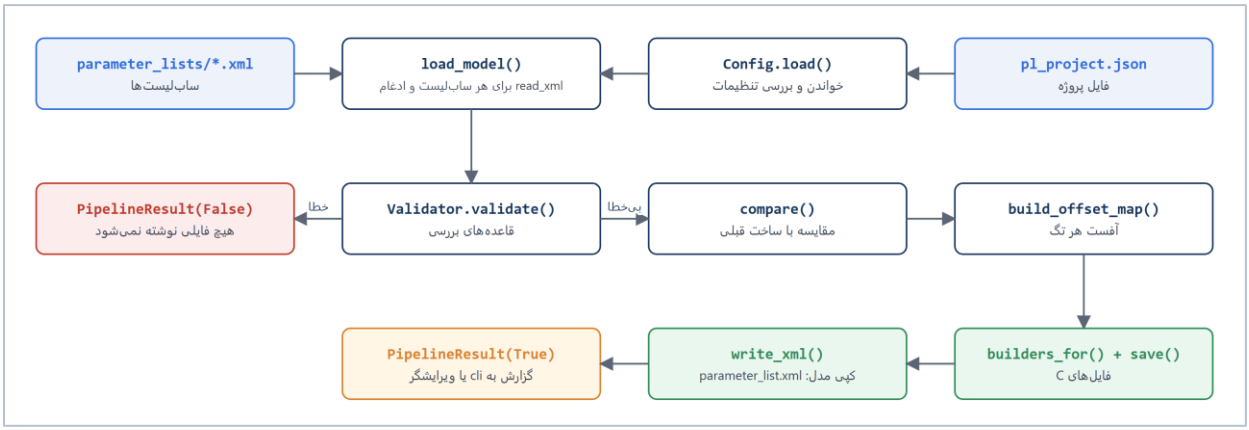
#endif /* NSRSYSTEMNAMES_H */
```

عددها مثل خود جدول بی `U` نوشته می‌شوند (`(16)`)، چون همان فرمور دست‌نویس هر دو را می‌خواند. ثابت: `FILE_NAME` (`nsrSystemNames.h`). دو نام با یک عدد اینجا هم `DEF005` است (فصل ۱۲).

کار	نام
سازنده‌ی این فایل: بنر، محافظ include، یک کامنت و `generate_guarded_block`.	`SystemNamesHeader`
`("(n)" ،نام)` برای هر نام سیستم، به ترتیب فهرست، از `Defines.numbered`.	`SystemNamesHeader._entries`

۱۵. خط‌لوله و خط فرمان

فصل‌های ۹ تا ۱۴ قطعه‌ها را شرح دادند. این فصل نشان می‌دهد چه کسی آن‌ها را به ترتیب کنار هم می‌گذارد: `pipeline.py` (هسته‌ی مشترک) و `cli.py` (خط فرمان، همان که `run.bat generate` و `bin/run_generate.py` اجرا می‌کنند). ویرایشگر هم همین `Pipeline` را صدا می‌زند؛ پس دکمه‌ی **Generate code** و خط فرمان **دقیقاً** یک کار را می‌کنند.



شکل ۶ — گام‌های یک ساخت: از فایل پروژه تا فایل‌های C و کپی مدل

هسته فقط یک ورودی می‌شناسد: فایل XML. اینکه XML از کجا آمده (ویرایشگر یا دست) کار جلوی ابزار است، نه هسته؛ همین است که اجازه می‌دهد روزی جلوی دیگری جای ویرایشگر بنشیند بی‌آنکه یک سطر هسته عوض شود.

۱۵.۱ `pipeline.py` — خط‌لوله‌ی مشترک

مسیر: `paramgen/src/pl_codegen/pipeline.py`

کلاس `PipelineResult`

```
@dataclass
class PipelineResult:
```

dataclass: هرچه یک فراخوان برای گزارش نتیجه‌ی اجرا لازم دارد.

- `success`: آیا تولید انجام شد (یا در `check`، آیا بررسی بی‌خطا بود).
- `diagnostics`: `DiagnosticCollector` همه‌ی پیام‌ها.
- `model_file`: اولین ساب‌لیست؛ `model_files`: همه، به ترتیب ساخت.
- `generated_files`: مسیر فایل‌های نوشته‌شده.
- `parameter_count`: تعداد مقدارها (برگ‌ها).
- `drift`: `LayoutDrift` (فصل ۱۳)، یا `None`.

ثابت `_READ_FROM_DISK` یک شیء یکتاست که به‌عنوان مقدار پیش‌فرض آرگومان `previous` به کار می‌رود؛ یعنی «ساخت قبلی را خودت از دیسک بخوان». چرا `None` نه: چون `None` معنای خودش را دارد («ساخت قبلی‌ای نیست»). این الگو در پایتون رایج است و «sentinel» نام دارد.

`Pipeline` کلاس

```
class Pipeline:
```

اجراکننده‌ی تولیدکننده. با یک `Config` ساخته می‌شود و logger را نگه می‌دارد.

`Pipeline.load_model` متد

```
def load_model(self, diagnostics: DiagnosticCollector,
               model_file: Optional[Path] = None) -> ParameterList:
```

مدل ادغام‌شده‌ی همه‌ی ساب‌لیست‌های پیکربندی‌شده (یا فقط `model_file` اگر داده شده). هر فایل با `read_xml` خوانده و مسیرش به `source_files` افزوده می‌شود؛ سپس `merge_lists` (فصل ۱۰) همه را یکی می‌کند. همان ادغامی است که ویرایشگر هم استفاده می‌کند؛ و همان جایی که تایپی که دو ساب‌لیست اعلان کرده‌اند یا ساب‌لیستی که تایپ دیگری را استفاده کرده، ثبت می‌شود. در پایان `apply_fixed_tags` (فصل ۱۴) تگ‌هایی را که قالب ثابت می‌کند - در smart data `ParamMode` برابر `PARAM_MODE_NONE` - در مدل ادغام‌شده می‌نویسد؛ فقط در حافظه، فایل‌ها دست نمی‌خورند.

خطا: `CodeGeneratorError` اگر یک ساب‌لیست وجود نداشته باشد (با راهنمای ساختن آن یا اصلاح `inputs.model_files`).

`Pipeline.generate` متد

```
def generate(self, model: ParameterList, diagnostics: DiagnosticCollector,
             previous: object = _READ_FROM_DISK) -> PipelineResult:
```

قلب ابزار. گام‌ها، به ترتیب:

- `apply_fixed_tags` دوباره، برای مدلی که از `load_model` نیامده؛ دو بار نوشتنش چیزی را عوض نمی‌کند.
- `Validator(...).validate`؛ اگر `False`، همان‌جا `PipelineResult(False, ...)` و هیچ فایلی نوشته نمی‌شود.
- ساخت قبلی (`previous_build` از `output.xml_file`، مگر اینکه `previous` داده شده باشد) و `compare_layout` (فصل ۱۳).
- `build_offset_map` و `CStyle.load`.
- برای هر کلاس `builders_for(config)`: ساختن سازنده و `save` در `output.code_directory`.
- نوشتن کپی مدل ادغام‌شده در `output.xml_file` (کلید `full_parameter_list_file` در فایل پروژه) **پس از** همه‌ی فایل‌ها؛ چون فقط اجرایی که تا اینجا رسیده «آن چیزی است که فرمور از آن ساخته شد». فایل تایپ‌های

کنارش فقط تایپ‌هایی را دارد که واقعاً استفاده می‌شوند (`types_used_by`). خطای نوشتن این کپی فقط هشدار لاگ است.

- اگر `drift.breaks_stored_data`، هشدار در لاگ.
- `._report_abandoned_copies`.

ورودی‌ها

`model`	مدل ادغام‌شده
`diagnostics`	جمع‌کننده‌ی پیام‌ها
`previous`	مدل ساخت قبلی؛ پیش‌فرض: از دیسک خوانده شود

خروجی: `PipelineResult`.

> **مهم:** جلویی که خودش `parameter_list.xml` را پیش از تولید می‌نویسد، باید نسخه‌ی قبلی را **پیش از نوشتن** بخواند و به‌عنوان `previous` بدهد؛ وگرنه مدل تازه با خودش مقایسه می‌شود و هیچ جابه‌جایی‌ای دیده نمی‌شود.

`Pipeline._report_abandoned_copies` متد

```
def _report_abandoned_copies(self, generated: List[Path],
                             diagnostics: DiagnosticCollector) -> None:
```

اگر `output.code_directory` عوض شده باشد، فایل‌های اجراهای قبلی سر جایشان می‌مانند و از آن به بعد هیچ‌کس در آن‌ها نمی‌نویسد. کسی که همیشه همان فایل قدیمی را باز می‌کند، می‌بیند «تغییر نکرد» و نمی‌فهمد چرا. این متد پوشه‌ی پروژه را می‌گردد (`os.walk`، بی پوشه‌های `lib`، `.git`، `.svn`، `__pycache__` و `node_modules`) و هر فایل هم‌نامِ یک خروجی را که جای دیگری است و در ۲۰۰۰ نویسه‌ی اولش `GENERATED_MARKER` دارد، با هشدار `OUT001` گزارش می‌کند. هدر دست‌نویسی که اتفاقاً هم‌نام است متهم نمی‌شود، چون نشانگر را ندارد.

`Pipeline.run` متد

```
def run(self, model_file: Optional[Path] = None) -> PipelineResult:
```

مسیر کامل خط فرمان: بنر «Reading the parameter list»، `load_model`، گزارش تعداد، بنر « Generating the C code» و `generate`.

خروجی: `PipelineResult`.

`Pipeline.check` متد

```
def check(self, model_file: Optional[Path] = None) -> PipelineResult:
```

فقط بررسی (`--check`): `load_model` و `Validator`، **بی‌نوشتن هیچ فایلی**. در `model_files` همان فایلی را می‌گذارد که واقعاً خوانده شد (فایل `-m` اگر داده شده).

خروجی: `PipelineResult` بی`generated_files`.

تابع `render_diagnostics`

```
def render_diagnostics(diagnostics: DiagnosticCollector, verbose: bool) -> str:
```

متن پیام‌ها برای کنسول؛ با `verbose` پیام‌های اطلاع هم، وگرنه از هشدار به بالا.

۱۵.۲ `cli.py` — خط فرمان تولیدکننده

مسیر: `paramgen/src/pl_codegen/cli.py`

آنچه یک اسکریپت ساخت یا CI صدا می‌زند. خود فایل `bin/run_generate.py` فقط پوشه‌ی `src` را به مسیر import اضافه می‌کند و `main` همین فایل را اجرا می‌کند (فصل ۲۱).

کدهای خروج

ثابت	مقدار	یعنی
`EXIT_OK`	۰	موفق (یا `--list-rules`)
`EXIT_VALIDATION_FAILED`	۱	بررسی خطا داشت (یا هشدار با `-W`)؛ فایلی نوشته نشد
`EXIT_TOOL_ERROR`	۲	خطای خود ابزار: پروژه پیدا نشد، فایل پروژه یا XML خراب، ساب‌لیست نیست

اسکریپت ساخت می‌تواند با همین کد تصمیم بگیرد؛ مثلاً در یک فایل bat:

```
py paramgen\bin\run_generate.py -q -c Projects\Flight_Computer
if errorlevel 1 (
    echo The parameter list has errors - see above.
    exit /b 1
)
```

گزینه‌ها

گزینه	کار
`-c PROJECT`، `--config`	پروژه: پوشه‌اش یا فایل `pl_project.json` آن
`-m FILE`، `--model`	یک ساب‌لیست به‌جای ساب‌لیست‌های پیکربندی‌شده
`--check`	فقط بررسی، بی‌نوشتن هیچ فایل
`--list-rules`	فهرست قاعده‌های بررسی و خروج
`-W`، `--warnings-as-errors`	هشدارها خطا حساب شوند
`-v`، `--verbose`	پیام‌های اطلاع هم نشان داده شوند
`-q`، `--quiet`	فقط مشکل‌ها
`--version`	نام و نسخه‌ی ابزار

یک اجرای واقعی

```
> py paramgen\bin\run_generate.py -c ...\Flight_Computer
INFO     parameter_list_code_generator v01.01.01
INFO     configuration: ...\Flight_Computer\pl_project.json
INFO     input: 2 parameter sub lists, merged:
         - ...\Flight_Computer\parameter_lists\core.xml
         - ...\Flight_Computer\parameter_lists\power.xml
================================================================================
Reading the parameter list
================================================================================
INFO       merged 2 sub list(s) into one parameter list
INFO       9 top level parameter(s), 21 value(s) after flattening
================================================================================
Generating the C code
================================================================================
INFO       generated cg_parameter_list_defines.h (129 lines)
INFO       generated cg_parameter_list_types.h (74 lines)
...
INFO       generated cg_parameter_list_table.h (129 lines)
================================================================================
Diagnostics
================================================================================
No problems found.

0 error(s), 0 warning(s), 0 info message(s)

Generated 7 file(s) for 21 value(s):
  ...
Done.
```

`resolve_config_path` تابع

```
def resolve_config_path(given: Optional[Path]) -> Path:
```

کدام پروژه ساخته شود. سه جواب، به ترتیبی که آدم منظور دارد: (۱) آنچه در خط فرمان آمده؛ **پوشه** یعنی فایل پروژه‌ی درونش، چون آدم پوشه را می‌کشد و رها می‌کند؛ (۲) پروژه‌ی پوشه‌ی جاری، که منظور اسکریپتی است که از ریشه‌ی پروژه اجرا می‌شود؛ (۳) آخرین پروژه‌ای که ویرایشگر باز کرده بود (`last_project`، فصل ۹). اگر هیچ‌کدام نبود، خطا با راهنمای هر سه راه.

خطا: `CodeGeneratorError`.

`build_parser` تابع

```
def build_parser() -> argparse.ArgumentParser:
```

تجزیه‌گر آرگومان‌ها با ماژول استاندارد `argparse` (بخش ۴.۱۷). متن راهنما و نمونه‌ها (`epilog`) همان است که `--help` چاپ می‌کند. **گزینه‌ی تازه** یعنی یک `parser.add_argument(...)` اینجا و خواندن `arguments.<نام>` در `main`.

`main` تابع

```
def main(argv: Optional[Sequence[str]] = None) -> int:
```

گام‌ها: تجزیه‌ی آرگومان‌ها؛ `setup_logging`؛ اگر `--list-rules`، چاپ و خروج؛ `Config.load(resolve_config_path(...))` (خطا ← کد ۲)؛ `-W` روی تنظیمات؛ چاپ نسخه و پیکربندی و ورودی؛ `Pipeline.check` یا `Pipeline.run` (خطای ابزار ← کد ۲)؛ و `_print_summary` (مگر در `-q` با نتیجه‌ی موفق).

ورودی‌ها

`argv`	آرگومان‌ها؛ `None` یعنی `sys.argv`. آزمون‌ها لیست خودشان را می‌دهند

خروجی: کد خروج (`0`، `1` یا `2`).

مثال:

```
code = main(["--check", "-c", "Projects/Flight_Computer"])
```

کار	نام
یک سطر درباره‌ی فایل‌هایی که خوانده می‌شوند: « no parameter sub list configured»، یک فایل، یا فهرست.	`_describe_input`
گزارش جابه‌جایی را **پیش از** فهرست فایل‌ها چاپ می‌کند (کسی که « 7 files written» را خوانده، دیگر نمی‌خواند): قرمز با «WARNING - THE MEMORY LAYOUT CHANGED SINCE THE LAST BUILD» اگر داده‌ی ذخیره‌شده خراب می‌شود، وگرنه زرد.	`_print_drift`
بنر «Diagnostics»، پیام‌ها، خلاصه‌ی تعداد، گزارش جابه‌جایی، فهرست فایل‌ها و «Done.»؛ یا «Code generation was aborted…» (و اگر خطایی نبود، «warnings are treated as errors»).	`_print_summary`

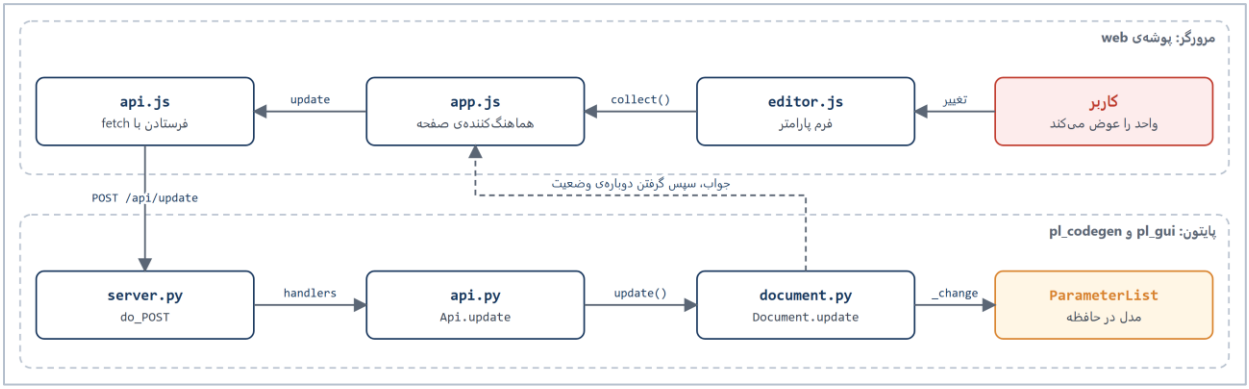
۱۶. ویرایشگر: نگاه کلی

ویرایشگر یک **برنامه‌ی وب محلی** است: یک سرور کوچک پایتون (فقط کتابخانه‌ی استاندارد) و یک صفحه‌ی وب (`web/`) که مرورگرِ خودِ رایانه نشانش می‌دهد. چیزی نصب نمی‌شود و هیچ چیزی از اینترنت خوانده نمی‌شود. سرور فقط به `127.0.0.1` گوش می‌دهد؛ یعنی از شبکه اصلاً در دسترس نیست.

لایه‌ها

کار	فایل	لایه
اسکلت صفحه و ظاهر	`web/index.html`، `style.css`	صفحه
رسم جدول و فرم، گرفتن کلیک‌ها و کلیدها	`web/app.js`، `grid.js`، `editor.js`	منطق صفحه
فرستادن هر درخواست به سرور با `fetch`	`web/api.js`	ارتباط
پخش فایل‌های `web/` و فرستادن هر `POST /api/<نام>` به متد هم‌نامش در `Api`	`server.py`	سرور
یک متد برای هر کار؛ نازک: درخواست JSON را به `Document` می‌دهد و جواب را JSON می‌کند	`api.py`	API
کدام پروژه باز است و پروژه‌های اخیر	`workspace.py`	پروژه
فهرست در حال ویرایش، Undo/Redo، و **همه‌ی قاعده‌های یک ویرایش معتبر**	`document.py`	سند
خواندن و نوشتن XML، بررسی، تولید (فصل‌های ۹ تا ۱۵)	`pl_codegen`	هسته

قانون طلایی این لایه‌ها: **هر تصمیمی درباره‌ی درستی یک ویرایش در `document.py` گرفته می‌شود، نه در JavaScript.** صفحه ممکن است دکمه‌ای را پنهان کند یا خانه‌ای را فقط‌خواندنی نشان دهد، ولی سند همان را دوباره رد می‌کند؛ چون صفحه‌ای که از قبل باز مانده (stale) یا یک اشتباه تایپی در JS نباید بتواند فهرست را خراب کند. برای همین بیشتر متدهای `Document` «refuse» (رد) دارند.



شکل ۷ — مسیر یک ویرایش: از تایپ در فرم تا رسم دوباره‌ی صفحه

مسیر یک ویرایش

فرض کنید کاربر واحد `Airspeed` را از `m/s` به `km/h` تغییر می‌دهد و از خانه بیرون می‌رود:

- `editor.js` تغییر را می‌بیند و همه‌ی خانه‌های فرم را جمع می‌کند (`collect`).
- `app.js` با `api.js` درخواست `POST /api/update` با بدنه‌ی JSON `{"path": "Airspeed", "changes": {...}}` می‌فرستد.
- `server.py` نام `update` را در `Api.handlers()` پیدا می‌کند و `Api.update` را صدا می‌زند.
- `Api.update` فقط `Document.update(path, changes)` را صدا می‌زند و `ValueError` را به `{"ok": false, "message": ...}` تبدیل می‌کند.
- `Document.update` درون `_change("edit")` تغییر را اعمال می‌کند: اگر چیزی رد شود، همه‌چیز برمی‌گردد؛ اگر متن فایل عوض نشده باشد، قدم Undo هم ثبت نمی‌شود.
- صفحه `POST /api/state` می‌فرستد و کل وضعیت (درخت، پیام‌های **Problems**، فهرست‌ها، نشانه‌ی `*` تغییر ذخیره‌نشده) را دوباره می‌گیرد و جدول و فرم را رسم می‌کند.

یعنی صفحه هیچ وضعیتی از فهرست را خودش نگه نمی‌دارد که با سرور فرق کند؛ هر بار همه را از `state` می‌گیرد. این ساده‌ترین راه برای اینکه صفحه و فایل هرگز اختلاف نداشته باشند.

افزودن یک کار تازه به ویرایشگر

الگو همیشه سه‌قسمتی است (جزئیات در فصل ۲۴): (۱) متدی در `Document` که کار را انجام می‌دهد و اگر لازم است رد می‌کند؛ (۲) متدی در `Api` که آن را صدا می‌زند و به `handlers()` اضافه می‌شود؛ (۳) دکمه یا منو در `app.js`/`index.html` که با `api.js` آن را می‌فرستد. و یک آزمون در `run_editor_test.py`.

۱۶.۱ `__init__.py` — درِ ورودی پکیج `pl_gui`

مسیر: `paramgen/src/pl_gui/__init__.py`

فقط توضیح بالای فایل و `__all__ = ["server", "api", "document"]`. تابعی ندارد. (توضیح بالای این فایل هنوز از `tree.js` نام می‌برد؛ آن فایل امروز `grid.js` است.)

۱۶.۲ `browser.py` — پیدا کردن مرورگر و باز کردن آن بی‌شبکه

مسیر: `paramgen/src/pl_gui/browser.py`

رایانه‌های شرکت اینترنت ندارند؛ این ماژول یک قول را صریح و آزمودنی می‌کند: «هیچ چیزی در این ابزار جز با `127.0.0.1` تماس نمی‌گیرد». دو چیز لازم است: صفحه‌ها به هیچ فایل بیرونی ارجاع ندهند (نمی‌دهند)، و مرورگر Chromium که ابزار باز می‌کند، کارهای شبکه‌ای پس‌زمینه‌اش (به‌روزرسانی، همگام‌سازی، گزارش خطا) را نکند؛ کار `OFFLINE_FLAGS`.

ثابت	محتوا
`BROWSER_COMMANDS`	نام‌هایی که در PATH جست‌وجو می‌شوند: `msedge`، `chrome`، `chromium`، `brave` و گونه‌هایشان
`BROWSER_PATHS`	مسیرهای نصب شناخته‌شده برای هر سیستم‌عامل (`win32`، `darwin`، `linux`)
`OFFLINE_FLAGS`	سوییچ‌های خط فرمان Chromium که هر تماس شبکه‌ای خودکار را خاموش می‌کنند: `--disable-background-networking`، `--disable-component-update`، `--disable-sync`، `--disable-extensions` ، ...

`find_browser` تابع

```
def find_browser() -> Optional[str]:
```

مسیر یک مرورگر Chromium یا `None`. متغیر محیطی `PL_CODEGEN_BROWSER` اگر به فایلی موجود اشاره کند، جست‌وجو را کنار می‌زند (برای مرورگر قابل‌حملی که در PATH نیست). بعد نام‌های `BROWSER_COMMANDS` با `shutil.which` و در پایان مسیرهای `BROWSER_PATHS`.

`app_window_command` تابع

```
def app_window_command(browser: str, url: str, profile_directory: str) -> List[str]:
```

فرمانی که صفحه را به شکل **پنجره‌ی برنامه** باز می‌کند: `--app=<url>` (بی‌نوار نشانی و زبانه)، یک پوشه‌ی پروفایل موقت (`--user-data-dir`) تا نشست خود کاربر دست نخورد، اندازه‌ی پنجره ۱۵۰۰×۹۵۰، و همه‌ی `OFFLINE_FLAGS`.

مثال:

```
app_window_command("msedge.exe", "http://127.0.0.1:8765/", "C:/Temp/pl_editor_x")
# ['msedge.exe', '--app=http://127.0.0.1:8765/',
#  '--user-data-dir=C:/Temp/pl_editor_x', '--window-size=1500,950',
#  '--disable-background-networking', ...]
```

۱۶.۳ `native_dialog.py` — پنجره‌های Open و Browse خود ویندوز

مسیر: `paramgen/src/pl_gui/native_dialog.py`

مرورگر نمی‌تواند مسیر واقعی یک فایل را به صفحه بدهد (فقط نام و محتوا). پس پنجره‌ی «باز کردن» را **سرور** نشان می‌دهد، با `tkinter` که همراه پایتون است و روی ویندوز همان پنجره‌ی واقعی ویندوز (IFileDialog) را نشان می‌دهد. جایی که `tkinter` نیست (لینوکس بی بسته‌ی `python3-tk` یا بی‌میزکار)، صفحه پنجره‌ی درون‌صفحه‌ای خودش را نشان می‌دهد.

پنجره در فرایند (process) جداگانه اجرا می‌شود: سرور با چند رشته (thread) جواب می‌دهد و Tk مال رشته‌ای است که ساختش؛ فرایند جدا Tk را کاملاً از سرور دور نگه می‌دارد و پنجره‌ای که گیر کند یا از کار بیفتد ویرایشگر را با خود پایین نمی‌کشد. خود همین فایل آن فرایند هم هست: اگر به‌صورت اسکریپت اجرا شود، یک پنجره نشان می‌دهد و جواب را به‌صورت یک سطر JSON چاپ می‌کند.

ثابت	معنی
`SWITCH`	نام متغیر محیطی `PL_EDITOR_NATIVE_DIALOGS`؛ مقدار `0` پنجره‌ی درون‌صفحه‌ای را اجباری می‌کند (برای آزمون آن مسیر)
`KIND_FILE`، `KIND_FOLDER`، `KIND_PROBE`	نوع پنجره؛ `probe` چیزی نشان نمی‌دهد و فقط ثابت می‌کند پنجره **ممکن** است
`FileTypes`	نام مستعار نوع: دنباله‌ی `(برچسب، الگوها)`
`_one_at_a_time`	قفل (`threading.Lock`): فقط یک پنجره در هر لحظه
`_can_show`	نتیجه‌ی یک‌باره‌ی «tkinter و میزکار هست؟»

نام	کار
`NativeDialogUnavailable`	استثنا: هیچ پنجره‌ی سیستمی نشان داده نشد؛ صفحه باید پنجره‌ی خودش را نشان دهد.
`DialogBusy`	استثنا: یک پنجره از قبل باز است.
`available`	آیا پنجره‌ی سیستمی ممکن است؛ `SWITCH` هر بار خوانده می‌شود، وجود tkinter یک بار.
`_has_tkinter_and_a_desktop`	آیا `tkinter` نصب است (`importlib.util.find_spec`) و (روی لینوکس) `DISPLAY` یا `WAYLAND_DISPLAY` هست.
`_existing`	خود مسیر، یا نزدیک‌ترین پوشه‌ی موجود بالای آن.
`child_command`	فرمان فرایند فرزند: همین پایتون + همین فایل + درخواست به‌صورت JSON فقط‌اسکی (تا نام پوشه‌ی فارسی در هر code page سالم برسد).
`_run`	فرایند فرزند را اجرا و آخرین سطر خروجی‌اش را به‌صورت JSON می‌خواند؛ هر شکستی `NativeDialogUnavailable`.
`_show`	در فرایند فرزند: آگاه از DPI در ویندوز، پنجره‌ی پنهان Tk که «همیشه رو» است (وگرنه پنجره پشت ویرایشگر باز می‌شد)، و `askdirectory` یا `askopenfilename`.
`main`	ورودی فرایند فرزند: درخواست را از `argv[1]` می‌خواند و `{"path": ...}` چاپ می‌کند.

ask تابع

```
def ask(kind: str, title: str, start: Path, filetypes: FileTypes = ()) -> str:
```

یک پنجره نشان می‌دهد و مسیر انتخاب‌شده را برمی‌گرداند؛ رشته‌ی خالی اگر لغو شد. پوشه‌ی شروعی که نیست، از نزدیک‌ترین پوشه‌ی موجود باز می‌شود.

ورودی‌ها

`kind`	`KIND_FILE` یا `KIND_FOLDER`
`title`	عنوان پنجره
`start`	پوشه‌ی شروع
`filetypes`	انواع فایل فهرست پایین پنجره

خطا: `NativeDialogUnavailable` یا `DialogBusy`.

مثال:

```
ask(KIND_FILE, "Open project", Path("F:/projects"),
    (("Parameter list project", ("pl_project.json",)),))
# 'F:\\projects\\demo\\pl_project.json'  یا ''
```

۱۷. سرور، فضای کار و فایل پروژه

این فصل سه فایل کوچک ویرایشگر را شرح می‌دهد که هیچ‌کدام چیزی از پارامترها نمی‌دانند: `server.py` (گرفتن درخواست‌ها)، `workspace.py` (کدام پروژه باز است) و `config_file.py` (نوشتن فهرست ساب‌لیست‌ها در `pl_project.json` بی‌آنکه کامنت‌هایش پاک شود).

۱۷.۱ `server.py` — سرور HTTP کوچک

مسیر: `paramgen/src/pl_gui/server.py`

با ماژول استاندارد `http.server` (بخش ۴.۱۷) ساخته شده؛ پس چیزی نصب نمی‌شود. دو کار دارد:

- **GET**: پخش فایل‌های پوشه‌ی `/web`. نشانی `/` یعنی `index.html`.
- **POST** به `/api/<نام>`: بدنه را JSON می‌خواند، متد هم‌نام را از `Api.handlers()` پیدا و صدا می‌زند، و جواب را JSON می‌فرستد.

ثابت `WEB_DIRECTORY` مسیر پوشه‌ی `web` است و `CONTENT_TYPES` نوع محتوای هر پسوند (`.html`، `.css`، `.js`، `.svg`، `.png`، `.ico`؛ `.png` برای لوگو و آیکون‌های پنجره). سرور به‌طور پیش‌فرض روی درگاه (port) ۸۷۶۵ و فقط `127.0.0.1` گوش می‌دهد. نمونه‌ی یک درخواست و جوابش، همان‌طور که روی سیم می‌رود:

```
POST /api/next_log_id HTTP/1.1
Content-Type: application/json

{}

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: no-store

{"ok": true, "value": 502}
```

(۵۰۲ چون آخرین شناسه‌ی پروژه‌ی نمونه ۵۰۱ است، مال `ChargeCurrent`.)

پاسخ به خطا

بدنه	کی	وضعیت
`not found`	GET فایلی که در `web/` نیست، یا بیرون آن (مثلاً `../`)	۴۰۴
`{"ok": false, "message": "Unknown action 'x'."}`	`POST /api/x` که `x` در `handlers()` نیست	۴۰۴
`{"ok": false, "message": "Bad request: ..."}`	بدنه JSON نیست	۴۰۰
`CodeGeneratorError`: فقط متنش؛ استثنای دیگر: نام کلاس + متن. خود استثنا با traceback در لاگ کنسول	متد API استثنا داد	۵۰۰

بررسی `WEB_DIRECTORY not in target.parents` جلوی خواندن هر فایل بیرون `web/` را می‌گیرد (مثلاً `GET /../../src/pl_gui/api.py`). هر استثنای متد API گرفته می‌شود تا **سرور هرگز نمیرد**؛ صفحه پیام را در یک toast نشان می‌دهد.

`GuiServer` کلاس

```
class GuiServer:
```

سرور ویرایشگر. هنگام ساخته شدن یک `Workspace` می‌سازد، اگر پروژه‌ای داده شده آن را باز می‌کند و یک `Api` روی آن می‌سازد. بی‌پروژه شروع کردن خطا نیست: ویرایشگر روی صفحه‌ی شروع باز می‌شود.

`GuiServer.__init__` متد

```
def __init__(self, project: Optional[Path] = None, host: str = "127.0.0.1",
             port: int = 8765) -> None:
```

ورودی‌ها

`project`	فایل پروژه برای باز کردن، یا `None`
`host`	نشانی؛ پیش‌فرض `127.0.0.1`
`port`	درگاه؛ پیش‌فرض ۸۷۶۵

`GuiServer.serve_forever` متد

```
def serve_forever(self, open_browser: bool = True, app_window: bool = True) -> None:
```

سرور را با `ThreadingHTTPServer` (هر درخواست در یک رشته) راه می‌اندازد، چند سطر لاگ چاپ می‌کند، و ۰٫۶ ثانیه بعد (`threading.Timer`) پنجره را باز می‌کند؛ تا Ctrl+C. یک پوشه‌ی پروفایل موقت برای مرورگر می‌سازد و در پایان پاکش می‌کند.

ورودی‌ها

`open_browser`	آیا مرورگر باز شود (آزمون‌ها `False` می‌دهند)
`app_window`	آیا پنجره‌ی برنامه امتحان شود

`GuiServer._open` متد

```
def _open(self, app_window: bool, profile_directory: str) -> None:
```

اول پنجره‌ی برنامه (`_open_app_window`)؛ اگر نشد، مرورگر پیش‌فرض با `webbrowser.open`؛ و اگر آن هم چیزی باز نکرد (رایانه‌ی بی‌میزکار یا WSL)، هشداری که می‌گوید نشانی را خودتان باز کنید.

متد `GuiServer._open_app_window`

```
def _open_app_window(self, profile_directory: str) -> bool:
```

مرورگر Chromium را با `app_window_command` اجرا می‌کند. خطای مرورگر به یک **فایل** موقت می‌رود نه pipe: pipe ای که کسی نمی‌خواند پر می‌شود (۶۴ کیلوبایت در لینوکس) و مرورگر و پنجره‌ی ویرایشگر را قفل می‌کند. اگر مرورگر در ۲ ثانیه بسته شود، یعنی شروع نشد؛ آخرین سطر خطایش در لاگ می‌آید و `False` برمی‌گردد.

کار	نام
`http://127.0.0.1:8765/` (property).	`GuiServer.url`
سرور را متوقف می‌کند (`shutdown`)؛ برای آزمون‌ها.	`GuiServer.stop`
کلاس handler را **درون یک تابع** می‌سازد تا به `api` دسترسی داشته باشد (closure، بخش ۴.۷). کلاس درونی `Handler` از `BaseHTTPRequestHandler` ارث می‌برد و `do_GET`، `do_POST`، `_json`، `_send` و `log_message` (که خاموش است تا کنسول تمیز بماند) را دارد.	`_make_handler`

۱۷.۲ `workspace.py` — پروژه‌ی باز و پروژه‌های اخیر

مسیر: `paramgen/src/pl_gui/workspace.py`

ویرایشگر یک بار روی هر رایانه نصب می‌شود و **پروژه‌ها** را ویرایش می‌کند، مثل یک ویرایشگر متن که فایل‌ها را. پس به کمی وضعیت نیاز دارد که مال هیچ پروژه‌ای نیست: پروژه‌ی باز و چند پروژه‌ی اخیر. این وضعیت در `paramgen/var/settings.json` است (ثابت `SETTINGS_FILE` در `app_home.py`) و فقط مسیر دارد؛ پاک کردنش بی‌خطر است. نمونه:

```
{
  "recent": [
    "../Projects/Flight_Computer/pl_project.json",
    "F:/firmware/power_board/pl_project.json"
  ],
  "current": "../Projects/Flight_Computer/pl_project.json"
}
```

مسیرها تا جای ممکن **نسبت به پوشه‌ی برنامه** نوشته می‌شوند (`store_recent_entry`، فصل ۹) تا جابه‌جا کردن پوشه‌ی کامل ابزار و پروژه‌ها با هم چیزی را خراب نکند. ثابت `RECENT_LIMIT` (۱۲) طول فهرست است.

کلاس `Workspace`

```
class Workspace:
```

پروژه‌ی باز (`document`) و فهرست پروژه‌های اخیر.

کار	نام
فایل تنظیمات را می‌خواند؛ هنوز پروژه‌ای باز نیست.	`Workspace.__init__`
آیا پروژه‌ای باز است (property).	`Workspace.is_open`
`Config` پروژه‌ی باز یا `None` (property).	`Workspace.config`

`Workspace.require` متد

```
def require(self) -> Document:
```

سند باز، یا خطا با جمله‌ی «Open'... < Project' No project is open. Use». هر کار ویرایشی از اینجا می‌گذرد (از طریق `Api.document`) تا «پروژه‌ای باز نیست» یک بار و با کلمات جواب داده شود، نه با `AttributeError` روی `None` در بیست جای مختلف.

خطا: `CodeGeneratorError`.

`Workspace.open` متد

```
def open(self, path: Path) -> Document:
```

پروژه‌ای را باز می‌کند و پروژه‌ی جاری می‌کند. پوشه یعنی فایل پروژه‌ی درونش (`project_file_in`، فصل ۹: نام فایل مهم نیست؛ اگر پوشه چند فایل پروژه‌ی تغییرنام‌داده دارد، خطا نام همه را می‌گوید تا یکی را باز کنید). پروژه‌ی قبلی **فقط وقتی** کنار می‌رود که تازه با موفقیت بار شده باشد؛ فایل پروژه‌ای با یک اشتباه تایپی نباید کار در دست را ببندد. در پایان `.remember`.

خطا: `CodeGeneratorError` اگر فایل نیست یا خراب است.

کار	نام
سندی را که جای دیگری باز شده جاری می‌کند **بی‌آنکه به اخیرها اضافه شود**؛ برای آزمون‌ها که پروژه را در پوشه‌ی موقت می‌سازند.	`Workspace.adopt`
ویرایشگر را به صفحه‌ی شروع برمی‌گرداند؛ چیزی ذخیره نمی‌شود.	`Workspace.close`
پروژه را به سر فهرست اخیر می‌برد (تکراری‌ها حذف)، فهرست را کوتاه و تنظیمات را ذخیره می‌کند.	`Workspace.remember`
پروژه‌ای را از فهرست اخیر برمی‌دارد.	`Workspace.forget`
پروژه‌های اخیری که هنوز هستند، تازه‌ترین اول. پروژه‌ای که پوشه‌اش پاک شده، به‌جای «missing»، از فهرست حذف و فراموش می‌شود؛ میان‌بُر به چیزی که نیست، اطلاع نیست.	`Workspace.recent`
آخرین پروژه‌ای که باز بود و هنوز هست؛ همان که `run.bat generate` بی‌آرگومان می‌سازد.	`Workspace.last`
فهرست اخیر را از فایل تنظیمات **خود همین workspace** می‌خواند.	`Workspace._read_settings`
`recent` و `current` را می‌نویسد.	`Workspace._write_settings`

`Workspace.create` متد

```
def create(self, folder: Path, name: str,
           output_format: str = FORMAT_SMART_DATA) -> Path:
```

یک پروژه‌ی تازه در یک پوشه می‌نویسد و مسیر فایل پروژه را برمی‌گرداند. گام‌ها:

- نام خالی، قالب ناشناخته، نویسه‌های `\ / : * ? " < > |` در نام و مسیر غیرکامل رد می‌شوند (`ValueError`).
- اگر در آن پوشه فایل پروژه‌ای هست - با هر نامی؛ `project_files_in` - `FileExistsError`: «New» راهی برای از دست دادن پروژه‌ی کسی نیست.
- الگوی `templates/pl_project.json` را می‌خواند (`load_jsonc`)، نام و قالب را **به‌صورت داده** در آن می‌گذارد و به‌صورت JSON ساده (بی‌کامنت) می‌نویسد.
- پوشه‌ی ساب‌لیست‌ها (`inputs.model_directory`) را می‌سازد.

ورودی‌ها

`folder`	پوشه‌ی پروژه (مسیر کامل؛ اگر نیست ساخته می‌شود)
`name`	نام پروژه
`output_format`	`smart_data` یا `param_table`

خروجی: مسیر `pl_project.json` تازه.

۱۷.۳ `config_file.py` — نوشتن فهرست ساب‌لیست‌ها در pl_project.json، با حفظ کامنت‌ها

مسیر: `paramgen/src/pl_gui/config_file.py`

ویرایشگر اجازه می‌دهد کاربر انتخاب کند **کدام** ساب‌لیست‌ها در ساخت باشند و **به چه ترتیبی**؛ و این انتخاب باید در فایل پروژه نوشته شود، چون `run_generate.py` همان را می‌خواند. راه ساده `json.load` و `json.dump` بود، ولی فایل پروژه ممکن است کامنت داشته باشد (JSONC، فصل ۶) و یک بار رفت‌وبرگشت با `json` همه‌ی کامنت‌ها را بی‌صدا پاک می‌کرد. پس این ماژول **متن** فایل را ویرایش می‌کند: فقط همان یک آرایه را عوض می‌کند و به بقیه دست نمی‌زند.

نمونه: فایل پیش و پس از اینکه کاربر ترتیب را عوض کند و `motors.xml` را اضافه کند:

```
  "inputs": {
    // the order is the order of the indices
    "model_files": [
      "core.xml",
      "power.xml"   // power board
    ],

  ... بعد از rewrite_model_files(path, ["power.xml", "core.xml", "motors.xml"]) ...

  "inputs": {
    // the order is the order of the indices
    "model_files": [
      "power.xml",
      "core.xml",
      "motors.xml"
      // power board
    ],
```

کامنت بیرون آرایه دست نخورده ماند؛ کامنتی که درون آرایه بود، زیر سطرهای تازه آمد (پس از جابه‌جایی، سطری نمانده که کامنت دنباله‌اش کنارش بنشیند). خروجی JSON ساده است: ویرگول بین مدخل‌ها و نه بعد از آخری.

تابع `rewrite_model_files`

```
def rewrite_model_files(config_path: Path, names: List[str], *,
                        key: str = "model_files", indent: str = "  ") -> None:
```

آرایه‌ی `inputs.model_files` فایل را با `names` جایگزین می‌کند.

ورودی‌ها

`config_path`	مسیر `pl_project.json`
`names`	نام یا مسیر ساب‌لیست‌ها، به ترتیب
`key`	نام کلید آرایه؛ پیش‌فرض `model_files`
`indent`	تورفتگی هر سطح

خطا: `CodeGeneratorError` اگر آرایه پیدا نشود.

`_array_span` تابع

```
def _array_span(text: str, key: str) -> Optional[Tuple[int, int]]:
```

اندیس `]` و `[` جفتش در `"key": [ ... ]`. هر دو نیمه با **راه رفتن روی متن** پیدا می‌شوند نه با عبارت منظم: فایل پر از کامنت است و یک `// "model_files": [ ... ]` کامنت‌شده (یادداشتی که کسی بعد از امتحان چیزی جا می‌گذارد) عیناً شبیه تنظیم واقعی است. نوشتن درون آن کامنت فایلی می‌ساخت که دیگر خوانده نمی‌شد.

کار	نام
همه‌ی کامنت‌های درون آرایه، به ترتیب. `//` درون یک رشته (مسیر) کامنت نیست؛ پس متن پیموده می‌شود، نه شکسته.	`_comments_in`
اندیس `]` بعد از کلید واقعی (نه کامنت‌شده): کلید، بعد `:`، بعد `[`.	`_find_key`
اندیس پایان یک کامنت `//` (سطرشکن).	`_skip_line_comment`
اندیس بعد از `*/` یک کامنت بلوکی.	`_skip_block_comment`
اندیس بعد از رشته‌ی JSON، با رعایت `\"`.	`_skip_string`
فاصله‌های ابتدای سطری که یک اندیس در آن است.	`_line_indent`

۱۸. سند در حال ویرایش: document.py

`document.py` بزرگ‌ترین و مهم‌ترین فایل ویرایشگر است: همه‌ی کارهای صفحه از کلاس `Document` می‌گذرد، پس **قاعده‌های یک ویرایش معتبر همه در یک جا هستند**. اگر می‌خواهید رفتار ویرایشگر را عوض کنید (چه چیزی مجاز است، پیش‌فرض یک پارامتر تازه چیست، کپی چه نامی می‌گیرد)، تقریباً همیشه جایش همین فایل است. نمونه‌های این فصل همه با اجرای واقعی کد روی کپی پروژه‌ی نمونه گرفته شده‌اند.

۱۸.۱ `document.py` — سند: فهرست باز، فایلش و Undo

مسیر: `paramgen/src/pl_gui/document.py`

ثابت‌ها

ثابت	معنی
`UNDO_DEPTH`	۶۰: چند قدم Undo نگه داشته می‌شود
`NO_LIST`	پیامی که هر کار نیازمند ساب‌لیست، وقتی هیچ ساب‌لیستی باز نیست، می‌دهد
`EDITABLE_FIELDS`	فیلدهای `Parameter` که فرم **مجاز** است بنویسد (همان `data-field`های `editor.js`): `user_name`، `param_type`، `value_type_name`، `array_size`، `unit`، `enable`، `default_value`، `minimum`، `maximum`، `resolution`، `log_id`، `description`، `address`. `kind` و `children` عمداً نیستند: مقدار غلط در آن‌ها بررسی را از کار می‌انداخت و چون بررسی درون `/api/state` اجرا می‌شود، کل ویرایشگر را
`DEFAULT_PARAMETER_TYPES`، `DEFAULT_SYSTEM_NAMES`، `DEFAULT_PARAM_MODES`	فهرست‌های پیش‌فرض قدیمی. **امروز هیچ کدی از آن‌ها استفاده نمی‌کند**؛ پیش‌فرض‌های واقعی در `templates/choices.json` هستند و `choices.py` می‌خواندشان (فصل ۹). عوض کردن این‌ها اثری ندارد
`_ELEMENT`	عبارت منظم یک تکه‌ی مسیر با اندیس: `Speed[3]` ← گروه ۱ `Speed`، گروه ۲ `3`
`_PATH_CHARACTERS`	`.`، `[` و `]`: نویسه‌هایی که نحو مسیر از آن‌ها ساخته شده و در نام ممنوع‌اند
`_COPY_NAME`	عبارت منظم نامی که خودش کپی است: نام اصلی + `_Copy` + عدد اختیاری

`Document` کلاس

```
class Document:
```

فهرست در حافظه به‌علاوه‌ی هرچه ویرایشگر دور آن لازم دارد. ویژگی‌ها:

- `config`: تنظیمات پروژه (`Config`).
- `model`: فهرست (`ParameterList`).
- `path`: فایل ساب‌لیست باز، یا `None` (پروژه‌ی بی‌ساب‌لیست).

- `_saved_text`: متن فایل در آخرین خواندن یا نوشتن؛ `dirty` با آن مقایسه می‌کند.
- `load_problems`: آنچه **خواننده‌ی XML** درباره‌ی فایل باز گفته، تا نوشتن بعدی. این‌ها با بررسی دوباره‌ی مدل برنمی‌گردند (خواننده مقدار ناخوانا را دور ریخته و مدل تمیز به نظر می‌رسد) ولی هنوز در فایل‌اند و تولیدکننده فایل را می‌خواند.
- `_undo` و `_redo`: پشته‌های (برچسب، کپی مدل).

بار کردن و ذخیره

`Document.reread_project` متد

```
def reread_project(self) -> Optional[str]:
```

فایل پروژه را دوباره می‌خواند تا تغییری که بیرون ویرایشگر (در یک ویرایشگر متن) داده شده دیده شود؛ پیش‌تر ویرایشگر `Config` روز باز شدن را نگه می‌داشت و Generate در جای قدیمی می‌نوشت. اگر فایل خراب است، تنظیمات فعلی می‌ماند و دلیل برگردانده می‌شود.

خروجی: متن خطا، یا `None`.

`Document.load` متد

```
def load(self, path: Optional[Path] = None) -> List[Dict[str, object]]:
```

ساب‌لیست و فایل پروژه را از دیسک می‌خواند («Reload» یعنی پروژه همان‌طور که روی دیسک است). گام‌ها: `reread_project`؛ `read_xml` (یا فهرست خالی اگر فایل نیست)؛ `_ensure_defines`؛ خالی کردن Undo/Redo؛ نگه داشتن پیام‌های خواننده در `load_problems` (و اگر فایل پروژه خراب است، یک خطای `CFG001` در ابتدا)؛ در پروژه‌ی `param_table` نوشتن زیرسیستم در تگ هر پارامتر (`_stamp_system_names`) با هشدار `PTB008`؛ `_give_every_row_its_text`؛ و در پایان همه‌ی این‌ها «ذخیره‌شده» حساب می‌شوند تا صرفِ باز کردن، `*` نگذارد.

ورودی‌ها

`path`	ساب‌لیستی برای باز کردن؛ `None` یعنی همان `self.path`

خروجی: `load_problems` (لیست دیکشنری پیام‌ها).

`Document.save` متد

```
def save(self, path: Optional[Path] = None) -> Path:
```

تگ زیرسیستم را (در `param_table`) می‌نویسد، `write_xml` (فایل و تایپ‌هایش) و `_saved_text` و `load_problems` را تازه می‌کند.

خروجی: مسیر فایل نوشته‌شده.

خطا: `ValueError(NO_LIST)` اگر ساب‌لیستی نیست.

`Document.dirty` ویژگی (property)

```
@property
def dirty(self) -> bool:
```

آیا فهرست با فایلش فرق دارد: همان * کنار نام فایل. **مقایسه می‌شود، نه به خاطر سپرده**: پرچمی که هر ویرایش بالا می‌برد، ویرایشی را هم که فرم فقط با ترک یک خانه می‌فرستد می‌شمرد، و بعد از Undo که فهرست را دقیقاً به فایل برگردانده بود، بالا می‌ماند.

`Document._file_text` متد

```
def _file_text(self) -> bytes:
```

فهرست و تایپ‌هایش همان‌طور که ذخیره می‌نویسد، به‌صورت یک متن (بایت)، بی زمان. با همان سازنده‌های عنصرِ نویسنده‌ها (`model_to_element` و `types_to_element`) ساخته می‌شود.

کار	نام
فهرست‌های کشویی رایانه را در مدل می‌گذارد (`load_choices`)؛ این فهرست‌ها مال برنامه‌اند، نه پروژه.	`Document._ensure_defines`
در `param_table`: زیرسیستمی که نام فایل ساب‌لیست می‌گوید، با املای فهرست `SYSTEM NAMES`؛ وگرنه `None`.	`Document._list_system`
تگ `SystemName` هر پارامتر سطح بالا را برابر زیرسیستم فایل می‌کند و پارامترهای تغییرکرده را برمی‌گرداند. فایل تصمیم می‌گیرد؛ تگی که زیرسیستم دیگری می‌گوید هم اصلاح می‌شود.	`Document._stamp_system_names`

Undo و Redo

Undo ساده‌ترین شکل ممکن را دارد: پیش از هر ویرایش، **کل مدل** با `copy.deepcopy` کپی و در پشته گذاشته می‌شود (بخش ۴.۱۷). برای فهرست‌های چند صد پارامتری سریع است و هیچ ویرایشی نمی‌تواند «Undo ناقص» داشته باشد.

کار	نام
پیش از تغییر: (برچسب، کپی مدل) را در `_undo` می‌گذارد، آن را به `UNDO_DEPTH` کوتاه و `_redo` را خالی می‌کند.	`Document.snapshot`
مدل را با آخرین کپی عوض می‌کند و مدل فعلی را در `_redo` می‌گذارد؛ برچسب یا `None`.	`Document.undo`
برعکس `undo`.	`Document.redo`
آیا قدمی برای Undo هست (property).	`Document.can_undo`
آیا قدمی برای Redo هست (property).	`Document.can_redo`

`Document._change` متد

```
@contextlib.contextmanager
def _change(self, label: str) -> Iterator[None]:
```

یک **مدیر زمینه** (`@contextlib.contextmanager`، بخش ۴.۱۲) که یک ویرایش را «همه یا هیچ» می‌کند:

- اول کپی پشته‌ها و متن فایل، و `snapshot`.
- بدنه‌ی `with` اجرا می‌شود.
- اگر بدنه استثنا داد: مدل و هر دو پشته **دقیقاً** به حال قبل برمی‌گردند و استثنا دوباره پرتاب می‌شود (تا صفحه پیامش را نشان دهد).
- اگر موفق بود: `_give_every_row_its_text`، و اگر متن فایل عوض نشده، قدم Undo پاک می‌شود؛ چون فرم با ترک هر خانه همه‌ی خانه‌ها را می‌فرستد و کلیک روی یک پارامتر پشته را از قدم‌های بی‌اثر پر می‌کرد.

مثال:

```
with self._change("delete"):
    siblings.remove(node)      # اگر اینجا خطا شود، چیزی عوض نشده است
```

برخی متدها (`add_parameter`، `duplicate`، `move`، `delete_struct_type`) هنوز به‌جای `_change` مستقیم `snapshot` را صدا می‌زنند؛ کارشان چیزی نیست که نیمه‌کاره رد شود. ویرایش تازه‌ای که ممکن است وسط کار رد شود، **باید** درون `_change` باشد.

پیدا کردن و ویرایش درخت

کار	نام
آیا ساب‌لیستی باز است (property).	`Document.has_list`
اگر ساب‌لیستی نیست، `ValueError(NO_LIST)`. قبلاً افزودن پارامتر به پروژه‌ی بی‌ساب‌لیست فقط در حافظه انجام می‌شد و Generate از پارامترهایی که روی دیسک نبودند کد می‌ساخت.	`Document._require_a_list`
گره‌ی یک مسیر نقطه‌دار (`model.find`)، یا `None`.	`Document.find`
(لیستی که گره در آن است، `None` گره پدر یا).	`Document.parent_of`

`Document.add_parameter` متد

```
def add_parameter(self, parent_path: str = "", kind: str = "primitive") -> str:
```

یک پارامتر یا استراکچر تازه زیر `parent_path` اضافه می‌کند. نام از `_unique_name` (`NewParameter`، `NewParameter2`، ...)، نوع داده `uint8_t` با `_give_starting_values`؛ و در سطح بالا Type اولِ فهرست `TYPE`، تگ‌های پیش‌فرض (`_default_tag`؛ حالت دسترسی اولین مقدار فهرست `PARAM MODES` یعنی `PARAM_MODE_NONE`)، در `param_table` زیرسیستم فایل، و در smart data تگ‌هایی که قالب ثابت می‌کند (`fixed_tags`، فصل ۱۴) - پس پارامتر تازه با `PARAM_MODE_NONE` ذخیره می‌شود.

خروجی: مسیر گره‌ی تازه.

مثال:

```
document.add_parameter("")        # 'NewParameter'
document.add_parameter("")        # 'NewParameter2'
document.add_parameter("Motor")
# ValueError: A field cannot be added to 'Motor' on its own: its fields
# belong to the structure type 'sMotor', ...
```

`Document._refuse_structures_in_a_table` متد

```
def _refuse_structures_in_a_table(self) -> None:
```

در پروژه‌ی `param_table` افزودن استراکچر را رد می‌کند.

`Document._refuse_reshaping` متد

```
def _refuse_reshaping(self, structure: Optional[Parameter], done: str) -> None:
```

افزودن، پاک کردن، کپی یا جابه‌جایی **یک فیلد** استراکچر را رد می‌کند؛ فیلدها مال **تایپ** هستند و هر پارامتر آن تایپ همان‌ها را دارد. این ویرایش‌ها قبلاً «انجام می‌شدند» و با اعمال دوباره‌ی تایپ دور ریخته می‌شدند. استراکچری که تایپش گم شده، هنوز اجازه‌ی پاک‌سازی دارد.

`Document.duplicate` متد

```
def duplicate(self, path: str) -> Optional[str]:
```

کپی عمیق گره، درست بعد از خودش، با نام `_copy_name`، **بی شناسه‌ی لاگ** (`_clear_log_ids`) و با آدرس صفر.

مثال:

```
document.duplicate("Airspeed")         # 'Airspeed_Copy'
document.duplicate("Airspeed")         # 'Airspeed_Copy2'
document.duplicate("Airspeed_Copy")    # 'Airspeed_Copy3'
```

`Document.delete` متد

```
def delete(self, path: str) -> bool:
```

گره را پاک می‌کند، **و هرگز تایپ استراکچرش را**: تایپ مال ساب‌لیست است تا وقتی کسی در **Structure types...** پاکش کند؛ تایپ بی‌استفاده هزینه‌ای ندارد.

خروجی: `True` اگر چیزی پاک شد.

`Document.move` متد

```
def move(self, path: str, offset: int) -> Optional[str]:
```

گره را `offset` خانه (مثبت پایین، منفی بالا) میان خواهر و برادرهایش جابه‌جا می‌کند؛ بیرون از لیست ← `None`. یادتان باشد جابه‌جایی آفست‌ها را عوض می‌کند (فصل ۱۳).

ردیف‌ها، اعلان‌ها و مقدارها

جدول ویرایشگر برای هر **مقدار** یک ردیف دارد (`Motor[1].Rpm`، `CellVoltage[2]`)، ولی درخت برای هر **اعلان** یک گره. بیشتر خانه‌ها مال اعلان‌اند و تغییرشان به همه‌ی عنصرها می‌رسد؛ فقط **نام نمایشی** و **توضیح** مال خود ردیف‌اند، و **شناسه‌ی لاگ** کل بلوک را جابه‌جا می‌کند.

`Document.value_at` متد

```
def value_at(self, path: str) -> Tuple[Optional[Parameter], Tuple[int, ...]]:
```

اعلانی که یک مسیر **مقدار** نام می‌برد، و اینکه کدام عنصر آن است. مسیر بی‌اندیس یک اعلان است و با مسیر عنصری خالی برمی‌گردد.

مثال:

```
node, indices = document.value_at("Motor[1].Rpm")
node.name, indices     # ('Rpm', (1,))
```

`Document.update` متد

```
def update(self, path: str, changes: Dict[str, object]) -> Optional[str]:
```

مقدارهای فرم را اعمال می‌کند، همه درون `_change("edit")`. اگر مسیر اندیس دارد، `_apply_to_one_value`؛ وگرنه `_apply_update`.

ورودی‌ها

`path`	مسیر ردیف
`changes`	دیکشنری فیلد ← مقدار، از فرم

خروجی: مسیر تازه (پس از تغییر نام)، یا `None` اگر مسیر نبود.

خطا: `ValueError` با جمله‌ای که دلیل رد را می‌گوید.

مثال:

```
document.update("Airspeed", {"unit": "km/h"})       # 'Airspeed'
document.update("Airspeed", {"name": "Air.speed"})
# ValueError: A name cannot contain '.'. ...
document.update("Airspeed", {"name": "MaxAltitude"})
# ValueError: 'MaxAltitude' is already used here. ...
```

Document._apply_to_one_value متد

```
def _apply_to_one_value(self, path: str, node: Parameter,
                        indices: Tuple[int, ...],
                        changes: Dict[str, object]) -> Optional[str]:
```

ویرایش روی یک ردیف اندیس‌دار: description و user_name مال همین ردیف‌اند (_describe_one_value و _name_one_value)، log_id بلوک را جابه‌جا می‌کند (_move_the_log_block) و هر چیز دیگر به اعلان می‌رود (_apply_update).

Document._describe_one_value متد

```
def _describe_one_value(self, path: str, node: Parameter,
                        indices: Tuple[int, ...], text: object) -> None:
```

به یک ردیف توضیح خودش را می‌دهد یا پس می‌گیرد. خالی کردن خانه، **یا** تایپ کردن دقیقاً همان متنی که ردیف از بالا می‌خواند، ردیف را پس می‌دهد (بی‌متن خودش). همین است که ویرایش عادی را عادی نگه می‌دارد: فرم همه‌ی خانه‌ها را می‌فرستد و ردیفِ بی‌توضیح نباید به‌خاطر ویرایش واحدش توضیح بگیرد.

Document._name_one_value متد

```
def _name_one_value(self, path: str, node: Parameter, indices: Tuple[int, ...],
                    text: object) -> None:
```

همان قاعده برای نام نمایشی.

Document.row_chain متد

```
def row_chain(self, path: str) -> List[Tuple[Parameter, Tuple[int, ...]]]:
```

همه‌ی ردیف‌ها از مدخل سطح بالا تا ردیفی که مسیر نام می‌برد، هرکدام با مسیر عنصری‌اش.

مثال:

```
[(n.name, i) for n, i in document.row_chain("Motor[1].Rpm")]
# [('Motor', (1,)), ('Rpm', (1,))]
```

Document.text_above متد

```
def text_above(self, path: str, field: str) -> str:
```

آنچه یک ردیف برای user_name یا description می‌خواند وقتی خودش چیزی ندارد: نزدیک‌ترین ردیف بالایی، همان‌طور که flatten() می‌خواند.

مثال:

```
document.text_above("Motor[1].Rpm", "user_name")     # 'Motor'
document.text_above("CellVoltage[2]", "user_name")   # 'Cell voltage'
```

`Document._give_every_row_its_text` متد

```
def _give_every_row_its_text(self) -> None:
```

متنی که یک فیلد میان همه‌ی عنصرهای یک آرایه به اشتراک داشت، روی تک‌تک ردیف‌ها نوشته می‌شود (جز ردیفی که متن خودش را دارد) و متن مشترک پاک می‌شود. آنچه هر مقدار می‌خواند عیناً همان می‌ماند؛ پس هیچ خروجی‌ای عوض نمی‌شود. با `itertools.product` همه‌ی ترکیب‌های اندیس ساخته می‌شوند.

`Document._move_the_log_block` متد

```
def _move_the_log_block(self, path: str, wanted: object) -> None:
```

شناسه‌ی لاگ **یک مقدار** را با جابه‌جا کردن بلوکی که در آن است تنظیم می‌کند: شناسه‌های یک مدخل پشت سر هم‌اند، پس مقدار وسطی نمی‌تواند عدد خودش را داشته باشد؛ ولی کل بلوک جابه‌جا می‌شود. خانه‌ی خالی یعنی «بی‌شناسه» برای کل بلوک.

مثال:

```
# CellVoltage شناسه‌های 200..203 دارد
document.update("CellVoltage[2]", {"log_id": 210})
# حالا 208, 209, 210, 211
document.update("CellVoltage[2]", {"log_id": 1})
# ValueError: 'CellVoltage[2]' is value 3 of 'CellVoltage', so a log id
# of 1 would start the block at -1. ...
```

`Document._offset_in_the_block` متد

```
def _offset_in_the_block(self, root_name: str, path: str) -> Optional[int]:
```

چند مقدار از آن مدخل پیش از این مسیر می‌آیند؛ `None` اگر مقدار خاموش است.

اعمال یک ویرایش: _apply_update

`Document._apply_update` متد

```
def _apply_update(self, path: str, node: Parameter,
                  changes: Dict[str, object]) -> Optional[str]:
```

کار اصلی `update`، درون `_change`. به ترتیب:

- در `param_table`، اندازه‌ی آرایه‌ی بیشتر از ۱ رد می‌شود.
- برای فیلد استراکچر (مسیر نقطه‌دار): `tags` و `param_type` دور ریخته می‌شوند (مال سطح بالا هستند) و تغییر `name`، `value_type_name`، `array_size` یا `enable` **رد** می‌شود با پیامی که به تایپ اشاره می‌کند. واحد و توضیح مجازند، چون معنای **این استفاده** از تایپ‌اند.
- تغییر نام: `_reject_path_characters` و `_reject_duplicate_name`؛ مسیر تازه ساخته می‌شود.
- تگ‌ها: در `param_table` تگ‌های `READ_ONLY_TAGS` (`SystemName`) و در smart data تگ‌هایی که قالب ثابت می‌کند (`fixed_tags`: `ParamMode`) همانی می‌مانند که در فایل بودند، هرچه فرستاده شود (فرمی که خانه‌ای

برای تگ ندارد، مقداری نمی‌فرستد و این خط آن را **پاک** می‌کرد؛ و صفحه‌ای که از قبل باز مانده ممکن است مقدار دیگری بفرستد)؛ تگ آزادِ متنی با `convert_tag` به همان شکلی درمی‌آید که خواننده‌ی XML می‌سازد.

- `address` با `_checked_address`؛ هر کلید `EDITABLE_FIELDS` با `setattr`.
- دو دنیا جدا می‌مانند: استراکچر نوع داده نمی‌گیرد و پارامتر ساده تایپ استراکچر نمی‌گیرد.
- استراکچری که تایپ تازه گرفته، فوراً با `_resync` تغییر شکل می‌دهد.
- تغییر نوع داده ← `_retune_for_type`؛ و در هر ویرایش `_hold_boolean_shape` و `_hold_number_shape`.

`Document._hold_boolean_shape` متد ایستا

```
@staticmethod
def _hold_boolean_shape(node: Parameter) -> None:
```

متد ایستا: نوع بولی را در تنها شکل ممکنش نگه می‌دارد: کمینه ۰، بیشینه ۱، رزولوشن ۱ و پیش‌فرض یک `bool` واقعی. فرم بولی خانه‌ای برای این سه ندارد؛ پس عددی که آنجا مانده بود هرگز دیده یا اصلاح نمی‌شد.

`Document._hold_number_shape` متد ایستا

```
@staticmethod
def _hold_number_shape(node: Parameter) -> None:
```

متد ایستا: JSON عدد صحیح را بی `.0` می‌نویسد؛ پس کمینه‌ی `0.0` یک `float32_t` از فرم به‌صورت `0` برمی‌گشت و فهرست «تغییرکرده» حساب می‌شد. نوع اعشاری اعشاری می‌ماند و نوع صحیح صحیح؛ همان چیزی که خواننده‌ی XML می‌سازد.

`Document._reject_duplicate_name` متد

```
def _reject_duplicate_name(self, path: str, node: Parameter, name: str) -> None:
```

نامی را که گره دیگری در همان سطح دارد رد می‌کند. گره با مسیرش شناخته می‌شود؛ دو هم‌نام یک مسیر دارند و هر جست‌وجو اولی را پیدا می‌کرد: تغییر نام یک پارامتر تازه به نام موجود و بعد پاک کردنش، **اصلی** را پاک می‌کرد.

`Document._retune_for_type` متد ایستا

```
@staticmethod
def _retune_for_type(node: Parameter, previous_type: Optional[str]) -> None:
```

متد ایستا: وقتی نوع داده عوض می‌شود، فیلدهایی را که به نوع وابسته‌اند به مقدارهایی می‌برد که نوع تازه می‌تواند نگه دارد. فقط آنچه برای نوع تازه **غلط** است عوض می‌شود:

اثر	تغییر
رزولوشنی که خالی یا برابر پیش‌فرض نوع قبلی بود، پیش‌فرض نوع تازه می‌شود؛ رزولوشن انتخابی کاربر می‌ماند	هر نوع
۰ / ۱ / ۱ و پیش‌فرض بولی	به `bool_t`
پیش‌فرض ۱ یا ۰، و «بی‌بازه» (کمینه و بیشینه ۰)	از `bool_t`
اگر کمینه یا بیشینه در نوع تازه جا نشود یا (برای نوع صحیح) صحیح نباشد، **هر دو** صفر (بی‌بازه)، نه بریده شدن به مرزی که کسی انتخاب نکرده؛ پیش‌فرضِ ناجور صفر؛ رزولوشن کسری روی نوع صحیح، پیش‌فرض نوع	عدد به عدد

ساب‌لیست‌های پروژه

پروژه یک فهرست است که در چند فایل (معمولاً یکی برای هر زیرماژول) تقسیم شده. همه در یک پوشه‌اند (`inputs.model_directory`) و فایل پروژه می‌گوید کدام‌ها و به چه ترتیبی در ساخت باشند؛ **ترتیب همان ترتیب اندیس‌ها در کد تولیدی است**. ویرایشگر این انتخاب را در `pl_project.json` می‌نویسد، چون خط فرمان همان را می‌خواند.

کار	نام
پوشه‌ی ساب‌لیست‌ها.	`Document.list_directory`
همه‌ی ساب‌لیست‌ها: اول انتخاب‌شده‌ها به ترتیب ساخت، بعد فایل‌های دیگر پوشه (انتخاب‌نشده). فایل `_types.xml` ساب‌لیست نیست.	`Document.known_lists`
دیکشنری یک ردیف پنجره: `path`، `name`، `file`، `selected`، `order`، `exists`، `current`، `inFolder`.	`Document._list_entry`

`Document._admit_to_the_build` متد

```
def _admit_to_the_build(self, build: List[Path], adding: List[Path]) -> None:
```

در `param_table`، ساب‌لیستی را که ساخت نمی‌تواند بسازد پیش از نوشتن یا افزودن رد می‌کند: نام فایلی که زیرسیستمی را نام نمی‌برد، یا زیرسیستمی که از قبل ساب‌لیست دارد (دو بلوک برای یک زیرسیستم که هر دو از صفر شماره می‌خورند). فقط ساب‌لیست‌های **تازه** داوری می‌شوند.

`Document.create_list` متد

```
def create_list(self, name: str, directory: str = "") -> Path:
```

ساب‌لیست خالی تازه‌ای می‌سازد، **فوراً روی دیسک** می‌نویسد و به ساخت اضافه می‌کند. نام از `_safe_stem` می‌گذرد؛ در `param_table` نام فایل `<system>_paramTable.xml` می‌شود.

خروجی: مسیر فایل تازه.

خطا: `ValueError` (نام نامعتبر)، `FileExistsError` (فایل هست).

`Document.inspect_list` متد

```
def inspect_list(self, path: str) -> Dict[str, object]:
```

دروازه‌ی جلوی «افزودن فایل موجود»: آیا این مسیر ساب‌لیستی است که ابزار می‌تواند بسازد؟ فایل `_types.xml`، فایل غیر `.xml`، XML خراب یا ریشه‌ی غلط، و هر دلیل `_why_not_added` رد می‌شود؛ وگرنه خلاصه‌ای از محتوا (`values`، `parameters`، `structures`، `types`) و یادداشت‌ها (مثلاً تایپی که در پروژه هم‌نام دارد).

خروجی: دیکشنری `{"ok": ..., "reason": ...}` یا خلاصه.

`Document.add_existing_list` متد

```
def add_existing_list(self, path: str) -> Path:
```

ساب‌لیست موجودی را به ساخت می‌آورد. اگر از پروژه‌ی دیگری است، **کپی** می‌شود (همراه فایل تایپ‌هایش) تا پروژه پوشه‌ای قابل‌جابه‌جایی بماند و دو پروژه مستقل بمانند. اگر از قبل در ساخت است، فقط همان برگردانده می‌شود (تا باز شود).

کار	نام
دلیل اینکه فایل به این ساخت اضافه نمی‌شود، یا `None`؛ یک جا تا بررسی پنجره و دکمه‌ی OK هرگز با هم اختلاف نداشته باشند.	`Document._why_not_added`
فایل (و تایپ‌هایش، با هر نامی که دارد) را در پوشه‌ی ساب‌لیست‌ها کپی می‌کند؛ هم‌نام موجود بازنویسی نمی‌شود (`_2`، `_3`، …). فایلی که زیر ریشه‌ی پروژه است، همان‌جا می‌ماند.	`Document._bring_into_the_project`

`Document.remove_list` متد

```
def remove_list(self, path: str, delete_file: bool = False) -> Dict[str, object]:
```

ساب‌لیستی را از ساخت بیرون می‌برد و در صورت `delete_file` فایلش (و فایل تایپ‌هایش با هر دو نام) را پاک می‌کند. ساخت بی هیچ ساب‌لیست مجاز نیست. اگر ساب‌لیست باز بود، می‌گوید کدام را به‌جایش باز کنیم.

خروجی: `{"removed", "deleted", "wasOpen", "openInstead"}`.

`Document.browse` متد

```
def browse(self, directory: str = "") -> Dict[str, object]:
```

زیرپوشه‌ها و فایل‌های `.xml` یک پوشه برای پنجره‌ی درون‌صفحه‌ای Open (مرورگر مسیر واقعی ندارد؛ پس پیمایش در سرور انجام می‌شود). هر فایل `inBuild` دارد و جواب، درایوهای رایانه (`machine_roots`) را هم دارد.

`Document.select_lists` متد

```
def select_lists(self, paths: List[str]) -> List[Path]:
```

تعیین می‌کند کدام ساب‌لیست‌ها و به چه ترتیبی در ساخت باشند: تکراری‌ها حذف (اولی می‌ماند)، `_admit_to_the_build` برای تازه‌ها، فایل درون پوشه‌ی ساب‌لیست‌ها با نام خالی و بقیه با مسیر نسبی، سپس `rewrite_model_files` (فصل ۱۷) **و** همان تغییر در `config` این نشست، تا بی راه‌اندازی دوباره اثر کند.

تایپ‌های استراکچر

تایپ مثل اعلان `struct` در C++ است. ویرایش تایپ، هر پارامتری را که آن را نام می‌برد ویرایش می‌کند: فیلدی اضافه شود، همه‌ی آن‌ها در جای درست رشد می‌کنند. **هر تایپ مال ساب‌لیستی است که اعلانش کرده**: فقط تایپ‌های ساب‌لیست باز نشان داده، ویرایش و استفاده می‌شوند، و نامی که ساب‌لیست دیگری دارد همان‌جا که تایپ می‌شود رد می‌شود.

کار	نام
تایپ‌های ساب‌لیست باز برای پنجره‌ی **Structure types...**: نام، فیلدها و `usedHere`.	`Document.struct_types`
مسیر استراکچرهای ساب‌لیست باز که از این تایپ‌اند.	`Document._uses_of_type`
یکی از تایپ‌های خود ساب‌لیست باز، یا `None`.	`Document.find_struct_type`
نام تایپ تازه، یا رد: خالی، شناسه‌ی نامعتبر C، نام یک نوع داده، تکراری، یا نامی که ساب‌لیست دیگری دارد.	`Document._new_type_name`
رد نامی که ساب‌لیست دیگری از ساخت دارد (به‌جای `TYP016` بعداً).	`Document._refuse_a_name_another_list_has`
یک فیلد از دیکشنری پنجره (`name`، `valueType`، `arraySize`، `enable`) به `StructField`؛ نام باید شناسه‌ی C باشد.	`Document._field_from`
تغییر نام تایپ و هر اشاره‌ای به آن در فیلدهای تایپ‌های دیگر و در استراکچرهای ساب‌لیست.	`Document._rename_struct_type`
کدام ساب‌لیست **دیگر** ساخت این نام را اعلان کرده (از فایل تایپ‌هایش).	`Document._list_defining_type`
گره را استراکچرِ آن تایپ می‌کند؛ واحد، پیش‌فرض و بازه پاک می‌شوند و `_resync`.	`Document._give_type`
همه‌ی فیلدهای همه‌ی استراکچرهای ساب‌لیست باز، در هر عمقی (generator، بخش ۴.۶).	`Document._fields`

متد `Document.create_struct_type`

```
def create_struct_type(self, name: str,
                       fields: Optional[List[Dict[str, object]]] = None) -> str:
```

تایپ تازه‌ای اعلان می‌کند، با فیلدهایش اگر داده شده؛ نام و فیلدها **با هم** بررسی می‌شوند، پیش از اینکه چیزی اعلان شود (قبلاً اول تایپ خالی ساخته می‌شد و اگر فیلدی رد می‌شد، تایپ خالی می‌ماند).

خروجی: نام تایپ.

متد `Document.update_struct_type`

```
def update_struct_type(self, name: str, changes: Dict[str, object]) -> str:
```

تایپی را تغییر می‌دهد (`changes` می‌تواند `name` و `fields` داشته باشد) و هر پارامتر آن را تغییر شکل می‌دهد. لیست فیلدها **کل حقیقت** است: هرچه در آن نیست حذف شده و ترتیبش ترتیب عضوهای C است. همه‌چیز پیش از تغییر بررسی می‌شود.

خروجی: نام (احتمالاً تازه‌ی) تایپ.

متد `Document._fields_from`

```
def _fields_from(self, raw: object) -> List[StructField]:
```

فیلدهای یک تایپ، به‌صورت یکجا بررسی‌شده: هرکدام نام‌دار و شناسه‌ی C، بی‌تکرار، با نوع داده یا تایپی از **همین** ساب‌لیست، و دست‌کم یک فیلد.

متد `Document.delete_struct_type`

```
def delete_struct_type(self, name: str) -> Tuple[bool, str]:
```

تایپی را حذف می‌کند، مگر اینکه پارامتری از آن استفاده کند یا فیلدی از تایپ دیگری باشد.

خروجی: `(حذف شد؟، دلیل اگر نه)`.

متد `Document.use_struct_type`

```
def use_struct_type(self, path: str, name: str) -> Optional[str]:
```

پارامتر یک مسیر را استراکچرِ تایپ داده‌شده می‌کند.

خروجی: مسیر، یا `None` اگر گره یا تایپ نبود.

متد `Document.add_structure`

```
def add_structure(self, name: str, type_name: str,
                  fields: Optional[List[Dict[str, object]]] = None) -> str:
```

یک استراکچر از یک تایپ اضافه می‌کند؛ اگر `fields` داده شده، تایپ را هم اعلان می‌کند. تگ‌هایش مثل `add_parameter` (با `fixed_tags`). یک قدم، بررسی‌شده به‌صورت کامل پیش از هر تغییر، و یک قدم Undo. (پنجره قبلاً پنج درخواست جدا می‌فرستاد و ردِ میانه حالت‌های شکسته باقی می‌گذاشت.)

خروجی: نام استراکچر تازه.

`Document._resync` متد

```
def _resync(self) -> None:
```

همه‌ی استراکچرهای ساب‌لیست باز را دوباره با تایپشان جور می‌کند (`apply_types`، فصل ۱۰)، **با تایپ‌های خود این ساب‌لیست**. فیلدی که تازه آمده مقدارهای شروع می‌گیرد (`_give_starting_values`) و فیلدی که نوعش عوض شده `_retune_for_type`. فیلدهای قدیمی همان مقدارهایشان را نگه می‌دارند.

`Document._give_starting_values` متد کلاس

```
@classmethod
def _give_starting_values(cls, node: Parameter) -> None:
```

متد کلاس: مقدارهای شروع یک مقدار تازه، تا بی‌هشدار شروع شود: پیش‌فرض ۰، بی‌بازه (۰ و ۰) و رزولوشن نوع؛ بولی `FALSE` با ۱/۱/۰. استراکچر چیزی نمی‌گیرد. **اگر شرکت بخواهد پارامتر تازه با مقدارهای دیگری شروع شود، اینجا عوض کنید** (فصل ۲۴).

`Document.set_defines` متد

```
def set_defines(self, column: str, values: List[str]) -> None:
```

یکی از فهرست‌های کشویی رایانه را عوض می‌کند و در `paramgen/var/choices.json` می‌نویسد (`save_choices`). عمداً روی پشته‌ی Undo **نیست**: Undo کردن یک ویرایش فهرست نباید بی‌صدا فهرست زیرسیستم‌های شرکت را برگرداند. برای System names و Parameter modes (`GENERATED_DEFINES`) سطرها `SYS_NAV = 16` هستند و با `parse_numbered` (فصل ۹) به نام‌ها و عددها تبدیل می‌شوند؛ سطری که خوانده نشود `ValueError` می‌دهد **پیش از** هر تغییر.

مثال:

```
document.set_defines("SYSTEM NAMES", ["SYS_UNKNOWN = 0", "SYS_NAV = 16", "SYS_NEXT"])
document.model.defines.numbered("SYSTEM NAMES")
# [('SYS_UNKNOWN', 0), ('SYS_NAV', 16), ('SYS_NEXT', 17)]
```

بررسی و تولید

`Document.merged_model` متد

```
def merged_model(self, diagnostics: Optional[DiagnosticCollector] = None
                 ) -> ParameterList:
```

این ساب‌لیست همان‌طور که در حال ویرایش است، به‌علاوه‌ی بقیه همان‌طور که روی دیسک‌اند، ادغام با همان `merge_lists` تولیدکننده و به ترتیب `inputs.model_files`؛ تا برخورد نام یا شناسه‌ی لاگ با ساب‌لیست دیگر **هنگام ویرایش** دیده شود و جای هر پارامتر همان اندیس واقعی‌اش باشد. در پایان `apply_fixed_tags` (فصل ۱۴)، مثل خط‌لوله؛ همه‌ی بخش‌ها کپی‌اند، و اگر چیزی برای ادغام نبود و مدل همان ساب‌لیست باز است، اول یک کپی گرفته می‌شود تا ساب‌لیست مقدار خودش را نگه دارد.

`Document.merged_parts` متد

```
def merged_parts(self, diagnostics: Optional[DiagnosticCollector] = None
                 ) -> List[Tuple[Path, ParameterList]]:
```

همان ساب‌لیست‌ها، هنوز جدا: `(فایل، فهرست)` برای هرکدام. ساب‌لیست باز از حافظه (کپی)، بقیه از دیسک. مسیرها resolve می‌شوند تا یک فایل با دو مسیر دو بار ادغام نشود. پیام‌های خواننده درباره‌ی ساب‌لیست‌های دیگر هم (اگر جمع‌کننده داده شود) به پنل **Problems** می‌رسد.

`Document.validate` متد

```
def validate(self) -> List[Dict[str, object]]:
```

همه‌ی مشکل‌های کل ساخت، همان‌طور که پنل **Problems** نشان می‌دهد: `load_problems` + پیام‌های خواننده درباره‌ی ساب‌لیست‌های دیگر + قاعده‌های بررسی روی مدل ادغام‌شده.

خروجی: لیست دیکشنری‌ها (`severity`، `code`، `message`، `location`، `hint`).

`Document.validate_codes` متد

```
def validate_codes(self, code: str) -> int:
```

چند پیام با یک کد؛ آزمون‌ها مدام می‌پرسند «آیا این گزارش شده، و فقط یک بار؟».

مثال:

```
document.validate_codes("TAG012")   # 0
```

`Document.generate` متد

```
def generate(self) -> Dict[str, object]:
```

اول فایل پروژه را دوباره می‌خواند، ساب‌لیست باز را **ذخیره** می‌کند، بعد همه را از دیسک می‌خواند (`Pipeline.load_model`) و `Pipeline.generate`؛ پس ویرایشگر و خط فرمان هرگز دو کد متفاوت نمی‌سازند.

خروجی: دیکشنری `success`، `message` (یک سطر، از `_refusal`)، `files`، `count`، `saved`، `modules`، `diagnostics` و `drift` (نسخه‌ی JSON شده‌ی `LayoutDrift`).

`Document.next_free_log_id` متد

```
def next_free_log_id(self) -> int:
```

اولین شناسه‌ی لاگی که هیچ مقداری در **کل ساخت** نگرفته: بیشینه‌ی `log_id + reserved_log_ids` روی برگ‌های مدل ادغام‌شده. دکمه‌ی **Next free** همین است.

مثال:

```
document.next_free_log_id()   # 502  (آخرین: ChargeCurrent = 501)
```

`Document.error_count` متد

```
def error_count(self) -> int:
```

تعداد پیام‌های با شدت خطا.

تابع‌های کمکی بیرون کلاس

کار	نام
چرا ساخت چیزی ننوشت، در یک سطر برای toast: اولین خطا و تعداد بقیه.	`_refusal`
نام دارای `.` یا `]` یا `[` را **هنگام نام‌گذاری** رد می‌کند؛ گره‌ای با چنین نامی دیگر با مسیر پیدا نمی‌شد و حتی پاک هم نمی‌شد.	`_reject_path_characters`
رزولوشن شروع یک نوع، از جدول نوع‌ها؛ نوع ناشناخته ۱.	`_default_resolution`
مقدار شروع یک تگ: بولی `False`، enum اولین مقدار فهرستش، بقیه `None`.	`_default_tag`
ساب‌لیستِ صاحب یک فایل تایپ: `power_types.xml` ← `power.xml`.	`_list_of_a_types_file`
آیا دو مسیر یک فایل‌اند (هر دو resolve).	`_same_file`
`NewParameter`، بعد `NewParameter2`، `NewParameter3`، … میان خواهر و برادرها.	`_unique_name`
شناسه‌ی لاگ گره و همه‌ی فرزندانش را پاک می‌کند (برای کپی).	`_clear_log_ids`

`_checked_address` تابع

```
def _checked_address(raw: object) -> int:
```

آدرس همان‌طور که فرم می‌فرستد، یا رد. خالی یعنی ۰.

مثال:

```
_checked_address("12")   # 12
_checked_address("")     # 0
_checked_address(7.0)    # 7
_checked_address("1.5")  # ValueError: The address is a whole number ...
```

`machine_roots` تابع

```
def machine_roots() -> List[str]:
```

همه‌ی جاهایی که پیمایش می‌تواند از آن شروع شود: درایوهای ویندوز (`A:\` تا `Z:\` که آماده‌اند) یا `/` در بقیه. بالا رفتن از یک پوشه در ریشه‌ی **همان** درایو تمام می‌شد و پروژه‌ی روی درایو دیگر در دسترس نبود.

`listing_target` تابع

```
def listing_target(raw: str, default: Path) -> Tuple[Path, str, str]:
```

پنجره‌ها هنگام تایپ در خانه‌ی مسیر، هر حالت نیمه‌تایپ‌شده را باید بفهمند:

خروجی: (`""` پوشه‌ای که فهرست شود، فایلی که انتخاب شود یا `""`، مسیر تایپ‌شده‌ای که نیست یا).

نتیجه	چیزی که تایپ شده
پوشه‌ی شروع خود پنجره	هیچ
همان پوشه	یک پوشه
پوشه‌اش، با فایل انتخاب‌شده	یک فایل
نزدیک‌ترین پوشه‌ی موجود بالای آن، و مسیر به‌عنوان «missing» (پروژه‌ی تازه ممکن است در پوشه‌ای ساخته شود که هنوز نیست)	مسیری که نیست
پوشه‌ی شروع؛ نسبی به هر جایی که سرور اجرا شده معنایی ندارد	مسیر نسبی
گیومه‌ها حذف می‌شوند (کار «Copy as path» ویندوز)	با گیومه

`_as_written` تابع

```
def _as_written(declaration: str, like: str) -> str:
```

مسیر اعلان، با اندیس‌های یک مسیر عنصری که دوباره رویش گذاشته شده؛ برای تغییر نامی که وقتی یک عنصر باز بود انجام شد.

مثال:

```
_as_written("Motor.Speed", "Motor[2].Rpm")   # 'Motor[2].Speed'
```

`_safe_stem` تابع

```
def _safe_stem(name: str) -> str:
```

متن تایپ‌شده را به نام فایلی تبدیل می‌کند که غافلگیر نکند: فقط حرف و رقم لاتین و `_` و `-`؛ پس هیچ نامی با `..` یا `/` از پوشه‌ی پروژه بیرون نمی‌رود. حداکثر ۶۴ نویسه.

مثال:

```
_safe_stem("Power board!")   # 'Power_board'
_safe_stem("../x/y")         # 'x_y'
_safe_stem("برق")            # ''  (پس create_list پیام «English letters» می‌دهد)
```

`_copy_name` تابع

```
def _copy_name(name: str, siblings: List[Parameter]) -> str:
```

نام کپی، مثل مدیر فایل ولی به شکل شناسه‌ی C: `_Copy` و سپس عدد. کپیِ یک کپی از همان نام اصلی می‌شمارد (`Airspeed_Copy` ← `Airspeed_Copy2`، نه `Airspeed_Copy_Copy`). «آزاد» یعنی آزاد میان خواهر و برادرها. (`_unique_name` عدد را به کل نام می‌چسباند، پس کپی `Airspeed2` می‌شد `Airspeed22`؛ برای همین کپی تابع خودش را دارد.)

مثال:

```
_copy_name("Motor1", siblings)          # 'Motor1_Copy'
_copy_name("Speed_Copy3", siblings)     # 'Speed_Copy' یا اولین آزاد
```

۱۹. API ویرایشگر: api.py

`api.py` لایه‌ی نازک بین صفحه و `Document` است: **یک متد برای هر کار**، همه کوچک. هر متد یک دیکشنری (JSON درخواست) می‌گیرد و یک دیکشنری (JSON جواب) برمی‌گرداند. قاعده‌ها اینجا نیستند (در `document.py` هستند)؛ کار این لایه سه چیز است: خواندن ورودی با پیش‌فرض‌های امن (`str(request.get("path") or "")`)، صدا زدن متد درست `Document`، و تبدیل خطای قابل‌انتظار (`ValueError` و…) به `{"ok": false, "message": ...}` به‌جای خطای ۵۰۰.

۱۹.۱ `api.py` — یک متد برای هر کار صفحه

مسیر: `paramgen/src/pl_gui/api.py`

نمونه‌ی واقعی

سه درخواست روی کپی پروژه‌ی نمونه و جواب واقعی‌شان:

```
api.update({"path": "Airspeed", "changes": {"unit": "km/h"}})
# {'ok': True, 'selected': 'Airspeed'}

api.update({"path": "Airspeed", "changes": {"name": "Max.x"}})
# {'ok': False, 'message': "A name cannot contain '.'. Parameters are addressed
#   by their dotted path - Position.Home.Lat - so a name with '.' in it could
#   not be found again."}

api.next_log_id({})
# {'ok': True, 'value': 502}
```

در صفحه همین‌ها با `api.js` فرستاده می‌شوند: `await api.update("Airspeed", {unit: "km/h"})`.

ثابت CHOICES

هر فایل یا پوشه‌ای که ویرایشگر از کاربر می‌خواهد انتخاب کند: `نام ← (نوع پنجره، عنوان، انواع فایل)`. چهار مدخل: `project` (فایل پروژه؛ هر `*.json`، چون صاحب پروژه ممکن است `pl_project.json` را تغییر نام داده باشد)، `project-location` (پوشه‌ی پروژه‌ی تازه)، `sub-list` (یک ساب‌لیست موجود) و `sub-list-location` (پوشه‌ی ساب‌لیست تازه). جای شروع هر پنجره را `start_folder` می‌گوید.

`Api` کلاس

```
class Api:
```

همه‌ی کارهایی که صفحه می‌تواند بخواهد. `self.workspace` را نگه می‌دارد.

کار	نام
`Workspace` را نگه می‌دارد.	`Api.__init__`
سند پروژه‌ی باز؛ اگر پروژه‌ای باز نیست، `workspace.require()` با جمله‌ای خطا می‌دهد (property).	`Api.document`

`Api.handlers` متد

```
def handlers(self) -> Dict[str, Callable[[Dict[str, Any]], Dict[str, Any]]]:
```

دیکشنری **نام کار ← متد**؛ همان که `server.py` با نام بخش آخر نشانی (`/api/<نام>`) در آن جست‌وجو می‌کند. **کار تازه‌ای که اینجا نباشد، از صفحه در دسترس نیست.** ۳۱ مدخل دارد؛ جدول زیر همه را با متد `Document` پشت هرکدام نشان می‌دهد.

کار و متد Document	درخواست	متد Api	نام کار
کل وضعیت صفحه	`allLists`	`state`	`state`
`Workspace.open`	`path`	`open_project`	`open_project`
`Workspace.create` و `open`	`folder`، `name`، `format`	`new_project`	`new_project`
`Workspace.close`	—	`close_project`	`close_project`
فهرست پوشه برای پنجره‌ی پروژه	`directory`، `for`	`browse_projects`	`browse_projects`
پنجره‌ی سیستم (`native_dialog.ask`)	`what`، `start`	`choose`	`choose`
`Document.load`	`path`	`open_file`	`open`
`Document.save`	`path`	`save`	`save`
`Document.add_parameter`	`parent`، `kind`	`add`	`add`
`Document.add_structure`	`name`، `type`، `fields`	`add_structure`	`add_structure`
`Document.duplicate`	`path`	`duplicate`	`duplicate`
`Document.delete`	`path`	`delete`	`delete`
`Document.move`	`path`، `offset`	`move`	`move`
`Document.update`	`path`، `changes`	`update`	`update`
`Document.undo`	—	`undo`	`undo`
`Document.redo`	—	`redo`	`redo`
`Document.set_defines`	`list`، `values`	`set_defines`	`defines`
`default_choices()`	—	`default_defines`	`default_defines`
`Document.generate`	—	`generate`	`generate`
`Document.next_free_log_id`	—	`next_log_id`	`next_log_id`

کار و متد Document	درخواست	متد Api	نام کار
Document.known_lists	—	lists	lists
Document.create_list	name، directory	new_list	new_list
Document.select_lists	paths	select_lists	select_lists
Document.browse	directory	browse	browse
Document.inspect_list	path	inspect_list	inspect_list
Document.add_existing_list	path	add_existing_list	add_existing_list
Document.remove_list	path، delete	remove_list	remove_list
Document.create_struct_type	name، fields	new_type	new_type
Document.update_struct_type	name، changes	update_type	update_type
Document.delete_struct_type	name	delete_type	delete_type
Document.use_struct_type	path، name	use_type	use_type

دو نام با نام متدشان فرق دارند: open ← open_file (چون open تابع داخلی پایتون است) و defines ← set_defines. جواب بیشتر کارها {"ok": true/false, ...} است؛ state و generate شکل خودشان را دارند.

state: هرچه صفحه برای رسم لازم دارد

متد **Api.state**

```
def state(self, request: Dict[str, Any]) -> Dict[str, Any]:
```

همه‌ی آنچه صفحه برای رسم یک صفحه‌ی کامل لازم دارد. اگر پروژه‌ای باز نیست، صفحه‌ی شروع (پروژه‌های اخیر و project: null)؛ این هم یک حالت عادی ویرایشگر است، نه خطا. اگر allLists خواسته شده، _all_lists_state.

کلیدهای واقعی جواب:

محتوا	کلید
نام، نسخه و launcher (run.bat یا ./run.sh)	tool
_project_info(): نام، توضیح، قالب، فایل و پوشه‌ها	project
پروژه‌های اخیر (Workspace.recent)	recent
از _dialogs()	startFolders ،nativeDialogs
در param_table، زیرسیستمی که نام فایل باز می‌گوید	systemFromFile
مسیر کامل ساب‌لیست باز و همان نسبت به ریشه‌ی پروژه (برای نوار بالا)	file ،fileShown
برای * و دکمه‌های Undo/Redo	dirty ،canUndo ،canRedo
درخت: یک دیکشنری _node برای هر پارامتر سطح بالا	tree
Document.validate(): پیام‌های پنل Problems	diagnostics

محتوا	کلید
فهرست‌های کشویی: `paramTypes`، `valueTypes`، `systemNames`، `paramModes`	`defines`
عدد هر نام System names و Parameter modes، `[[نام, عدد], …]` (از `_numbers_of`)؛ پنجره‌ی Drop-down lists سطرها را با آن می‌نویسد	`defineNumbers`
تگ‌هایی که قالب ثابت می‌کند (`fixed_tags`، فصل ۱۴): در smart data `{"ParamMode": "PARAM_MODE_NONE"}`؛ فرم آن‌ها را فقط‌خواندنی نشان می‌دهد	`fixedTags`
تایپ‌های ساب‌لیست باز	`structTypes`
توصیف هر تگ (از `TAG_REGISTRY`) و شناسه‌ی لاگ؛ صفحه فرم تگ‌ها را از روی همین می‌سازد	`tagSpecs`، `logIdSpec`
پوشه‌ی خروجی و ساب‌لیست‌های پروژه	`outputDirectory`، `modules`

بخشی از گره‌ی واقعی `CellVoltage` در `tree`:

```
{"path": "CellVoltage", "name": "CellVoltage", "kind": "primitive",
 "valueType": "float32_t", "arraySize": 4, "unit": "V",
 "logId": 200, "logIds": [200, 201, 202, 203],
 "values": [{"index": 3, "name": "CellVoltage[0]", "logId": 200,
             "description": "Voltage of one battery cell.", "userName": "Cell voltage"},
            ... ],
 ...}
```

پیشنهاد: افزودن یک اطلاع تازه به صفحه (مثلاً «تعداد کل بایت‌های ResetProof») یعنی یک کلید تازه در دیکشنری `state` و خواندنش در `app.js` از `this.state.<کلید>`. هزینه‌ی `state` مهم است چون پس از **هر** کلیک اجرا می‌شود؛ قبلاً یک فهرست ۴۰۰تایی پس از هر کلیک ۱۸ ثانیه طول می‌کشید، چون برای هر ردیف کل ساخت دوباره flatten می‌شد. محاسبه‌ی سنگین را یک بار برای کل درخت انجام دهید (مثل `_log_ids_by_declaration`).

متد `Api._all_lists_state`

```
def _all_lists_state(self) -> Dict[str, Any]:
```

کل ساخت به‌صورت یک درخت **فقط‌خواندنی**، به ترتیب ساخت (نمای «همه‌ی ساب‌لیست‌ها»). هر گره `ownerList` دارد (ساب‌لیستی که از آن آمده، از روی **جایگاه**، نه نام) تا کلیک روی آن، همان ساب‌لیست را باز و ردیف را انتخاب کند. فقط‌خواندنی است چون ویرایش باید تصمیم بگیرد در کدام فایل بنویسد.

پروژه و پنجره‌ها

کار	نام
اطلاعات پروژه‌ی باز: `name`، `description`، `format`، `file`، `folder`، `listDirectory`، `outputDirectory`.	`Api._project_info`
`Workspace.open`؛ خطا ← `{ok: false, message}`.	`Api.open_project`
پروژه را می‌سازد **و باز می‌کند**؛ کسی پروژه نمی‌سازد که بسته بماند.	`Api.new_project`
`Workspace.close`.	`Api.close_project`
آنچه صفحه پیش از باز کردن هر پنجره لازم دارد: `nativeDialogs` و `startFolders`.	`Api._dialogs`
`Document.load` روی یک ساب‌لیست (یا همان باز)؛ پیام‌های خواننده را برمی‌گرداند.	`Api.open_file`
`Document.save`؛ پروژه‌ی بی‌ساب‌لیست ← پیامی برای آدم، نه خطای سرور.	`Api.save`

`Api.browse_projects` متد

```
def browse_projects(self, request: Dict[str, Any]) -> Dict[str, Any]:
```

پوشه‌ها و فایل‌های `.json` یک پوشه برای پنجره‌ی درون‌صفحه‌ای پروژه. فایلی که پروژه است و پوشه‌ای که پروژه دارد علامت می‌خورند (`isProject`، `projectFile`) - از روی **محتوا** (`is_project_file`)، نه نام - ولی به پوشه همیشه می‌شود وارد شد. جواب `drives` هم دارد. `for: "new"` یعنی شروع از کنار برنامه.

`Api.choose` متد

```
def choose(self, request: Dict[str, Any]) -> Dict[str, Any]:
```

پنجره‌ی خود سیستم برای یکی از `CHOICES`. اگر ممکن نیست، `{ok: false, unavailable: true}` و صفحه پنجره‌ی خودش را نشان می‌دهد؛ اگر پنجره‌ای باز است، `busy: true`.

خروجی: `{ok: true, path, cancelled}`.

`Api.generate` متد

```
def generate(self, _request: Dict[str, Any]) -> Dict[str, Any]:
```

`Document.generate`؛ خطای ابزار (مثلاً ساب‌لیستی که در فایل پروژه هست و روی دیسک نیست) به جواب «ناموفق» با دلیل تبدیل می‌شود، نه خطای ۵۰۰ که صفحه را روی «Generating...» نگه می‌داشت.

ویرایش درخت

کار	نام
`add_parameter(parent, kind)` ← `{ok, selected}` (مسیر گره‌ی تازه، تا صفحه انتخابش کند).	`Api.add`
`add_structure(name, type, fields)`؛ اگر `fields` لیست باشد تایپ هم اعلان می‌شود.	`Api.add_structure`
`duplicate(path)` ← `{ok, selected}`.	`Api.duplicate`
`delete(path)`؛ اگر چیزی نبود پیام «There is nothing called ... to delete».	`Api.delete`
`move(path, offset)`.	`Api.move`
`update(path, changes)`؛ ویرایش ردشده **باید** دلیلش را بگوید، وگرنه شبیه بی‌اثری بی‌صدا می‌شد.	`Api.update`
`{ok}`: آیا قدمی برگشت.	`Api.undo`
`{ok}`.	`Api.redo`
نام صفحه‌ای فهرست (`paramTypes`، `systemNames`، `paramModes`، `valueTypes`) را به ستون مدل تبدیل و `Document.set_defines` را صدا می‌زند. سطری که خوانده نشود (`SYS_NAV = abc`) ← `{ok: false, message}` و هیچ چیز ذخیره نمی‌شود.	`Api.set_defines`
فهرست‌های همراه برنامه برای دکمه‌ی «بازگرداندن پیش‌فرض‌ها»، با **همان کلیدهایی که پنجره می‌خواهد** (قبلاً `types` به‌جای `valueTypes` برمی‌گشت و OK فهرست نوع‌ها را پاک می‌کرد)، به‌اضافه‌ی `defineNumbers` آن‌ها.	`Api.default_defines`
`{ok, value}` با `next_free_log_id`.	`Api.next_log_id`

ساب‌لیست‌ها و تایپ‌ها

کار	نام
`directory`، `config` و `lists` (`known_lists`).	`Api.lists`
`create_list(name, directory)` ← `{ok, path}`.	`Api.new_list`
`Document.browse(directory)`.	`Api.browse`
`Document.inspect_list(path)`؛ خطای خواندن ← `{ok: false, reason}`.	`Api.inspect_list`
`add_existing_list(path)` ← `{ok, path, alreadyInBuild}` (ساب‌لیستی که از قبل در ساخت بود فقط باز می‌شود).	`Api.add_existing_list`
`remove_list(path, delete)` ← `{ok, removed, deleted, wasOpen, openInstead}`.	`Api.remove_list`
`select_lists(paths)` ← `{ok, paths}`.	`Api.select_lists`
`create_struct_type(name, fields)` ← `{ok, name}`.	`Api.new_type`

کار	نام
`update_struct_type(name, changes)` ← `{ok, name}`.	`Api.update_type`
`delete_struct_type(name)` ← `{ok, message}`.	`Api.delete_type`
`use_struct_type(path, name)` ← `{ok, selected, message}`.	`Api.use_type`

تابع‌های بیرون کلاس

`_node` تابع

```
def _node(parameter: Parameter, path: str,
          inherited: Dict[str, Any], inherited_type: Any,
          owner: str = "", inherited_log_id: Any = None,
          log_ids: Optional[Dict[str, List[int]]] = None,
          indices: Optional[Dict[str, List[Dict[str, Any]]]] = None,
          parent_enabled: bool = True,
          owner_type: str = "",
          above: Optional[Dict[Tuple[int, ...], Tuple[str, str]]] = None,
          fixed: Optional[Dict[str, Any]] = None
          ) -> Dict[str, Any]:
```

یک گره‌ی درخت همان‌طور که صفحه می‌خواهد (بازگشتی برای فرزندان). مهم‌ترین تابع این فایل برای کسی که صفحه را تغییر می‌دهد: هر چیزی که فرم یا جدول نشان می‌دهد از کلیدهای همین دیکشنری می‌آید. کلیدهای اصلی:

- شناسه: `path`، `name`، `kind`، `isStruct`، `isField`، `owner` (نام مدخل سطح بالا برای فیلد)، `ownerType` (تایپی که شکل فیلد از آن می‌آید).
- خانه‌ها: `enable`، `userName`، `address`، `paramType`، `valueType`، `arraySize`، `defaultValue`، `minimum`، `maximum`، `resolution`، `unit`، `logId`، `description`.
- مقدارهای به ارث رسیده (برای فیلدها فقط‌خواندنی): `effectiveTags`، `effectiveType`، `effectiveLogId`، `effectiveEnable`، `parentEnabled`.
- مقدارهای محاسبه‌شده از `flatten()`: `logIds` و `values` (اندیس، نام، شناسه، توضیح و نام نمایشی هر مقدار)؛ صفحه هرگز این حساب‌ها را خودش دوباره انجام نمی‌دهد.
- متن ردیف‌ها: `elementDescriptions`، `elementUserNames`، `sharedUserName`، `inheritedUserName` و `rowTexts` (از `_row_texts`).
- `tags` و `children`.

پارامتر `fixed`: تگ‌هایی که قالب برای همه‌ی پارامترها ثابت می‌کند (`fixed_tags`). در گره‌ی سطح بالا جای مقدار فایل را در `tags` و `effectiveTags` می‌گیرند - جدول و فرم همان را نشان می‌دهند که کد تولیدی می‌گیرد - و فیلدها آن را از `effectiveTags` گره‌ی بالایی به ارث می‌برند. خود ساب‌لیست مقدار خودش را نگه می‌دارد.

`_numbers_of` تابع

```
def _numbers_of(defines: Any) -> Dict[str, List[List[Any]]]:
```

{"systemNames": [["SYS_UNKNOWN", 0], …], "paramModes": […]}: هر نام با عددی که `#define` می‌شود - نام‌های بی‌عدد هم، همان‌طور که سازنده حساب می‌کند (`Defines.numbered`) - تا پنجره هر سطر را `SYS_NAV = 16` نشان دهد.

ورودی‌ها

`defines`	یک `Defines`

`_row_texts` تابع

```
def _row_texts(parameter: Parameter, above: Dict[Tuple[int, ...], Tuple[str, str]]
               ) -> Tuple[Dict[str, Dict[str, str]], Dict[Tuple[int, ...], Tuple[str,
str]]]:
```

آنچه هر ردیف جدول یک گره نشان می‌دهد (نام نمایشی و توضیح، و آنچه از بالا می‌خواند)، به کلید مسیر عنصری (`""` ردیف اعلان، `"1"` برای `Wheels[1]`، `"1.5"` برای `Wheels[1].Samples[5]`)؛ و برای استراکچر، آنچه هر عنصرش به فیلدهایش می‌دهد. همان خواندنی است که `flatten()` می‌کند، تا جدول و فرم هرگز خودشان حساب نکنند.

`_indices_by_row` تابع

```
def _indices_by_row(document, per_list: bool = False,
                    merged_view: bool = False) -> List[RowValues]:
```

برای هر ردیف سطح بالا، دیکشنری «مسیر اعلان ← یک مدخل برای هر مقدار»: اندیس در جدول تولیدی، نام عنصر و شناسه‌ی لاگ. شمارش روی **کل ساخت** است، به ترتیب ساخت، چون اندیس یک پارامتر به همه‌ی ساب‌لیست‌های قبلی بستگی دارد. با `per_list` (در `param_table`) ساب‌لیست باز به‌تنهایی از صفر شمرده می‌شود. برای هر ردیف یک دیکشنری جدا، چون نام فقط در یک ساب‌لیست یکتاست.

نام	کار
`_mark_owner`	`ownerList` را روی گره و همه‌ی زیرگره‌ها می‌گذارد.
`_log_ids_by_declaration`	«مسیر اعلان ← شناسه‌هایی که مقدارهایش واقعاً گرفته‌اند»، از `flatten()` ساب‌لیست باز؛ یک بار برای کل درخت.
`_count_rows`	مقدارهای هر پارامتر، شماره‌خورده از `first`؛ هر مدخل سطح بالا جدا flatten می‌شود.
`_row`	مقدارهای ردیف یک جایگاه، یا دیکشنری خالی.

`start_folder` تابع

```
def start_folder(workspace: Workspace, what: str) -> Path:
```

جای شروع هر پنجره: Open project از پوشه‌ی پروژه‌ی باز (یا آخرین پروژه، یا کنار برنامه)؛ New project کنار برنامه (پوشه‌ای که `paramgen/` در آن است؛ **هرگز درون آن**، چون در پوشه‌ی برنامه هنگام کار نوشته نمی‌شود)؛ و ساب‌لیست‌ها پوشه‌ی ساب‌لیست‌های پروژه.

۲۰. صفحه‌ی ویرایشگر: پوشه‌ی web

شش فایل، بی هیچ کتابخانه‌ی بیرونی (نه React، نه jQuery): HTML و CSS و JavaScript ساده که هر مرورگری اجرا می‌کند. اصول این سه زبان در فصل ۵ آمده است. ترتیب بار شدن اسکریپت‌ها در پایان `index.html` مهم است: `api.js`، `grid.js`، `editor.js` و در آخر `app.js` که بقیه را به هم وصل می‌کند و `app.start()` را اجرا می‌کند.

کار	شیء اصلی	فایل
اسکلت صفحه: نوار بالا، صفحه‌ی شروع، جدول، فرم، پنل Problems، نوار وضعیت، پنجره‌ی modal	—	`index.html`
همه‌ی ظاهر؛ رنگ‌ها متغیرهای CSS هستند و تم تیره همان متغیرها را عوض می‌کند	—	`style.css`
تنها جایی که با سرور حرف می‌زند	`api`	`api.js`
جدول (نمای صفحه‌گسترده) سمت چپ	`gridView`	`grid.js`
فرم پارامتر سمت راست	`editorView`	`editor.js`
هماهنگ‌کننده: نوار ابزار، پنل‌ها، پنجره‌ها، میان‌برها	`app`	`app.js`

هر فایل JS یک **شیء** (object literal، بخش ۵.۴) تعریف می‌کند که متدهایش با `this` به هم دسترسی دارند: `app.refresh()`، `gridView.render(...)`. چون اسکریپت‌ها module نیستند، این شیءها و تابع‌های بیرونی (مثل `escapeHtml`) سراسری‌اند و هر فایل بعدی آن‌ها را می‌بیند.

> **مهم:** صفحه هیچ قاعده‌ای را **تصمیم** نمی‌گیرد؛ فقط نشان می‌دهد. هر دکمه‌ای که پنهان یا خاموش می‌شود، سرور همان کار را دوباره رد می‌کند. اگر قاعده‌ای تازه لازم است، اول در `document.py` بنویسید؛ صفحه فقط پیامش را نشان می‌دهد.

۲۰.۱ `index.html` — اسکلت صفحه

مسیر: `paramgen/src/pl_gui/web/index.html`

بخش‌ها به ترتیب:

- `<head>`: عنوان «Parameter List Editor»، آیکن پنجره و نوار وظیفه با دو `<link rel="icon">` (`icon-32.png` و `icon-256.png`؛ بی آن، مرورگر خودش `/favicon.ico` می‌خواهد و خطای ۴۰۴ می‌گیرد)، `style.css`، و یک اسکریپت کوچک که تم را **پیش از** رسم صفحه از `localStorage` (کلید `pl-theme`، پیش‌فرض `dark`) می‌خواند تا صفحه یک لحظه سفید نشود.
- `header.topbar` با دو ردیف: ردیف سند (لوگوی شرکت `img.brand-mark`، `btn-project` و منوی `menu-project`، `format-badge`، `module-picker`، `btn-lists-select`، `btn-save`، `btn-reload`، `file-name`،

`btn-generate`، منوی تنظیمات `btn-settings`) و ردیف ویرایش (`btn-add-menu` و `menu-add`، `btn-types`، `btn-duplicate`، `btn-up`، `btn-down`، `btn-delete`، `btn-undo`، `btn-redo`، جعبه‌ی جست‌وجوی `filter`، `btn-clear-filters`).

- `section#start-screen`: صفحه‌ی شروع (وقتی پروژه‌ای باز نیست).
- `main#layout`: جدول (`#grid`) و فرم (`#editor` با `#editor-title` و `#editor-path`).
- `section#problems-pane`: پنل Problems با سه شمارنده‌ی شدت که فیلتر هم هستند.
- `footer.statusbar`، پنجره‌ی `#modal`، پرده‌ی `#dialog-veil` (وقتی پنجره‌ی Open ویندوز باز است) و `#toast`.

سه تصویر کنار `index.html`: `logo.png` لوگوی شرکت در گوشه‌ی نوار بالا (پس‌زمینه‌ی شفاف، ۳۰ پیکسل ارتفاع از `style.css`)، و `icon-32.png` و `icon-256.png` همان لوگو در مربع شفاف، برای آیکون پنجره و نوار وظیفه. هر سه یک بار از تصویر لوگو ساخته شده‌اند (پس‌زمینه‌ی سفید برداشته شده)؛ برنامه هیچ کتابخانه‌ی تصویری لازم ندارد و سرور آن‌ها را مثل هر فایل دیگر `web` می‌فرستد (`.png` در `CONTENT_TYPES`، فصل ۱۷). عوض کردن لوگو یعنی جایگزین کردن همین سه فایل با همین نام‌ها.

عنصرهایی که کلاس `needs-project` دارند، وقتی پروژه‌ای باز نیست پنهان می‌شوند (`app.paintProject`). شناسه‌ی (`id`) هر دکمه همان است که `app.bindToolbar` با `document.getElementById` پیدا می‌کند؛ **تغییر یک `id` یعنی تغییر همان نام در `app.js`.**

۲۰.۲ `style.css` — همه‌ی ظاهر

مسیر: `paramgen/src/pl_gui/web/style.css`

بالای فایل، در `:root`، همه‌ی رنگ‌ها و اندازه‌ها متغیر CSS (custom property) هستند؛ مثلاً `--accent: #0b4f8a;` (آبی اصلی) و `--error: #b3261e;`. نزدیک پایان فایل، `:root[data-theme="dark"]` همان متغیرها را برای تم تیره دوباره تعریف می‌کند. پس:

- برای عوض کردن یک رنگ در **هر دو** تم، هر دو بلوک را عوض کنید.
- در قاعده‌های دیگر همیشه `var(--accent)` بنویسید، نه کد رنگ؛ وگرنه تم تیره آن را نمی‌شناسد.
- قاعده‌ی `[hidden] { display: none !important; }` را برندارید: بدون آن هر قاعده‌ای که `display` می‌دهد، ویژگی `hidden` را بی‌اثر می‌کند.
- `--topbar-h` را `app.js` اندازه می‌گیرد و می‌نویسد (`trackTopbarHeight`)؛ دستی ثابتش نکنید.
- قلم‌ها: `--mono` (Consolas / Cascadia Mono) برای نام‌ها و عددها، `--sans` (Segoe UI) برای بقیه. هیچ قلمی از اینترنت بار نمی‌شود.

۲۰.۳ `api.js` — تنها جایی که با سرور حرف می‌زند

مسیر: `paramgen/src/pl_gui/web/api.js`

`api.call` متد

```
async call(action, payload = {})
```

یک درخواست `POST /api/<action>` با بدنه‌ی JSON می‌فرستد (`fetch`، بخش ۵.۶) و جواب JSON را برمی‌گرداند. اگر وضعیت HTTP خطا باشد، `message` جواب را (نه متن خام JSON) در یک `Error` پرتاب می‌کند تا toast جمله‌ی خوانا نشان دهد.

مثال:

```
const answer = await api.call("next_log_id");
// {ok: true, value: 502}
```

بقیه‌ی متدها همه یک‌خطی‌اند: نام JS را به نام کار سرور و آرگومان‌ها را به بدنه تبدیل می‌کنند. جدول کامل نام‌های سرور در فصل ۱۹ است.

کار سرور	متد
`state` ← `state(allLists)`	`api.state`
`open_project` ←	`api.openProject`
`new_project` ← (`folder`، `name`، `format`)	`api.newProject`
`close_project` ←	`api.closeProject`
`browse_projects` ←	`api.browseProjects`
`choose` ←	`api.choose`
`open` ←	`api.open`
`save` ←	`api.save`
`add` ←	`api.add`
`add_structure` ←	`api.addStructure`
`duplicate` ←	`api.duplicate`
`delete` ← (نام `delete` در JS کلمه‌ی رزرو است)	`api.remove`
`move` ←	`api.move`
`update` ←	`api.update`
`undo` ←	`api.undo`
`redo` ←	`api.redo`
`defines` ←	`api.setDefines`
`default_defines` ←	`api.defaultDefines`
`generate` ←	`api.generate`
`next_log_id` ←	`api.nextLogId`
`lists` ←	`api.lists`

کار سرور	متد
← `new_list`	`api.newList`
← `select_lists`	`api.selectLists`
← `browse`	`api.browse`
← `inspect_list`	`api.inspectList`
← `add_existing_list`	`api.addExistingList`
← `remove_list` (`delete` در بدنه)	`api.removeList`
← `new_type`	`api.newType`
← `update_type`	`api.updateType`
← `delete_type`	`api.deleteType`
← `use_type`	`api.useType`

۲۰.۴ `grid.js` — جدول سمت چپ

مسیر: `paramgen/src/pl_gui/web/grid.js`

جدول یک **نما** است، نه ویرایشگر دوم: کلیک روی ردیف آن را انتخاب می‌کند و فرم سمت راست جای تغییر دادن است. هر مقدار یک ردیف؛ استراکچرها و آرایه‌ها باز و بسته می‌شوند.

ستون‌ها

آرایه‌ی `gridView.columns` هر ستون را با `key` (کلیدی که خانه از ردیف می‌خواند)، `label`، `width` (یک track در CSS grid)، و اختیاری `align` و `mono` تعریف می‌کند. سرستون، خانه‌ها، فیلترها و قالب CSS همه از همین آرایه ساخته می‌شوند؛ **افزودن ستون یعنی یک مدخل اینجا و یک کلید در** `rows()`. ستونی که پهنایش `0px` است اصلاً رسم نمی‌شود (`owner` بیرون از نمای ادغام‌شده و `address` بیرون از `param_table`؛ `app.paint` این دو را تنظیم می‌کند).

```
// gridView.columns
{ key: "unit",  label: "Unit",  width: "68px" },
{ key: "def",   label: "Default", width: "82px", align: "right", mono: true },

// در rows() → emit(): همان کلید روی ردیف
unit: node.unit || "",
```

ستون‌های `CHOICE_COLUMNS` (مثل `paramType`، `valueType`، `system`) به‌جای جعبه‌ی متن، فهرست کشویی مقدارهای موجود در جدول دارند و «دقیقاً برابر» فیلتر می‌کنند؛ بقیه «شامل».

وضعیت باز و بسته: پیش‌فرض، استراکچر باز و آرایه بسته است؛ `toggled` فقط استثناهایی را نگه می‌دارد که کاربر عوض کرده، و `forceOpen` مسیرهایی را که چیز دیگری (پنل Problems) باز کرده. `filters` متن هر فیلتر ستون و `search` جعبه‌ی جست‌وجوی کلی است.

کار	نام
عنصر جدول و دو callback (`onSelect`، `onToggleEnable`) را نگه می‌دارد و کلیک‌ها را با `onSteadyClick` روی ظرف جدول می‌گیرد.	`gridView.init`
باز یا بسته کردن یک سطح (کلید ▸ / ▾)؛ مدل دست نمی‌خورد.	`gridView.toggleRow`
پیوندهای زیر جدول: expand all، collapse all، clear the filters.	`gridView.followLink`
مسیری را به `forceOpen` اضافه می‌کند.	`gridView.open`
مقدارهای متمایز یک ستون برای فهرست کشویی فیلترش، مرتب با مقایسه‌ی عددی.	`gridView.choicesFor`
همه‌ی `values` (از `_node` در api.py) گره‌ها و زیرگره‌ها.	`gridView.valueIndex`
مقدارهایی که در یک مسیر یا زیر آن‌اند (`Motor[0]` صاحب `Motor[0].Rpm` است، نه `Motor[01].Rpm`).	`gridView.under`
آیا یک ردیف باز است: `forceOpen`، یا پیش‌فرض با استثنای `toggled`.	`gridView.isOpen`
همه‌ی ردیف‌های قابل رسم: برای آرایه، ردیف اعلان و یک ردیف برای هر عنصر؛ برای استراکچر، فیلدها زیرش. هر ردیف یک شیء با یک کلید برای هر ستون (و کلیدهای `...Search` برای فیلتر روی معنای ردیف، مثلاً همه‌ی اندیس‌های پشت `3-18`). هنگام جست‌وجو همه‌چیز باز فرض می‌شود.	`gridView.rows`
آیا یک ردیف از جست‌وجو و همه‌ی فیلترها می‌گذرد.	`gridView.matches`
کل جدول را می‌سازد (HTML رشته‌ای). سرستون و ردیف فیلتر اگر تغییر نکرده‌اند نگه داشته می‌شوند تا کلیک روی فیلتر بلافاصله پس از تایپ در فرم گم نشود؛ اسکرول سر جایش برمی‌گردد. کلاس‌های ردیف: `struct-row`، `value-row`، `selected`، `disabled`، `has-error`.	`gridView.render`
ردیف یک مسیر را اگر بیرون دید است، به دید می‌آورد (با در نظر گرفتن سرستون چسبان).	`gridView.reveal`
باز یا بسته کردن همه؛ «expand all» جدول را دقیقاً مثل جدول تولیدی نشان می‌دهد.	`gridView.openEverything`
رویدادهای جعبه‌های فیلتر را وصل می‌کند و پس از هر رسم دوباره، مکان‌نما را سر جایش برمی‌گرداند.	`gridView.wire`
`& < > " '` را escape می‌کند. **هر متنی که از مدل در HTML گذاشته می‌شود باید از این بگذرد**؛ وگرنه نام یا توضیحی که `<` دارد صفحه را می‌شکند.	`escapeHtml`
کلیک از روی دو نیمه‌ی `mousedown` و `mouseup`، نه رویداد `click`: ترک یک خانه‌ی فرم صفحه را دوباره رسم می‌کند و رهاسازی روی عنصر تازه‌ای می‌افتاد؛ مرورگر آن را کلیک حساب نمی‌کرد و «اولین کلیک کار نمی‌کرد».	`onSteadyClick`
عدد برای جدول: بی‌صفرهای اضافه، بولی `FALSE`/`TRUE`، خالی برای «تنظیم نشده».	`fmtNumber`
ساب‌لیست صاحب یک گره در نمای ادغام‌شده.	`ownerOf`
مسیر عنصری یک ردیف: `Wheels[1].Samples[5]` ← `"1.5"`.	`rowKey`
خلاصه‌ی ردیف بسته: بازه‌ی پیوسته `3-6`، وگرنه شمارش `8×` (مقدارهای یک فیلد آرایه‌ی استراکچر پیوسته نیستند).	`fmtSpan`

کار	نام
FALSE/TRUE یا خالی.	fmtBool

۲۰.۵ editor.js — فرم سمت راست

مسیر: paramgen/src/pl_gui/web/editor.js

فرم یک پارامتر را **از بالا به پایین** به‌صورت HTML می‌نویسد (structForm یا primitiveForm) و collect() آن را برمی‌خواند. هر خانه‌ی قابل‌ویرایش ویژگی data-field="<نام فیلد>" Parameter" (یا برای تگ data-tag="<شناسه>") دارد؛ collect() همه‌ی این‌ها را جمع می‌کند و سرور فقط نام‌های EDITABLE_FIELDS را می‌پذیرد. ثابت IGNORED_TAGS باید با همان ثابت در param_table_h.py هم‌گام بماند.

افزودن یک خانه به فرم یعنی سه چیز: یک سطر در primitiveForm (مثلاً html += this.text("unit", "Unit", node.unit);)، تبدیل درستش در collect() اگر عدد یا بولی است، و نام فیلد در EDITABLE_FIELDS در document.py.

مهم‌ترین ظرافت این فایل: فرم **زیر دست کسی که تایپ می‌کند دوباره ساخته نمی‌شود**. هر ویرایش، صفحه را از مدل دوباره رسم می‌کند و آن رسم چند میلی‌ثانیه بعد از رفتن مکان‌نما به خانه‌ی بعدی می‌رسد؛ ساختن دوباره‌ی فرم، خانه‌ی در حال تایپ و هرچه در آن تایپ شده بود را دور می‌ریخت. ownEdit، sentBy و keepsWhatIsOnScreen همین را مدیریت می‌کنند.

کار	نام
عنصرهای فرم، عنوان و مسیر و callback onChange را نگه می‌دارد.	editorView.init
«Nothing selected»، یا اگر ساب‌لیستی نیست، توضیح اینکه اول باید ساب‌لیست ساخت.	editorView.clear
فرم یک گره را نشان می‌دهد (عنوان: Parameter/Structure/Array/Element)، مگر اینکه تغییر از خود فرم آمده و باید دست نخورد؛ مکان‌نما را نگه می‌دارد و برمی‌گرداند.	editorView.show
اگر ردیف انتخاب‌شده اندیس دارد: اطلاعات همان ردیف (اندیس، شناسه، متن‌ها و اینکه مال خودش‌اند یا از بالا).	editorView.valueShown
سطر کوچک زیر User Name یا Description یک ردیف: متن مال کیست و خالی کردنش چه می‌کند.	editorView.rowHint
خانه‌ی User Name، برای ردیف یا برای اعلان (با placeholder آنچه از استراکچر می‌خواند).	editorView.userNameField
خانه‌ی Description، با همان قاعده.	editorView.descriptionFiel d
آیا عنصر جعبه‌ی تایپ است (نه فهرست کشویی یا تیک).	editorView.isTextBox
آیا صفحه‌کلید درون یک جعبه‌ی متن همین فرم است.	editorView.isBeingTypedIn

کار	نام
آیا پاسخ تغییرِ خود فرم می‌تواند فرم را دست‌نخورده بگذارد.	`editorView.keepsWhatIsOnScreen`
خانه‌ی دارای فوکوس را **با نامش** (`id`/`data-tag`/`data-field`) و جای مکان‌نما به خاطر می‌سپارد؛ خود عنصر از رسم دوباره جان سالم به در نمی‌برد.	`editorView.captureFocus`
فوکوس و مکان‌نما را برمی‌گرداند.	`editorView.restoreFocus`
فرم استراکچر: Identity (نام، User Name، Type، تایپ، اندازه، Enabled)، Description، تگ‌ها (یا برای استراکچر درونی، به ارث رسیده‌ها) و Fields با دکمه‌ی ویرایش تایپ.	`editorView.structForm`
فرم پارامتر: Identity (برای فیلد، نام و نوع و اندازه فقط‌خواندنی‌اند)، Value (برای بولی فقط پیش‌فرض)، تگ‌ها و Description. در `param_table` اندازه فقط‌خواندنی و خانه‌ی Address نمایان است.	`editorView.primitiveForm`
مقداری که نشان داده می‌شود ولی قابل‌ویرایش نیست، با راهنمای اختیاری.	`editorView.readOnly`
فهرست نوع: برای استراکچر فقط تایپ‌های استراکچر، برای پارامتر فقط نوع‌های داده؛ هرگز مخلوط.	`editorView.dataTypeSelect`
برای فیلد: Type و تگ‌ها و شناسه‌ی لاگ به ارث رسیده، فقط‌خواندنی، با دکمه‌ی « Edit them on <استراکچر>».	`editorView.inheritedGroup`
خانه‌های تگ‌ها از روی `state.tagSpecs`: بولی فهرست TRUE/FALSE، enum فهرست مقدارها، آزاد جعبه‌ی متن؛ در `param_table` تگ SystemName فقط‌خواندنی، و هر تگی که قالب ثابت می‌کند (`state.fixedTags`؛ در smart data `ParamMode` برابر `PARAM_MODE_NONE`) هم فقط‌خواندنی و بی `data-tag`، پس فرستاده نمی‌شود. خانه‌ی شناسه‌ی لاگ درست بعد از Tag Loggable.	`editorView.tagsGroup`
خانه‌ی شناسه‌ی لاگ با دکمه‌ی **Next free** و توضیح بازه‌ی شناسه‌ها؛ اگر Loggable نیست کم‌رنگ.	`editorView.logIdField`
سازنده‌ی HTML یک جعبه‌ی متن با `data-field`.	`editorView.text`
جعبه‌ی عدد (`step="any"`).	`editorView.number`
فهرست کشویی با `optionList`.	`editorView.select`
تیک.	`editorView.check`
جعبه‌ی چندخطی.	`editorView.area`
رویدادها: تیک و فهرست با `change`، جعبه‌ها با `blur` (و Enter یعنی blur)؛ دکمه‌ی Next free؛ فهرست نوع که برای استراکچر از `app.useType` می‌گذرد؛ دکمه‌های ویرایش تایپ و رفتن به صاحب.	`editorView.wire`
همه‌ی فرم را فوراً می‌فرستد؛ Ctrl+S پیش از ذخیره صدایش می‌زند، چون مکان‌نما هنوز در خانه است.	`editorView.commit`
`sentBy` را ثبت و `onChange(collect())` را صدا می‌زند.	`editorView.send`

کار	نام
دیکشنری تغییرها: خانه‌ی خالی `null`؛ `array_size` خالی ۱؛ `log_id` خالی `null` (صفر شناسه‌ی معتبر است)؛ `address` همان‌طور که تایپ شده (سرور رد می‌کند)؛ عددها با `Number`؛ تگ‌ها در `changes.tags`.	`editorView.collect`
`<option>`های یک فهرست، با گزینه‌ی خالی – اختیاری.	`optionList`

۲۰.۶ `app.js` — هماهنگ‌کننده‌ی صفحه

مسیر: `paramgen/src/pl_gui/web/app.js`

تنها فایلی که همه‌ی بخش‌ها را می‌شناسد. ویژگی‌های اصلی `app`: `state` (آخرین جواب سرور؛ **تنها منبع حقیقت صفحه**)، `selected` (مسیر ردیف انتخاب‌شده)، `allLists` (آیا نمای ادغام‌شده است)، `hidden` (شدت‌هایی که در Problems خاموش شده‌اند) و `SHORTCUTS` (کلید ← فرمان: Ctrl+S ذخیره، Ctrl+G تولید، Ctrl+N افزودن، Ctrl+D کپی، Ctrl+Z و Ctrl+Y). ثابت `ALL_LISTS` مقدار فهرست ساب‌لیست‌ها برای «همه، ادغام‌شده» است.

الگوی تقریباً همه‌ی کارها یکی است:

```
async remove() {
  try {
    const answer = await api.remove(this.selected);   // ۱. درخواست
    if (!answer.ok) { toast(answer.message, "bad"); return; }   // ۲. رد با دلیل
    this.selected = "";
    await this.refresh();                                 // ۳. وضعیت تازه و رسم
    setStatus("Deleted");
  } catch (error) { this.failed("Deleting", error); }   // ۴. خطای غیرمنتظره
}
```

بار کردن و رسم

کار	نام
جدول و فرم را راه می‌اندازد، نوار ابزار و میان‌برها را وصل و اولین `reload` را اجرا می‌کند.	`app.start`
هر درخواستی که استثنا داد: toast قرمز و نوار وضعیت. کاری که بی‌صدا شکست بخورد، کاربر را به اشتباه می‌اندازد.	`app.failed`
`state` را از سرور می‌گیرد و `paint`.	`app.refresh`
`refresh` که خطا را گزارش می‌کند به‌جای پرتاب.	`app.reload`
همه‌چیز را از `state` رسم می‌کند: پروژه، نام فایل و *، عنوان پنجره، فهرست ساب‌لیست‌ها، پهنای ستون‌های `address`/`owner`، جدول، فرم، Problems، شمارنده‌ها و دکمه‌ها.	`app.paint`
گره‌ی اعلان پشت `selected` (اندیس‌ها برداشته می‌شوند)؛ اگر عنصر دیگر وجود ندارد، به اعلان برمی‌گردد.	`app.selectedNode`

کار	نام
صفحه‌ی شروع یا ویرایشگر، نام پروژه، نشان «parameter table» و خاموش کردن دکمه‌های استراکچر در آن قالب، فهرست پروژه‌های اخیر.	`app.paintProject`
آیا پروژه‌ی باز `param_table` است.	`app.isParamTable`
متن tooltip قالب پروژه.	`app.formatName`
عنوان پنجره: مسیر کامل فایل پروژه و نام ویرایشگر، `*D:\Fc\pl_project.json — Parameter List Editor` (`*` تا ساب‌لیست باز ذخیره نشده). فقط از `state.project.file`؛ پیش‌تر نام ساب‌لیست را هم در صفحه از فهرست ساب‌لیست‌ها حدس می‌زد و با چند ساب‌لیست گاهی دیگری را نشان می‌داد.	`app.windowTitle`
فهرست کشویی ساب‌لیست‌ها (با «every sub list, merged») ؛ فقط وقتی متنش عوض شده دوباره نوشته می‌شود.	`app.paintModules`
شمارنده‌ی زیر جدول و نوار وضعیت.	`app.paintCounts`
پنل Problems: مرتب بر اساس شدت، با کد و پیام و مکان و راهنما؛ پنل خالی خودش جمع می‌شود.	`app.paintProblems`
خاموش و روشن کردن دکمه‌ها با tooltip دلیل: در نمای ادغام‌شده، روی فیلد استراکچر، روی یک عنصر، یا بی‌ساب‌لیست.	`app.updateButtons`
تابع خالی؛ از نسخه‌ی قبلی منوی Add مانده و `paint` هنوز صدایش می‌زند.	`app.paintAddMenu`
ارتفاع واقعی پنل Problems را در `--problems-now` می‌نویسد.	`app.sizeProblems`
مسیر پارامترهایی که خطا دارند (برای رنگ قرمز ردیف‌ها).	`app.problemPaths`

پروژه، ساب‌لیست و رفتن به یک مشکل

کار	نام
اگر تغییر ذخیره‌نشده هست، `confirm` می‌پرسد؛ هر مسیری که چیز باز را عوض می‌کند اول از اینجا می‌گذرد.	`app.mayLeaveTheList`
باز کردن یک پروژه‌ی اخیر یا انتخاب‌شده.	`app.openProject`
بستن پروژه و بازگشت به صفحه‌ی شروع.	`app.closeProject`
کلیک روی ردیف Problems: رفتن به پارامتر (`goToProblem`) یا تایپ (`goToType`).	`app.bindProblemRows`
پیامِ یک تایپ، ساب‌لیستی را که آن تایپ را اعلان کرده باز می‌کند و پنجره‌ی تایپ‌ها را روی آن.	`app.goToType`
سه شمارنده‌ی شدت به‌عنوان فیلتر.	`app.bindSeverityFilters`
کلیک در نمای ادغام‌شده: ساب‌لیست صاحب ردیف را باز و ردیف را انتخاب می‌کند.	`app.editWhereItLives`
پیامی که مال ساب‌لیست دیگری است: اول آن را باز می‌کند، بعد ردیف را.	`app.goToProblem`
انتخاب یک مسیر، باز کردن والدهایش و آوردن به دید.	`app.select`

کار	نام
دو پرسش جدا: بیرون بردن از ساخت (برگشت‌پذیر) و پاک کردن فایل (برگشت‌ناپذیر).	`app.removeList`
تیک Enabled یک ویرایش عادی از مسیر `update` است.	`app.setEnabled`
پنجره‌ی ☰: انتخاب و ترتیب ساب‌لیست‌های ساخت، افزودن و برداشتن.	`app.editListSelection`
پنجره‌ی ساب‌لیست تازه با خانه‌ی Location (در `param_table` فهرست زیرسیستم‌ها).	`app.newList`
پنجره‌ی ⚙ ▸ Drop-down lists…: چهار فهرست کشویی، یک مقدار در هر سطر، با دکمه‌ی بازگرداندن پیش‌فرض‌های شرکت. System names و Parameter modes سطر به سطر با عددشان (`SYS_NAV = 16`، از `state.defineNumbers`)؛ سطری که سرور رد کند پیامش را نشان می‌دهد و پنجره با همان متن باز می‌ماند (`return false`).	`app.editLists`

پنجره‌های فایل و پوشه

کار	نام
آیا پنجره‌ی خود سیستم ممکن است (`state.nativeDialogs`).	`app.nativeDialogs`
پوشه‌ی شروع یک پنجره (`state.startFolders`).	`app.startFolder`
پنجره‌ی سیستم با `api.choose`؛ در این مدت پرده‌ی `dialog-veil` صفحه را می‌پوشاند. `null` یعنی ممکن نبود و پنجره‌ی خود صفحه باید نشان داده شود.	`app.pickNative`
Project ▸ Open: پنجره‌ی سیستم، وگرنه `openProjectDialog`.	`app.openProjectFlow`
افزودن ساب‌لیست موجود: پنجره‌ی سیستم، وگرنه `openList`.	`app.addExistingSubList`
یک مسیر را به ساخت می‌آورد و بازش می‌کند؛ دلیل رد را برمی‌گرداند.	`app.addExistingListAt`
پنجره‌ی درون‌صفحه‌ای Open project با فهرست زنده‌ی پوشه.	`app.openProjectDialog`
پنجره‌ی New project: نام، قالب خروجی و Location.	`app.newProjectDialog`
پنجره‌ی درون‌صفحه‌ای افزودن ساب‌لیست؛ هر فایل همان لحظه با `inspectList` بررسی می‌شود.	`app.openList`

ویرایش

کار	نام
تغییرهای فرم را با `api.update` برای ردیفی که **هنگام فرستادن** انتخاب بود می‌فرستد؛ رد ← toast و رسم دوباره (خانه به مقدار واقعی برمی‌گردد)؛ موفق ← `ownEdit` و `refresh`. `pendingEdit` را تنظیم می‌کند تا Ctrl+S منتظرش بماند.	`app.applyChanges`
پنجره‌ی «Add a structure»: نام و تایپ موجود، یا تایپ تازه با جدول فیلدهایش؛ همه در **یک** درخواست `addStructure`.	`app.addStructure`
جدول فیلدها (مشترک «Add a structure» و پنجره‌ی تایپ‌ها) با دکمه‌های ↑ ↓ 🗑.	`app.renderDraftFields`

کار	نام
مقدارهای جدول فیلدها را به `draftFields` برمی‌گرداند.	`app.readDraftFields`
افزودن پارامتر و آوردن آن به دید (پارامتر تازه پایان فهرست است).	`app.addNode`
دادن یک تایپ به استراکچر (`api.useType`).	`app.useType`
پنجره‌ی Structure types: همه‌ی تایپ‌های ساب‌لیست باز و استفاده‌هایشان.	`app.editTypes`
پنجره‌ی اعلان تایپ تازه.	`app.newType`
ویرایش فیلدهای یک تایپ در جدول؛ کل لیست با OK فرستاده می‌شود.	`app.editOneType`
پاک کردن ردیف انتخاب‌شده پس از `confirm`.	`app.remove`
جابه‌جایی بالا/پایین؛ اگر ممکن نبود می‌گوید «Already first/last».	`app.moveNode`
ذخیره؛ پروژه‌ی بی‌ساب‌لیست پیام می‌گیرد، نه «Saved to undefined».	`app.save`
تولید؛ هرگز روی «Generating...» نمی‌ماند. در موفقیت پنجره‌ای با فایل‌ها و گزارش جابه‌جایی (`driftHtml`) و اگر چیدمان جابه‌جا شده، عنوان « CHECK THE MEMORY LAYOUT».	`app.generate`
HTML گزارش جابه‌جایی آفست‌ها در پنجره‌ی پایان تولید.	`app.driftHtml`
همان برای `param_table`: شماره‌های تغییرکرده در زیرسیستم.	`app.tableDriftHtml`

نوار ابزار، منوها، میان‌برها و تم

کار	نام
همه‌ی دکمه‌ها و منوها را با `id` به متدهایشان وصل می‌کند؛ **دکمه‌ی تازه اینجا وصل می‌شود.**	`app.bindToolbar`
یک دکمه و جعبه‌ی منو: باز با کلیک، بسته با کلیک جای دیگر یا Escape؛ هر آیتم به تابعی در `data-action` `actions`.	`app.bindMenu`
میان‌برها؛ وقتی پنجره‌ای باز است یا در جعبه‌ای تایپ می‌شود، رفتار مناسب (Ctrl+Z درون جعبه مال خود جعبه است؛ AltGr که Ctrl+Alt است ذخیره نمی‌کند).	`app.bindShortcuts`
اجرای یک فرمان؛ پیش از save و generate خانه‌ی فعال فرم فرستاده و منتظر `pendingEdit` می‌شود.	`app.runShortcut`
تم تیره/روشن؛ در `localStorage` با کلید `pl-theme`.	`app.toggleTheme`
متن آیتم منوی تم.	`app.paintThemeLabel`

تابع‌های کمکی بیرون از app

نام	کار
`trackTopbarHeight`	ارتفاع نوار بالا را با `ResizeObserver` اندازه می‌گیرد و در `--topbar-h` می‌نویسد.
`countBySeverity`	تعداد پیام‌های هر شدت.
`findNode`	گره‌ی یک مسیر در درخت `state.tree`.
`countLeaves`	تعداد مقدارها، همان‌طور که جدول تولیدی می‌شمارد (هر عنصر آرایه یکی).
`countStructs`	تعداد استراکچرها.
`samePath`	آیا دو مسیر یک فایل‌اند (بی‌توجه به `\` و `/` و بزرگی حروف).
`expandTo`	همه‌ی والدهای یک مسیر را در جدول باز می‌کند.
`driveRow`	ردیف دکمه‌های درایوها بالای فهرست پوشه.
`liveListing`	فهرست زنده‌ی پوشه زیر یک جعبه‌ی مسیر؛ دو طرفه: پیمایش جعبه را می‌نویسد و تایپ فهرست را می‌برد.
`folderListHtml`	HTML درایوها، «..» و پوشه‌های یک فهرست.
`fileListHtml`	HTML فایل‌های یک فهرست.
`missingNote`	یادداشت «این مسیر نیست؛ ساخته می‌شود».
`markPicked`	علامت زدن یک ردیف فایل به‌عنوان انتخاب‌شده.
`declarationPath`	مسیر بی‌اندیس: `Wheels[1].Speed` ← `Wheels.Speed`.
`firstRowPath`	اولین ردیف جدول یک اعلان درون آرایه‌ی استراکچر: `Wheels.Samples` ← `Wheels[0].Samples`.
`valuesUnder`	همه‌ی مقدارهای یک گره و زیرگره‌ها.
`folderFor`	نام پوشه‌ی پروژه‌ی تازه از نامش (فاصله ← _).
`systemOf`	زیرسیستم از نام فایل؛ همتای JS تابع `system_of`.
`tableFileName`	نام فایل ساب‌لیست یک زیرسیستم: `sys_rhs_paramTable.xml`.
`setStatus`	متن نوار وضعیت.
`toast`	پیام کوتاه گوشه‌ی صفحه؛ `kind` می‌تواند `good` یا `bad` باشد.
`showModal`	پنجره‌ی modal با عنوان و HTML؛ `onOk` اگر `false` برگرداند پنجره باز می‌ماند.
`closeModal`	بستن پنجره.

نکته: دو کامنت قدیمی در کد مانده که با رفتار امروز نمی‌خواند: بالای بخش «reusable structure types» در `app.js` (می‌گوید پارامتر کپی تایپ را دارد و پیوندی نیست؛ امروز تایپ همه‌ی استفاده‌هایش را تغییر شکل

> می‌دهد) و توضیح بالای `pl_gui/__init__.py` (از `tree.js` نام می‌برد). به کد اعتماد کنید، نه به این دو کامنت.

۲۱. راه‌اندازی ابزار: run.bat، run.sh و پوشه‌ی bin

کاربر فقط یک فایل را اجرا می‌کند: `run.bat` در ویندوز یا `run.sh` در لینوکس. این دو پایتون را پیدا می‌کنند و `bin/run.py` را اجرا می‌کنند که منو را نشان می‌دهد؛ هر گزینه‌ی منو یک فایل `run_*.py` جداست که در **فرایند جداگانه** اجرا می‌شود. ترتیب:

```
run.bat  /  run.sh
  └─ python bin\run.py [action] [args...]
       ├─ 1 editor    → bin\run_gui.py       → pl_gui.server.GuiServer
       ├─ 2 generate  → bin\run_generate.py  → pl_codegen.cli.main
       ├─ 3 tests     → bin\run_tests.py     → tests\run_*_test.py
       └─ 4 setup     → bin\run_setup.py     → setup_environment.main

bin\set_version.bat → python set_version.py
```

هر مجموعه آزمون در فرایند جدا اجرا می‌شود، و `set_version.bat` جدا از منوست و فقط برای نگهدارنده‌ی ابزار.

هر `run_*.py` دقیقاً یک کار را با یک الگو انجام می‌دهد: پوشه‌ی `src` را به مسیر import اضافه می‌کند و تابع `main` ماژول واقعی را صدا می‌زند. پس وقتی import شکست خورد، یک جای روشن برای نگاه کردن هست:

```
APP_DIR = Path(__file__).resolve().parent.parent      # ...\paramgen
sys.path.insert(0, str(APP_DIR / "src"))

from pl_codegen.cli import main  # noqa: E402

if __name__ == "__main__":
    raise SystemExit(main())
```

(`# noqa: E402` به ابزارهای بررسی سبک می‌گوید import بعد از کد، عمدی است.)

۲۱.۱ `run.bat` — تنها فایلی که کاربر ویندوز اجرا می‌کند

مسیر: `paramgen/run.bat`

یک فایل دسته‌ای (batch) ویندوز؛ اصول نحو آن در فصل ۷. کارها به ترتیب:

- `setlocal` و `cd /d "%~dp0"`: پوشه‌ی جاری همان پوشه‌ی خود فایل می‌شود؛ پس مسیر نسبی‌ای که به یک کار داده شود (مثل `editor ..\projects\demo`) از همین‌جا حساب می‌شود.
- `call "%~dp0bin\find_python.bat"`: پایتونی را پیدا می‌کند که واقعاً اجرا شود و ۳٫۱۱ یا تازه‌تر باشد (پایین‌تر). اگر نبود، پیام روشن، `pause` (پنجره‌ای که بسته شود کمکی نمی‌کند) و خروج با کد ۹.
- `"%PYTHON%" bin\run.py %*`: همان پایتون، و همه‌ی آرگومان‌ها (`%*`) به منو می‌رسند. هر کاری که منو اجرا می‌کند هم با همان پایتون (`sys.executable`) اجرا می‌شود.
- اگر آرگومان داده شده (مثل `run.bat tests`) یا کار شکست خورده، `pause` تا پیام دیده شود؛ دابل‌کلیک ساده خودش در منو منتظر می‌ماند.

نمونه‌ها: `run.bat` (منو)، `run.bat editor ..\projects\demo`، `run.bat generate -c ..\projects\demo --check`، `run.bat tests`.

> **هشدار:** این فایل باید پایان سطر CRLF داشته باشد؛ `.gitattributes` همین را برای `*.bat` تعیین می‌کند. فایل bat با پایان سطر LF در برخی نسخه‌های ویندوز رفتار عجیب دارد.

۲۱.۲ `find_python.bat` — پیدا کردن پایتونی که واقعاً اجرا شود

مسیر: `paramgen/bin/find_python.bat`

`run.bat` و `bin/set_version.bat` هر دو آن را `call` می‌کنند و بعد از آن متغیر `PYTHON` مسیر کامل یک `python.exe` است، یا خالی اگر پایتونی پیدا نشد. نحو و ترتیب جست‌وجو در فصل ۷ آمده است. چرا لازم است: روی رایانه‌ای که «Python install manager» دارد، میان‌بر `python.exe` در `WindowsApps` اول PATH است؛ وقتی آن میان‌بر خراب باشد اجرا نمی‌شود، و `run.bat` که فقط اولین `python` را امتحان می‌کرد می‌گفت « Python was not found on this machine» روی رایانه‌ای که پایتون سالم داشت. حالا هر نامزد امتحان می‌شود و اولین خوب برداشته می‌شود.

آزمون `every door asks for the same Python` (فصل ۲۳) بررسی می‌کند که کف نسخه‌ی ۳٫۱۱ در این فایل و در `run.sh` یکی است و هیچ‌کدام از دو فایل bat خودش `python` خالی را اجرا نمی‌کند.

۲۱.۳ `run.sh` — همتای لینوکسی run.bat

مسیر: `paramgen/run.sh`

یک اسکریپت `sh` ساده (POSIX؛ چیزی مخصوص bash ندارد تا روی dash، `sh` اوبونتو، هم اجرا شود). همان کارهای `run.bat` را می‌کند، به‌علاوه‌ی چند ظرافت لینوکس:

- پیوند نمادین (symlink) به خودش را دنبال می‌کند تا پوشه‌ی واقعی را پیدا کند.
- اگر با دابل‌کلیک در مدیر فایل و بی‌ترمینال اجرا شده، یک پنجره‌ی ترمینال (`x-terminal-emulator`، `gnome-terminal`، `konsole`، …) باز می‌کند و خودش را در آن دوباره اجرا می‌کند.
- `trap : INT`: بدون آن، Ctrl+C که ویرایشگر را متوقف می‌کند، خود اسکریپت را هم می‌کشت (dash این کار را دیرتر و پس از پایان منو می‌کند) و ترمینال بی‌echo می‌ماند. **این سطر را برندارید**؛ آزمون اسناد هم وجودش را بررسی می‌کند.
- تابع `pause` همتای `pause` ویندوز (با `stty` و `dd`).
- اول `python3` و بعد `python`، و بررسی نسخه‌ی ۳٫۱۱.

> **هشدار:** پایان سطر این فایل **باید LF** باشد؛ با CRLF، لینوکس سطر اول را `/bin/sh\r` می‌خواند و اجرا نمی‌شود. `.gitattributes` این را تعیین می‌کند و `run_docs_test.py` بررسی می‌کند. اگر با ویرایشگر ویندوزی بازش کردید، پیش از ذخیره پایان سطر را LF نگه دارید.

۲۱.۴ `run.py` — منوی همه‌ی کارها

مسیر: `paramgen/bin/run.py`

پیش‌تر چهار فایل bat کنار هم بود و آدم باید حدس می‌زد کدام را می‌خواهد؛ حالا یک در با کارهای نوشته‌شده روی آن. منوی واقعی:

```
================================================================================
  Parameter list code generator 01.01.01
================================================================================
  project: Flight Computer  (D:\Projects\Flight_Computer)

   1  Editor          open a project and edit its parameter lists
   2  Generate code   write the C files of the project opened last
   3  Tests           run every self test of the tool
   4  Check this PC   is the Python on this machine complete enough?

   0  Quit

  Select an option [1]:
```

هر کار در فرایند جداگانه اجرا می‌شود و این عمدی است: ویرایشگر تا Ctrl+C منتظر می‌ماند و Ctrl+C باید ویرایشگر را متوقف کند و منو را برگرداند، نه منو را هم بکشد؛ کاری که از کار بیفتد منو را سر پا می‌گذارد؛ و هر کار کد خروج و آرگومان‌ها و `--help` خودش را دارد.

`Action` کلاس

```
class Action(NamedTuple):
```

`NamedTuple` (بخش ۴.۵) یک کار منو: `key` (عددی که تایپ می‌شود)، `names` (نام‌های جایگزین، مثل `editor`، `gui`، `e`)، `title`، `what` (توضیح کنار عنوان)، `script` (فایل `run_*.py`)، `arguments` (آرگومان همیشگی؛ برای generate، `-v`) و `endings` (کد خروج ← جمله‌ی پایانی؛ کلید `None` یعنی «هر کد دیگر»).

لیست `ACTIONS` چهار کار منوست؛ **افزودن یک گزینه‌ی منو یعنی یک** `Action` **تازه در این لیست و یک فایل** `run_*.py` **تازه در** `bin/` (فصل ۲۴). ثابت `QUIT` کلمه‌هایی است که منو را می‌بندند (`0`، `q`، `quit`، `exit`، `bye`).

کار	نام
کاری که کاربر منظور داشته، با عدد یا نام (بی‌توجه به بزرگی حروف).	`find`
منو را چاپ می‌کند، با نام پروژه‌ای که کارها روی آن انجام می‌شوند (`last_project`).	`show_menu`
یک کار را با `subprocess.call` و پوشه‌ی جاری برنامه اجرا می‌کند و کد خروجش را برمی‌گرداند؛ Ctrl+C پایان عادی است. اگر فایل اسکریپت نیست، کد ۲.	`run`
جمله‌ی پایانی کار (`endings`)؛ فقط وقتی کار بی‌آرگومان اضافه اجرا شده (برای `generate --check` «فایل‌ها در پوشه‌ی خروجی‌اند» دروغ بود).	`report`
حلقه‌ی بپرس، اجرا کن، دوباره بپرس؛ Enter خالی یعنی ۱ (ویرایشگر).	`menu`
بی‌آرگومان ← `menu()`؛ با نام کار ← همان کار با بقیه‌ی آرگومان‌ها (`argparse.REMAINDER`)؛ نام ناشناخته ← کد ۲.	`main`

۲۱.۵ چهار درِ ورودی: run_gui، run_generate، run_tests و run_setup

`run_gui.py` — ویرایشگر

مسیر: `paramgen/bin/run_gui.py`

آرگومان‌ها را می‌خواند، logging را راه می‌اندازد و `GuiServer(...).serve_forever(...)`. پروژه به ترتیب: آرگومان مکانی، `-c`، یا آخرین پروژه (`last_project`)؛ بی‌پروژه هم شروع می‌شود (صفحه‌ی شروع).

کار	گزینه
پروژه برای باز کردن (پوشه یا `pl_project.json`)	`project` یا `-c`
درگاه؛ پیش‌فرض ۸۷۶۵. اگر برنامه‌ی دیگری آن را گرفته: `run.bat editor -p 8800`	`-p PORT`
مرورگری باز نشود؛ فقط نشانی چاپ شود (برای آزمون یا رایانه‌ی بی‌میزکار)	`--no-browser`
زبانه‌ی مرورگر پیش‌فرض به‌جای پنجره‌ی جدا	`--tab`

`run_generate.py` — تولیدکننده از خط فرمان

مسیر: `paramgen/bin/run_generate.py`

فقط `pl_codegen.cli.main` را اجرا می‌کند؛ همه‌ی گزینه‌ها در فصل ۱۵ آمده‌اند. تابعی ندارد.

`run_tests.py` — همه‌ی آزمون‌ها

مسیر: `paramgen/bin/run_tests.py`

لیست `SUITES` هشت مجموعه‌ی آزمون را به ترتیب دارد؛ هرکدام در فرایند جدا اجرا می‌شود و در پایان یک حکم: «All 8 suite(s) passed.» یا «N suite(s) FAILED: ...» با کد ۱. مجموعه‌ای که فایلش نیست «missing» گزارش می‌شود، نه «passed». `-l` فهرست را بی‌اجرا چاپ می‌کند:

```
> py paramgen\bin\run_tests.py -l
  [ok ] validation rules                  tests/run_validation_test.py
  [ok ] the editor                        tests/run_editor_test.py
  [ok ] every project setting             tests/run_settings_test.py
  [ok ] the documentation                 tests/run_docs_test.py
  [ok ] setting the version               tests/run_version_test.py
  [ok ] structure types                   tests/run_struct_test.py
  [ok ] the parameter-table format        tests/run_paramtable_test.py
  [ok ] generated code compiles and runs  tests/run_compile_test.py
```

خروجی‌اش با `errors="replace"` تنظیم شده تا اگر به فایل یا pipe فرستاده شد، نویسه‌ای که code page ویندوز ندارد اجرا را با `UnicodeEncodeError` متوقف نکند. **آزمون تازه یعنی یک ردیف در** `SUITES` (فصل ۲۳).

`run_setup.py` — بررسی رایانه

مسیر: `paramgen/bin/run_setup.py`

فقط `setup_environment.main` را اجرا می‌کند. تابعی ندارد.

۲۱.۶ `setup_environment.py` — آیا این رایانه می‌تواند ابزار را اجرا کند؟

مسیر: `paramgen/src/setup_environment.py`

چیزی نصب نمی‌کند: ابزار **فقط از کتابخانه‌ی استاندارد پایتون** استفاده می‌کند (نه `pip`، نه `requirements.txt`، نه اینترنت). این یک تصمیم است، نه اتفاق: رایانه‌های شرکت بی‌اینترنت‌اند و ابزاری که بسته‌ای لازم دارد، آنجا قابل استفاده نیست. این اسکریپت فقط با کلمات ساده می‌گوید پایتون این رایانه کافی است یا نه. خروجی واقعی:

```
================================================================================
Parameter list code generator - environment check
================================================================================
Python      : 3.11.9 (C:\Users\...\python.exe)
Platform    : Windows 10
App folder  : F:\...\paramgen

Checking the standard library modules the tool uses:
  [ok]      xml.etree.ElementTree    - reading and writing the parameter sub list XML files
  [ok]      http.server              - the local web server behind the graphical editor
  ...
Checking the tool:
  [ok]      pl_codegen.pipeline      - the code generator
  [ok]      pl_gui.server            - the graphical editor
--------------------------------------------------------------------------------
Everything is ready - no package needs to be installed.
```

ثابت	معنی
`MINIMUM_PYTHON`	`(3, 11)`: کمترین نسخه‌ی پایتون. **بالا بردن آن یعنی تغییر همین‌جا، در `run.bat`، در `run.sh` و در راهنماها**
`TOOLS_DIR`	پوشه‌ی `src`
`STANDARD_MODULES`	ماژول‌های استانداردی که بررسی می‌شوند، با دلیل استفاده؛ روی پایتون‌های «سبک» یا embedded ممکن است نباشند

نام	کار
`_print_header`	سرآیند: نسخه‌ی پایتون، سیستم‌عامل و پوشه‌ی برنامه.
`_red`	متن قرمز روی کنسول؛ رنگ را از `logging_utils` می‌گیرد ولی import را درون تابع انجام می‌دهد، چون همین بررسی‌ها هستند که می‌گویند ابزار import می‌شود یا نه.
`check_python_version`	آیا نسخه کافی است؛ اگر نه، پیام قرمز و توقف.
`check_standard_modules`	هر ماژول `STANDARD_MODULES` را import می‌کند و `[ok]` یا `[MISSING]` چاپ می‌کند.
`check_tool_imports`	خود ابزار را import می‌کند (`pl_codegen.pipeline` و `pl_gui.server`)؛ همان بررسی‌ای که واقعاً مهم است.
`_print_footer`	دستورهای راه‌اندازی، به شکلی که **این** رایانه دارد (`run.bat` یا `./run.sh`).
`main`	سه بررسی به ترتیب؛ اولین شکست ← کد ۲؛ همه موفق ← پانویس و کد ۰. `--check-only` فقط برای سازگاری پذیرفته می‌شود.

۲۱.۷ تغییر نسخه‌ی ابزار: set_version.bat و set_version.py

نسخه‌ی ابزار فقط در **یک سطر** نوشته شده: `__version__` در `src/pl_codegen/__init__.py` (فصل ۹). همه‌ی جاهای دیگر از آنجا می‌خوانند: سطر اول منوی `run.bat`، `run.bat generate --version`، سرآیند هر فایل C تولیدی و `generatorVersion` هر ساب‌لیستی که ویرایشگر ذخیره می‌کند. این دو فایل فقط همان یک سطر را عوض می‌کنند؛ **راهنماها (همین راهنما و راهنمای کاربر) با دست به‌روز می‌شوند.**

`set_version.bat` — دابل‌کلیک برای تغییر نسخه

مسیر: `paramgen/bin/set_version.bat`

مثل `run.bat`: پوشه‌ی خودش، پیدا کردن پایتون با `find_python.bat`، سپس `"%PYTHON%" set_version.py %*`. اگر بی‌آرگومان اجرا شده (دابل‌کلیک) یا شکست خورده، `pause`. جزو منوی `run.bat` نیست چون کسی که فقط فهرست ویرایش می‌کند هرگز به آن نیاز ندارد.

نمونه‌ی اجرا (دابل‌کلیک و تایپ `01.01.02`):

```
  ParamGen is version 01.01.01 now.

  Type the new version as three pairs of digits, like xx.xx.xx
  Press Enter on its own to keep 01.01.01.

  New version: 01.01.02

  Done: ParamGen is now version 01.01.02 (it was 01.01.01).

  The run.bat menu and 'run.bat generate --version' show it right away.
  Generated files and saved sub lists carry it from the next time they
  are generated or saved.
```

و با قالب غلط:

```
> set_version.bat 1.2.3
  ParamGen is version 01.01.01 now.

  '1.2.3' is not a valid version. Use three pairs of digits separated by dots, like
01.01.85.
  Nothing changed.
```

`set_version.py` — خواندن و نوشتن سطر نسخه

مسیر: `paramgen/bin/set_version.py`

ثابت	معنی
`INIT_FILE`	`paramgen/src/pl_codegen/__init__.py`
`VERSION_FORMAT`	`\d{2}\.\d{2}\.\d{2}`: دقیقاً دو رقم در هر بخش (`01.01.85`)
`EXAMPLE`	`01.01.85`؛ در همه‌ی پیام‌ها یکسان
`VERSION_LINE`	عبارت منظم سطر `__version__ = "..."` با سه گروه (پیش، نسخه، گیومه‌ی پایانی)

نام	کار
`VersionError`	استثنا: نسخه تنظیم نشد؛ پیامش دلیل را می‌گوید.
`check_format`	قالب را بررسی می‌کند؛ غلط ← `VersionError`.
`current_version`	نسخه‌ی فعلی؛ اگر سطر نسخه دقیقاً یک بار نیامده باشد، `VersionError`.
`_read`	فایل را با `newline=""` می‌خواند تا CRLF همان CRLF بماند.

`set_version` تابع

```
def set_version(version: str, init_file: Path = INIT_FILE) -> str:
```

نسخه را می‌نویسد و نسخه‌ی قبلی را برمی‌گرداند. فقط متن بین دو گیومه عوض می‌شود؛ بقیه‌ی فایل، پایان سطرها هم، بایت به بایت همان می‌ماند. اگر قالب غلط است یا سطر دقیقاً یک بار نیست، هیچ چیزی نوشته نمی‌شود؛ و اگر نسخه همان است، فایل اصلاً لمس نمی‌شود.

ورودی‌ها

`version`	نسخه‌ی تازه، مثل `01.01.02`
`init_file`	فایل؛ آزمون‌ها یک کپی می‌دهند

خروجی: نسخه‌ی قبلی.

خطا: `VersionError`.

`main` تابع

```
def main(argv: list) -> int:
```

اگر آرگومان داده شده همان را، وگرنه از کاربر می‌پرسد (Enter خالی یعنی بی‌تغییر). `lstrip("\ufeff")` نشانه‌ی BOM را که PowerShell جلوی متن pipeشده می‌گذارد حذف می‌کند. `INIT_FILE` درون تابع خوانده می‌شود (نه به‌صورت مقدار پیش‌فرض) تا آزمون بتواند آن را به یک کپی ببرد.

خروجی: کد خروج: ۰ (تغییر کرد یا لازم نبود)، ۱ (قالب غلط، لغو).

> **مهم:** این ابزار را روی درخت واقعی فقط وقتی اجرا کنید که واقعاً نسخه‌ی تازه می‌خواهید. آزمون `run_version_test.py` همیشه روی کپی کار می‌کند و یکی از آزمون‌هایش بررسی می‌کند که هیچ فایل دیگری نسخه را با دست ننوشته باشد.

۲۲. کتابخانه‌ی C: smart_data و parameter_descriptor

کدی که ابزار تولید می‌کند روی دو کتابخانه‌ی **دست‌نویس** فرمور ساخته می‌شود؛ تولید نمی‌شوند و ابزار به آن‌ها دست نمی‌زند:

- `smart_data`: «داده‌ی هوشمند»؛ هر مقدار یک شیء با بازه، پیش‌فرض، رزولوشن، واحد، زمان آخرین نوشتن و پرچم «تازه» برای هر خواننده.
- `parameter_descriptor`: شکل یک ردیف جدول پارامترها (`sParameterDescriptor`) و تابع‌های نوشتن که پس از نوشتن موفق یک رویداد می‌فرستند.

> **مهم:** این دو پوشه مال فرمور است. هر تغییری در آن‌ها باید با صاحب فرمور هماهنگ شود، و هر تغییر در **شکل** `sParameterDescriptor` یا **نام** تابع‌های smart data باید هم‌زمان در تولیدکننده هم اعمال شود (`descriptor_layout.py`، `parameter_list_c.py`، `types.py`). دستورالعمل در فصل ۲۴.

۲۲.۱ `smart_data.h` — رابط smart data

مسیر: `smart_data/smart_data.h`

نوع‌ها

اگر پیش‌تر تعریف نشده باشند (`#ifndef float32_t__`)، `float32_t` (`float`)، `float64_t` (`double`) و `bool_t` (`bool`) و ماکروهای `TRUE` و `FALSE` را تعریف می‌کند. کد تولیدی همین نام‌ها را به کار می‌برد.

مقدارها و معنی	enum
`eSMART_DATA_OK` (۰)؛ `_ERROR_NULL_POINTER`؛ `_ERROR_DEFAULT_VALUE_OUT_OF_RANGE` (سازنده)؛ `_ERROR_VALUE_OUT_OF_RANGE` (نوشتن یا تغییر حد)؛ `_ERROR_NOT_APPLICABLE`؛ `_ERROR_INVALID_USER_NUMBER`؛ `_ERROR_DATA_TYPE_NOT_SUPPORTED`؛ `_ERROR_NO_NEW_DATA` (پاسخ عادیِ «چیز تازه‌ای نیست»، نه خطا)؛ `_ERROR_DATA_NOT_CHANGED`	`eSmartDataStatus`
`eSMART_DATA_TYPE_B8` تا `_F64` و `_ENUMERATION`؛ در `_core.TypeInfo.Type` هر شیء	`eSmartDataType`
`eSMART_DATA_USER_ID_1` تا `_32` و `_QTY`؛ هر خواننده‌ای که «تازه برای من؟» می‌پرسد یک شناسه می‌گیرد (یک بیت از `UserFlags`)	`eSmartDataUserId`

ساختارها

همه‌ی شیءها با `sSmartDataCore _core` شروع می‌شوند (و **باید** اولین عضو باشد): `TypeInfo` (نوع، `SmartDataSize`، `ValueSize`)، `TimestampUs`، `LastReadTimestampUs` و `ReadWriteStatus` (`UserFlags`، `HasBeenWrittenOnce`). بعد از آن، نمونه‌ی `sSmartDataU16`:

```
typedef struct {
    sSmartDataCore _core;      /* همیشه اول */
    uint16_t Value;            /* مقدار */
    uint16_t _min;
    uint16_t _max;
    uint16_t _defaultValue;
    uint16_t _resolution;
    const char *pUnit;         /* واحد؛ اگر نیست "" */
} sSmartDataU16;
```

`sSmartDataB8` فقط `Value` و `_defaultValue` دارد (بی بازه، رزولوشن و واحد).

خانواده‌ی تابع‌ها

برای هر نوع، یک خانواده‌ی **یکسان** از تابع‌ها هست؛ فقط پسوند و نوع C فرق می‌کند. نام همیشه `fSmartData_<پسوند>_<کار>` است و اولین آرگومان `me` اشاره‌گر به شیء است (شبیه `self` در پایتون). همه جز `HasBeenWrittenSoFar` یک `eSmartDataStatus` برمی‌گردانند.

سازنده‌ی تولیدی برای	struct	نوع C	پسوند
`bool_t`	`sSmartDataB8`	`bool_t`	`B8`
`int8_t` / `uint8_t`	`sSmartDataI8` / `sSmartDataU8`	`int8_t` / `uint8_t`	`I8` / `U8`
…	`sSmartDataI16` / `sSmartDataU16`	`int16_t` / `uint16_t`	`I16` / `U16`
…	`sSmartDataI32` / `sSmartDataU32`	`int32_t` / `uint32_t`	`I32` / `U32`
…	`sSmartDataI64` / `sSmartDataU64`	`int64_t` / `uint64_t`	`I64` / `U64`
…	`sSmartDataF64` / `sSmartDataF32`	`float64_t` / `float32_t`	`F64` / `F32`

این پسوندها همان `suffix` جدول `VALUE_TYPES` در `types.py` (فصل ۱۰) است؛ تولیدکننده نام struct و سازنده را از روی آن می‌سازد. کارها، با نمونه‌ی `U16`:

آرگومان‌ها پس از `me`	چه می‌کند	کار
`(me, min, max, default, resolution, pUnit)` برای B8؛ فقط `(me, default)`	سازنده: نوع، اندازه‌ها، زمان صفر، `UserFlags` همه یک، «هنوز نوشته نشده»، مقدار = پیش‌فرض. اگر پیش‌فرض بیرون بازه باشد یا کمینه > بیشینه، `..._DEFAULT_VALUE_OUT_OF_RANGE` ← واحد `NULL` → `""`.	`Ctor`
`(me, pOut)`	مقدار فعلی (یا پیش‌فرض اگر هنوز کسی ننوشته). `LastReadTimestampUs` را تازه می‌کند.	`Read`

کار	چه می‌کند	آرگومان‌ها پس از me
ReadAndGetTimestampUs	همان، به‌علاوه‌ی زمان آخرین نوشتن (میکروثانیه).	(me, pOut, pOutTs)
ReadIfChanged	فقط اگر مقدار با مقدار مرجع فرق دارد؛ وگرنه ..._DATA_NOT_CHANGED.	(me, pRef, pOut)
ReadIfChangedAndGetTimestampUs	همان با زمان.	(me, pRef, pOut, pOutTs)
ReadIfIsNewerThanTimestampUs	فقط اگر زمان آخرین نوشتن تازه‌تر از زمان داده‌شده است؛ وگرنه ..._NO_NEW_DATA. مقایسه در برابر سرریز شمارنده امن است (_IS_NEWER).	(me, pOut, ts)
ReadIfIsNewerThanTimestampUsAndGetTimestampUs	همان با زمان.	(me, pOut, ts, pOutTs)
ReadIfIsUpdatedWithinUs	فقط اگر در duration میکروثانیه‌ی اخیر نوشته شده؛ وگرنه ..._NO_NEW_DATA.	(me, pOut, durationUs)
ReadIfIsUpdatedWithinUsAndGetTimestampUs	همان با زمان.	(me, pOut, durationUs, pOutTs)
ReadIfIsNewForId	فقط اگر از آخرین خواندنِ **این خواننده** چیزی نوشته شده؛ بیت آن خواننده پاک می‌شود. شناسه‌ی بیرون ۰ تا ۳۱ ← ..._INVALID_USER_NUMBER.	(me, pOut, userId)
ReadIfIsNewForIdAndGetTimestamp	همان با زمان.	(me, pOut, userId, pOutTs)
Write	مقدار را اگر در بازه است می‌نویسد، زمان را از fChrono_GetContinuousTickUs() می‌گذارد، «نوشته شد» و همه‌ی بیت‌های خواننده‌ها را روشن می‌کند. بیرون بازه ← ..._VALUE_OUT_OF_RANGE و مقدار قبلی می‌ماند. رزولوشن بررسی **نمی‌شود**.	(me, pValue)
WriteWithTimestampUs	همان، با زمانی که فراخوان می‌دهد.	(me, pValue, ts)
HasBeenWrittenSoFar	TRUE اگر از زمان ساخت دست‌کم یک بار نوشته شده (خروجی bool_t، نه وضعیت).	(me)
Verify	آیا مقدار فعلی در بازه است. (B8 ندارد.)	(me)
SetMinValue / GetMinValue	کمینه را عوض می‌کند (اگر از بیشینه بزرگ‌تر نباشد) / می‌خواند. (B8 ندارد.)	(me, pNew) / (me, pOut)
SetMaxValue / GetMaxValue	بیشینه، با همان قاعده. (B8 ندارد.)	(me, pNew) / (me, pOut)
SetDefaultValue / GetDefaultValue	پیش‌فرض (باید در بازه باشد).	(me, pNew) / (me, pOut)
SetResolution / GetResolution	رزولوشن؛ فقط نگه داشته می‌شود. (B8 ندارد.)	(me, pNew) / (me, pOut)
GetTimestamp	زمان آخرین نوشتن.	(me, pOutTsUs)

آرگومان‌ها پس از me	چه می‌کند	کار
`(me, pOutSize)`	`sizeof` کل شیء / `sizeof` مقدار.	`GetSmartDataSize / GetValueSize`

نمونه‌ی امضای کامل: `eSmartDataStatus fSmartData_U16_Write(sSmartDataU16 * const me, uint16_t *pValue);` و نمونه‌ی استفاده:

```
uint16_t altitude = 4500U;
if (fSmartData_U16_Write(&Cg_SmartData_MaxAltitude, &altitude) != eSMART_DATA_OK) {
    /* بیرون بازه‌ی 0..6000 است؛ مقدار قبلی مانده */
}

uint16_t now = 0U;
(void)fSmartData_U16_Read(&Cg_SmartData_MaxAltitude, &now);
```

۲۲.۲ `smart_data.c` — پیاده‌سازی smart data

مسیر: `smart_data/smart_data.c`

حدود ۵۰۰۰ سطر، ولی ساده: کار مشترک همه‌ی خانواده‌ها در **ماکروهای خصوصی** بالای فایل است و هر تابع چند سطر است که همان ماکروها را صدا می‌زند. نمونه:

```
eSmartDataStatus fSmartData_U16_Write(sSmartDataU16 * const me, uint16_t *pValue){

    CHECK_RANGE_(me, pValue);
    WRITE_(me, pValue);

    return eSMART_DATA_OK;
}
```

هر ماکرو در `do { ... } while (0)` پیچیده شده تا مثل یک دستور رفتار کند، و برای خطا با `return` از **تابعی که آن را صدا زده** خارج می‌شود.

کار	ماکرو
اشاره‌گر `NULL` ← `eSMART_DATA_ERROR_NULL_POINTER`	`ASSERT_NOT_NULL_`
کمینه > بیشینه یا پیش‌فرض بیرون بازه ← `..._DEFAULT_VALUE_OUT_OF_RANGE`	`ASSERT_DEFAULT_VALUE_`
مقدار بیرون `[_min, _max]` ← `..._VALUE_OUT_OF_RANGE`	`CHECK_RANGE_`
خواندن‌ها؛ هرکدام `LastReadTimestampUs` را تازه می‌کند	`READ_`، `READ_AND_GET_TIMESTAMP_US_`، `READ_IF_...`
`(int64_t)(ts - ref) > 0`: «تازه‌تر» حتی وقتی شمارنده‌ی ۶۴بیتی سرریز کند	`IS_NEWER_`
نوشتن؛ `WRITE_` زمان را از `fChrono_GetContinuousTickUs()` می‌گیرد	`WRITE_`، `WRITE_WITH_TIMESTAMP_US_`
حدها، پیش‌فرض، رزولوشن، زمان و اندازه‌ها	`SET_..._`، `GET_..._`
شناسه‌ی خواننده باید کمتر از `eSMART_DATA_USER_ID_QTY` باشد	`CHECK_USER_ID_`

وابستگی به ساعت: فایل `#include "chrono.h"` دارد و `fChrono_GetContinuousTickUs()` را صدا می‌زند؛ این ماژول مال فرم‌ور است و در این مخزن نیست. آزمون `run_compile_test.py` اگر `chrono.h` واقعی پیدا نشود، یک stub می‌سازد که همیشه صفر برمی‌گرداند (فصل ۲۳).

> **نکته:** توضیح بالای `CHECK_RANGE_` می‌گوید شرط به شکل «نه (max => value => min)» نوشته شده تا NaN رد شود؛ ولی کد فعلی `(value < min) || (value > max)` است و چون هر مقایسه با NaN نادرست است، **NaN از این بررسی می‌گذرد**. آزمونی که این را بررسی می‌کرد به درخواست صاحب پروژه برداشته شد. اگر روزی رد کردن NaN لازم شد، جایش همین ماکرو است (با هماهنگی صاحب فرم‌ور).

۲۲.۳ `parameter_descriptor.h` — شکل یک ردیف جدول پارامترها

مسیر: `parameter_descriptor/parameter_descriptor.h`

دست‌نویس است و **تولید نمی‌شود**؛ شکل جدولی را تعریف می‌کند که تولیدکننده پر می‌کند. توضیح بالای خود فایل یادآوری می‌کند که هر فیلد تازه باید در `descriptor_layout.py` هم اضافه شود.

فیلد	نوع	معنی
pName	const char *	نام پارامتر همان‌طور که در ویرایشگر نوشته شده؛ فیلد با نام نقطه‌دار
pUserName	const char *	نامی که آدم می‌خواند (ایستگاه زمینی، لاگ)؛ برای هر مقدار جدا؛ "" نه NULL
Type	eParameterType	MONITORING، SETTING یا COMMAND
pSmartData	void *	شیء smart data همین یک مقدار
LogId	uint16_t	شناسه‌ی لاگ؛ فقط وقتی TagLoggable درست است؛ وگرنه ۰
Tag4 … TagSystemName	uint32_t	یک فیلد برای هر تگ، به ترتیب TAG_REGISTRY
pDescription	const char *	توضیح؛ "" نه NULL

ترتیب این فیلدها **باید** با descriptor_field_order() در descriptor_layout.py یکی باشد (فصل ۱۴). ArraySize عمداً نیست: هر توصیف‌گر یک مقدار است و اندازه‌ی آرایه ماکروی <NAME>_ARRAY_SIZE است.

تعریف	معنی
eParameterDescriptorStatus	ePARAMETER_DESCRIPTOR_OK، _ERROR_NULL_REFERENCE، _ERROR_WRITE
eParameterType	ePARAMETER_TYPE_MONITORING (۰)، _SETTING، _COMMAND؛ تولیدکننده نام را با type_enum_member می‌سازد و قاعده‌ی GRP002 فقط همین سه را می‌پذیرد (KNOWN_TYPES)
PARAMETER_LIST_WEAK	ویژگی weak مستقل از کامپایلر (همان تعریف هدر تولیدی؛ هر دو با #ifndef محافظت شده‌اند)
PARAMETER_LIST_UNUSED_(x)	(void)(x) برای خاموش کردن هشدار پارامتر بی‌استفاده

۲۲.۴ parameter_descriptor.c — نوشتن از راه جدول و رویداد تغییر

مسیر: parameter_descriptor/parameter_descriptor.c

یازده تابع fParameterDescriptor_Write<پسوند>(pParamDesc, value) با یک الگو: اگر توصیف‌گر NULL است _ERROR_NULL_REFERENCE؛ pSmartData را به struct همان نوع تبدیل (cast) و fSmartData_<پسوند>_Write را صدا می‌زند؛ اگر رد شد _ERROR_WRITE؛ وگرنه fParameterDescriptor_ParameterWriteCompletedCallback(pParamDesc, NULL) و OK. پس رویداد «تغییر کرد» فقط پس از نوشتنی که smart data **پذیرفته** فرستاده می‌شود.

```
eParameterDescriptorStatus fParameterDescriptor_WriteU16(const sParameterDescriptor *
pParamDesc,
                                                         uint16_t value) {
  if(pParamDesc == NULL) {
    return ePARAMETER_DESCRIPTOR_ERROR_NULL_REFERENCE;
  }
  sSmartDataU16 *pSd = (sSmartDataU16*)(pParamDesc->pSmartData);
  if(fSmartData_U16_Write(pSd, &value) != eSMART_DATA_OK) {
    return ePARAMETER_DESCRIPTOR_ERROR_WRITE;
  }
  fParameterDescriptor_ParameterWriteCompletedCallback(pParamDesc, NULL);
  return ePARAMETER_DESCRIPTOR_OK;
}
```

هشدار: این تابع‌ها نوع را بررسی **نمی‌کنند**: `fParameterDescriptor_WriteU16` روی توصیف‌گر یک پارامتر `float32_t`، حافظه‌ی شیء را با نوع غلط می‌نویسد. فراخوان باید تابع درست نوع پارامتر را صدا بزند (نوع را از نام یا از `_core.TypeInfo.Type` شیء می‌شود فهمید).

`fParameterDescriptor_ParameterWriteCompletedCallback` به‌صورت weak و خالی تعریف شده؛ برنامه نسخه‌ی خودش را بی `PARAMETER_LIST_WEAK` می‌نویسد (مثلاً برای لاگ یا ارسال تله‌متری).

۲۲.۵ `parameter_list_usage_example.c` — نمونه‌ی استفاده از کد تولیدی در فرم‌ور

مسیر: `paramgen/docs/example/parameter_list_usage_example.c`

یک فایل دست‌نویس که با هشت تابع کوتاه به هشت پرسش رایج برنامه‌نویس فرم‌ور جواب می‌دهد. آزمون `run_compile_test.py` آن را **با کد تازه تولیدشده کامپایل و اجرا می‌کند**؛ پس هر تغییری در نام‌های تولیدی که این نمونه را بشکند، آزمون را رد می‌کند. هدر کنارش (`parameter_list_usage_example.h`) پروتوتایپ همین هشت تابع است. پارامترهایی که نمونه نام می‌برد مال **مدل آزمون** است (`_EXAMPLE_XML` در `run_compile_test.py`، فصل ۲۳)، نه پروژه‌ی نمونه‌ی این راهنما؛ نمونه با هر پروژه‌ای که همان پارامترها را داشته باشد هم کامپایل می‌شود (`_EXAMPLE_NEEDS`).

پرسش	تابع
کتابخانه را چطور راه بیندازم؟ (`fParameterList_Init()`، یک بار پیش از هر چیز)	`fExample_Start`
یک مقدار را چطور بخوانم؟ (`fSmartData_F32_Read`)	`fExample_Read`
چطور بنویسم؟ (بیرون بازه رد می‌شود و مقدار قبلی می‌ماند)	`fExample_Write`
چطور بنویسم و رویداد تغییر بفرستم؟ (`fParameterDescriptor_WriteF32`)	`fExample_WriteAndNotify`
فیلدی درون استراکچر؟ (`&Cg_SmartData_<Structure>.<Field>`؛ استراکچر درون استراکچر یک نام بیشتر)	`fExample_StructAccess`
همه‌ی عنصرهای یک آرایه؟ (حلقه تا `<NAME>_ARRAY_SIZE`، هر عنصر در یک بافر هم‌اندازه)	`fExample_ArrayAccess`
همه‌ی پارامترها بی دانستن نامشان؟ (حلقه روی `fParameterList_GetByIndex`)	`fExample_WalkTable`
فقط وقتی برای **من** تازه است؟ (`ReadIfIsNewForId` با شناسه‌ی خواننده)	`fExample_ReadIfNew`

و یک `fParameterList_InitCompletedCallback` که نسخه‌ی weak تولیدی را بازنویسی می‌کند؛ جای مناسب برای بازگرداندن مقدارهای Reset-proof. خلاصه‌ی راه‌اندازی در `main` یک فرمور:

```
#include "cg_parameter_list.h"

int main(void) {
    if (fParameterList_Init() != 0U) {   /* همه‌ی سازنده‌ها + callback */
        Error_Handler();
    }
    for (;;) {
        float32_t airspeed = 0.0f;
        (void)fSmartData_F32_Read(&Cg_SmartData_Airspeed, &airspeed);
        ...
    }
}
```

۲۳. آزمون‌ها: پوشه‌ی tests

ابزار هشت مجموعه آزمون دارد، **بی هیچ چارچوب آزمون** (نه pytest، نه unittest): هر فایل یک اسکریپت ساده‌ی پایتون است که چند تابع آزمون را اجرا می‌کند و کد خروج ۰ (همه گذشت) یا ۱ برمی‌گرداند. چیزی نصب نمی‌شود و اینترنت لازم نیست. **بعد از هر تغییری در ابزار، همه را اجرا کنید:**

```
run.bat tests

rem یا مستقیم:
py paramgen\bin\run_tests.py

rem یا فقط یک مجموعه:
py paramgen\tests\run_editor_test.py
```

هیچ آزمونی به پروژه‌ی واقعی شما دست نمی‌زند: هر آزمون پروژه‌ی خودش را در یک پوشه‌ی موقت (`tempfile.TemporaryDirectory`) می‌سازد و در پایان پاکش می‌کند. خروجی هر مجموعه برای هر آزمون یک سطر `[PASS]` یا `[FAIL]` با دلیل است.

مجموعه	چه چیزی را می‌آزماید	نیاز
`run_validation_test.py`	هر قاعده‌ی بررسی با یک مدل خراب در حافظه؛ و کیفیت پیام‌ها	—
`run_editor_test.py`	ویرایشگر از راه `Api` (همان که مرورگر صدا می‌زند) و چند قاعده‌ی خوانده‌شده از متن JS	—
`run_settings_test.py`	هر کلید `pl_project.json` واقعاً اثری دارد	—
`run_docs_test.py`	راهنمای کاربر و READMEها نام چیزی را نمی‌برند که دیگر نیست؛ `run.sh` سالم است	—
`run_version_test.py`	`set_version.py` فقط نسخه را عوض می‌کند	—
`run_struct_test.py`	تایپ‌های استراکچر: اعلان، استفاده، تغییر، حذف	—
`run_paramtable_test.py`	قالب `param_table`	—
`run_compile_test.py`	کد تولیدی با `gcc` (C99) و `g++` کامپایل و اجرا می‌شود	`gcc` و `g++` در PATH

هشدار: اگر `gcc` یا `g++` در PATH نباشد، `run_compile_test.py` با پیام زرد «SKIPPED» و **کد ۰** تمام می‌شود؛ یعنی «همه گذشت» گفته می‌شود بی‌آنکه کدی کامپایل شده باشد. روی رایانه‌ای که تغییر را تحویل می‌دهید، کامپایلر (مثلاً MinGW) را نصب کنید و مطمئن شوید این مجموعه واقعاً اجرا شده است.

یک آزمون چه شکلی است

الگوی همه‌ی مجموعه‌ها (جز validation و settings که جدول دارند) یکی است: تابعی به نام `test_...` که پوشه‌ی موقت می‌گیرد، کاری انجام می‌دهد و با `equal` یا `check` انتظار را می‌سنجد؛ و یک ردیف در لیست `CASES` پایین فایل. نمونه‌ی واقعی از `run_editor_test.py`:

```
def test_a_copy_is_named_like_a_copy(work: Path) -> None:
    """Duplicate names the copy the way a file manager does ..."""
    document, api, _config = open_copy(work, empty=True)
    for name in ("Speed", "Motor1"):
        api.add({"parent": "", "kind": "primitive"})
        api.update({"path": "NewParameter", "changes": {"name": name}})

    def copy(path: str) -> str:
        answer = api.duplicate({"path": path})
        check(answer["ok"], f"{path} was not duplicated: {answer.get('message')}")
        return answer["selected"]

    equal(copy("Speed"), "Speed_Copy", "the first copy")
    equal(copy("Speed"), "Speed_Copy2", "the second copy of the same parameter")
    ...

CASES = [
    ...
    ("a copy is named like a copy", test_a_copy_is_named_like_a_copy),
]
```

افزودن آزمون تازه = نوشتن تابع `test_...` در فایل مناسب + افزودن یک ردیف به `CASES` همان فایل. **افزودن مجموعه‌ی تازه** = یک فایل `run_..._test.py` که مثل بقیه `main()` دارد و کد خروج برمی‌گرداند + یک ردیف در `SUITES` در `bin/run_tests.py` (فصل ۲۱).

قاعده‌ی کار این پروژه: هر باگی که رفع می‌شود، یک آزمون می‌گیرد که **پیش از رفع رد** می‌شد. بسیاری از توضیح‌های آزمون‌ها با «(Reported.)» یا «(Asked for.)» تمام می‌شوند: یعنی از گزارش یا درخواست کاربر آمده‌اند. پیش از تغییر رفتاری که آزمونی برایش هست، توضیح همان آزمون را بخوانید؛ معمولاً می‌گوید چرا رفتار فعلی عمدی است.

۲۳.۱ `run_validation_test.py` — قاعده‌های بررسی

مسیر: `paramgen/tests/run_validation_test.py`

هر مورد یک مدل کوچک **در حافظه** می‌سازد (بی فایل)، `Validator` را اجرا می‌کند و می‌پرسد کد خطای مورد انتظار آمده یا نه (`expect: None` یعنی هیچ خطایی نباید باشد). مورد تازه یعنی یک دیکشنری در `CASES` (نمونه در فصل ۱۲). بخش دوم (`check_messages`) کیفیت پیام‌ها را می‌سنجد: پیام باید **کجا** را بگوید (تا قابل کلیک باشد)، **چه کنم** را بگوید (`hint`)، و برای یک اشتباه فقط یک بار بیاید. ثابت‌های `SYSTEM_NAMES`، `PARAM_MODES` و `PARAMETER_TYPES` فهرست‌های کشویی مدل آزمون‌اند.

کار	نام
تگ‌های پیش‌فرض معتبر، با جایگزینی هر کلیدی که داده شود.	`tags`

کار	نام
یک پارامتر `uint8_t` معتبر (۵ در بازه‌ی ۰ تا ۱۰)؛ هر فیلد قابل بازنویسی.	`value`
یک استراکچر با فرزندان داده‌شده.	`struct`
فیلد استراکچر: بی تگ و Type.	`field`
زیراستراکچر، مثل فیلد.	`inner`
مدل از پارامترها، با فهرست‌های کشویی و تایپ‌هایی که استراکچرها لازم دارند (`adopt_types` و `apply_types`).	`model`
استراکچری که تایپش هرگز تنظیم نشده (فایل دست‌کاری‌شده).	`_typeless_struct`
پارامتر ساده با نام تایپ استراکچر.	`_parameter_of_struct_type`
استراکچر با `float32_t`.	`_struct_of_value_type`
تایپی که خودش را دارد.	`_self_containing_type`
دو ساب‌لیست که هر دو `S` را اعلان کرده‌اند.	`_one_type_in_two_lists`
ساب‌لیستی که تایپ ساب‌لیست دیگر را استفاده کرده.	`_another_lists_type`
یک مورد را اجرا می‌کند و مجموعه‌ی کدهای **خطا** را برمی‌گرداند.	`run_case`
یک مدل که از هر خانواده یک قاعده را می‌شکند؛ برای بررسی پیام‌ها.	`broken_model`
بررسی کیفیت پیام‌ها روی مدل خراب و فایل؛ تعداد شکست‌ها.	`check_messages`
همه‌ی `CASES` و `check_messages`.	`main`

۲۳.۲ `run_editor_test.py` — ویرایشگر

مسیر: `paramgen/tests/run_editor_test.py`

همه‌چیز از `pl_gui.api.Api` می‌گذرد، دقیقاً همان که مرورگر صدا می‌زند؛ پس آزمونی که اینجا می‌گذرد یعنی دکمه‌ی نوار ابزار کار می‌کند. برخی آزمون‌ها رفتاری را که فقط در مرورگر رخ می‌دهد (کلیک، میان‌بر) از **متن** فایل‌های JS بررسی می‌کنند؛ پس تغییر نام تابع یا ثابتی در JS ممکن است یکی از این‌ها را رد کند. `APP_DIR` پوشه‌ی برنامه و `ROOT_DIR` ریشه‌ی مخزن است.

کار	نام
استثنای «یک انتظار برآورده نشد».	`Failure`
اگر شرط نادرست است، `Failure` با پیام.	`check`
اگر دو مقدار برابر نیستند، `Failure` با هر دو مقدار.	`equal`

کار	نام
یک `Document` و `Api` روی پروژه‌ی تازه‌ساز در پوشه‌ی موقت؛ با `empty=True` همه‌ی ساب‌لیست‌ها خالی می‌شوند تا آزمون دقیقاً بداند چه دارد. workspace فایل تنظیمات خودش را دارد تا به اخیرهای کاربر چیزی اضافه نشود.	`open_copy`
یک پروژه‌ی آزمون از صفر می‌سازد (قبلاً پروژه‌ی دمو کپی می‌شد و آزمون‌ها به محتوای امروزش وابسته بودند).	`_a_project`
ساخت کپی را به ساب‌لیست‌های پوشه‌ی `parameter_lists/` می‌برد.	`_build_from_the_list_folder`
همه‌ی ساب‌لیست‌های دیگر ساخت را روی دیسک خالی می‌کند.	`_empty_the_other_lists`
نام مقدارهای یک ساب‌لیست به ترتیب.	`_parameter_names`
همه‌ی آنچه `collect()` فرم برای خانه‌های دست‌نخورده می‌فرستد، با همان تبدیل JSON مرورگر.	`_as_the_form_sends`
آنچه جدول برای هر مقدار نشان می‌دهد همان است که `flatten()` می‌دهد.	`_texts_agree`
هر مورد `CASES` در پوشه‌ی موقت خودش؛ `[ERROR]`/`[FAIL]`/`[PASS]` و حکم پایانی.	`main`

آزمون‌ها:

کار	نام
استراکچر **از** یک تایپ است و تغییر تایپ به همه‌ی آن‌ها می‌رسد.	`test_structure_types`
آرایه‌ی استراکچر برای هر عنصر یک دسته مقدار می‌سازد.	`test_structure_arrays`
ساختن، انتخاب و ترتیب ساب‌لیست‌های پروژه.	`test_parameter_lists`
فایل پروژه‌ی دست‌کاری‌شده با حالت‌های عجیب (تکراری، ناموجود).	`test_list_selection_edge_cases`
تایپ مال ساب‌لیست خودش است؛ ساب‌لیست دیگر آن را نمی‌بیند و استفاده نمی‌کند.	`test_a_type_belongs_to_its_list`
تولید وقتی آفست مقداری را جابه‌جا کرده، می‌گوید (فصل ۱۳).	`test_layout_warning`
هیچ import بیرون از کتابخانه‌ی استاندارد و خود پروژه (با **خواندن** importها، نه اجرا).	`test_no_external_dependency`
هیچ تغییری فقط نیمی از صفحه را دوباره رسم نمی‌کند (از متن `app.js`).	`test_every_pane_redraws`
یک مفهوم همه‌جا یک نام دارد؛ نام‌های قدیمی (مثل `GroupRule`، `COLUMN_GROUPS`، `data_type_name`، `TagLogId`) در هیچ فایلی نمانده‌اند. **تغییر نام تازه را به فهرست همین آزمون اضافه کنید.**	`test_one_name_for_one_thing`
فهرست کشویی که بلافاصله پس از تایپ باز می‌شود، باز می‌ماند.	`test_a_drop_down_opens_the_first_time`
کلیک بلافاصله پس از تایپ در فرم، کار می‌کند (`onSteadyClick`).	`test_the_first_click_is_never_lost`

کار	نام
فیلد تازه همان مقدارهای شروع پارامتر تازه را می‌گیرد.	`test_a_new_field_starts_with_no_warning`
تغییر نوع، چیزی که نوع تازه نمی‌تواند نگه دارد باقی نمی‌گذارد (`_retune_for_type`).	`test_changing_the_value_type`
بولی همیشه ۱/۱/۰ و پیش‌فرض بولی واقعی دارد.	`test_a_boolean_keeps_its_shape`
پروژه روی درایو دیگر قابل پیمایش، باز کردن و ساختن است.	`test_every_drive_can_be_reached`
Open از پروژه، New کنار برنامه، ساب‌لیست از پوشه‌ی ساب‌لیست‌ها شروع می‌شود.	`test_the_dialogs_start_where_the_project_is`
مسیری که در جعبه‌ی پنجره تایپ می‌شود فوراً فهرست می‌شود (`listing_target`).	`test_a_typed_path_is_listed_at_once`
پنجره‌ی خود سیستم، و پیام روشن وقتی نیست.	`test_the_systems_own_dialog`
هر Open و Browse اول پنجره‌ی سیستم را امتحان می‌کند (از متن JS).	`test_the_dialogs_try_the_system_first`
فایل JSON که پروژه نیست، به‌عنوان پروژه باز نمی‌شود.	`test_only_a_project_file_opens`
پروژه‌ی بی‌ساب‌لیست پارامتر نمی‌پذیرد و می‌گوید چرا.	`test_no_list_no_parameter`
باز کردن ساب‌لیستی که در ساخت هست، آن را باز می‌کند (رد نمی‌کند).	`test_opening_a_list_already_in_the_build`
هر ردیف نمای ادغام‌شده ساب‌لیستش را می‌گوید.	`test_the_merged_view_names_every_rows_list`
ساب‌لیست smart data هم Address و userName و size را ذخیره می‌کند؛ یک شکل فایل برای هر دو قالب.	`test_one_file_shape_for_both_formats`
فایل‌های تولیدی ساب‌لیست‌ها را نسبت به ریشه‌ی پروژه نام می‌برند، نه با مسیر کامل.	`test_the_output_names_lists_from_the_project_root`
* دقیقاً وقتی هست که فهرست با فایل فرق دارد (`dirty`).	`test_the_unsaved_mark_means_different_from_the_file`
هر عنصر آرایه می‌تواند توضیح خودش را داشته باشد.	`test_a_value_has_a_description_of_its_own`
هر ردیف فقط به آنچه زیر آن است می‌رسد، در آرایه‌ی استراکچر هم.	`test_a_field_of_a_struct_array_too`
فهرست قدیمی با متن مشترک، همان‌طور خوانده می‌شود (`_give_every_row_its_text`).	`test_an_old_shared_text_goes_onto_every_row`
هر مقدار می‌تواند نام نمایشی خودش را داشته باشد.	`test_a_value_has_a_user_name_of_its_own`
ردیفی که یک مقدار است، همان مقدار را انتخاب می‌کند (از متن JS).	`test_the_grid_selects_one_value`
هر سه پنجره‌ی پوشه درایوها را نشان می‌دهند.	`test_the_dialogs_offer_the_drives`
کاری که انتخاب را جای تازه‌ای می‌برد، ردیف را به دید می‌آورد.	`test_a_row_put_somewhere_new_is_brought_into_sight`

کار	نام
میان‌برها با **کلید** (`event.code`) کار می‌کنند، نه حرف؛ تا روی صفحه‌کلید فارسی هم کار کنند.	`test_the_shortcuts_are_bound_to_the_key`
متن فرم خالی دکمه را با برچسب واقعی‌اش (`+ Add variable`) نام می‌برد.	`test_the_empty_form_names_the_add_button`
هر پیامی که می‌گوید «'+ چیزی' را بزن»، دکمه‌ی واقعی را نام می‌برد.	`test_a_hint_names_a_button_the_editor_has`
`bin/find_python.bat`، `run.sh` و بررسی رایانه همه یک کمینه‌ی پایتون (۳٫۱۱) دارند؛ `run.bat` و `set_version.bat` پایتون را با `find_python.bat` پیدا می‌کنند و هیچ‌کدام `python` خالی اجرا نمی‌کند.	`test_every_door_asks_for_the_same_python`
پایتون قدیمی‌تر از ۳٫۱۱ با پیام قرمز گفته می‌شود.	`test_an_old_python_is_told_in_red`
پنجره‌ی ساب‌لیست‌ها از #1 می‌شمارد، نه #0.	`test_the_sub_lists_are_numbered_from_one`
ستون Address بعد از تگ‌هاست و ستون پنهان پنهان می‌ماند.	`test_the_address_column_comes_before_the_description`
پروژه‌ای با نام قدیمی فایل (`project.json`) هنوز باز می‌شود.	`test_a_project_file_with_the_old_name_still_opens`
هر کاری که ساب‌لیست باز را عوض می‌کند، اگر تغییر ذخیره‌نشده هست، اول می‌پرسد.	`test_nothing_replaces_the_open_list_in_silence`
پیامِ ساب‌لیست دیگر، آن ساب‌لیست را باز می‌کند.	`test_a_problem_in_another_list_can_be_reached`
پیام‌ها launcher همین رایانه را نام می‌برند (`run.bat` یا `./run.sh`).	`test_the_messages_name_this_machines_launcher`
هر مسیر مرورگر در `browser.py` جایی است که مرورگر می‌تواند باشد.	`test_every_browser_path_can_exist`
خطاهای مرورگر به فایل می‌روند، نه pipe؛ پنجره قفل نمی‌شود.	`test_the_browser_can_always_write_its_errors`
بی‌مرورگر، ویرایشگر نشانی را می‌گوید.	`test_no_browser_says_so`
workspace فقط در فایل تنظیمات خودش می‌نویسد.	`test_a_workspace_keeps_to_its_own_settings_file`
مدخل اخیر روی ویندوز و لینوکس یکسان است.	`test_a_recent_entry_works_on_any_system`
فایل پروژه‌ای که برنامه می‌نویسد JSON ساده است.	`test_the_project_file_is_plain_json`
عدد تایپ‌شده در Tag1..Tag4 عدد است، مثل خواندن از فایل.	`test_a_free_tag_takes_a_number`
خانه‌ی Index فقط مقدارهای مدخل خودش را می‌خواند (دو هم‌نام در دو ساب‌لیست).	`test_the_grid_finds_a_rows_values_in_its_own_entry`
کپی `Name_Copy`، `Name_Copy2`… نام می‌گیرد.	`test_a_copy_is_named_like_a_copy`
در smart data حالت دسترسی `PARAM_MODE_NONE` است و عوض نمی‌شود: صفحه NONE نشان می‌دهد، کد NONE می‌نویسد، ویرایش (حتی با مقدار دیگر) مقدار فایل را عوض نمی‌کند و پارامتر تازه با NONE ذخیره می‌شود.	`test_a_smart_data_project_has_no_param_mode`

کار	نام
System names و Parameter modes با `#define` عددند و فقط `#ifndef`: `SYS_NAV = 16`، سطر بی‌عدد عدد بعدی، سطرهای غلط رد می‌شوند، enum دیگر نیست و دو نام یک عدد `DEF005` است. فهرست رایانه را هرگز نمی‌نویسد (`CHOICES_FILE` به پوشه‌ی موقت).	`test_the_lists_are_numbered_defines`
عنوان پنجره مسیر کامل فایل پروژه است (و *)، فقط از `state.project.file`.	`test_the_title_names_the_project_file`
فایل پروژه‌ی تغییرنام‌داده با فایلش، با پوشه‌اش و از خط فرمان باز می‌شود؛ پنجره‌ی Open همه‌ی JSONها را نشان می‌دهد؛ دو فایل پروژه در یک پوشه نام هر دو را می‌گوید؛ New در پوشه‌ی پروژه چیزی نمی‌نویسد.	`test_a_renamed_project_file_opens`

۲۳.۳ `run_settings_test.py` — هر تنظیم اثری دارد؟

مسیر: `paramgen/tests/run_settings_test.py`

یک پروژه‌ی دورانداختنی، و برای هر کلید: تغییرش بده، تولید کن (`run_generate.py` در فرایند جدا) و خروجی را نگاه کن. تنظیمی که خوانده می‌شود ولی هیچ اثری ندارد بدترین نوع باگ است. هر مورد با `case(کلید، مقدار، بررسی، چرا)` ثبت می‌شود؛ بررسی یک `lambda` است که ریشه‌ی پروژه و متن خروجی را می‌گیرد:

```
case("codegen.index_define_prefix", "IDX_",
     lambda root, text: "IDX_SPEED" in read(root, "out/cg_parameter_list_defines.h"),
     "the index macros")
```

کلید تازه در فایل پروژه یعنی یک `case` تازه اینجا (فصل ۲۴). ثابت‌های `LIST_XML` و `BASE` ساب‌لیست و تنظیمات پایه‌ی پروژه‌ی آزمون‌اند.

کار	نام
پوشه‌ی پروژه‌ای که `pl_project.json` آن دقیقاً همین تنظیمات است.	`make_project`
`run_generate.py -c ... -q` را اجرا می‌کند؛ (`کد خروج، متن`).	`generate`
کپی تنظیمات با یک کلید نقطه‌دار (`output.file_prefix`) تغییرکرده.	`deep`
یک مورد را به `CASES` اضافه می‌کند.	`case`
متن یک فایل نسبت به ریشه‌ی پروژه.	`read`
هر مورد را در پروژه‌ی جدا اجرا می‌کند.	`main`

۲۳.۴ `run_docs_test.py` — راهنماها با کد می‌خوانند؟

مسیر: `paramgen/tests/run_docs_test.py`

سه نوع بررسی روی فایل‌های `WHERE` (READMEها، **راهنمای کاربر Word**، الگوی پروژه، `run.bat` و `run.sh`؛ نه این راهنما):

- `GONE`: دیکشنری «متنی که دیگر نباید باشد ← آنچه امروز هست»؛ مثل `"plcogen": "paramgen"` یا `"snapshot_file": "full_parameter_list_file"`. هر بار چیزی تغییر نام می‌دهد، نام قدیمی را اینجا اضافه کنید.
- `KNOWN_PATHS`: هر مسیر فایلی که راهنما نام می‌برد باید در درخت باشد.
- `check_the_launchers`: `run.sh` پایان سطر LF دارد، سطر اولش `#!/bin/sh` است و `trap : INT` را دارد.

> **مهم:** چون راهنمای کاربر Word در `WHERE` است، **ویرایش دستی راهنمای کاربر هم می‌تواند این آزمون را رد کند**؛ مثلاً اگر نام قدیمی یک کلید تنظیمات در آن نوشته شود. پیام آزمون می‌گوید کدام متن و جایگزینش چیست.

کار	نام
متن یک فایل راهنما؛ از Word، متن `<w:t>`های `word/document.xml` بی نشانه‌های جهت نامرئی.	`read_doc`
سلامت `run.sh` برای لینوکس؛ تعداد مشکل‌ها.	`check_the_launchers`
همه‌ی بررسی‌ها روی همه‌ی فایل‌ها.	`main`

۲۳.۵ `run_version_test.py` — set_version فقط نسخه را عوض می‌کند

مسیر: `paramgen/tests/run_version_test.py`

`set_version.py` را با `importlib.util.spec_from_file_location` بار می‌کند (نه `import`؛ آزمون «no external dependency» ویرایشگر هر import ناشناخته را رد می‌کند) و همیشه روی **کپی** `__init__.py` کار می‌کند.

کار	نام
کپی `__init__.py` واقعی در پوشه‌ی موقت.	`copy_of_init`
آیا یک فراخوان `VersionError` داد.	`refused`
نسخه‌ی تازه نوشته و نسخه‌ی قبلی برگردانده می‌شود.	`test_the_new_version_is_written`
CRLF و LF دست نمی‌خورند.	`test_line_endings_survive`
قالب غلط (`1.2.3`، `01.01.1`، …) هیچ بایتی را عوض نمی‌کند.	`test_a_malformed_version_changes_nothing`
فایل بی سطر نسخه (یا با دو سطر) دست نمی‌خورد.	`test_no_version_line_changes_nothing`

کار	نام
نسخه‌ی یکسان فایل را لمس نمی‌کند.	`test_the_same_version_writes_nothing`
`main()` با آرگومان و با جواب تایپ‌شده.	`test_the_command_line`
فایل واقعی دقیقاً یک سطر نسخه با قالب درست دارد.	`test_the_real_file_has_one_version_line`
هیچ فایل دیگری نسخه را با دست تکرار نکرده؛ همه از `__version__` می‌خوانند.	`test_nothing_else_writes_the_version_by_hand`
همه‌ی موردها.	`main`

۲۳.۶ `run_struct_test.py` — تایپ‌های استراکچر

مسیر: `paramgen/tests/run_struct_test.py`

کار	نام
یک مورد برآورده نشد.	`Failure`
برابری با پیام.	`equal`
درستی با پیام.	`true`
پروژه با یک ساب‌لیست خالی، باز.	`new_project`
اعلان یک تایپ با فیلدها، مثل پنجره.	`a_type`
ساب‌لیست خالی دوم در ساخت، باز.	`_second_list`
تغییر نام فایل تایپ‌ها به نام قدیمی روی دیسک.	`_rename_on_disk`
همه‌ی `CASES`.	`main`
دو تایپ هم‌نام نمی‌شوند.	`test_one_name_one_type`
پارامتر ساده تایپ استراکچر نمی‌گیرد؛ ویرایشگر رد می‌کند و فایل دست‌کاری‌شده `TYP004` می‌گیرد.	`test_a_parameter_cannot_be_a_structure`
پس از حذف آخرین استراکچر یک تایپ، تایپ قابل حذف است.	`test_deleting_a_structure`
فیلد تازه به همه‌ی پارامترهای آن تایپ می‌رسد.	`test_editing_a_type_reaches_every_parameter`
تایپ فیلدها را می‌گوید؛ بازه مال پارامتر است.	`test_limits_are_the_parameters_own`
تغییر نام تایپ همه‌جا اعمال می‌شود.	`test_renaming_a_type`
تایپ خودش را در بر ندارد (`TYP015`).	`test_a_type_cannot_contain_itself`
ساب‌لیست با فایل تایپ‌هایش ذخیره می‌شود.	`test_the_types_file_travels_with_the_list`
ساب‌لیست بازشده از جای دیگر استراکچرهایش را نگه می‌دارد.	`test_a_list_from_another_project`

کار	نام
افزودن استراکچر، اگر تایپی هست، تایپ می‌دهد.	`test_a_new_parameter_is_never_a_broken_structure`
نام، نوع و اندازه‌ی فیلد روی پارامتر قابل تغییر نیست؛ واحد و توضیح هستند.	`test_a_fields_shape_belongs_to_the_type`
پاک کردن ساب‌لیست فایل تایپ‌هایش را هم پاک می‌کند.	`test_deleting_a_list_takes_its_structures`
ساب‌لیست پروژه‌ی دیگر با تایپ‌هایش کپی می‌شود.	`test_a_list_from_elsewhere_is_copied_in`
یک تایپ، دو استفاده، دو معنا (واحد و توضیح).	`test_unit_and_description_belong_to_the_use`
روشن/خاموش بودن فیلد مال تایپ است.	`test_enabling_a_field_belongs_to_the_type`
افزودن استراکچری که رد شد، چیزی باقی نمی‌گذارد.	`test_a_refused_structure_leaves_nothing`
حذف استراکچر تایپش را حذف نمی‌کند.	`test_deleting_a_structure_keeps_its_type`
فایل تایپ‌های ساخت فقط تایپ‌های استفاده‌شده را دارد.	`test_the_merged_types_file_holds_what_the_project_uses`
ایراد تایپ بی‌استفاده فقط هشدار است.	`test_a_type_nothing_uses_cannot_block_generate`
ویرایش ردشده‌ی تایپ چیزی را عوض نمی‌کند.	`test_a_refused_type_edit_changes_nothing`
فیلد به‌تنهایی حذف، کپی، جابه‌جا یا اضافه نمی‌شود.	`test_a_fields_place_belongs_to_the_type`
فایل تایپ با نام قدیمی (`_structures.xml`) خوانده و منتقل می‌شود.	`test_the_types_file_was_renamed`
تایپ ساب‌لیست دیگر شکل این ساب‌لیست را عوض نمی‌کند.	`test_another_lists_type_never_reshapes_this_one`
استراکچر فقط تایپ ساب‌لیست خودش را دارد (`TYP017`).	`test_a_list_cannot_use_another_lists_type`
تایپ بی‌استفاده تا حذف می‌ماند.	`test_an_unused_type_stays_until_it_is_deleted`
ساب‌لیستی با فایل تایپ قدیمی کامل کپی می‌شود.	`test_a_list_with_an_older_types_file_comes_whole`
نام بلند در `cg_parameter_list_table.h` کامل نوشته می‌شود.	`test_a_long_name_reaches_the_table_in_full`
یادداشت‌های زیر جدول‌ها نام ستون‌هایی را می‌برند که هستند.	`test_the_table_notes_name_what_is_there`

۲۳.۷ `run_paramtable_test.py` — قالب param_table

مسیر: `paramgen/tests/run_paramtable_test.py`

ثابت `REBUILT_ROWS` شش ردیف واقعی از فایل دست‌نویس قدیمی است و `REBUILT_LIST` ساب‌لیستی که باید دقیقاً همان‌ها را بسازد؛ آزمون `test_the_file_it_replaces` ادعای اصلی این قالب را می‌سنجد: خروجی ابزار می‌تواند جای فایل دست‌نویس را بگیرد. `EMPTY_LIST` ساب‌لیست خالی است.

کار	نام
یک مورد برآورده نشد.	`Failure`
برابری با پیام.	`equal`
درستی با پیام.	`true`
پروژه‌ای با قالب داده‌شده و یک ساب‌لیست خالی برای هر نام.	`new_project`
یک پارامتر، همان‌طور که ویرایشگر پر می‌کند.	`add`
سطر بی padding ستون‌ها، برای بررسی محتوا.	`squeeze`
کدهای پیام‌های یک بررسی.	`codes`
ساختن استراکچر به روش عادی (فقط در پروژه‌ی smart_data ممکن است).	`a_structure_in`
کپی ساب‌لیست و تایپ‌هایش از پروژه‌ای به پروژه‌ی دیگر.	`hand_over`
سیزده آرگومان یک `ADD_PARAM`، با رعایت گیومه‌ها.	`cells_of`
«نام ← اندیسی که ویرایشگر کنارش نشان می‌دهد».	`indices_shown`
`(اندیس، نام عنصر)` هر مقداری که ویرایشگر نشان می‌دهد.	`every_value_shown`
ردیف‌های یادداشت آفست‌ها برای هر زیرسیستم.	`offset_rows`
همه‌ی `CASES`.	`main`
دو ساب‌لیست، یک هدر، یک بلوک `#if` برای هرکدام، اندیس از صفر.	`test_one_file_per_project`
هر ستون از جای درستش می‌آید.	`test_the_columns`
بلوک `#define`ها فهرست PARAM MODES رایانه است.	`test_the_modes_come_from_the_machine`
ویرایشگر چیزی که قالب جا ندارد نمی‌سازد.	`test_the_editor_will_not_build_one`
استراکچری که از پروژه‌ی smart_data آمده، `PTB001` می‌گیرد.	`test_a_structure_that_arrived_from_elsewhere`
همین برای آرایه (`PTB002`).	`test_an_array_that_arrived_from_elsewhere`
بولی رد می‌شود (`PTB006`).	`test_a_boolean_is_refused`
تگ‌های بی‌ستون بی‌صدا تنظیم می‌شوند.	`test_the_ignored_tags_say_nothing`
فرم آن تگ‌ها و LogID را تنظیم می‌کند، ولی زیرسیستم را نه.	`test_the_editor_sets_them_but_not_the_sub_system`
بزرگی حروف نام فایل مهم نیست؛ نام ناشناخته رد می‌شود.	`test_the_file_name_names_the_system`
`code_directory` خالی یعنی پوشه‌ی خود پروژه.	`test_the_output_lands_where_it_is_told`
یک فایل در دو پروژه: در یکی ساخته می‌شود، در دیگری خطاست.	`test_the_same_list_in_both_formats`
شش ردیف واقعی فایل دست‌نویس، از روی یک ساب‌لیست بازسازی می‌شوند.	`test_the_file_it_replaces`

کار	نام
جابه‌جایی ساب‌لیست‌ها شماره‌ی هیچ پارامتری را در زیرسیستمش عوض نمی‌کند.	`test_reordering_the_lists_moves_nothing`
ساب‌لیست به‌تنهایی از صفر شمرده می‌شود.	`test_a_sub_list_counts_from_zero`
هم‌نام در دو ساب‌لیست، هرکدام اندیس خودش.	`test_the_same_name_in_two_sub_lists`
ردیف تازه وسط زیرسیستم، هشدار جابه‌جایی شماره می‌دهد.	`test_a_new_row_in_the_middle_is_the_one_that_warns`
فایل قدیمی پوشه‌ی خروجی پیشین `OUT001` می‌گیرد.	`test_an_abandoned_copy_is_reported`
یادداشت `nsrParamOffsets.h` درست است.	`test_the_offsets_note`
خاموش کردن یکی، اندیس و آفست بعدی‌ها را جابه‌جا می‌کند.	`test_a_disabled_parameter_takes_nothing`
پروژه‌ی تازه پوشه‌ی `lib/` نمی‌گیرد.	`test_a_new_project_gets_no_library`
ساب‌لیست تازه باید زیرسیستمی را نام ببرد.	`test_a_new_list_must_name_a_sub_system`
هر زیرسیستم یک ساب‌لیست.	`test_a_sub_system_has_one_list`
فایل موجود هم همین‌طور داوری می‌شود.	`test_an_existing_file_must_name_a_sub_system`
ساخت قدیمی با نام بد هنوز قابل جابه‌جایی است.	`test_an_old_badly_named_list_still_reorders`
این بررسی فقط مال param_table است.	`test_a_smart_data_project_takes_any_name`
پارامتر تازه زیرسیستم فایل را در تگ می‌گیرد.	`test_a_new_parameter_carries_its_lists_sub_system`
باز کردن ساب‌لیست تگ‌ها را اصلاح می‌کند.	`test_opening_a_list_writes_in_its_sub_system`
ساب‌لیست روی دیسک که چنین ذخیره نشده، `PTB008` می‌گیرد.	`test_a_list_on_disk_that_disagrees_warns`
فایل بی‌زیرسیستم چیزی در تگ نمی‌نویسد.	`test_a_file_that_names_no_system_is_not_stamped`
بررسی پنجره‌ی Open همان را می‌گوید که OK می‌گفت.	`test_the_check_and_ok_agree`
آدرس فیلد خودش است، نه Tag1.	`test_the_address_is_a_field_of_its_own`
آدرس نامعتبر دست‌نویس `PTB009` می‌گیرد.	`test_a_hand_written_address_is_checked`
`PARAM_MODE_NONE` اول فهرست همراه برنامه است و `(0)`؛ در param_table حالت دسترسی هنوز قابل انتخاب است.	`test_none_is_the_first_mode`
در param_table بلوک `PARAM_MODE` با جایگاه شماره می‌خورد، نه عدد پنجره، و بی `#ifndef`؛ عدد تکراری حالت‌ها آنجا خطا نیست.	`test_the_table_numbers_modes_by_their_place`
`nsrSystemNames.h` کنار دو فایل دیگر ساخته می‌شود: هر نام با عدد پنجره، فقط `#ifndef`، با محافظ include و هیچ چیز دیگر؛ جدول آن را include نمی‌کند؛ دو نام سیستم با یک عدد اینجا هم `DEF005`.	`test_the_system_names_have_a_file_of_their_own`

۲۳.۸ `run_compile_test.py` — کد تولیدی کامپایل و اجرا می‌شود

مسیر: `paramgen/tests/run_compile_test.py`

آزمون سرتاسری: برای چند «گونه‌ی ورودی» (`input_variants`)، در پوشه‌ی موقت تولید می‌کند، با `smart_data.c` و `parameter_descriptor.c` به‌صورت **C99** با `-Wall -Wextra -Wpedantic` کامپایل می‌کند، هدرها را به‌صورت **C++11** کامپایل می‌کند، و برنامه‌ای کوچک (`main_source`) را که `fParameterList_Init()` را صدا می‌زند، جدول را می‌پیماید و نمونه‌ی استفاده (فصل ۲۲) را اجرا می‌کند، می‌سازد و اجرا می‌کند. گونه‌ی `param_table` جدا (`_check_param_table`) جدول `ADD_PARAM` را درون یک تابع include و اجرا می‌کند. اگر `chrono.h` واقعی فرمور در دسترس نیست، stub ثابت `_CHRONO_STUB_H` (ساعتی که همیشه صفر است) در پوشه‌ای ساخته می‌شود که **آخر** جست‌وجو می‌شود.

خروجی برای هر گونه:

```
=== 1/N  the usage example ===
  1/5 generating the code ...
  2/5 compiling as C99 ...
  3/5 compiling the headers as C++ ...
  4/5 a system name the firmware defines first ...
  5/5 running the generated parameter list ...
```

گام ۴ (`compile_firmware_defines`) یک فایل C می‌سازد که مثل فرمور اول خودش یکی از `SYS_...`ها را با عدد دیگری تعریف می‌کند و بعد `cg_parameter_list_defines.h` را include می‌کند، و آن را با `-Werror` کامپایل می‌کند: نه هشدار redefinition، نه عوض شدن عدد فرمور؛ و هیچ `typedef enum` دیگر نباید در هدر باشد. گونه‌ی `param_table` همین را برای `nsrSystemNames.h` می‌کند: برنامه‌ی شبیه فرمور آن را بعد از `SYS_GS_MANAGER` خودش include می‌کند.

گونه‌ی ورودی تازه (مثلاً مدلی با نوع داده‌ی تازه) یعنی یک دیکشنری تازه در `input_variants`.

کار	نام
`subprocess.run` با خروجی گرفته‌شده.	`_run`
آیا کامپایلر در PATH هست.	`have_compiler`
فایل پروژه‌ای که در پوشه‌ی کار تولید می‌کند.	`write_config`
فهرست گونه‌ها: مدل آزمون، مدل‌های ویژه (نوع‌ها، آرایه‌ها، ...) و param_table.	`input_variants`
نام همه‌ی پارامترهای سطح بالای چند فایل.	`_names_in`
ساب‌لیست پروژه که نمونه‌ی استفاده با آن جور است، یا `None`.	`_list_holding_the_example`
ساب‌لیست‌های پروژه‌ی واقعی برای ساختن؛ عمداً خالی (جای خالی نگه داشته شده).	`_project_lists`

کار	نام
کامپایل C99 فایل‌های تولیدی و کتابخانه (و نمونه، اگر جور است).	`compile_c`
کامپایل هدرها به‌صورت C++.	`compile_cxx`
نامی که فرم‌ور زودتر تعریف کرده، با `-Werror`، عدد خود فرم‌ور را نگه می‌دارد (بالا).	`compile_firmware_defines`
متن C برنامه‌ی آزمون، با بررسی‌های اضافه‌ی هر گونه.	`main_source`
تولید، کامپایل و اجرای جدول `ADD_PARAM`.	`_check_param_table`
یک گونه: تولید، کامپایل C، کامپایل C++، اجرا.	`check_one`
اگر کامپایلری نیست «SKIPPED» و کد ۰؛ وگرنه همه‌ی گونه‌ها.	`main`

۲۴. دستورالعمل‌های تغییر

این فصل برای رایج‌ترین تغییرها، گام‌به‌گام می‌گوید **کدام فایل‌ها و کجای آن‌ها** باید عوض شوند. هر دستورالعمل فرض می‌کند روال فصل ۳ (آزمون پیش و پس از تغییر، آزمودن روی کپی پروژه) را رعایت می‌کنید، و به فصلی که آن فایل‌ها را شرح می‌دهد ارجاع می‌دهد.

بخش	تغییر	#
۲۴.۱	افزودن یک نوع داده‌ی C	۱
۲۴.۲	افزودن یک تگ	۲
۲۴.۳	افزودن یک قاعده‌ی بررسی	۳
۲۴.۴	افزودن یک فایل تولیدی	۴
۲۴.۵	تغییر محتوای یک فایل تولیدی	۵
۲۴.۶	تغییر بنر، نشانگرها و کپی‌رایت	۶
۲۴.۷	افزودن کلید به فایل پروژه	۷
۲۴.۸	افزودن فیلد به پارامتر	۸
۲۴.۹	تغییر نام یک ویژگی XML	۹
۲۴.۱۰	افزودن یک Type	۱۰
۲۴.۱۱	افزودن مقدار به فهرست‌های کشویی پیش‌فرض	۱۱
۲۴.۱۲	تغییر مقدارهای شروع پارامتر تازه	۱۲
۲۴.۱۳	افزودن ستون به جدول ویرایشگر	۱۳
۲۴.۱۴	افزودن خانه به فرم ویرایشگر	۱۴
۲۴.۱۵	افزودن کار تازه به ویرایشگر	۱۵
۲۴.۱۶	افزودن میان‌بر صفحه‌کلید	۱۶
۲۴.۱۷	تغییر رنگ‌ها و ظاهر	۱۷
۲۴.۱۸	افزودن گزینه به منوی run.bat	۱۸
۲۴.۱۹	افزودن گزینه به خط فرمان تولیدکننده	۱۹
۲۴.۲۰	تغییر پیشوند نام‌های تولیدی	۲۰
۲۴.۲۱	تغییر نام یک مفهوم در همه‌جا	۲۱
۲۴.۲۲	بالا بردن کمینه‌ی نسخه‌ی پایتون	۲۲
۲۴.۲۳	تغییر یک پیام خطا	۲۳
۲۴.۲۴	آماده کردن نسخه‌ی تازه برای تحویل	۲۴

۲۴.۱ افزودن یک نوع داده‌ی C

مثلاً اگر روزی کتابخانه‌ی smart data نوع تازه‌ای پیدا کند. شرط لازم: خود کتابخانه (فصل ۲۲) struct و خانواده‌ی تابع آن نوع را داشته باشد؛ این کار صاحب فرم‌ور است.

- **۱.** در `model/types.py` یک مدخل به `VALUE_TYPES` اضافه کنید. برای نوع صحیح، `_int(c_name, suffix, bits, signed, literal_suffix)` همه‌چیز را می‌سازد (ماکروهای بازه از `stdint.h`)؛ برای نوع دیگر یک `CValueType` کامل با همه‌ی فیلدها (فصل ۱۰). `suffix` باید **دقیقاً** پسوند نام‌های کتابخانه باشد (`sSmartData<suffix>`، `fSmartData_<suffix>_Ctor`).
- **۲.** اگر نام‌های کوتاه هم می‌خواهید (مثل `u16`)، آن‌ها را به دیکشنری نام‌های مستعار همان فایل اضافه کنید.
- **۳.** نوع را به آخر فهرست `valueTypes` در `templates/choices.json` اضافه کنید؛ روی هر رایانه هم از پنجره‌ی **Drop-down lists…** (یا با «Restore the company defaults») بزنید.
- **۴.** اگر پروژه‌ی `param_table` هم باید این نوع را داشته باشد: `_C_NAME` و `_TYPE_ENUM` در `codegen/files/param_table_h.py` (اگر نام C یا enum آن با قاعده‌ی پیش‌فرض `TYPE_<NAME>` نمی‌خواند).
- **۵.** اگر لیترال‌های این نوع شکل خاصی دارند، `codegen/literals.py` (فصل ۱۴).
- **۶.** در `tests/run_compile_test.py` یک گونه‌ی ورودی با پارامتری از این نوع اضافه کنید تا کد آن واقعاً کامپایل و اجرا شود.

فایل‌ها: `model/types.py`، `templates/choices.json`، `param_table_h.py`، `run_compile_test.py`

آزمون: همه‌ی مجموعه‌ها، به‌ویژه `run_compile_test.py` (با کامپایلر).

۲۴.۲ افزودن یک تگ

تگ یک فیلد `uint32_t` در `sParameterDescriptor` است. بیشتر جاها از روی `TAG_REGISTRY` خودکار درست می‌شوند؛ چند جا نه.

- **۱. فرم‌ور**: فیلد تازه‌ی `uint32_t` در `sParameterDescriptor` (`parameter_descriptor.h`)، **در همان جایگاهی** که تگ در `TAG_REGISTRY` خواهد داشت (معمولاً بعد از آخرین تگ و پیش از `pDescription`). با صاحب فرم‌ور هماهنگ کنید.
- **۲.** در `model/tags.py` یک `TagSpec` به `TAG_REGISTRY` در همان جایگاه اضافه کنید:
  `TagSpec("Calibrated", "Tag Calibrated", "TagCalibrated", TagKind.BOOLEAN, True, description="...")`
  سومی نام فیلد C است.
- **۳.** خودکار درست می‌شوند: خواندن و نوشتن XML، ترتیب مقداردهی جدول (`descriptor_field_order`)، ردیف‌های جدول C، ناحیه‌ی آفست و ماکروی `PARAMETER_LIST_TAG_<TAG>_SIZE` و تابع `fParameterList_Get<Tag>Size`، قاعده‌ی تگ‌های اجباری، و خانه‌ی فرم ویرایشگر (از `state.tagSpecs`).

- **۴. جدول ویرایشگر خودکار نیست**: در `web/grid.js` یک ستون به `columns` (کنار ستون‌های `Tag ...`) و کلید هم‌نامش در `emit` درون `rows()` اضافه کنید (بخش ۲۴.۱۳).
- **۵.** اگر تگ از نوع ENUM با فهرست کشویی **تازه** است: ستون تازه در `model/defines.py` (`COLUMN_...`)، و اگر نام‌هایش باید `#define` عددی شوند در `GENERATED_DEFINES`)، کلید تازه در `COLUMNS` و `FALLBACK` در `choices.py` و در `templates/choices.json`، کلید در `defines` جواب `Api.state` و در `Api.set_defines`، نگاشت فهرست در `tagsGroup` در `editor.js` (که امروز فقط `SYSTEM NAMES` و `PARAM MODES` را می‌شناسد) و پنجره‌ی `editLists` در `app.js`.
- **۶.** اگر پروژه‌ی `param_table` ستونی برای این تگ ندارد، نامش را به `IGNORED_TAGS` در `param_table_h.py` **و** همان ثابت در `editor.js` اضافه کنید.
- **۷.** آزمون: پارامتری با تگ تازه بسازید، تولید و کامپایل کنید.

فایل‌ها: `parameter_descriptor.h` (فرم و ر)، `model/tags.py`، `web/grid.js`

آزمون: `run_compile_test.py` ترتیب و نام فیلد را با کامپایل می‌سنجد؛ `run_editor_test.py` و `run_validation_test.py` را هم اجرا کنید.

۲۴.۳ افزودن یک قاعده‌ی بررسی

- **۱.** کلاسی از `ValidationRule` با `rule_id`، `description` و متد `check` در فایل مناسب `validation/rules/` بنویسید؛ نمونه‌ی کامل در بخش ۱۲.۱۲.
- **۲.** کد پیام تازه‌ای انتخاب کنید که در پیوست ب نیست.
- **۳.** کلاس را در `validation/rules/__init__.py` import و به `ALL_RULES` اضافه کنید.
- **۴.** مورد آزمون در `CASES` فایل `tests/run_validation_test.py` (برای خطا) یا آزمونی در `run_editor_test.py` با `validate_codes` (برای هشدار).
- **۵.** اگر کاربر این پیام را می‌بیند، جدول پیام‌های راهنمای کاربر را هم به‌روز کنید.

فایل‌ها: `validation/rules/*.py`، `validation/rules/__init__.py`

۲۴.۴ افزودن یک فایل تولیدی

- **۱.** فایل تازه در `codegen/files/` با کلاسی از `FileBuilder` که `file_name` (property) و `generate()` را دارد؛ از بلوک‌های `self.include`، `self.define`، ... استفاده کنید و فایل را از بالا به پایین با `text += ...` بنویسید (فصل ۱۴). برای نام، `self.config.generated_file_name("x", "h")` نام `cg_parameter_list_x.h` می‌سازد.
- **۲.** کلاس را در `codegen/files/__init__.py` import و به `SMART_DATA_BUILDERS` یا `PARAM_TABLE_BUILDERS` اضافه کنید.

- **۳.** اگر فایل `.c` است، در فایل `tests/run_compile_test.py` به لیست فایل‌هایی که کامپایل می‌شوند اضافه‌اش کنید (آن لیست صریح است، نه همه‌ی `cg_*.c`).
- **۴.** اگر هدرهای دیگر باید آن را include کنند، `generate()` آن‌ها را هم عوض کنید.

فایل‌ها: `codegen/files/<new>.py`، `codegen/files/__init__.py`، `run_compile_test.py`

۲۴.۵ تغییر محتوای یک فایل تولیدی

مثلاً افزودن یک تابع دسترسی تازه به `cg_parameter_list.c`.

- **۱.** سازنده‌ی آن فایل را از جدول ابتدای فصل ۱۴ پیدا کنید و `generate()` آن را از بالا بخوانید تا جای سطر مورد نظر را بیابید.
- **۲.** متن را با همان بلوک‌ها بنویسید؛ برای تابع: `self.comment.generate_doc(...)`، `self.function.generate_begin(...)`، `generate_body(...)`، `generate_end()`؛ و پروتوتایپ در هدر هم‌نامش (`parameter_list_h.py`). نام‌های مشترک را در ثابت‌های بالای `parameter_list_h.py` بگذارید.
- **۳.** کد تولید و کامپایل کنید؛ هشدار کامپایلر را جدی بگیرید (فرمور شرکت با `-Werror` می‌سازد).

فایل‌ها: `codegen/files/*.py`

آزمون: `run_compile_test.py`؛ اگر رفتار تابع مهم است، فراخوانی آن را به `main_source` همان آزمون اضافه کنید.

۲۴.۶ تغییر بنر، نشانگرها و کپی‌رایت

شکل این‌ها از `templates/c.json` (snippetهای VS Code) می‌آید، نه از پایتون (بخش ۶.۵).

- **۱.** بنر: بلوک `/** ... */` اول snippet «Create new header file» را عوض کنید. جای‌خالی‌های `$TM_FILENAME` و `$CURRENT_YEAR` پر می‌شوند و سطر `@brief` جایگزین می‌شود.
- **۲.** نشانگرها و سطر `=== EOF`: در همان snippetها.
- **۳.** نام شرکت در یک پروژه: کلید `style.company` در `pl_project.json` (همه‌ی `FAS`ها عوض می‌شوند)؛ نیازی به تغییر کد نیست.

فایل‌ها: `templates/c.json`

۲۴.۷ افزودن کلید به فایل پروژه

مثلاً کلید `codegen.emit_log_ids` برای خاموش کردن شناسه‌های لاگ.

- **۱.** در `config.py` فیلدی با مقدار پیش‌فرض به dataclass بخش مربوط اضافه کنید (مثلاً `CodegenConfig`: `emit_log_ids: bool = True`). خواننده (`_dataclass_from_dict`) کلید را خودکار می‌شناسد و کلید ناشناخته را هشدار می‌دهد.

- **۲.** اگر مقدار مسیر است یا نیاز به تبدیل دارد، در `Config.load` مثل کلیدهای `output` تبدیلش کنید.
- **۳.** جایی که اثر دارد، آن را بخوانید: `self.config.codegen.emit_log_ids`.
- **۴.** کلید را با مقدار پیش‌فرضش به `templates/pl_project.json` اضافه کنید تا پروژه‌های تازه داشته باشند.
- **۵.** یک `case(...)` در `tests/run_settings_test.py` که اثرش را در خروجی می‌سنجد.
- **۶.** در جدول کلیدهای فصل ۹ و راهنمای کاربر بنویسید.

فایل‌ها: `config.py`، `templates/pl_project.json`، `run_settings_test.py`

۲۴.۸ افزودن فیلد به پارامتر

مثلاً فیلد «scale» که در جدول C نوشته شود. این بزرگ‌ترین تغییر رایج است چون از XML تا صفحه می‌رود.

- **۱.** `model/parameter.py`: فیلد با پیش‌فرض به `Parameter` dataclass (`scale: Optional[float] = None`). `copy_as_leaf` با `replace` کار می‌کند، پس فیلد خودکار به برگ‌ها می‌رسد.
- **۲.** `xml_model/schema.py`: ثابت `ATTR_SCALE = "scale"`.
- **۳.** `xml_model/xml_reader.py` در `_read_parameter`: خواندن و تبدیل؛ مقدار ناخوانا را با پیام گزارش کنید، نه دور بریزید.
- **۴.** `xml_model/xml_writer.py` در `_parameter_to_element`: `_set(element, sch.ATTR_SCALE, parameter.scale)`.
- **۵.** `pl_gui/document.py`: نام فیلد را به `EDITABLE_FIELDS` اضافه کنید؛ اگر قاعده‌ای دارد، در `_apply_update` بررسی کنید.
- **۶.** `pl_gui/api.py` در `_node`: کلید `"scale": parameter.scale`.
- **۷.** `web/editor.js`: خانه در `primitiveForm` (`this.number("scale", "Scale", node.scale)`) و تبدیل عددی در `collect()` (فهرست `default_value`، `minimum`، … را ببینید).
- **۸.** اختیاری: ستون جدول (بخش ۲۴.۱۳)، قاعده‌ی بررسی (۲۴.۳) و استفاده در سازنده‌ی فایل (۲۴.۵). اگر در `sParameterDescriptor` هم می‌رود: فرم‌ور، `descriptor_layout.py` و `_table_entry`.
- **۹.** آزمون: در `run_editor_test.py` مقدار را از راه `api.update` بنویسید، ذخیره کنید، دوباره بخوانید و برابری را بسنجید (رفت‌وبرگشت XML).

فایل‌ها: `model/parameter.py`، `xml_model/schema.py`، `xml_reader.py`، `xml_writer.py`، `document.py`، `api.py`، `editor.js`

۲۴.۹ تغییر نام یک ویژگی XML بی شکستن فایل‌های قدیمی

نمونه‌ی واقعی: `gsName` که `userName` شد.

- **۱.** در `schema.py` ثابت تازه را با نام تازه بگذارید و نام قدیمی را در ثابتی مثل `ATTR_OLD_USER_NAME` نگه دارید.
- **۲.** در `xml_reader.py` اول نام تازه و اگر نبود نام قدیمی را بخوانید: `element.get(sch.ATTR_USER_NAME) or element.get(sch.ATTR_OLD_USER_NAME)`.
- **۳.** `xml_writer.py` **فقط** نام تازه را بنویسد؛ پس فایل قدیمی با اولین ذخیره به‌روز می‌شود.
- **۴.** نام قدیمی را به `GONE` در `run_docs_test.py` و به فهرست آزمون `test_one_name_for_one_thing` در `run_editor_test.py` اضافه کنید تا جایی در راهنما یا کد نماند.
- **۵.** آزمون: یک فایل XML با نام قدیمی بخوانید و مطمئن شوید مقدار رسیده است.

فایل‌ها: `xml_model/schema.py`، `xml_reader.py`، `xml_writer.py`، `run_docs_test.py`

۲۴.۱۰ افزودن یک Type

Typeها (`COMMAND`، `SETTING`، `MONITORING`) enumerator در `eParameterType` در `parameter_descriptor.h`اند.

- **۱. فرمور**: enumerator تازه در **آخر** `eParameterType` (مثلاً `ePARAMETER_TYPE_DIAGNOSTIC`)؛ با صاحب فرمور.
- **۲.** در `validation/rules/type_rules.py` نام را به `KNOWN_TYPES` اضافه کنید؛ وگرنه `GRP002`.
- **۳.** به `paramTypes` در `templates/choices.json`، و روی هر رایانه از **Drop-down lists…**.
- **۴.** نام enumerator را `type_enum_member` خودکار می‌سازد (`DIAGNOSTIC` ← `ePARAMETER_TYPE_DIAGNOSTIC`).

فایل‌ها: `parameter_descriptor.h` (فرمور)، `type_rules.py`، `templates/choices.json`

۲۴.۱۱ افزودن مقدار به فهرست‌های کشویی پیش‌فرض

مثلاً زیرسیستم تازه‌ی `SYS_RADAR`.

- **۱.** به **آخر** `systemNames` در `templates/choices.json` اضافه کنید؛ نامی که عدد ندارد عدد نام بالایی به‌اضافه‌ی یک را می‌گیرد (بخش ۶.۴). اگر عدد دیگری می‌خواهید، آن را زیر `numbers.systemNames` همان فایل بنویسید (`"SYS_RADAR": 40`).
- **۲.** روی هر رایانه‌ای که قبلاً `var/choices.json` دارد، از پنجره‌ی **Drop-down lists…** همان را اضافه کنید - `SYS_RADAR = 40`، یا بی عدد در آخر (فایل کاربر بر الگو برنده است).
- **۳.** مطمئن شوید فرمور همان عدد را می‌شناسد؛ `#define SYS_RADAR` از همین فهرست تولید می‌شود (`cg_parameter_list_defines.h` در smart data، `nsrSystemNames.h` در param_table)، فقط `#ifndef`. دو نام با یک عدد `DEF005` می‌گیرد.

فایل‌ها: `templates/choices.json`

۲۴.۱۲ تغییر مقدارهای شروع پارامتر تازه

- **۱.** نوع داده‌ی پیش‌فرض (`uint8_t`) و Type و تگ‌های شروع: `Document.add_parameter`.
- **۲.** پیش‌فرض، بازه و رزولوشن شروع: `Document._give_starting_values` (برای پارامتر تازه **و** فیلدی که تایپ تازه به استراکچر می‌دهد).
- **۳.** مقدار شروع هر تگ: `_default_tag` (بولی `False`، enum اولین مقدار فهرست).
- **۴.** نام شروع: `_unique_name("NewParameter", ...)` در `add_parameter`.

فایل‌ها: `pl_gui/document.py`

آزمون: `test_a_new_field_starts_with_no_warning` در `run_editor_test.py` می‌سنجد مقدارهای شروع هیچ هشداری ندهند؛ اگر مقدار تازه هشدار می‌سازد، آن آزمون می‌گوید.

۲۴.۱۳ افزودن ستون به جدول ویرایشگر

- **۱.** در `web/grid.js` یک مدخل به `gridView.columns` در جای دلخواه: `{ key: "scale", label: "Scale", width: "70px", align: "right", mono: true }`.
- **۲.** در `rows()`، درون `emit`، کلید هم‌نام را به شیء ردیف بدهید: `scale: fmtNumber(node.scale)`. داده باید در گره‌ی `state.tree` باشد (`_node` در `api.py`).
- **۳.** اگر مقدارهای ستون محدودند و فیلتر باید فهرست کشویی باشد، `key` را به `CHOICE_COLUMNS` اضافه کنید.
- **۴.** F5 در ویرایشگر.

فایل‌ها: `web/grid.js`، `pl_gui/api.py`

آزمون: `test_the_address_column_comes_before_the_description` نمونه‌ای از آزمون جای ستون از روی متن JS است.

۲۴.۱۴ افزودن خانه به فرم ویرایشگر

- **۱.** در `web/editor.js`، `primitiveForm` یا `structForm`، با سازنده‌های `this.text`، `this.number`، `this.select`، `this.check` یا `this.area`؛ نام اول `data-field` است و باید نام فیلد `Parameter` باشد.
- **۲.** اگر عدد یا بولی است، تبدیلش را در `collect()` اضافه کنید.
- **۳.** نام فیلد را به `EDITABLE_FIELDS` در `document.py` اضافه کنید؛ وگرنه سرور آن را نادیده می‌گیرد.
- **۴.** اگر فیلد مال تایپ است (برای فیلدهای استراکچر)، در `_apply_update` رد کنید و در فرم `readOnly` نشان دهید.

فایل‌ها: `web/editor.js`، `pl_gui/document.py`

۲۴.۱۵ افزودن کار تازه به ویرایشگر

مثلاً «مرتب کردن پارامترها بر اساس نام». الگوی سه‌لایه (فصل ۱۶):

- **۱.** `document.py`: متد `sort_by_name(self)` که درون `with self._change("sort"):` لیست را مرتب می‌کند و هر رد را با `ValueError` و جمله‌ی روشن می‌گوید.
- **۲.** `api.py`: متد `sort_by_name(self, request)` که آن را صدا می‌زند و `ValueError` را به `{"ok": False, "message": ...}` تبدیل می‌کند؛ و مدخل `"sort_by_name": self.sort_by_name` در `handlers()`.
- **۳.** `web/api.js`: `sortByName() { return this.call("sort_by_name"); },`.
- **۴.** `web/index.html`: دکمه با `id` یکتا (مثلاً `btn-sort`) و `title`.
- **۵.** `web/app.js` در `bindToolbar`: `on("btn-sort", () => this.sortByName());` و متد `async sortByName()` با الگوی درخواست، بررسی `ok`، `refresh` و `setStatus` (بخش ۲۰.۶).
- **۶.** در `updateButtons` اگر دکمه گاهی نباید فعال باشد (نمای ادغام‌شده، بی‌ساب‌لیست).
- **۷.** آزمون در `run_editor_test.py` که `api.sort_by_name({})` را صدا می‌زند و نتیجه و Undo را می‌سنجد.

فایل‌ها: `document.py`، `api.py`، `api.js`، `index.html`، `app.js`، `run_editor_test.py`

۲۴.۱۶ افزودن میان‌بر صفحه‌کلید

- **۱.** در `web/app.js` به `SHORTCUTS` کلید **فیزیکی** را بدهید (`KeyR: "sort"`)، نه حرف؛ تا روی صفحه‌کلید فارسی هم کار کند.
- **۲.** در `runShortcut` فرمان را اجرا کنید.
- **۳.** در `title` دکمه‌ی مربوط در `index.html` میان‌بر را بنویسید.

فایل‌ها: `web/app.js`، `web/index.html`

آزمون: `test_the_shortcuts_are_bound_to_the_key`.

۲۴.۱۷ تغییر رنگ‌ها و ظاهر

- **۱.** رنگ را در **هر دو** بلوک متغیرها در `web/style.css` عوض کنید: `:root` (روشن) و `:root[data-theme="dark"]` (تیره).
- **۲.** در قاعده‌ی تازه همیشه `var(--name)` به کار ببرید.
- **۳.** F5 و هر دو تم را از منوی ⚙ ببینید.

فایل‌ها: `web/style.css`

۲۴.۱۸ افزودن گزینه به منوی run.bat

- **۱.** فایل `bin/run_<name>.py` با الگوی سه‌خطی بقیه (پوشه‌ی `src` به `sys.path`، import، `raise SystemExit(main())`).
- **۲.** یک `Action("5", ("name", ...), "Title", "what it does", "run_<name>.py", endings={...})` به `ACTIONS` در `bin/run.py`.
- **۳.** متن راهنمای بالای `run.bat` و `run.sh` و README را به‌روز کنید.

فایل‌ها: `bin/run.py`، `bin/run_<name>.py`

۲۴.۱۹ افزودن گزینه به خط فرمان تولیدکننده

- **۱.** در `cli.py`، `build_parser`: `parser.add_argument("--name", ...)`.
- **۲.** در `main`: `arguments.name` را بخوانید و اثرش را بدهید (اغلب با تغییر `config` پیش از `Pipeline`).
- **۳.** اگر نمونه‌ی مفیدی است، به `epilog` اضافه کنید.

فایل‌ها: `pl_codegen/cli.py`

۲۴.۲۰ تغییر پیشوند نام‌های تولیدی

این تغییر **کد نمی‌خواهد**: کلیدهای `codegen` در `pl_project.json` پروژه.

- **۱.** `index_define_prefix` (ماکروهای اندیس)، `offset_define_prefix` (ماکروهای اندازه‌ی تگ)، `descriptor_symbol`، `struct_type_prefix` (`sCgStruct_`)، `smart_data_prefix` (`Cg_SmartData_`)، `quantity_define`؛ و `output.file_prefix` و `output.base_name` برای نام فایل‌ها.
- **۲.** پس از تغییر، قاعده‌ی `NAME004` برخورد احتمالی نام‌ها را می‌گیرد.

۲۴.۲۱ تغییر نام یک مفهوم در همه‌جا

مثلاً تغییر نام «Type» به «Category». خطرناک‌ترین تغییر از نظر نیمه‌کاره ماندن.

- **۱.** همه‌جا را جست‌وجو کنید: کد پایتون، JS، متن پیام‌ها و راهنماها (بخش ۳.۴).
- **۲.** نام XML را مثل بخش ۲۴.۹ با پشتیبانی از نام قدیمی عوض کنید.
- **۳.** نام‌های قدیمی را به فهرست `test_one_name_for_one_thing` و `GONE` در `run_docs_test.py` اضافه کنید.
- **۴. کد پیام‌ها را عوض نکنید** (مثل `GRP...`)؛ شناسه‌اند.

۲۴.۲۲ بالا بردن کمینه‌ی نسخه‌ی پایتون

- **۱.** `MINIMUM_PYTHON` در `src/setup_environment.py`.

- **۲.** `sys.version_info < (3, 11)` در `run.bat`، `run.sh` و `bin/set_version.bat` (اگر بررسی نسخه دارد) و متن پیام‌هایشان.
- **۳.** README‌ها و راهنماها.

فایل‌ها: `setup_environment.py`، `run.bat`، `run.sh`

آزمون: `test_every_door_asks_for_the_same_python` می‌سنجد همه یک عدد بگویند.

۲۴.۲۳ تغییر یک پیام خطا

- **۱.** کد پیام را جست‌وجو کنید (مثل `"TAG012"`)؛ پیام و `hint` کنار هم‌اند.
- **۲.** متن را عوض کنید ولی **کد را نه**.
- **۳.** آزمون‌هایی که تکه‌ای از متن را می‌جویند (`check_messages` در `run_validation_test.py`) را اجرا کنید.
- **۴.** اگر راهنمای کاربر متن پیام را نقل کرده، آن را هم عوض کنید.

۲۴.۲۴ آماده کردن نسخه‌ی تازه برای تحویل

- **۱.** همه‌ی آزمون‌ها با کامپایلر: `run.bat tests`؛ هیچ «SKIPPED» نباید باشد.
- **۲.** ویرایشگر را روی کپی یک پروژه‌ی واقعی باز کنید، ذخیره و تولید کنید و کد را با کامپایلر فرم‌ور بسازید.
- **۳.** `bin\set_version.bat` و نسخه‌ی تازه (بخش ۲۱.۶).
- **۴.** راهنمای کاربر و این راهنما (شماره‌ی نسخه روی جلد و هر بخشی که رفتارش عوض شده).
- **۵.** اگر چیدمان حافظه عوض شده (فصل ۱۳)، به تیم فرم‌ور خبر دهید.

۲۵. اشکال‌زدایی

این فصل راه‌های پیدا کردن علت یک مشکل را، از ساده به پیچیده، شرح می‌دهد؛ و در پایان جدولی از مشکل‌های رایج و راه‌حلشان.

۲۵.۱ خواندن traceback پایتون

وقتی استثنایی گرفته نشود، پایتون یک **traceback** چاپ می‌کند: زنجیره‌ی فراخوانی‌ها از بیرونی‌ترین تا جایی که خطا رخ داد. نمونه‌ی واقعی (دادن NaN به نوشتن لیترال):

```
Traceback (most recent call last):
  File "...\tb_demo.py", line 6, in <module>
    print(numeric_literal(float("nan"), resolve_value_type("float32_t")))
  File "...\paramgen\src\pl_codegen\codegen\literals.py", line 141, in numeric_literal
    return float_literal(float(effective), value_type)
  File "...\paramgen\src\pl_codegen\codegen\literals.py", line 71, in float_literal
    raise ValueError(f"'{value}' cannot be written as a C floating point literal.")
ValueError: 'nan' cannot be written as a C floating point literal.
```

- **از پایین بخوانید.** سطر آخر نوع و متن استثناست.
- هر جفت سطر یک قاب است: فایل، شماره‌ی سطر و تابع، و خود آن سطر. پایین‌ترین قاب جایی است که استثنا ساخته شد؛ بالاتری‌ها کسانی‌اند که صدایش زدند.
- علت معمولاً در **پایین‌ترین قابِ کدِ خودمان** یا یکی بالاتر است: اینجا `numeric_literal` مقدار NaN را رد نکرده تا به `float_literal` برسد.
- استثناهای رایج: `AttributeError: 'NoneType' object has no attribute ...` (چیزی `None` بود که انتظار نمی‌رفت؛ مثلاً `parameter.value_type` برای نوع ناشناخته)، `KeyError` (کلیدی در دیکشنری نبود)، `TypeError` (نوع آرگومان غلط)، `ModuleNotFoundError` (مسیر import؛ بخش ۴.۱۳).

۲۵.۲ پیام‌های ابزار

بیشتر مشکل‌ها traceback ندارند؛ ابزار یک **پیام با کد** می‌دهد (`TAG012`، `RNG003`، …). پیوست ب برای هر کد متن، شدت و جای آن در کد را دارد. برای فهم دقیق اینکه چه چیزی پیام را ساخته، کد را در `src` جست‌وجو کنید (بخش ۳.۴) و شرط بالای `diagnostics.error(...)` را بخوانید.

اگر **ویرایشگر** پیامی نشان می‌دهد ولی خط فرمان نه (یا برعکس)، تقریباً همیشه یکی فایل ذخیره‌نشده را می‌بیند: ویرایشگر ساب‌لیست باز را از حافظه و خط فرمان از دیسک می‌خواند. ذخیره کنید و دوباره بسنجید.

۲۵.۳ لاگ و چاپ

- `run.bat generate -v` (یا `--verbose`) پیام‌های اطلاع و لاگ سطح DEBUG را هم نشان می‌دهد.

- در کد پایتون: `from pl_codegen.logging_utils import get_logger` و `get_logger().debug("offset of %s is %d", name, offset)`؛ فقط با `-v` دیده می‌شود.
- برای یک بررسی سریع، `print(...)` ساده هم کار می‌کند؛ در ویرایشگر خروجی‌اش در پنجره‌ی کنسولی است که سرور در آن اجرا شده. **پیش از تحویل همه‌ی `print`های موقت را بردارید.**
- `repr(x)` شکل دقیق یک مقدار را نشان می‌دهد (`'5'` در برابر `5`)؛ برای اشکال‌های نوع مفید است.

۲۵.۴ اشکال‌زدایی ویرایشگر

ویرایشگر دو نیمه دارد و هر نیمه جای خودش را برای دیدن خطا:

خطا کجا دیده می‌شود	نیمه
پنجره‌ی کنسولی که `run.bat editor` در آن اجرا شده: `api 'update' failed` با traceback کامل؛ و در صفحه یک toast قرمز با نوع و متن استثنا	سرور (پایتون)
F12 ▸ Console: خطای JS با فایل و سطر (بخش ۵.۷)	صفحه (JavaScript)
F12 ▸ Network: هر `api/...` با بدنه‌ی درخواست و جواب	ارتباط

برای دیدن کنسول سرور، ویرایشگر را از یک پنجره‌ی Command Prompt اجرا کنید (`run.bat editor`)، نه با دابل‌کلیک، تا پنجره بماند. تغییر کد پایتون فقط پس از بستن (Ctrl+C) و اجرای دوباره‌ی سرور، و تغییر `web/` با F5 دیده می‌شود.

۲۵.۵ آزمودن تکه‌ای از کد در پایتون تعاملی

برای فهمیدن اینکه مدل یک پروژه واقعاً چه دارد، لازم نیست ویرایشگر را باز کنید. از پوشه‌ی `paramgen` پایتون را بی‌فایل اجرا کنید و دستورها را یکی‌یکی بنویسید:

```
> cd paramgen
> py -3.11
>>> import sys; sys.path.insert(0, "src")
>>> from pl_codegen.config import Config
>>> from pl_codegen.pipeline import Pipeline
>>> from pl_codegen.errors import DiagnosticCollector
>>> config = Config.load(r"D:\Projects\Flight_Computer\pl_project.json")
>>> diagnostics = DiagnosticCollector()
>>> model = Pipeline(config).load_model(diagnostics)
>>> [leaf.name for leaf in model.flatten()][:5]
['Airspeed', 'MaxAltitude', 'FlightMode', 'CellVoltage[0]', 'CellVoltage[1]']
>>> from pl_codegen.layout.tag_offset import build_offset_map
>>> build_offset_map(model).total("ResetProof")
7
>>> exit()
```

`Pipeline(config).load_model` هیچ فایلی نمی‌نویسد؛ پس این کار روی پروژه‌ی واقعی هم بی‌خطر است. `Pipeline.run` و `Document.save` **می‌نویسند**؛ آن‌ها را فقط روی کپی صدا بزنید.

۲۵.۶ توقف و قدم‌به‌قدم: breakpoint

نوشتن `breakpoint()` در هر جای کد پایتون، اجرا را همان‌جا متوقف می‌کند و اعلان `(Pdb)` را در کنسول نشان می‌دهد:

کار	فرمان
چاپ مقدار یک متغیر (`p parameter.tags`)	`p name`
سطر بعد (بی ورود به تابع)	`n`
ورود به تابعی که در این سطر صدا زده می‌شود	`s`
ادامه تا breakpoint بعدی	`c`
زنجیره‌ی فراخوانی‌ها (مثل traceback)	`w`
خروج	`q`

در ویرایشگر، breakpoint درخواست همان کار را نگه می‌دارد تا در کنسول سرور ادامه دهید؛ صفحه در این مدت منتظر می‌ماند. **پیش از تحویل همه‌ی `breakpoint()`ها را بردارید.**

۲۵.۷ وقتی کد C تولیدی کامپایل نمی‌شود

- پیام کامپایلر را بخوانید و سطر فایل تولیدی را باز کنید؛ سپس سازنده‌ی آن فایل را (فصل ۱۴) و جایی که آن سطر را می‌نویسد پیدا کنید.
- `run.bat tests` را با کامپایلر اجرا کنید؛ `run_compile_test.py` همان خطا را با گونه‌های ورودی مختلف بازتولید می‌کند و گونه‌ی کوچک‌تری برای اشکال‌زدایی می‌دهد.
- نام تکراری یا نامعتبر: قاعده‌های `NAME00x` باید آن را پیش از تولید می‌گرفتند؛ اگر نگرفتند، باگ در قاعده است (فصل ۱۲).
- `APP_FLAG_X undeclared`: تگ آزادی که کلمه است و هدرش در `codegen.extra_includes` نیست.
- `chrono.h: No such file` :`smart_data.c` ماژول ساعت فرمور را می‌خواهد (فصل ۲۲).
- هشدار به‌جای خطا (مثلاً متغیر بی‌استفاده) هم مهم است: فرمور شرکت با `-Werror` می‌سازد.

۲۵.۸ ردیابی یک مشکل از صفحه تا فایل تولیدی

فرض کنید کاربر می‌گوید «واحد Airspeed را km/h کردم ولی در کد تولیدی هنوز m/s است». مسیر داده را گام‌به‌گام دنبال کنید و در هر گام ببینید مقدار درست است یا نه؛ اولین گامی که غلط است، جای باگ است:

گام	کجا را ببینید	چه باید ببینید
۱. صفحه فرستاد؟	Payload ▸ api/update ▸ Network ▸ F12	"unit": "km/h" در changes
۲. سند پذیرفت؟	جواب همان درخواست	{"ok": true}؛ اگر نه، message می‌گوید چرا
۳. وضعیت درست برگشت؟	tree ▸ Response ▸ api/state ▸ Network	"unit": "km/h" در گره
۴. ذخیره شد؟	فایل core.xml در ویرایشگر متن	unit="km/h" روی <Parameter name="Airspeed">؛ * کنار نام فایل نیست
۵. خوانده شد؟	پایتون تعاملی (بخش ۲۵.۵)	model.find("Airspeed").unit == "km/h"
۶. نوشته شد؟	cg_smart_data.c و cg_smart_data.h	"km/h" در سازنده و کامنت

مثلاً اگر گام ۳ درست است و گام ۴ نه، مشکل در Document.save یا xml_writer.py است؛ اگر گام ۶ غلط است ولی ۵ درست، در literals.constructor_arguments یا تنظیم codegen.emit_unit. یک علت رایج دیگر: ساب‌لیست ذخیره نشده و تولید از خط فرمان انجام شده.

۲۵.۹ مشکل‌های رایج

نشانه	علت	راه‌حل
«Python was not found» در run.bat	هیچ python.exe سالمی پیدا نشد: نه در PATH، نه با py، نه در پوشه‌های نصب پایتون (find_python.bat همه را امتحان می‌کند؛ میان‌بر خراب WindowsApps رد می‌شود)	نصب از python.org با « Add python.exe to PATH»؛ اگر نصب است، در cmd بزنید where python.exe و هر کدام را با -- version اجرا کنید
ModuleNotFoundError: No module named 'pl_codegen'	اسکریپتی بیرون bin/ بی افزودن src به sys.path	از bin/run_*.py اجرا کنید یا sys.path.insert(0, "src")
منو می‌گوید « The editor could not start. Try another port» (در کنسول OSError درباره‌ی socket)	درگاه ۸۷۶۵ را برنامه‌ی دیگری گرفته (این پیام روی لینوکس)	run.bat editor -p 8800

نشانه	علت	راه‌حل
ویرایشگر تازه پروژه‌ی **قبلی** را نشان می‌دهد و ویرایش می‌کند	ویرایشگر قبلی هنوز در کنسولش روشن است (بستن پنجره سرور را نمی‌بندد). روی ویندوز دومی بی‌خطا روی همان درگاه ۸۷۶۵ بالا می‌آید، ولی همه‌ی درخواست‌ها به اولی می‌رسند	کنسول ویرایشگر قبلی را ببندید (Ctrl+C) و دوباره اجرا کنید؛ یا ویرایشگر دوم را با `-p 8800`
پنجره‌ی ویرایشگر باز نمی‌شود ولی کنسول نشانی را می‌گوید	مرورگر Chromium پیدا نشد یا شروع نشد	نشانی را در مرورگر باز کنید، یا `PL_CODEGEN_BROWSER` را به مسیر مرورگر بدهید
تغییر کد پایتون دیده نمی‌شود	سرور قبلی هنوز اجرا می‌شود	Ctrl+C در کنسول و اجرای دوباره
تغییر `web/` دیده نمی‌شود	صفحه بار نشده	F5
`run.sh`: «bad interpreter» یا «not found»	پایان سطر CRLF	در ویرایشگر به LF برگردانید (فصل ۷)
`UnicodeEncodeError` هنگام چاپ	خروجی به فایل یا pipe با code page ویندوز	`set PYTHONIOENCODING=utf-8` یا `sys.stdout.reconfigure(encoding="utf-8")`
فارسی در فایل خراب است	نوشتن یا خواندن بی `encoding="utf-8"`	همیشه `encoding="utf-8"` (بخش ۴.۱۵)
همه‌ی آزمون‌ها گذشت ولی کد فرمور کامپایل نمی‌شود	`run_compile_test` بی کامپایلر «SKIPPED» شده	نصب MinGW و اجرای دوباره
`chrono.h: No such file or directory`	ماژول ساعت فرمور در مسیر include نیست	مسیر include فرمور؛ در آزمون، stub خودکار ساخته می‌شود
پیام « Unknown configuration key»	اشتباه تایپی در `pl_project.json`	پیام نزدیک‌ترین نام را پیشنهاد می‌دهد
* کنار نام فایل بی هیچ تغییری	فایل با شکل دیگری از آنچه ذخیره می‌نویسد ذخیره شده (مثلاً با دست)	یک بار ذخیره کنید
پروژه پس از جابه‌جایی پوشه کار نمی‌کند	مسیر کاملی در فایل پروژه (با دست) نوشته شده	مسیرها را نسبی کنید (نسبت به پوشه‌ی `pl_project.json`)

پیوست الف. فهرست همه‌ی فایل‌ها

همه‌ی 116 فایل مخزن (جز راهنماهای Word و PDF و فایل‌های موقت پایتون)، با بخشی از این راهنما که شرحشان داده است. کد هر فایل را در خود فایل، در همین مسیر، بخوانید: این راهنما شرح می‌دهد و به کد ارجاع می‌دهد، کد را تکرار نمی‌کند.

ریشه‌ی مخزن

چیست	بخش	فایل
پایان سطرها در git	۲.۵	`.gitattributes`
فایل‌هایی که در git نمی‌روند	۲.۵	`.gitignore`
معرفی مخزن	۲.۵	`README.md`

کتابخانه‌ی C

چیست	بخش	فایل
نوشتن از راه جدول و رویداد تغییر	۲۲.۴	`parameter_descriptor/parameter_descriptor.c`
شکل یک ردیف جدول پارامترها	۲۲.۳	`parameter_descriptor/parameter_descriptor.h`
پیاده‌سازی smart data	۲۲.۲	`smart_data/smart_data.c`
رابط smart data	۲۲.۱	`smart_data/smart_data.h`

برنامه: راه‌اندازی

چیست	بخش	فایل
معرفی برنامه	۲.۵	`paramgen/README.md`
تنها فایلی که کاربر ویندوز اجرا می‌کند	۲۱.۱	`paramgen/run.bat`
همتای لینوکسی run.bat	۲۱.۳	`paramgen/run.sh`

برنامه: bin

چیست	بخش	فایل
پیدا کردن پایتونی که واقعاً اجرا شود	۲۱.۲	paramgen/bin/find_python.bat
منوی همه‌ی کارها	۲۱.۴	paramgen/bin/run.py
تولیدکننده از خط فرمان	۲۱.۵	paramgen/bin/run_generate.py
ویرایشگر	۲۱.۵	paramgen/bin/run_gui.py
بررسی رایانه	۲۱.۵	paramgen/bin/run_setup.py
همه‌ی آزمون‌ها	۲۱.۵	paramgen/bin/run_tests.py
دابل‌کلیک برای تغییر نسخه	۲۱.۷	paramgen/bin/set_version.bat
خواندن و نوشتن سطر نسخه	۲۱.۷	paramgen/bin/set_version.py

برنامه: الگوها

چیست	بخش	فایل
snippetهای VS Code و قالب فایل‌های C شرکت	۶.۵	paramgen/templates/c.json
فهرست‌های کشویی پیش‌فرض شرکت	۶.۴	paramgen/templates/choices.json
الگوی فایل پروژه	۶.۳	paramgen/templates/pl_project.json

برنامه: نمونه‌ی C

چیست	بخش	فایل
نمونه‌ی استفاده از کد تولیدی در فرمور	۲۲.۵	paramgen/docs/example/parameter_list_usage_example.c
پروتوتایپ تابع‌های نمونه‌ی استفاده	۲۲.۵	paramgen/docs/example/parameter_list_usage_example.h

تولیدکننده: pl_codegen

چیست	بخش	فایل
شناسنامه‌ی پکیج	۹.۱	paramgen/src/pl_codegen/__init__.py
برنامه کجاست	۹.۲	paramgen/src/pl_codegen/app_home.py
لیست‌های کشویی رایانه	۹.۷	paramgen/src/pl_codegen/choices.py

چیست	بخش	فایل
خط فرمان تولیدکننده	۱۵.۲	paramgen/src/pl_codegen/cli.py
درِ ورودی پکیج codegen	۱۴.۱	paramgen/src/pl_codegen/codegen/__init__.py
فهرست بلوک‌ها	۱۴.۶	paramgen/src/pl_codegen/codegen/blocks/__init__.py
کامنت‌ها	۱۴.۹	paramgen/src/pl_codegen/codegen/blocks/comment.py
محافظ C++	۱۴.۱۷	paramgen/src/pl_codegen/codegen/blocks/cpp_guard.py
سطرهای #define	۱۴.۸	paramgen/src/pl_codegen/codegen/blocks/define.py
typedef enum	۱۴.۱۰	paramgen/src/pl_codegen/codegen/blocks/enum.py
بنر بالای فایل	۱۴.۱۴	paramgen/src/pl_codegen/codegen/blocks/file_banner.py
سطر آخر فایل	۱۴.۱۵	paramgen/src/pl_codegen/codegen/blocks/file_footer.py
پروتوتایپ و بدنه‌ی تابع	۱۴.۱۲	paramgen/src/pl_codegen/codegen/blocks/function.py
سطر #include	۱۴.۷	paramgen/src/pl_codegen/codegen/blocks/include.py
محافظ include دوباره	۱۴.۱۶	paramgen/src/pl_codegen/codegen/blocks/include_guard.py
نشانگر بخش	۱۴.۱۸	paramgen/src/pl_codegen/codegen/blocks/section_marker.py
جدول متنی	۱۴.۱۳	paramgen/src/pl_codegen/codegen/blocks/table.py
اعلان و تعریف متغیر	۱۴.۱۱	paramgen/src/pl_codegen/codegen/blocks/variable.py
شکل فایل‌های شرکت، از روی c.json	۱۴.۲	paramgen/src/pl_codegen/codegen/c_style.py
ترتیب فیلدهای sParameterDescriptor	۱۴.۳	paramgen/src/pl_codegen/codegen/descriptor_layout.py
کدام فایل‌ها ساخته شوند	۱۴.۲۰	paramgen/src/pl_codegen/codegen/files/__init__.py
کلاس پایه‌ی همه‌ی سازنده‌ها	۱۴.۲۱	paramgen/src/pl_codegen/codegen/files/file_builder.py
قالب param_table: فایل nsrParamOffsets.h	۱۴.۳۰	paramgen/src/pl_codegen/codegen/files/param_offsets_h.py
قالب param_table: فایل nsrParamTables.h	۱۴.۲۹	paramgen/src/pl_codegen/codegen/files/param_table_h.py
فایل cg_parameter_list.c	۱۴.۲۷	paramgen/src/pl_codegen/codegen/files/parameter_list_c.py
فایل cg_parameter_list_defines.h	۱۴.۲۲	paramgen/src/pl_codegen/codegen/files/parameter_list_defines_h.py
فایل cg_parameter_list.h	۱۴.۲۶	paramgen/src/pl_codegen/codegen/files/parameter_list_h.py

چیست	بخش	فایل
فایل cg_parameter_list_table.h	۱۴.۲۸	paramgen/src/pl_codegen/codegen/files/parameter_list_table_h.py
فایل cg_parameter_list_types.h	۱۴.۲۳	paramgen/src/pl_codegen/codegen/files/parameter_types_h.py
فایل cg_smart_data.c	۱۴.۲۵	paramgen/src/pl_codegen/codegen/files/smart_data_c.py
فایل cg_smart_data.h	۱۴.۲۴	paramgen/src/pl_codegen/codegen/files/smart_data_h.py
قالب param_table: فایل nsrSystemNames.h	۱۴.۳۱	paramgen/src/pl_codegen/codegen/files/system_names_h.py
نوشتن عددها و رشته‌ها به زبان C	۱۴.۴	paramgen/src/pl_codegen/codegen/literals.py
خواندن فایل پروژه	۹.۶	paramgen/src/pl_codegen/config.py
خطاها و هشدارها	۹.۳	paramgen/src/pl_codegen/errors.py
درِ ورودی پکیج layout	۱۳.۱	paramgen/src/pl_codegen/layout/__init__.py
آیا این ساخت چیزی را جابه‌جا کرده است؟	۱۳.۳	paramgen/src/pl_codegen/layout/drift.py
آفست هر مقدار در ناحیه‌ی هر تگ	۱۳.۲	paramgen/src/pl_codegen/layout/tag_offset.py
کنسول و رنگ	۹.۴	paramgen/src/pl_codegen/logging_utils.py
درِ ورودی پکیج model	۱۰.۱	paramgen/src/pl_codegen/model/__init__.py
لیست‌های کشویی در حافظه	۱۰.۴	paramgen/src/pl_codegen/model/defines.py
یک گره از درخت	۱۰.۵	paramgen/src/pl_codegen/model/parameter.py
کل فهرست	۱۰.۶	paramgen/src/pl_codegen/model/parameter_list.py
تبدیل استراکچرهای قدیمی	۱۰.۹	paramgen/src/pl_codegen/model/struct_migration.py
هماهنگ کردن استراکچرها با تایپشان	۱۰.۸	paramgen/src/pl_codegen/model/struct_sync.py
تایپ‌های استراکچر	۱۰.۷	paramgen/src/pl_codegen/model/struct_type.py
فهرست تگ‌ها	۱۰.۳	paramgen/src/pl_codegen/model/tags.py
نوع‌های داده‌ی C	۱۰.۲	paramgen/src/pl_codegen/model/types.py
از نام پارامتر تا نام‌های C	۹.۵	paramgen/src/pl_codegen/naming.py
خط‌لوله‌ی مشترک	۱۵.۱	paramgen/src/pl_codegen/pipeline.py
معرفی کوتاه پکیج pl_codegen	۹.۱	paramgen/src/pl_codegen/README.md
درِ ورودی پکیج validation	۱۲.۱	paramgen/src/pl_codegen/validation/__init__.py
فهرست همه‌ی قاعده‌ها	۱۲.۲	paramgen/src/pl_codegen/validation/rules/__init__.py
قاعده‌ی فهرست‌های کشویی	۱۲.۵	paramgen/src/pl_codegen/validation/rules/defines_rules.py

چیست	بخش	فایل
قاعده‌های نام‌ها و شناسه‌های ساخته‌شده از آن‌ها	۱۲.۶	paramgen/src/pl_codegen/validation/rules/name_rules.py
قاعده‌های قالب param_table	۱۲.۱۱	paramgen/src/pl_codegen/validation/rules/param_table_rules.py
قاعده‌های پیش‌فرض، کمینه، بیشینه و رزولوشن	۱۲.۹	paramgen/src/pl_codegen/validation/rules/range_rules.py
کلاس پایه‌ی قاعده‌ها و چند کمک‌تابع	۱۲.۴	paramgen/src/pl_codegen/validation/rules/rule_base.py
قاعده‌های تایپ‌های استراکچر	۱۲.۷	paramgen/src/pl_codegen/validation/rules/struct_type_rules.py
قاعده‌های تگ‌ها، نام نمایشی و شناسه‌ی لاگ	۱۲.۱۰	paramgen/src/pl_codegen/validation/rules/tag_rules.py
قاعده‌های نوع داده، Type و اندازه‌ی آرایه	۱۲.۸	paramgen/src/pl_codegen/validation/rules/type_rules.py
اجرای همه‌ی قاعده‌ها	۱۲.۳	paramgen/src/pl_codegen/validation/validator.py
درِ ورودی پکیج xml_model	۱۱.۱	paramgen/src/pl_codegen/xml_model/__init__.py
نام عنصرها و ویژگی‌ها	۱۱.۲	paramgen/src/pl_codegen/xml_model/schema.py
فایل تایپ‌های استراکچر	۱۱.۵	paramgen/src/pl_codegen/xml_model/types_xml.py
از XML به مدل	۱۱.۳	paramgen/src/pl_codegen/xml_model/xml_reader.py
از مدل به XML	۱۱.۴	paramgen/src/pl_codegen/xml_model/xml_writer.py

ویرایشگر: pl_gui

چیست	بخش	فایل
درِ ورودی پکیج pl_gui	۱۶.۱	paramgen/src/pl_gui/__init__.py
یک متد برای هر کار صفحه	۱۹.۱	paramgen/src/pl_gui/api.py
پیدا کردن مرورگر و باز کردن آن بی‌شبکه	۱۶.۲	paramgen/src/pl_gui/browser.py
نوشتن فهرست ساب‌لیست‌ها در pl_project.json، با حفظ کامنت‌ها	۱۷.۳	paramgen/src/pl_gui/config_file.py
سند: فهرست باز، فایلش و Undo	۱۸.۱	paramgen/src/pl_gui/document.py
پنجره‌های Open و Browse خود ویندوز	۱۶.۳	paramgen/src/pl_gui/native_dialog.py
سرور HTTP کوچک	۱۷.۱	paramgen/src/pl_gui/server.py
تنها جایی که با سرور حرف می‌زند	۲۰.۳	paramgen/src/pl_gui/web/api.js
هماهنگ‌کننده‌ی صفحه	۲۰.۶	paramgen/src/pl_gui/web/app.js
فرم سمت راست	۲۰.۵	paramgen/src/pl_gui/web/editor.js

چیست	بخش	فایل
جدول سمت چپ	۲۰.۴	paramgen/src/pl_gui/web/grid.js
لوگوی شرکت و آیکون پنجره	۲۰.۱	paramgen/src/pl_gui/web/icon-256.png
لوگوی شرکت و آیکون پنجره	۲۰.۱	paramgen/src/pl_gui/web/icon-32.png
اسکلت صفحه	۲۰.۱	paramgen/src/pl_gui/web/index.html
لوگوی شرکت و آیکون پنجره	۲۰.۱	paramgen/src/pl_gui/web/logo.png
همه‌ی ظاهر	۲۰.۲	paramgen/src/pl_gui/web/style.css
پروژه‌ی باز و پروژه‌های اخیر	۱۷.۲	paramgen/src/pl_gui/workspace.py

بررسی رایانه

چیست	بخش	فایل
آیا این رایانه می‌تواند ابزار را اجرا کند؟	۲۱.۶	paramgen/src/setup_environment.py

آزمون‌ها

چیست	بخش	فایل
کد تولیدی کامپایل و اجرا می‌شود	۲۳.۸	paramgen/tests/run_compile_test.py
راهنماها با کد می‌خوانند؟	۲۳.۴	paramgen/tests/run_docs_test.py
ویرایشگر	۲۳.۲	paramgen/tests/run_editor_test.py
قالب param_table	۲۳.۷	paramgen/tests/run_paramtable_test.py
هر تنظیم اثری دارد؟	۲۳.۳	paramgen/tests/run_settings_test.py
تایپ‌های استراکچر	۲۳.۶	paramgen/tests/run_struct_test.py
قاعده‌های بررسی	۲۳.۱	paramgen/tests/run_validation_test.py
set_version فقط نسخه را عوض می‌کند	۲۳.۵	paramgen/tests/run_version_test.py

پیوست ب. پیام‌های ابزار

همه‌ی 80 کد پیامی که ابزار ممکن است بدهد، مستقیم از کد منبع خوانده شده: کد، شدت، متن انگلیسی پیام (هر … جایی است که مقداری مثل نام پارامتر یا عدد نشانده می‌شود) و تابعی که پیام را می‌سازد. «خطا/هشدار» یعنی شدت به این بستگی دارد که تایپ استراکچر در ساخت استفاده می‌شود یا نه (_TypeProblems، فصل ۱۲). پیشوندها: ARR آرایه، CFG فایل پروژه، DEF فهرست‌های کشویی، GRP Type، META واحد، NAME نام‌ها، OUT خروجی، PTB قالب param_table، RNG بازه، STR استراکچر، TAG تگ‌ها و شناسه‌ی لاگ، TYP تایپ‌ها، TYPE نوع داده، VAL کل فهرست، XML خواندن فایل.

کد	شدت	پیام	کجا
ARR001	خطا	Array size … is invalid; it must be 1 or greater.	type_rules.py: ArraySizeRule.check
ARR002	خطا	Array size … exceeds the maximum of ….	type_rules.py: ArraySizeRule.check
CFG001	خطا	(متن پیام از جای دیگری می‌آید)	document.py: Document.load
DEF001	خطا	The '…' list is empty, so none of its defines can be generated.	defines_rules.py: DefinesSheetRule.check
DEF002	خطا	'…' in the '…' column is not a valid C identifier, so it becomes a # define in the generated header.	defines_rules.py: DefinesSheetRule.check
DEF004	خطا	'…' and under '…' is listed both under '…'; the two defines would collide.	defines_rules.py: DefinesSheetRule.check
DEF005	خطا	… and … in the '…' list are both …, so the firmware cannot tell them apart.	defines_rules.py: DefinesSheetRule._check_numbers
DEF006	خطا	… in the '…' list is …, which does not fit the uint32_t tag it is stored in (0 to …).	defines_rules.py: DefinesSheetRule._check_numbers
GRP001	خطا	No type is set (one and no structure above it sets).	type_rules.py: ParameterTypeRule.check
GRP002	خطا	Type '…' has no matching enumerator (…) in parameter_descriptor.h.	type_rules.py: ParameterTypeRule.check
GRP003	هشدار	Type '…' list is not in the TYPE'.	type_rules.py: ParameterTypeRule.check
META002	خطا	The '…' cell is empty in Unit'.	range_rules.py: MetadataRule.check
NAME001	خطا	Name '…' is not a valid C identifier.	name_rules.py: NameSyntaxRule.check

کجا	پیام	شدت	کد
name_rules.py: NameSyntaxRule.check	Name '…' is a reserved C/C++ keyword.	خطا	NAME002
name_rules.py: NameUniquenessRule._check_level	Name '…' is used … times in …; this one is in ….	خطا	NAME003
name_rules.py: IndexMacroCollisionRule.check	'…' is used for more than one thing: ….	خطا	NAME004
name_rules.py: GeneratedIdentifierRule.check	The array size macro derived from '…' is '…', which is not a usable C identifier.	خطا	NAME005
pipeline.py: Pipeline._report_abandoned_copies	An older copy of … is still at …, and nothing writes to it any more.	هشدار	OUT001
param_table_rules.py: ParamTableShapeRule.check	'…' is a structure, and this project generates a parameter table, which has none.	خطا	PTB001
param_table_rules.py: ParamTableShapeRule.check	'…' has array size …, and a parameter table holds one value per row.	خطا	PTB002
param_table_rules.py: ParamTableSystemRule.check	'…' does not name a sub-system: '…' is not in the 'SYSTEM NAMES' list.	خطا	PTB005
param_table_rules.py: ParamTableTypeRule.check	'…' is a bool_t, and the parameter table has no boolean type.	خطا	PTB006
param_table_rules.py: ParamTableSystemTagRule.check؛ document.py :Document.load	… parameter(s) of '…' do not say in Tag '…' - System Name says '…'.	هشدار	PTB008
param_table_rules.py: ParamTableAddressRule.check	'…' has address …, and an address is a whole number from 0 to ….	خطا	PTB009
range_rules.py: NativeRangeRule.check	'…' = … does not fit into '…' (allowed: … .. …).	خطا	RNG001
range_rules.py: MinMaxOrderRule.check	Minimum (…) is greater than maximum (…). The smart-data constructor would refuse every value.	خطا	RNG002
range_rules.py: DefaultInRangeRule.check	Default value … is outside the allowed range […,…]. The smart-data constructor would refuse it at run time.	خطا	RNG003
range_rules.py: RequiredValuesRule._check_boolean	…: ignored, because '…' has no minimum, maximum or resolution.	هشدار	RNG010
range_rules.py: RequiredValuesRule.check	No default value; 0 is used instead.	هشدار	RNG011

کجا	پیام	شدت	کد
range_rules.py: RequiredValuesRule.check	0 is ;Minimum' is empty' but … Maximum' is' .used as the minimum	هشدار	RNG012
range_rules.py: RequiredValuesRule.check	0 is ;Maximum' is empty' but … Minimum' is' .used as the maximum	هشدار	RNG013
range_rules.py: RequiredValuesRule.check	the default for) is used instead … ;No resolution .('…').	هشدار	RNG014
range_rules.py: RequiredValuesRule.check	.is not positive … Resolution	هشدار	RNG015
range_rules.py: RequiredValuesRule.check	,…] is larger than the whole range … Resolution only one value would ever be ;(… span) […] .reachable	هشدار	RNG016
range_rules.py: IntegralValueRule.check	'…' and ,which is not a whole number ,… is '…' … cannot hold it: the generated code would use .instead	خطا	RNG017
name_rules.py: StructRule.check	.has no enabled field '…' Structure	خطا	STR001
name_rules.py: StructRule.check	it must be 1 or ;… has array size '…' Structure .more	خطا	STR002
tag_rules.py: MandatoryTagRule.check	.empty … … Mandatory tag…	خطا	TAG001
tag_rules.py: TagEnumValueRule.check	.is not a valid C identifier '…' value '…' Tag	خطا	TAG002
tag_rules.py: TagEnumValueRule.check	.list '…' is not listed in the '…' value '…' Tag	خطا	TAG003
tag_rules.py: TagValueRangeRule.check	0) does not fit into uint32_t … value ('…') '…' Tag .(… ..	خطا	TAG004
tag_rules.py: FreeTagSyntaxRule.check	is neither a number nor a valid '…' value '…' Tag .C identifier	خطا	TAG005
tag_rules.py: LogIdRule.check	.The parameter is loggable but has no log id	خطا	TAG010
tag_rules.py: UserNameRule.check	and the ,is loggable '…' .has no User Name '…' so the cell ,log names a value by its User Name has to be filled in.…	خطا	TAG011
tag_rules.py: LogIdRule.check	.'…' is already used by … Log id	خطا	TAG012
tag_rules.py: LogIdRule.check	the ;Tag Loggable' is FALSE' but … The log id is .id is ignored and 0 is written into the table	اطلاع	TAG014

کجا	پیام	شدت	کد
tag_rules.py: `LogIdRule.check`	Log id '...' of ... does not fit into the uint16_t field' LogId' (0.....).	خطا	TAG015
struct_migration.py: `_adopt`	Structure type '...' had no structure type '...' one called ;was made from the fields it already has.	اطلاع	TYP001
struct_type_rules.py: `StructTypeRule.check`	Structure '...' has no structure type.	خطا	TYP002
struct_type_rules.py: `StructTypeRule.check`	Structure '...' is of type '...', this project which does not have.	خطا	TYP003
struct_type_rules.py: `StructTypeRule.check`	'...' is a parameter but its data type '...' is a structure type.	خطا	TYP004
struct_type_rules.py: `StructTypeRule.check`	Structure '...' is of type '...', which is a data type and not a structure.	خطا	TYP005
struct_type_rules.py: `StructTypeDeclarationRule.check`	Structure type '...' is not a valid C identifier.	خطا/هشدار	TYP006
struct_type_rules.py: `StructTypeDeclarationRule.check`	Structure type '...' has no fields.	خطا/هشدار	TYP007
struct_type_rules.py: `StructTypeDeclarationRule.check`	Structure type '...' has two fields called '...'.	خطا/هشدار	TYP008
struct_type_rules.py: `StructTypeDeclarationRule.check`	Field '.....' has no data type.	خطا/هشدار	TYP009
types_xml.py: `read_types`	A <StructType element has no 'name' attribute.	خطا	TYP010
types_xml.py: `read_types`	A field of '...' has no 'name' attribute.	خطا	TYP011
types_xml.py: `read_types`	'...' is declared twice in this file.	خطا	TYP012
struct_type_rules.py: `StructTypeDeclarationRule.check`	Field '.....' is of type '...', which is neither a data type nor a structure type.	خطا/هشدار	TYP013
struct_type_rules.py: `StructTypeDeclarationRule.check`	Field '.....' has array size ;... it must be 1 or more.	خطا/هشدار	TYP014
struct_type_rules.py: `StructTypeCycleRule.check`	Structure type '...' contains itself:	خطا/هشدار	TYP015
struct_type_rules.py: `StructTypeClashRule.check`	Structure type '...' is declared by more than one parameter sub list: .... A structure type belongs to one sub list only.	خطا	TYP016

کجا	پیام	شدت	کد
struct_type_rules.py: StructTypeOwnerRule.check	Structure '...' of type ... which ,'...' is not declared - it is a type of .... A parameter sub list uses only its own structure types.	خطا، خطا/ه شدار	TYP017
type_rules.py: ValueTypeRule.check	No data type is set.	خطا	TYPE001
type_rules.py: ValueTypeRule.check	Data type '...' is not supported.	خطا	TYPE002
type_rules.py: DeclaredTypeRule.check	Data type '...' is not listed in the 'TYPES' column ' of the 'TYPES' list.	هشدار	TYPE003
validator.py: Validator.validate	The parameter list is empty; nothing would be generated....	خطا	VAL001
validator.py: Validator.validate	This project has no parameter sub list, so there is nothing to generate.	خطا	VAL002
xml_reader.py: read_xml	XML schema version '...' differs from the ('...') version supported by this tool.	هشدار	XML001
xml_reader.py: read_xml	Parameter '...' element is missing.	خطا	XML002
xml_reader.py: _read_parameter	A <Parameter> element has no ... attribute' ' name.	خطا	XML010
xml_reader.py: _read_parameter	Unknown tag '...' is ignored.	هشدار	XML011
xml_reader.py: _report_bad_number	Field '...' of structure '...' is '...' in the XML model, which	خطا	XML012
xml_reader.py: _drop_inherited	Field '...' of structure '...' carried the tag(s '...' ...); they were dropped.	هشدار	XML013
xml_reader.py: _drop_inherited	Field '...' of structure '...' carried the type '...'; it was dropped.	هشدار	XML014
xml_reader.py: _drop_inherited	Field '...' of structure '...' carried the log id '...'; it was dropped.	هشدار	XML015
xml_reader.py: _read_parameter	Unknown parameter kind '...'; primitive' is ' assumed.	هشدار	XML016
xml_reader.py: _read_parameter	Parameter '...' has address '...', which is not a whole number; 0 is used.	هشدار	XML017
xml_reader.py: _read_parameter	Parameter '...' has a description for element '...', which is not an element path; it is ignored.	هشدار	XML018

پیوست پ. واژه‌نامه

واژه‌هایی که در این راهنما به معنای خاص به کار رفته‌اند، با معادل انگلیسی‌شان در کد و در ویرایشگر.

معنی	انگلیسی	واژه
کارهایی که صفحه از سرور می‌خواهد؛ `api.py` (فصل ۱۹)	`API`	API
MONITORING، SETTING یا COMMAND؛ `eParameterType`	`parameter type`	Type
بازگرداندن آخرین ویرایش؛ با کپی کامل مدل	`undo`	Undo
پارامتری با اندازه‌ی بیشتر از ۱؛ هر عنصر یک مقدار و یک ردیف جدول	`array`	آرایه
اسکریپتی که درستی رفتاری را می‌سنجد (فصل ۲۳)	`test`	آزمون
جای یک مقدار در ناحیه‌ی یک تگ، بر حسب بایت (فصل ۱۳)	`MemoryOffset`	آفست حافظه
خطایی که با `raise` پرتاب و با `except/try` گرفته می‌شود	`exception`	استثنا
پارامتری که فیلد دارد و از یک تایپ است	`structure`	استراکچر
یک گره‌ی درخت؛ آرایه‌ی ۴تایی یک اعلان و چهار مقدار است	`declaration`	اعلان
پوشه‌ی `paramgen/`؛ یک بار روی هر رایانه	`app`	برنامه
یک مقدار تکی در فهرست تخت `flatten()`	`leaf`	برگ
شکل یک استراکچر: نام و فیلدها؛ مثل `struct` در C	`structure type`	تایپ استراکچر
یک ردیف `sParameterDescriptor` جدول `ParameterList`	`descriptor`	توصیف‌گر
`pl_codegen`؛ از XML کد C می‌سازد	`generator`	تولیدکننده
طبقه‌بندی یک پارامتر که در توصیف‌گر یک فیلد `uint32_t` است (SystemName، ResetProof، …)	`tag`	تگ
تغییر آفست یا اندیس نسبت به ساخت قبلی (فصل ۱۳)	`layout drift`	جابه‌جایی چیدمان
`DiagnosticCollector`؛ همه‌ی پیام‌های یک اجرا	`diagnostic collector`	جمع‌کننده‌ی پیام
ترتیب خواندن، بررسی، آفست و تولید؛ `pipeline.py`	`pipeline`	خط‌لوله
شماره‌ی سرور محلی ویرایشگر؛ پیش‌فرض ۸۷۶۵	`port`	درگاه
جدول کلید ← مقدار در پایتون	`dict`	دیکشنری
یک فایل XML فهرست؛ پروژه چند ساب‌لیست دارد	`sub list`	ساب‌لیست
یک اجرای تولید کد از همه‌ی ساب‌لیست‌های پروژه	`build`	ساخت
کلاسی که یک فایل تولیدی را می‌نویسد (فصل ۱۴)	`file builder`	سازنده‌ی فایل
`Document`؛ ساب‌لیست باز در ویرایشگر با Undo	`document`	سند
عددی که لاگ مقدار را با آن می‌شناسد؛ `LogId`	`log id`	شناسه‌ی لاگ

معنی	انگلیسی	واژه
متغیر `sSmartData..` که مقدار و بازه و زمان را دارد	`smart data object`	شیء smart data
بخش مرورگری ویرایشگر؛ پوشه‌ی `web/`	`page`	صفحه
SYSTEM NAMES، PARAM MODES، TYPE، VALUE TYPES؛ مال رایانه	`drop-down list / choices`	فهرست کشویی
عضو یک استراکچر	`field`	فیلد
کلاسی که یک بررسی را انجام می‌دهد (فصل ۱۲)	`rule`	قاعده
`smart_data` یا `param_table`	`output format`	قالب خروجی
یک فایل `.py`	`module`	ماژول
تابعی که مال یک کلاس است	`method`	متد
شکل فهرست در حافظه؛ `ParameterList` و `Parameter`	`model`	مدل
نام کامل یک گره: `Position.Lat` یا `Motor[1].Rpm`	`dotted path`	مسیر نقطه‌دار
بایت‌هایی که مقدارهای دارای یک تگ پشت سر هم در آن می‌نشینند	`tag area`	ناحیه‌ی تگ
نامی که آدم می‌خواند؛ `pUserName`	`User Name`	نام نمایشی
`__version__`؛ قالب `xx.xx.xx`	`version`	نسخه
سطر `/* Includes ---- */` فایل‌های C	`section marker`	نشانگر بخش
همه‌ی ساب‌لیست‌ها با هم، فقط‌خواندنی، در ویرایشگر	`merged view`	نمای ادغام‌شده
`pl_gui`؛ سرور محلی و صفحه	`editor`	ویرایشگر
پوشه‌ای با `pl_project.json`، ساب‌لیست‌ها و خروجی	`project`	پروژه
پوشه‌ای از ماژول‌های پایتون با `__init__.py`	`package`	پکیج
خطا، هشدار یا اطلاعی که بررسی‌ها می‌دهند؛ با کد (پیوست ب)	`diagnostic`	پیام
ماژول‌هایی که همراه پایتون نصب می‌شوند	`standard library`	کتابخانه‌ی استاندارد
شناسه‌ی ثابت یک پیام، مثل `TAG012`	`diagnostic code`	کد پیام
struct با تابع‌ها در پایتون یا JS	`class`	کلاس

پیوست ت. نمایه

هر تابع، کلاس، متد و فایلی که در این راهنما شرح داده شده، به ترتیب الفبا، با شماره‌ی صفحه‌ی شرحش. کنار نام هر تابع، در پرانتز، نام فایلی آمده که آن را دارد.

.

.gitattributes 15
.gitignore 15

_

__contains__ (struct_type.py) 90
__init__ (api.py) 194
__init__ (comment.py) 138
__init__ (cpp_guard.py) 141
__init__ (define.py) 137
__init__ (enum.py) 138
__init__ (errors.py) 58
__init__ (file_banner.py) 140
__init__ (file_builder.py) 143
__init__ (file_footer.py) 140
__init__ (function.py) 139
__init__ (include.py) 137
__init__ (include_guard.py) 141
__init__ (rule_base.py) 108
__init__ (section_marker.py) 141
__init__ (server.py) 169
__init__ (struct_type.py) 90
__init__ (struct_type_rules.py) 113
__init__ (tag_offset.py) 126
__init__ (validator.py) 107
__init__ (variable.py) 139
__init__ (workspace.py) 171
__init__.py 50, 74, 95, 106, 123, 131, 137, 142, 164
__iter__ (parameter_list.py) 85
__iter__ (struct_type.py) 90
__len__ (errors.py) 58
__len__ (parameter_list.py) 85
__len__ (struct_type.py) 90
__str__ (errors.py) 57
__str__ (parameter.py) 82
__str__ (rule_base.py) 108
__str__ (struct_type.py) 90
_a_project (run_editor_test.py) 234
_address_of (param_table_h.py) 154
_admit_to_the_build (document.py) 184
_adopt (struct_migration.py) 94
_all_entries (parameter_list_table_h.py) 152
_all_lists_state (api.py) 196
_all_nodes (name_rules.py) 112
_another_lists_type (run_validation_test.py) 233
_apply_field (struct_sync.py) 92
_apply_to_one_value (document.py) 181
_apply_update (document.py) 182
_array_size_entries (parameter_list_defines_h.py) 146
_array_span (config_file.py) 174
_as_float32 (literals.py) 134
_as_template (xml_reader.py) 100
_as_the_form_sends (run_editor_test.py) 234
_as_written (document.py) 191
_block (param_offsets_h.py) 155
_block (param_table_h.py) 154
_blocks (param_offsets_h.py) 155
_blocks (param_table_h.py) 154
_bring_into_the_project (document.py) 185
_build_from_the_list_folder (run_editor_test.py) 234
_cells (param_table_h.py) 154
_change (document.py) 178
_check_param_table (run_compile_test.py) 244
_checked_address (document.py) 190
_clear_log_ids (document.py) 190
_closest (config.py) 69
_collect (parameter_list.py) 88
_collect_arrays (parameter_list.py) 89
_collect_structs (parameter_list.py) 89
_ColourFormatter (logging_utils.py) 61
_comments_in (config_file.py) 174
_copy_name (document.py) 191
_count_rows (api.py) 200
_cycle_from (struct_type_rules.py) 114
_dataclass_from_dict (config.py) 69
_default_resolution (document.py) 190
_default_tag (document.py) 190
_describe_input (cli.py) 162
_describe_one_value (document.py) 181
_dialogs (api.py) 197
_distribute_log_ids (parameter_list.py) 89
_drop_inherited (xml_reader.py) 100
_effective_limit (parameter.py) 83
_element_order (xml_writer.py) 102
_empty_the_other_lists (run_editor_test.py) 234
_enable_windows_ansi (logging_utils.py) 60
_ensure_defines (document.py) 177
_entries (system_names_h.py) 156
_existing (native_dialog.py) 166
_extract_banner (c_style.py) 132
_field_from (document.py) 186

```
                186  _fields (document.py)
             187  _fields_from (document.py)
             112  _fields_of (name_rules.py)
               112  _file_of (name_rules.py)
               177  _file_text (document.py)
             174  _find_key (config_file.py)
          94  _free_name (struct_migration.py)
_generate_extra_includes (parameter_list_c.py)
                                         151
151  _generate_get_by_index (parameter_list_c.py)
 151  _generate_get_by_name (parameter_list_c.py)
151  _generate_get_quantity (parameter_list_c.py)
        151  _generate_init (parameter_list_c.py)
  _generate_init_callback (parameter_list_c.py)
                                         151
   147  _generate_struct (parameter_types_h.py)
     151  _generate_table (parameter_list_c.py)
  151  _generate_tag_sizes (parameter_list_c.py)
     182  _give_every_row_its_text (document.py)
        188  _give_starting_values (document.py)
                186  _give_type (document.py)
 _has_tkinter_and_a_desktop (native_dialog.py)
                                         166
       183  _hold_boolean_shape (document.py)
        183  _hold_number_shape (document.py)
          113  _holder (struct_type_rules.py)
            118  _how_to_fill (tag_rules.py)
                 104  _indent (types_xml.py)
               103  _indent (xml_writer.py)
 152  _index_by_path (parameter_list_table_h.py)
146  _index_entries (parameter_list_defines_h.py)
      152  _index_of (parameter_list_table_h.py)
              200  _indices_by_row (api.py)
           148  _init_body (smart_data_c.py)
                        75  _int (types.py)
         84  _is_zero_or_empty (parameter.py)
                  139  _line (variable.py)
            174  _line_indent (config_file.py)
 146  _list_defines (parameter_list_defines_h.py)
        186  _list_defining_type (document.py)
             184  _list_entry (document.py)
 _list_holding_the_example (run_compile_test.py)
                                         243
       114  _list_name (struct_type_rules.py)
      190  _list_of_a_types_file (document.py)
             177  _list_system (document.py)
                118  _listed (tag_rules.py)
  152  _log_id_range (parameter_list_table_h.py)
        200  _log_ids_by_declaration (api.py)
               170  _make_handler (server.py)
                    200  _mark_owner (api.py)
          147  _members (parameter_types_h.py)
                154  _modes (param_table_h.py)
       182  _move_the_log_block (document.py)
          181  _name_one_value (document.py)
           111  _name_segments (name_rules.py)
          92  _names_a_struct (struct_sync.py)
          243  _names_in (run_compile_test.py)

                92  _new_field (struct_sync.py)
             186  _new_type_name (document.py)
                          199  _node (api.py)
      152  _number (parameter_list_table_h.py)
         129  _numbered_per_system (drift.py)
                     200  _numbers_of (api.py)
       182  _offset_in_the_block (document.py)
  152  _offset_note (parameter_list_table_h.py)
  152  _offset_rows (parameter_list_table_h.py)
_one_type_in_two_lists (run_validation_test.py)
                                         233
                       169  _open (server.py)
             170  _open_app_window (server.py)
    234  _parameter_names (run_editor_test.py)
                  _parameter_of_struct_type
             233  (run_validation_test.py)
    102  _parameter_to_element (xml_writer.py)
                  162  _print_drift (cli.py)
     220  _print_footer (setup_environment.py)
     220  _print_header (setup_environment.py)
                162  _print_summary (cli.py)
                 197  _project_info (api.py)
     243  _project_lists (run_compile_test.py)
        114  _reachable (struct_type_rules.py)
                       72  _read (choices.py)
                  221  _read (set_version.py)
              99  _read_defines (xml_reader.py)
            99  _read_parameter (xml_reader.py)
             172  _read_settings (workspace.py)
               220  _red (setup_environment.py)
                    190  _refusal (document.py)
_refuse_a_file_that_is_not_a_project (config.py)
                                          69
  _refuse_a_name_another_list_has (document.py)
                                         186
          179  _refuse_reshaping (document.py)
179  _refuse_structures_in_a_table (document.py)
      183  _reject_duplicate_name (document.py)
     190  _reject_path_characters (document.py)
      239  _rename_on_disk (run_struct_test.py)
         186  _rename_struct_type (document.py)
    159  _report_abandoned_copies (pipeline.py)
        100  _report_bad_number (xml_reader.py)
             178  _require_a_list (document.py)
                  92  _reshape (struct_sync.py)
                     188  _resync (document.py)
           184  _retune_for_type (document.py)
         111  _root_identifier (name_rules.py)
                           200  _row (api.py)
                       200  _row_texts (api.py)
                  154  _rows (param_table_h.py)
                   166  _run (native_dialog.py)
               243  _run (run_compile_test.py)
                 191  _safe_stem (document.py)
                 190  _same_file (document.py)
         239  _second_list (run_struct_test.py)
  _self_containing_type (run_validation_test.py)
                                         233
```

178 add_parameter (document.py)
198 add_structure (api.py)
187 add_structure (document.py)
205 addExistingList (api.js)
211 addExistingListAt (app.js)
211 addExistingSubList (app.js)
212 addNode (app.js)
204 addStructure (api.js)
211 addStructure (app.js)
172 adopt (workspace.py)
93 adopt_types (struct_migration.py)
90 all (struct_type.py)
193 Api (api.py)
204 api.js
193 api.py
209 app.js
51 app_home.py
165 app_window_command (browser.py)
143 apply_fixed_tags (__init__.py)
91 apply_types (struct_sync.py)
211 applyChanges (app.js)
208 area (editor.js)
126 area (tag_offset.py)
126 areas (tag_offset.py)
87 array_declarations (parameter_list.py)
87 array_nodes (parameter_list.py)
63 array_size_macro (naming.py)
144 array_size_of (file_builder.py)
144 array_size_of_name (file_builder.py)
115 ArraySizeRule (type_rules.py)
166 ask (native_dialog.py)
86 assign_paths (parameter_list.py)
166 available (native_dialog.py)

B

62 banner (logging_utils.py)
132 banner_lines (c_style.py)
212 bindMenu (app.js)
210 bindProblemRows (app.js)
210 bindSeverityFilters (app.js)
212 bindShortcuts (app.js)
212 bindToolbar (app.js)
154 blocks_by_system (param_table_h.py)
134 bool_literal (literals.py)
128 breaks_stored_data (drift.py)
233 broken_model (run_validation_test.py)
205 browse (api.js)
198 browse (api.py)
186 browse (document.py)
197 browse_projects (api.py)
204 browseProjects (api.js)
164 browser.py
126 build_offset_map (tag_offset.py)
161 build_parser (cli.py)
142 builders_for (__init__.py)
85 by_name (parameter_list.py)
101 _set (xml_writer.py)
135 _shortest_exact (literals.py)
166 _show (native_dialog.py)
139 _signature (function.py)
104 _size_of (types_xml.py)
174 _skip_block_comment (config_file.py)
174 _skip_line_comment (config_file.py)
174 _skip_string (config_file.py)
132 _snippet_body (c_style.py)
102 _source_file (xml_writer.py)
89 _stamp (parameter_list.py)
177 _stamp_system_names (document.py)
_struct_of_value_type (run_validation_test.py)
233
150 _table_entry (parameter_list_c.py)
152 _tag (parameter_list_table_h.py)
151 _tag_rows (parameter_list_table_h.py)
_tag_total_entries (parameter_list_defines_h.py)
146
71 _take (choices.py)
101 _text (xml_writer.py)
234 _texts_agree (run_editor_test.py)
104 _to_bool (types_xml.py)
100 _to_bool (xml_reader.py)
104 _to_int (types_xml.py)
100 _to_int (xml_reader.py)
101 _to_number (xml_reader.py)
101 _to_optional_int (xml_reader.py)
94 _type_from_node (struct_migration.py)
233 _typeless_struct (run_validation_test.py)
112 _TypeProblems (struct_type_rules.py)
88 _undeclared_types (parameter_list.py)
190 _unique_name (document.py)
60 _use_utf8 (logging_utils.py)
186 _uses_of_type (document.py)
152 _value_note (parameter_list_table_h.py)
151 _value_rows (parameter_list_table_h.py)
109 _walk_enabled (rule_base.py)
109 _walk_paths (rule_base.py)
69 _warn_unknown_keys (config.py)
69 _warn_unknown_sections (config.py)
185 _why_not_added (document.py)
154 _widths (param_table_h.py)
148 _without_element (smart_data_c.py)
172 _write_settings (workspace.py)

A

241 a_structure_in (run_paramtable_test.py)
239 a_type (run_struct_test.py)
217 Action (run.py)
204 add (api.js)
198 add (api.py)
79 add (defines.py)
58 add (errors.py)
85 add (parameter_list.py)
241 add (run_paramtable_test.py)
198 add_existing_list (api.py)
185 add_existing_list (document.py)

C

41 c.json
131 c_style.py
154 c_type_of (param_table_h.py)
204 call (api.js)
177 can_redo (document.py)
177 can_undo (document.py)
208 captureFocus (editor.js)
237 case (run_settings_test.py)
241 cells_of (run_paramtable_test.py)
208 check (editor.js)
159 check (pipeline.py)
108 check (rule_base.py)
233 check (run_editor_test.py)
221 check_format (set_version.py)
233 check_messages (run_validation_test.py)
244 check_one (run_compile_test.py)
220 check_python_version (setup_environment.py)
check_standard_modules (setup_environment.py) 220
238 check_the_launchers (run_docs_test.py)
220 check_tool_imports (setup_environment.py)
166 child_command (native_dialog.py)
40 choices.json
70 choices.py
206 choicesFor (grid.js)
204 choose (api.js)
197 choose (api.py)
207 clear (editor.js)
160 cli.py
172 close (workspace.py)
197 close_project (api.py)
213 closeModal (app.js)
204 closeProject (api.js)
210 closeProject (app.js)
67 CodegenConfig (config.py)
56 CodeGeneratorError (errors.py)
241 codes (run_paramtable_test.py)
209 collect (editor.js)
60 colour_supported (logging_utils.py)
138 Comment (comment.py)
138 comment.py
208 commit (editor.js)
128 compare (drift.py)
244 compile_c (run_compile_test.py)
244 compile_cxx (run_compile_test.py)
compile_firmware_defines (run_compile_test.py) 244
67 Config (config.py)
171 config (workspace.py)
64 config.py
173 config_file.py
75 constructor (types.py)
136 constructor_arguments (literals.py)
79 contains (defines.py)
100 convert_tag (xml_reader.py)
91 ,90 copy (struct_type.py)
84 copy_as_leaf (parameter.py)
54 copy_lib_into (app_home.py)
238 copy_of_init (run_version_test.py)
58 count (errors.py)
213 countBySeverity (app.js)
213 countLeaves (app.js)
213 countStructs (app.js)
141 cpp_guard.py
141 CppGuard (cpp_guard.py)
172 create (workspace.py)
185 create_list (document.py)
187 create_struct_type (document.py)
132 CStyle (c_style.py)
221 current_version (set_version.py)
74 CValueType (types.py)

D

208 dataTypeSelect (editor.js)
213 declarationPath (app.js)
108 declared_values (rule_base.py)
115 DeclaredTypeRule (type_rules.py)
237 deep (run_settings_test.py)
70 default_choices (choices.py)
198 default_defines (api.py)
70 default_defines (choices.py)
53 default_snippet_file (app_home.py)
204 defaultDefines (api.js)
116 DefaultInRangeRule (range_rules.py)
137 Define (define.py)
137 define.py
79 Defines (defines.py)
78 defines.py
144 defines_header (file_builder.py)
109 defines_rules.py
109 DefinesSheetRule (defines_rules.py)
198 delete (api.py)
179 delete (document.py)
187 delete_struct_type (document.py)
199 delete_type (api.py)
205 deleteType (api.js)
84 description_at (parameter.py)
144 description_of (file_builder.py)
84 description_over (parameter.py)
207 descriptionField (editor.js)
descriptor_field_order (descriptor_layout.py) 133
133 descriptor_layout.py
144 descriptor_symbol (file_builder.py)
57 Diagnostic (errors.py)
58 DiagnosticCollector (errors.py)
166 DialogBusy (native_dialog.py)
177 dirty (document.py)
85 disabled (parameter_list.py)
194 document (api.py)
175 Document (document.py)
175 document.py
109 dotted_names (rule_base.py)
126 drift.py

212 driftHtml (app.js)
213 driveRow (app.js)
204 duplicate (api.js)
198 duplicate (api.py)
179 duplicate (document.py)

E

211 editLists (app.js)
211 editListSelection (app.js)
212 editOneType (app.js)
207 editor.js
212 editTypes (app.js)
210 editWhereItLives (app.js)
83 effective_log_id (parameter.py)
83 effective_maximum (parameter.py)
83 effective_minimum (parameter.py)
82 effective_tag (parameter.py)
80 element_key (parameter.py)
85 enabled (parameter_list.py)
82 enabled_children (parameter.py)
109 enabled_leaves (rule_base.py)
109 enabled_nodes (rule_base.py)
109 enabled_structs (rule_base.py)
138 Enum (enum.py)
138 enum.py
75 enum_name (types.py)
233 equal (run_editor_test.py)
241 equal (run_paramtable_test.py)
239 equal (run_struct_test.py)
58 error (errors.py)
190 error_count (document.py)
56 errors.py
134 escape_string (literals.py)
206 escapeHtml (grid.js)
241 every_value_shown (run_paramtable_test.py)
104 existing_types_file_for (types_xml.py)
213 expandTo (app.js)
86 extend (parameter_list.py)

F

209 failed (app.js)
233 Failure (run_editor_test.py)
241 Failure (run_paramtable_test.py)
239 Failure (run_struct_test.py)
233 field (run_validation_test.py)
90 field_named (struct_type.py)
140 file_banner.py
143 file_builder.py
140 file_footer.py
143 file_name (file_builder.py)
140 FileBanner (file_banner.py)
143 FileBuilder (file_builder.py)
140 FileFooter (file_footer.py)
213 fileListHtml (app.js)
178 find (document.py)
86 find (parameter_list.py)
218 find (run.py)
165 find_browser (browser.py)
216 find_python.bat
186 find_struct_type (document.py)
213 findNode (app.js)
213 firstRowPath (app.js)
142 fixed_tags (__init__.py)
86 flatten (parameter_list.py)
135 float_literal (literals.py)
207 fmtBool (grid.js)
206 fmtNumber (grid.js)
206 fmtSpan (grid.js)
213 folderFor (app.js)
213 folderListHtml (app.js)
206 followLink (grid.js)
132 footer_line (c_style.py)
85 ForeignType (parameter_list.py)
172 forget (workspace.py)
61 format (logging_utils.py)
210 formatName (app.js)
119 FreeTagSyntaxRule (tag_rules.py)
139 Function (function.py)
139 function.py

G

204 generate (api.js)
197 generate (api.py)
212 generate (app.js)
137 generate (define.py)
189 generate (document.py)
138 generate (enum.py)
140 generate (file_banner.py)
144 generate (file_builder.py)
140 generate (file_footer.py)
137 generate (include.py)
158 generate (pipeline.py)
237 generate (run_settings_test.py)
141 generate (section_marker.py)
140 generate (table.py)
137 generate_aligned_block (define.py)
139 generate_begin (function.py)
138 generate_block (comment.py)
139 generate_body (function.py)
141 generate_close (cpp_guard.py)
141 generate_close (include_guard.py)
139 generate_declaration (variable.py)
139 generate_definition (variable.py)
138 generate_doc (comment.py)
139 generate_end (function.py)
137 generate_guarded_block (define.py)
138 generate_line (comment.py)
141 generate_open (cpp_guard.py)
141 generate_open (include_guard.py)
139 generate_prototype (function.py)
138 generate_separator (comment.py)
68 generated_file_name (config.py)
111 GeneratedIdentifierRule (name_rules.py)
144 generation_notes (file_builder.py)
79 get (defines.py)

90 get (struct_type.py)
61 get_logger (logging_utils.py)
210 goToProblem (app.js)
210 goToType (app.js)
205 grid.js
144 guard_name (file_builder.py)
169 GuiServer (server.py)

H

241 hand_over (run_paramtable_test.py)
194 handlers (api.py)
83 has_active_tag (parameter.py)
126 has_entry (tag_offset.py)
58 has_errors (errors.py)
75 has_limit_macros (types.py)
178 has_list (document.py)
75 has_range (types.py)
58 has_warnings (errors.py)
243 have_compiler (run_compile_test.py)

I

137 Include (include.py)
137 include.py
64 include_guard (naming.py)
141 include_guard.py
141 IncludeGuard (include_guard.py)
202 index.html
63 index_macro (naming.py)
144 index_of (file_builder.py)
110 IndexMacroCollisionRule (name_rules.py)
241 indices_shown (run_paramtable_test.py)
58 info (errors.py)
208 inheritedGroup (editor.js)
207 init (editor.js)
206 init (grid.js)
233 inner (run_validation_test.py)
243 input_variants (run_compile_test.py)
66 InputConfig (config.py)
198 inspect_list (api.py)
185 inspect_list (document.py)
205 inspectList (api.js)
135 integer_literal (literals.py)
117 IntegralValueRule (range_rules.py)
82 is_array (parameter.py)
75 is_bool (types.py)
77 is_boolean (tags.py)
128 is_empty (drift.py)
75 is_float (types.py)
171 is_open (workspace.py)
82 is_primitive (parameter.py)
52 is_project_file (app_home.py)
62 is_reserved_keyword (naming.py)
82 is_struct (parameter.py)
103 is_types_file (types_xml.py)
62 is_valid_c_identifier (naming.py)
207 isBeingTypedIn (editor.js)
206 isOpen (grid.js)
210 isParamTable (app.js)
207 isTextBox (editor.js)
58 items (errors.py)

K

208 keepsWhatIsOnScreen (editor.js)
184 known_lists (document.py)

L

56 label (errors.py)
172 last (workspace.py)
55 last_project (app_home.py)
128 LayoutDrift (drift.py)
104 legacy_types_file_for (types_xml.py)
136 limit_headers (literals.py)
184 list_directory (document.py)
144 list_header (file_builder.py)
144 list_source (file_builder.py)
190 listing_target (document.py)
204 lists (api.js)
198 lists (api.py)
134 literals.py
213 liveListing (app.js)
132 load (c_style.py)
67 load (config.py)
176 load (document.py)
71 load_choices (choices.py)
65 load_jsonc (config.py)
158 load_model (pipeline.py)
125 log_id_span (tag_offset.py)
59 logging_utils.py
208 logIdField (editor.js)
119 LogIdRule (tag_rules.py)
203 logo.png

M

190 machine_roots (document.py)
162 main (cli.py)
166 main (native_dialog.py)
218 main (run.py)
244 main (run_compile_test.py)
238 main (run_docs_test.py)
234 main (run_editor_test.py)
241 main (run_paramtable_test.py)
237 main (run_settings_test.py)
239 main (run_struct_test.py)
233 main (run_validation_test.py)
239 main (run_version_test.py)
222 main (set_version.py)
220 main (setup_environment.py)
244 main_source (run_compile_test.py)
237 make_project (run_settings_test.py)
78 mandatory_tags (tags.py)
118 MandatoryTagRule (tag_rules.py)
132 marker (c_style.py)
213 markPicked (app.js)

206 matches (grid.js)
210 mayLeaveTheList (app.js)
218 menu (run.py)
79 merge (defines.py)
91 merge (struct_type.py)
87 merge_lists (parameter_list.py)
188 merged_model (document.py)
189 merged_parts (document.py)
117 MetadataRule (range_rules.py)
116 MinMaxOrderRule (range_rules.py)
213 missingNote (app.js)
233 model (run_validation_test.py)
66 model_file (config.py)
101 model_to_element (xml_writer.py)
204 move (api.js)
198 move (api.py)
179 move (document.py)
212 moveNode (app.js)

N

110 name_rules.py
90 names (struct_type.py)
110 NameSyntaxRule (name_rules.py)
110 NameUniquenessRule (name_rules.py)
62 naming.py
165 native_dialog.py
211 nativeDialogs (app.js)
166 NativeDialogUnavailable (native_dialog.py)
116 NativeRangeRule (range_rules.py)
80 nearest_text (parameter.py)
198 new_list (api.py)
197 new_project (api.py)
241 new_project (run_paramtable_test.py)
239 new_project (run_struct_test.py)
198 new_type (api.py)
205 newList (api.js)
211 newList (app.js)
204 newProject (api.js)
211 newProjectDialog (app.js)
205 newType (api.js)
212 newType (app.js)
189 next_free_log_id (document.py)
198 next_log_id (api.py)
204 nextLogId (api.js)
208 number (editor.js)
79 number_of (defines.py)
79 numbered (defines.py)
135 numeric_literal (literals.py)

O

126 offset (tag_offset.py)
241 offset_rows (run_paramtable_test.py)
64 offset_total_macro (naming.py)
144 offset_total_of (file_builder.py)
125 OffsetArea (tag_offset.py)
125 OffsetEntry (tag_offset.py)
125 OffsetMap (tag_offset.py)
128 OffsetMove (drift.py)
206 onSteadyClick (grid.js)
204 open (api.js)
206 open (grid.js)
171 open (workspace.py)
234 open_copy (run_editor_test.py)
197 open_file (api.py)
197 open_project (api.py)
206 openEverything (grid.js)
211 openList (app.js)
204 openProject (api.js)
210 openProject (app.js)
211 openProjectDialog (app.js)
211 openProjectFlow (app.js)
78 optional_tags (tags.py)
209 optionList (editor.js)
67 OutputConfig (config.py)
206 ownerOf (grid.js)

P

209 paint (app.js)
61 paint (logging_utils.py)
210 paintAddMenu (app.js)
210 paintCounts (app.js)
210 paintModules (app.js)
210 paintProblems (app.js)
210 paintProject (app.js)
212 paintThemeLabel (app.js)
79 param_modes (defines.py)
154 param_offsets_h.py
152 param_table_h.py
120 param_table_rules.py
79 param_types (defines.py)
81 Parameter (parameter.py)
80 parameter.py
228 parameter_descriptor.c
227 parameter_descriptor.h
85 parameter_list.py
150 parameter_list_c.py
145 parameter_list_defines_h.py
149 parameter_list_h.py
151 parameter_list_table_h.py
229 parameter_list_usage_example.c
147 parameter_types_h.py
81 ParameterKind (parameter.py)
85 ParameterList (parameter_list.py)
ParameterListDefinesHeader
146 (parameter_list_defines_h.py)
149 ParameterListHeader (parameter_list_h.py)
150 ParameterListSource (parameter_list_c.py)
ParameterListTableHeader
151 (parameter_list_table_h.py)
144 parameters (file_builder.py)
115 ParameterTypeRule (type_rules.py)
147 ParameterTypesHeader (parameter_types_h.py)
155 ParamOffsetsHeader (param_offsets_h.py)
121 ParamTableAddressRule (param_table_rules.py)
154 ParamTableHeader (param_table_h.py)

120 ParamTableRule (param_table_rules.py)
120 ParamTableShapeRule (param_table_rules.py)
121 ParamTableSystemRule (param_table_rules.py)
ParamTableSystemTagRule (param_table_rules.py)
121
120 ParamTableTypeRule (param_table_rules.py)
178 parent_of (document.py)
71 parse_numbered (choices.py)
211 pickNative (app.js)
158 Pipeline (pipeline.py)
157 pipeline.py
157 PipelineResult (pipeline.py)
40 pl_project.json
154 plain_number (param_table_h.py)
129 previous_build (drift.py)
208 primitiveForm (editor.js)
210 problemPaths (app.js)
53 project_file_in (app_home.py)
53 project_files_in (app_home.py)
55 project_name (app_home.py)
66 ProjectConfig (config.py)
90 put (struct_type.py)

Q

144 quantity (file_builder.py)

R

82 range_disabled (parameter.py)
116 range_rules.py
237 read (run_settings_test.py)
238 read_doc (run_docs_test.py)
54 read_settings (app_home.py)
104 read_types (types_xml.py)
98 read_xml (xml_reader.py)
212 readDraftFields (app.js)
15 README.md
208 readOnly (editor.js)
172 recent (workspace.py)
204 redo (api.js)
198 redo (api.py)
177 redo (document.py)
209 refresh (app.js)
238 refused (run_version_test.py)
66 relative_to_root (config.py)
209 reload (app.js)
172 remember (workspace.py)
204 remove (api.js)
212 remove (app.js)
90 remove (struct_type.py)
198 remove_list (api.py)
185 remove_list (document.py)
205 removeList (api.js)
211 removeList (app.js)
91 rename (struct_type.py)
59 ,57 render (errors.py)
206 render (grid.js)
160 render_diagnostics (pipeline.py)
211 renderDraftFields (app.js)
128 report (drift.py)
218 report (run.py)
113 report (struct_type_rules.py)
171 require (workspace.py)
117 RequiredValuesRule (range_rules.py)
176 reread_project (document.py)
125 reserved_log_ids (tag_offset.py)
83 reserved_size (parameter.py)
124 reserved_size (tag_offset.py)
161 resolve_config_path (cli.py)
55 resolve_recent_entry (app_home.py)
76 resolve_value_type (types.py)
208 restoreFocus (editor.js)
206 reveal (grid.js)
173 rewrite_model_files (config_file.py)
181 row_chain (document.py)
207 rowHint (editor.js)
206 rowKey (grid.js)
206 rows (grid.js)
107 rule_base.py
107 rules (validator.py)
159 run (pipeline.py)
218 run (run.py)
215 run.bat
217 run.py
216 run.sh
233 run_case (run_validation_test.py)
243 run_compile_test.py
238 run_docs_test.py
233 run_editor_test.py
218 run_generate.py
218 run_gui.py
240 run_paramtable_test.py
237 run_settings_test.py
219 run_setup.py
239 run_struct_test.py
219 run_tests.py
232 run_validation_test.py
238 run_version_test.py
212 runShortcut (app.js)

S

134 safe_comment (literals.py)
213 samePath (app.js)
204 save (api.js)
197 save (api.py)
212 save (app.js)
176 save (document.py)
144 save (file_builder.py)
72 save_choices (choices.py)
95 schema.py
141 section_marker.py
141 SectionMarker (section_marker.py)
72 seed_from (choices.py)
210 select (app.js)
208 select (editor.js)
198 select_lists (api.py)

208 structForm (editor.js)
111 StructRule (name_rules.py)
90 StructType (struct_type.py)
114 StructTypeClashRule (struct_type_rules.py)
113 StructTypeCycleRule (struct_type_rules.py)
StructTypeDeclarationRule (struct_type_rules.py)
113
90 StructTypeLibrary (struct_type.py)
114 StructTypeOwnerRule (struct_type_rules.py)
112 StructTypeRule (struct_type_rules.py)
85 structures_by_type (parameter_list.py)
203 style.css
67 StyleConfig (config.py)
128 summary (drift.py)
59 summary (errors.py)
76 supported_type_names (types.py)
79 system_names (defines.py)
155 system_names_h.py
153 system_of (param_table_h.py)
156 SystemNamesHeader (system_names_h.py)
213 systemOf (app.js)

T

140 Table (table.py)
140 table.py
144 table_header (file_builder.py)
212 tableDriftHtml (app.js)
213 tableFileName (app.js)
82 tag (parameter.py)
123 tag_offset.py
118 tag_rules.py
64 tag_size_function (naming.py)
144 tag_size_function_of (file_builder.py)
136 tag_value_literal (literals.py)
118 TagEnumValueRule (tag_rules.py)
77 TagKind (tags.py)
232 tags (run_validation_test.py)
76 tags.py
208 tagsGroup (editor.js)
77 TagSpec (tags.py)
119 TagValueRangeRule (tag_rules.py)
test_a_boolean_is_refused
241 (run_paramtable_test.py)
test_a_boolean_keeps_its_shape
235 (run_editor_test.py)
test_a_copy_is_named_like_a_copy
236 (run_editor_test.py)
test_a_disabled_parameter_takes_nothing
242 (run_paramtable_test.py)
test_a_drop_down_opens_the_first_time
234 (run_editor_test.py)
test_a_field_of_a_struct_array_too
235 (run_editor_test.py)
test_a_fields_place_belongs_to_the_type
240 (run_struct_test.py)
test_a_fields_shape_belongs_to_the_type
240 (run_struct_test.py)
186 select_lists (document.py)
209 selectedNode (app.js)
205 selectLists (api.js)
208 send (editor.js)
169 serve_forever (server.py)
168 server.py
198 set_defines (api.py)
188 set_defines (document.py)
222 set_version (set_version.py)
220 set_version.bat
221 set_version.py
204 setDefines (api.js)
211 setEnabled (app.js)
213 setStatus (app.js)
219 setup_environment.py
61 setup_logging (logging_utils.py)
56 Severity (errors.py)
90 shape (struct_type.py)
207 show (editor.js)
218 show_menu (run.py)
213 showModal (app.js)
126 size (tag_offset.py)
144 sized_tags (file_builder.py)
210 sizeProblems (app.js)
226 smart_data.c
223 smart_data.h
148 smart_data_c.py
68 smart_data_file_name (config.py)
147 smart_data_h.py
144 smart_data_header (file_builder.py)
144 smart_data_of (file_builder.py)
144 smart_data_source (file_builder.py)
75 smart_data_struct (types.py)
64 smart_data_symbol (naming.py)
148 SmartDataHeader (smart_data_h.py)
148 SmartDataSource (smart_data_c.py)
177 snapshot (document.py)
57 SourceLocation (errors.py)
241 squeeze (run_paramtable_test.py)
209 start (app.js)
201 start_folder (api.py)
211 startFolder (app.js)
204 state (api.js)
195 state (api.py)
170 stop (server.py)
54 store_recent_entry (app_home.py)
63 strip_array_indices (naming.py)
233 struct (run_validation_test.py)
93 struct_migration.py
87 struct_parameters (parameter_list.py)
91 struct_sync.py
89 struct_type.py
64 struct_type_name (naming.py)
144 struct_type_of (file_builder.py)
112 struct_type_rules.py
186 struct_types (document.py)
87 struct_types_in_use (parameter_list.py)
89 StructField (struct_type.py)

```
test_a_file_that_names_no_system_is_not_stamped
                242  (run_paramtable_test.py)
test_a_free_tag_takes_a_number
                236  (run_editor_test.py)
test_a_hand_written_address_is_checked
                242  (run_paramtable_test.py)
test_a_hint_names_a_button_the_editor_has
                236  (run_editor_test.py)
test_a_list_cannot_use_another_lists_type
                240  (run_struct_test.py)
test_a_list_from_another_project
                239  (run_struct_test.py)
test_a_list_from_elsewhere_is_copied_in
                240  (run_struct_test.py)
test_a_list_on_disk_that_disagrees_warns
                242  (run_paramtable_test.py)
test_a_list_with_an_older_types_file_comes_whole
                240  (run_struct_test.py)
test_a_long_name_reaches_the_table_in_full
                240  (run_struct_test.py)
test_a_malformed_version_changes_nothing
                238  (run_version_test.py)
test_a_new_field_starts_with_no_warning
                235  (run_editor_test.py)
test_a_new_list_must_name_a_sub_system
                242  (run_paramtable_test.py)
test_a_new_parameter_carries_its_lists_sub_system
                242  (run_paramtable_test.py)
test_a_new_parameter_is_never_a_broken_structure
                240  (run_struct_test.py)
test_a_new_project_gets_no_library
                242  (run_paramtable_test.py)
test_a_new_row_in_the_middle_is_the_one_that_warn
                242  s (run_paramtable_test.py)
test_a_parameter_cannot_be_a_structure
                239  (run_struct_test.py)
test_a_problem_in_another_list_can_be_reached
                236  (run_editor_test.py)
test_a_project_file_with_the_old_name_still_opens
                236  (run_editor_test.py)
test_a_recent_entry_works_on_any_system
                236  (run_editor_test.py)
test_a_refused_structure_leaves_nothing
                240  (run_struct_test.py)
test_a_refused_type_edit_changes_nothing
                240  (run_struct_test.py)
test_a_renamed_project_file_opens
                237  (run_editor_test.py)
test_a_row_put_somewhere_new_is_brought_into_sigh
                235  t (run_editor_test.py)
test_a_smart_data_project_has_no_param_mode
                236  (run_editor_test.py)
test_a_smart_data_project_takes_any_name
                242  (run_paramtable_test.py)
test_a_structure_that_arrived_from_elsewhere
                241  (run_paramtable_test.py)
test_a_sub_list_counts_from_zero
                242  (run_paramtable_test.py)
test_a_sub_system_has_one_list
                242  (run_paramtable_test.py)
test_a_type_belongs_to_its_list
                234  (run_editor_test.py)
test_a_type_cannot_contain_itself
                239  (run_struct_test.py)
test_a_type_nothing_uses_cannot_block_generate
                240  (run_struct_test.py)
test_a_typed_path_is_listed_at_once
                235  (run_editor_test.py)
test_a_value_has_a_description_of_its_own
                235  (run_editor_test.py)
test_a_value_has_a_user_name_of_its_own
                235  (run_editor_test.py)
test_a_workspace_keeps_to_its_own_settings_file
                236  (run_editor_test.py)
test_an_abandoned_copy_is_reported
                242  (run_paramtable_test.py)
test_an_array_that_arrived_from_elsewhere
                241  (run_paramtable_test.py)
test_an_existing_file_must_name_a_sub_system
                242  (run_paramtable_test.py)
test_an_old_badly_named_list_still_reorders
                242  (run_paramtable_test.py)
test_an_old_python_is_told_in_red
                236  (run_editor_test.py)
test_an_old_shared_text_goes_onto_every_row
                235  (run_editor_test.py)
test_an_unused_type_stays_until_it_is_deleted
                240  (run_struct_test.py)
test_another_lists_type_never_reshapes_this_one
                240  (run_struct_test.py)
test_changing_the_value_type (run_editor_test.py)
                                        235
test_deleting_a_list_takes_its_structures
                240  (run_struct_test.py)
test_deleting_a_structure (run_struct_test.py)
                                        239
test_deleting_a_structure_keeps_its_type
                240  (run_struct_test.py)
test_editing_a_type_reaches_every_parameter
                239  (run_struct_test.py)
test_enabling_a_field_belongs_to_the_type
                240  (run_struct_test.py)
test_every_browser_path_can_exist
                236  (run_editor_test.py)
test_every_door_asks_for_the_same_python
                236  (run_editor_test.py)
test_every_drive_can_be_reached
                235  (run_editor_test.py)
234  test_every_pane_redraws (run_editor_test.py)
234  test_layout_warning (run_editor_test.py)
test_limits_are_the_parameters_own
                239  (run_struct_test.py)
test_line_endings_survive (run_version_test.py)
                                        238
test_list_selection_edge_cases
                234  (run_editor_test.py)
```

```
test_the_first_click_is_never_lost
      234  (run_editor_test.py)
test_the_grid_finds_a_rows_values_in_its_own_entr
      236  y (run_editor_test.py)
test_the_grid_selects_one_value
      235  (run_editor_test.py)
test_the_ignored_tags_say_nothing
      241  (run_paramtable_test.py)
test_the_lists_are_numbered_defines
      237  (run_editor_test.py)
test_the_merged_types_file_holds_what_the_project
      240  _uses (run_struct_test.py)
test_the_merged_view_names_every_rows_list
      235  (run_editor_test.py)
test_the_messages_name_this_machines_launcher
      236  (run_editor_test.py)
test_the_modes_come_from_the_machine
      241  (run_paramtable_test.py)
test_the_new_version_is_written
      238  (run_version_test.py)
test_the_offsets_note (run_paramtable_test.py)
      242
test_the_output_lands_where_it_is_told
      241  (run_paramtable_test.py)
test_the_output_names_lists_from_the_project_root
      235  (run_editor_test.py)
test_the_project_file_is_plain_json
      236  (run_editor_test.py)
test_the_real_file_has_one_version_line
      239  (run_version_test.py)
test_the_same_list_in_both_formats
      241  (run_paramtable_test.py)
test_the_same_name_in_two_sub_lists
      242  (run_paramtable_test.py)
test_the_same_version_writes_nothing
      239  (run_version_test.py)
test_the_shortcuts_are_bound_to_the_key
      236  (run_editor_test.py)
test_the_sub_lists_are_numbered_from_one
      236  (run_editor_test.py)
test_the_system_names_have_a_file_of_their_own
      242  (run_paramtable_test.py)
test_the_systems_own_dialog (run_editor_test.py)
      235
test_the_table_notes_name_what_is_there
      240  (run_struct_test.py)
test_the_table_numbers_modes_by_their_place
      242  (run_paramtable_test.py)
test_the_title_names_the_project_file
      237  (run_editor_test.py)
test_the_types_file_travels_with_the_list
      239  (run_struct_test.py)
test_the_types_file_was_renamed
      240  (run_struct_test.py)
test_the_unsaved_mark_means_different_from_the_fi
      235  le (run_editor_test.py)
test_unit_and_description_belong_to_the_use
      240  (run_struct_test.py)
test_no_browser_says_so (run_editor_test.py)
      236
test_no_external_dependency (run_editor_test.py)
      234
test_no_list_no_parameter (run_editor_test.py)
      235
test_no_version_line_changes_nothing
      238  (run_version_test.py)
test_none_is_the_first_mode
      242  (run_paramtable_test.py)
test_nothing_else_writes_the_version_by_hand
      239  (run_version_test.py)
test_nothing_replaces_the_open_list_in_silence
      236  (run_editor_test.py)
test_one_file_per_project
      241  (run_paramtable_test.py)
test_one_file_shape_for_both_formats
      235  (run_editor_test.py)
test_one_name_for_one_thing (run_editor_test.py)
      234
test_one_name_one_type (run_struct_test.py)
      239
test_only_a_project_file_opens
      235  (run_editor_test.py)
test_opening_a_list_already_in_the_build
      235  (run_editor_test.py)
test_opening_a_list_writes_in_its_sub_system
      242  (run_paramtable_test.py)
test_parameter_lists (run_editor_test.py)
      234
test_renaming_a_type (run_struct_test.py)
      239
test_reordering_the_lists_moves_nothing
      242  (run_paramtable_test.py)
test_structure_arrays (run_editor_test.py)
      234
test_structure_types (run_editor_test.py)
      234
test_the_address_column_comes_before_the_descript
      236  ion (run_editor_test.py)
test_the_address_is_a_field_of_its_own
      242  (run_paramtable_test.py)
test_the_browser_can_always_write_its_errors
      236  (run_editor_test.py)
test_the_check_and_ok_agree
      242  (run_paramtable_test.py)
test_the_columns (run_paramtable_test.py)
      241
test_the_command_line (run_version_test.py)
      239
test_the_dialogs_offer_the_drives
      235  (run_editor_test.py)
test_the_dialogs_start_where_the_project_is
      235  (run_editor_test.py)
test_the_dialogs_try_the_system_first
      235  (run_editor_test.py)
test_the_editor_sets_them_but_not_the_sub_system
      241  (run_paramtable_test.py)
test_the_editor_will_not_build_one
      241  (run_paramtable_test.py)
test_the_empty_form_names_the_add_button
      236  (run_editor_test.py)
test_the_file_it_replaces
      241  (run_paramtable_test.py)
test_the_file_name_names_the_system
      241  (run_paramtable_test.py)
```

208 text (editor.js)
181 text_above (document.py)
58 ,57 to_dict (errors.py)
58 to_list (errors.py)
63 to_macro_case (naming.py)
213 toast (app.js)
206 toggleRow (grid.js)
212 toggleTheme (app.js)
144 top_level_objects (file_builder.py)
126 total (tag_offset.py)
213 trackTopbarHeight (app.js)
241 true (run_paramtable_test.py)
239 true (run_struct_test.py)
80 type_enum_member (defines.py)
154 type_enum_of (param_table_h.py)
92 type_of (struct_sync.py)
114 type_rules.py
79 types (defines.py)
74 types.py
104 types_file_for (types_xml.py)
144 types_header (file_builder.py)
104 types_to_element (types_xml.py)
93 types_used_by (struct_sync.py)
103 types_xml.py

U

206 under (grid.js)
204 undo (api.js)
198 undo (api.py)
177 undo (document.py)
145 unit_of (file_builder.py)
134 unsigned_literal (literals.py)
204 update (api.js)
198 update (api.py)
180 update (document.py)
187 update_struct_type (document.py)
199 update_type (api.py)
210 updateButtons (app.js)
205 updateType (api.js)
170 url (server.py)
187 use_struct_type (document.py)
199 use_type (api.py)
85 used_data_type_suffixes (parameter_list.py)
84 user_name_at (parameter.py)
84 user_name_over (parameter.py)
207 userNameField (editor.js)
119 UserNameRule (tag_rules.py)
91 uses_of (struct_type.py)
205 useType (api.js)
212 useType (app.js)

V

68 validate (config.py)
189 validate (document.py)
107 validate (validator.py)
189 validate_codes (document.py)
67 ValidationConfig (config.py)
107 ValidationRule (rule_base.py)
106 Validator (validator.py)
106 validator.py
233 value (run_validation_test.py)
180 value_at (document.py)
75 value_size (types.py)
82 value_type (parameter.py)
206 valueIndex (grid.js)
207 valueShown (editor.js)
213 valuesUnder (app.js)
115 ValueTypeRule (type_rules.py)
139 Variable (variable.py)
139 variable.py
221 VersionError (set_version.py)

W

82 walk (parameter.py)
85 walk (parameter_list.py)
58 warning (errors.py)
210 windowTitle (app.js)
208 wire (editor.js)
206 wire (grid.js)
170 Workspace (workspace.py)
170 workspace.py
243 write_config (run_compile_test.py)
54 write_settings (app_home.py)
104 write_types (types_xml.py)
102 write_xml (xml_writer.py)

X

98 xml_reader.py
101 xml_writer.py